

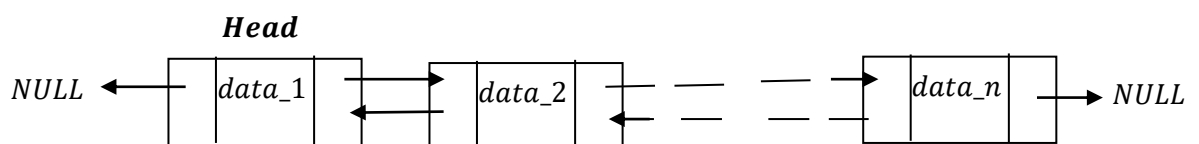
Doubly Linked List – Introduction

A doubly linked list is a linked list in which a node contains a pointer to the previous as well as the next node in the sequence. Therefore, a doubly linked list is also called two – way linked list. It consists of three parts : data, a pointer to the next node in sequence (next pointer), a pointer to the previous node(previous pointer). Each node has two links: one point to the previous node, or points to a NULL value or empty list if it is the first node, and one point to the next, or points to a NULL value or empty list if it is the final node.

The advantage of a doubly linked list is that given a node in the list, we can navigate in both directions. A node in a singly linked list cannot be removed unless we have the pointer to its predecessor. But in a doubly linked list, we can delete a node even if we don't have the previous node address (since each node has a left pointer pointing to the previous node and can remove backward).

The primary disadvantages of doubly linked lists are:

- → *Each node requires an extra pointer ,requiring more space.*
- → *The insertion or deletion of a node takes a bit longer(for more pointer operation).*



***A simple representation of Doubly Data
Linked List .***

Doubly Linked List – Creation of Empty List And Time Complexity.

Program :

```
#include <cstdlib>
#include <iostream>

using namespace std;

typedef struct DLL
{
    int data;
    struct DLL *next;
    struct DLL *prev;
}Node;

Node *head;

void createEmptyList()
{
    head = nullptr;
}

int main()
{
    createEmptyList();
    return 0;
}
```

→ *Node, is a structure variable i. e. the name given to struct DLL achieved through typedef keyword, gives new name to existing data types, here for struct DLL. Struct is a composite data type which contains group of several variables or different data types , grouped under a single name or single type.*

Node is given name to struct DLL ,As shown below:

```
typedef struct DLL
{...} Node;
```



We can also write it as:

```
typedef struct DLL
{
    int data;
    DLL *next;
    DLL *prev;
}Node;
```

And , now :

```
Node *head;
```

Without , typedef we can rewrite it as:

```
struct Node
{
    int data;
    Node *next;
    Node *prev;
};

Node *head;
```

OR

```
struct Node
{
    int data;
    struct Node *next;
    struct Node *prev;
};

struct Node *head;
```

Now ,

```
Node *head;
```

is a global variable, which are accessible to any part of the program, and as we need to start from head and keep track of it , its easy to perform by making the variable global otherwise with local scope we have to create each seperate head variable while calling each modular functions.

```
#include <cstdlib>
#include <iostream>

using namespace std;

typedef struct DLL
{
    int data;
    DLL *next;
    DLL *prev;
}Node;

Node *head;
```

The Rule: Zero – Initialization

In C ++, any variable declared at the global scope (outside of all functions) has static storage duration. The C ++ standard guarantees that variables with static storage duration are zero – initialized by the compiler if you do not explicitly initialize them.

For a pointer, "zero – initialization" means it is automatically set to a null pointer.

Node * head

Address Head

ff21

NULLPTR

*And , then * head initialized to null pointer(explicitly).*

```
void createEmptyList()
{
    head = nullptr;
}
```



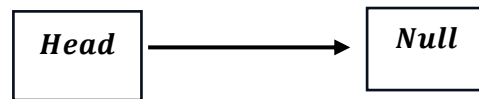
*Note: It is just explicitly assigning Node * head to Nullptr, which is already assigned with NULLPTR ,since Node * head is global pointer variable, following :The Zero Initialization Rule.*

C + + version 11 → Null Pointer vs MACRO NULL

In C + +03 and earlier versions, there was no dedicated null pointer literal. Developers often used the constant 0 or macros like NULL to represent null pointers. The problem with this approach is that an integer value of 0 could accidentally be assigned to a pointer variable, leading to unexpected behavior and potential crashes.

In essence, nullptr is a special literal value that represents the absence of a valid pointer in C + +. It was introduced in the C + +11 standard to enhance type safety and improve code clarity. The nullptr, however, is a distinct type, std :: nullptr_t, designed explicitly for pointer absence. By using nullptr, the compiler can enforce type safety and distinguish between an intentionally null pointer and an integer value. This reduces the potential for errors caused by accidental assignment of integers to pointers. The nullptr enhances code readability and self – documentation. Its explicit representation of a null pointer makes the code's intent clearer. Using nullptr makes the code more readable and less prone to errors. This helps prevent subtle bugs that could arise from accidentally assigning the wrong value to a pointer. The nullptr is a dedicated keyword that represents the null pointer value.

The nullptr is a keyword introduced in C + +11 to represent a null pointer. It is used to indicate that a pointer does not point to any object or function. The nullptr has its own type, std :: nullptr_t, which can be implicitly converted to any pointer type but not to integral types. This helps prevent errors where a null pointer might be confused with an integer. We can use nullptr wherever you need to assign or compare a pointer to a null value. Hence Creation of Empty Linked List , according to Linked List representation:



Head points to Null, is creation of empty linked list.

Time Complexity of the program

```
#include <cstdlib>
#include <iostream>

using namespace std;

typedef struct DLL
{
    int data; → O(1)
    DLL *next; → O(1)
    DLL *prev; → O(1)
}Node;

Node *head; → O(1)

void createEmptyList()
{
    head = nullptr; → O(1)
}

int main()
{
    createEmptyList(); → O(1)
    return 0; → O(1)
}
```

To discuss about time complexity of the program module of creation of

empty linked list :

→ Declaration of data , pointer variable * next, pointer variable * prev takes $O(1)$ i. e. constant time complexity.

→ Renaming of struct DLL to Node takes $O(1)$ i. e. constant time complexity.

→ Function call takes constant time complexity i. e. $O(1)$, including initialization of global Head pointer variable to NULL takes constant amount of time.

→ Returns 0, which is another constant – time operation takes $O(1)$.

Hence , total amount of time complexity : $O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(1)$ i. e. constant amount of time.

<i>Doubly Linked List</i>	<i>Time Complexity</i>
<i>Creation of Empty List</i>	<i>$O(1)$</i>

Doubly Linked List – Creation of Empty List –Space Complexity and Algorithm.

Program :

```
#include <cstdlib>
#include <iostream>

using namespace std;

typedef struct DLL
{
    int data;
    DLL *next;
    DLL *prev;
}Node;

Node *head;

void createEmptyList()
{
    head = nullptr;
}

int main()
{
    createEmptyList();
    return 0;
}
```

Space Complexity

The space complexity of a program refers to the amount of memory space required by the program as a function of the input size. In this particular program, the space complexity is determined by the following factors:

1. **Node Structure:** The DLL structure is defined to hold an integer (`int data`), a pointer to the next node (`DLL * next`), a pointer to the previous node (`DLL * prev`).
The space required for each node is constant, i. e., $O(1)$.
2. **Head Pointer:** The head pointer is a global variable that points to the start of the linked list. It requires a constant amount of space, i. e., $O(1)$.
3. **Function Call:** The `createEmptyList` function takes a pointer to a pointer as an argument. The space required for this function call is also constant, i. e., $O(1)$.

Since the program only initializes an empty list and does not create any nodes dynamically, the overall space complexity of this program is $O(1)$.

1. Input Space Complexity: $O(1)$

Input space is the memory required to store the parameters passed to a function.

- **Parameter-Only & Full-Data Definition:** The `createEmptyList()` function takes no parameters. Since there is no input, the space required for it is zero, which is a constant amount.

Therefore, under both definitions, the input space complexity is $O(1)$.

2. Auxiliary Space Complexity: $O(1)$

Auxiliary space is the extra memory an algorithm uses during its execution.

1. **No New Variables:** The `createEmptyList` function does not declare any of its own local variables.
2. **Single Operation:** It performs a single assignment (`head = nullptr;`) on a global variable. This operation does not consume any additional memory.

Because the function uses no extra memory to perform its task, its auxiliary space complexity is constant.

Therefore, the auxiliary space complexity is $O(1)$.

Algorithm of Creation of Empty List

Algorithm of Creation of Empty Doubly Linked List

Step 1: DEFINE STRUCTURE

- Define a structure `DLL` with three members:
 - An integer `data` to store the value of the node.
 - A pointer `next` of type `DLL*` to point to the next node in the list.
 - A pointer `prev` of type `DLL*` to point to the previous node in the list.

Step 2: Type Alias

- Use `typedef` to create an alias `Node` for `struct DLL` to simplify the code.

Step 3: DECLARE GLOBAL POINTER

- Declare a global pointer `head` of type `Node*` to keep track of the head of the list.

Step 4: CREATE FUNCTION

- Define a function `createEmptyList` that takes no arguments.
- `SET HEAD := NULLPTR`

Step 5: Call Function

- Call Function → `createEmptyList`.
-

Doubly Linked List – Insert At Beginning And Time Complexity.

Program :

```
void insertAtBeginning(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }

    newNode->data = data;

    if (head == nullptr)
    {
        newNode->prev = nullptr;
        newNode->next = nullptr;
        head = newNode;
        cout << data << " inserted at the beginning." <<
endl;
    }

    else
    {
        newNode->prev = nullptr;
        newNode->next = head;
        head->prev = newNode;
        head = newNode;
        cout << data << " inserted at the beginning." <<
endl;
    }
}
```

```
.....
case 1:
    cout << "Enter data to insert: ";
    cin >> data;
    insertAtBeginning(data);
    break;
```

Program Analysis:

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ *The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.*

→ *The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

→ *The size of the Node structure itself is determined by the sum of the sizes of its members:*

- *data(an integer) is typically 4 bytes.*
- *next(a pointer to another Node) is typically 8 bytes.*
- *prev(a pointer to previous Node) is typically 8 bytes.*

→ *Therefore ,the size of the Node structure is typically 20 bytes(4 bytes for data + 8 bytes for next + 8 bytes for prev) .*

And we already know:

```

#include <cstdlib>
#include <iostream>

using namespace std;

typedef struct DLL
{
    int data;
    DLL *next;
    DLL *prev;
}Node;

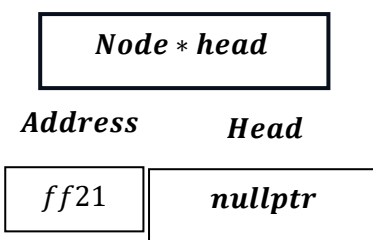
Node *head;

void createEmptyList()
{
    head = nullptr;
}

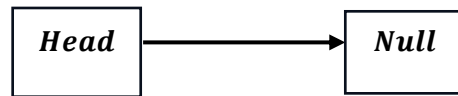
int main()
{
    createEmptyList();
    return 0;
}

```

Head will point to nullptr ,at creation of linked list.



i.e.



Now coming to insert at beginning :

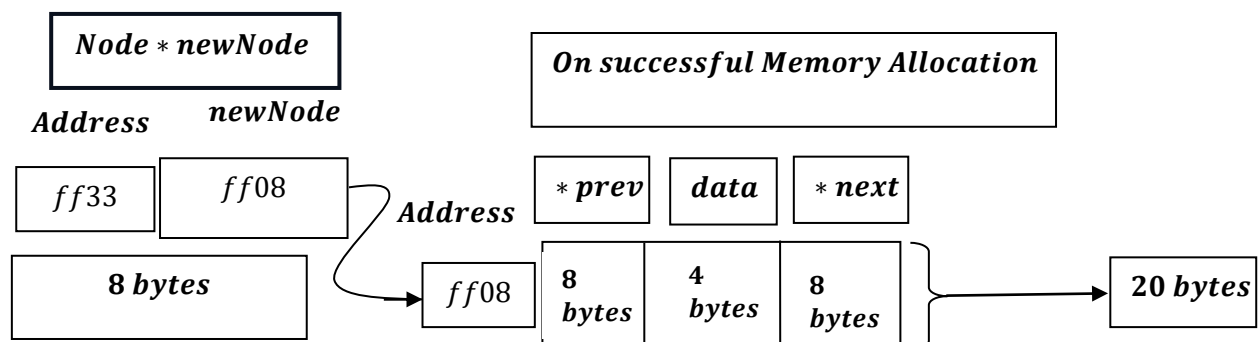
```

Node *newNode = (Node *)malloc(sizeof(Node));
if (newNode == nullptr)
{
    cout << "Error: Memory allocation failed." << endl;
    return;
}
  
```

What does it mean?

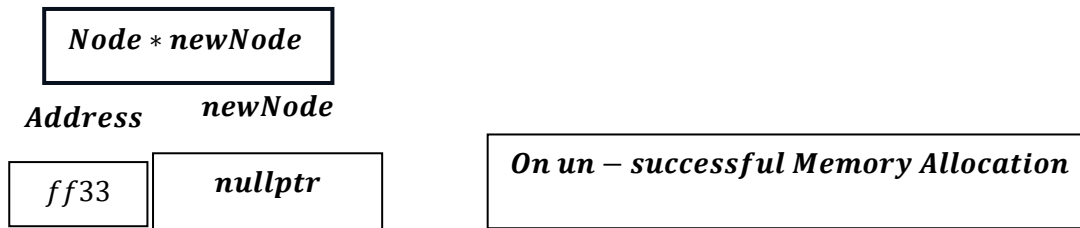
→ *newNode will try to allocate memory for data i.e. 4 bytes ,next pointer variable i.e. typically 8 bytes, and prev pointer variable that is typically 8 bytes i.e. total 20 bytes, as discussed earlier , the size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

Say,



But on Un – Successful memory allocation:

→ if the allocation fails due to insufficient memory or other reasons, malloc will return a null pointer.



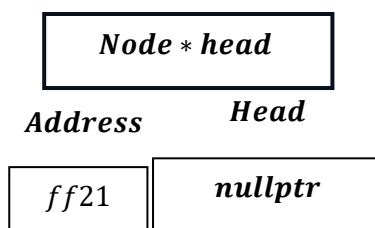
Now, if the value of the pointer variable newNode is nullptr, then:

One get output : `Error: Memory Allocation Failed as a message.`

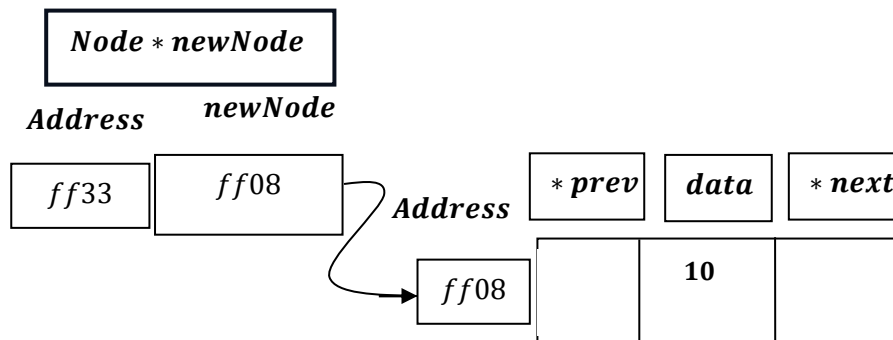
Now,

```
case 1:
    cout << "Enter data to insert: ";
    cin >> data;
    insertAtBeginning(data);
    break;
```

Say , data = 10 and currently head is pointed at nullptr.




```
newNode->data = data;
```

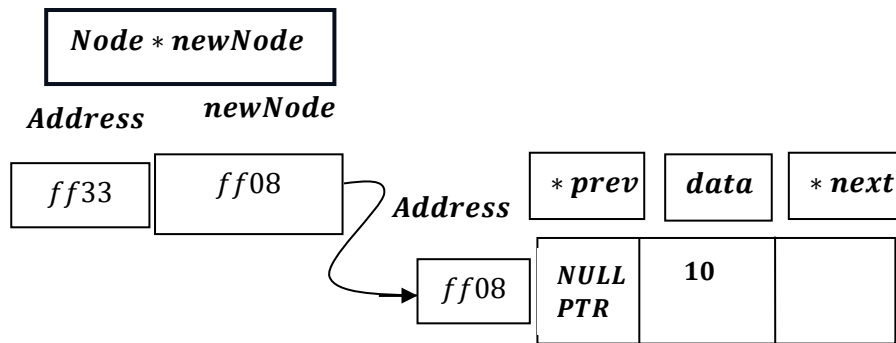


So, head is having value nullpointer or NULL , hence :

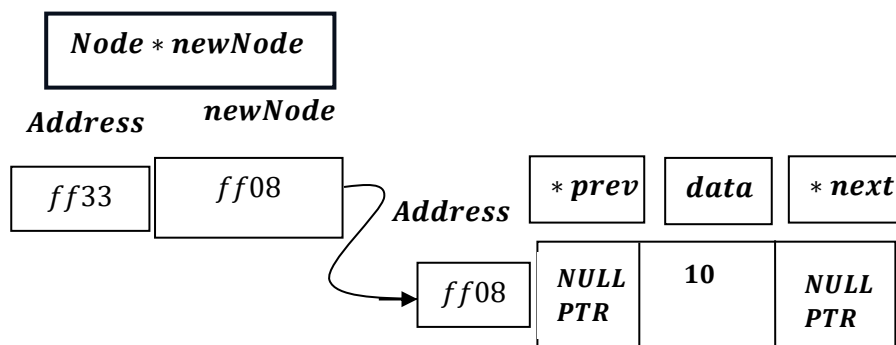
```
if (head == nullptr)
{
    newNode->prev = nullptr;
    newNode->next = nullptr;
    head = newNode;
    cout << data << " inserted at the beginning." <<
endl;
}
```

∴

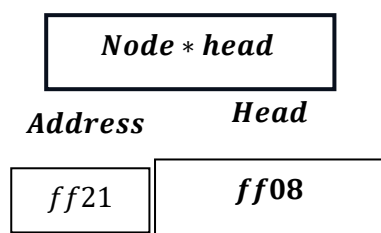
```
newNode->prev = nullptr;
```



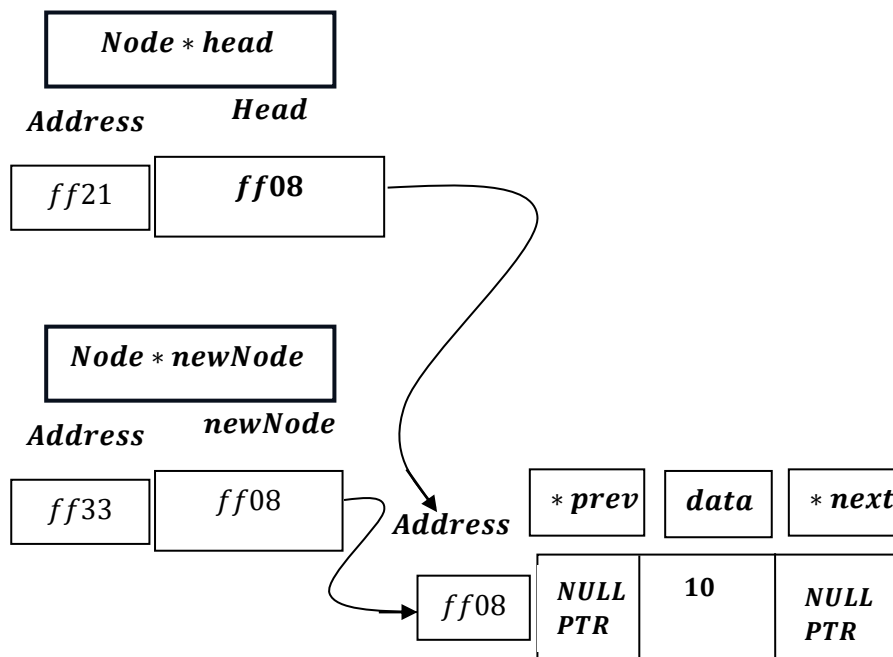
```
newNode->next = nullptr;
```



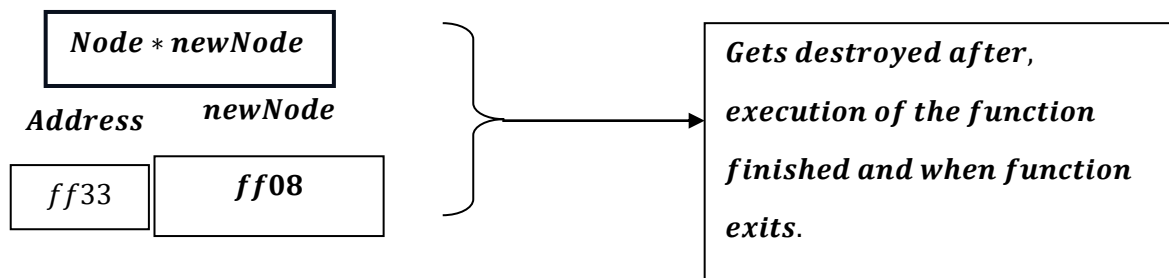
```
head = newNode;
```



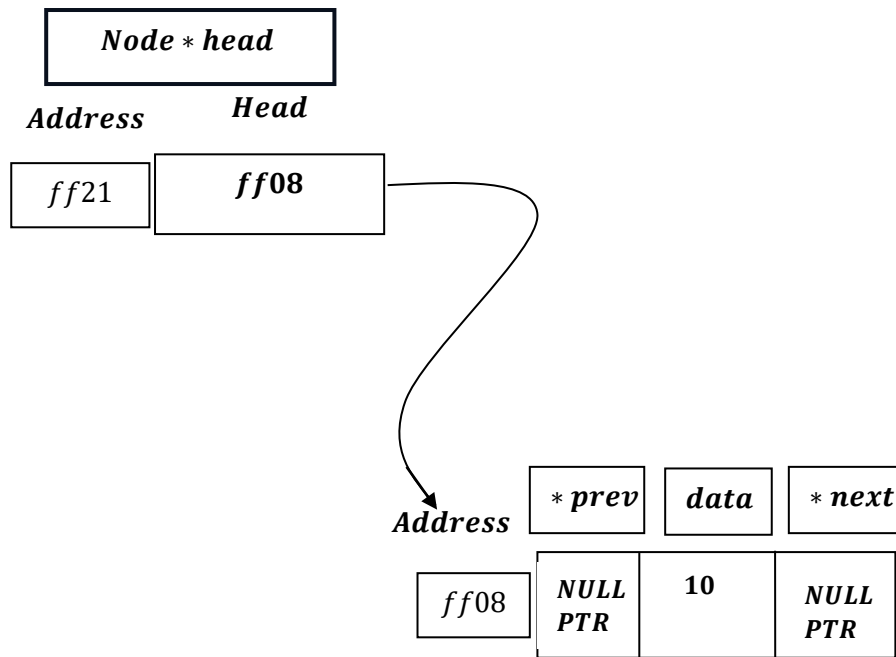
Now, `head` will point to memory address obtained by pointer `newNode`.



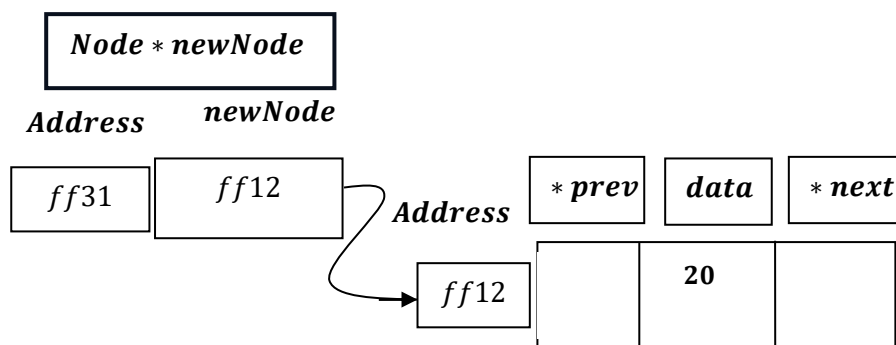
Now, **Node * newNode** temporary variable, therefore gets destroyed after function execution get finished .



Hence , we are left with :



Now say we try to enter data : 20 .



As, head is not having ``nullptr``, it will now go to else part:

```
else
{
    newNode->prev = nullptr;
    newNode->next = head;
    head->prev = newNode;
    head = newNode;
```

```

        cout << data << " inserted at the beginning." <<
endl;
    }

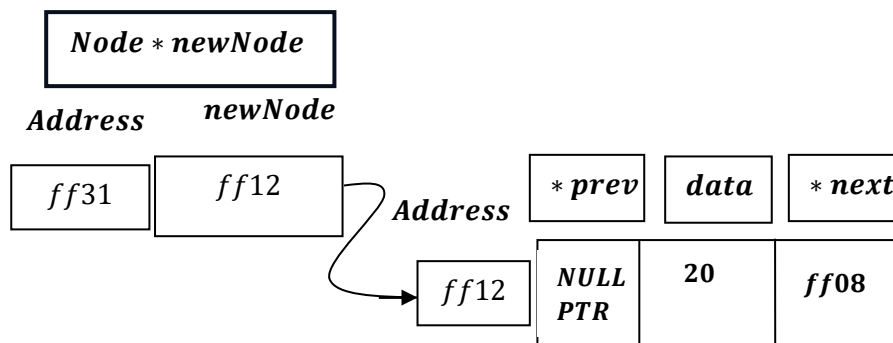
```

∴

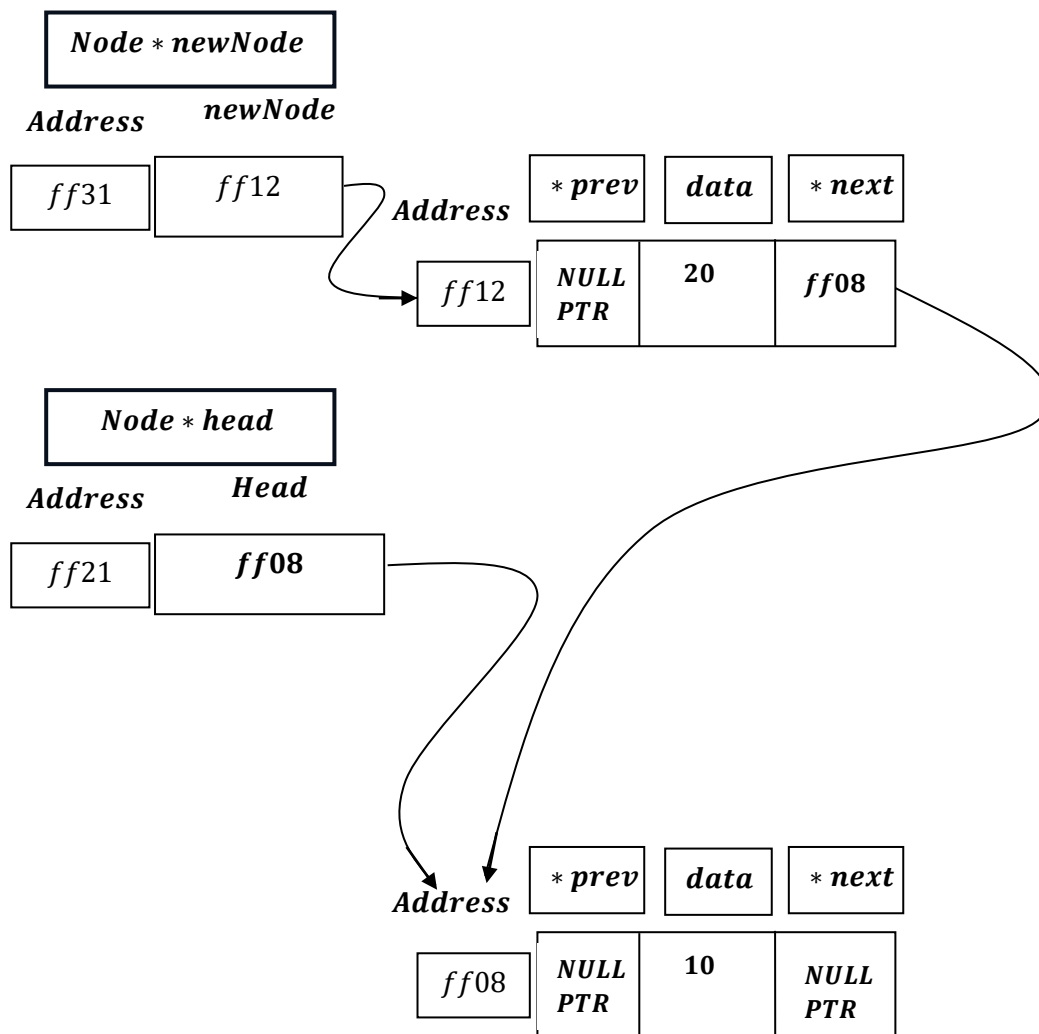
```

newNode->prev = nullptr;
newNode->next = head;

```



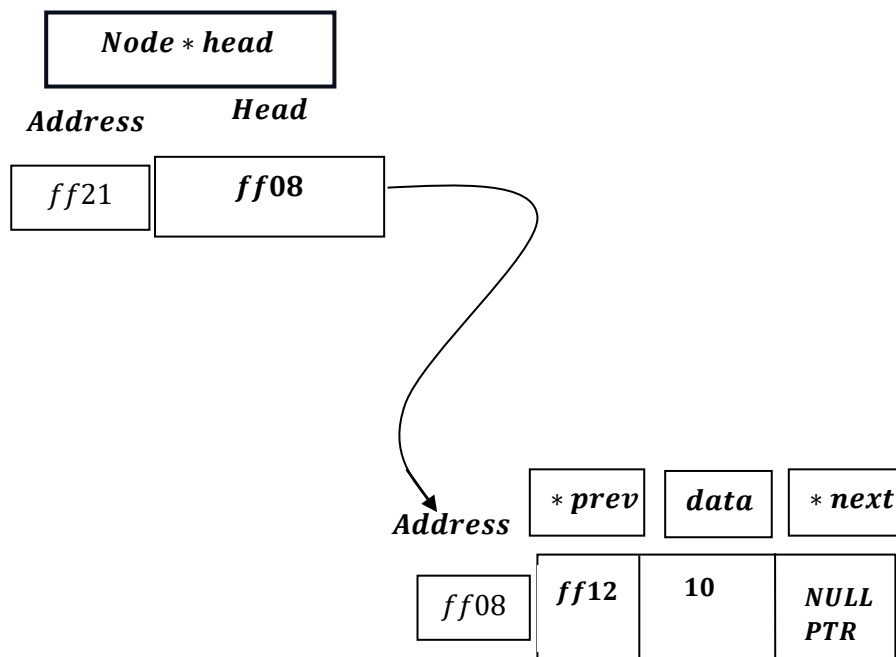
Hence ,now ,this new node will point to previous Node.



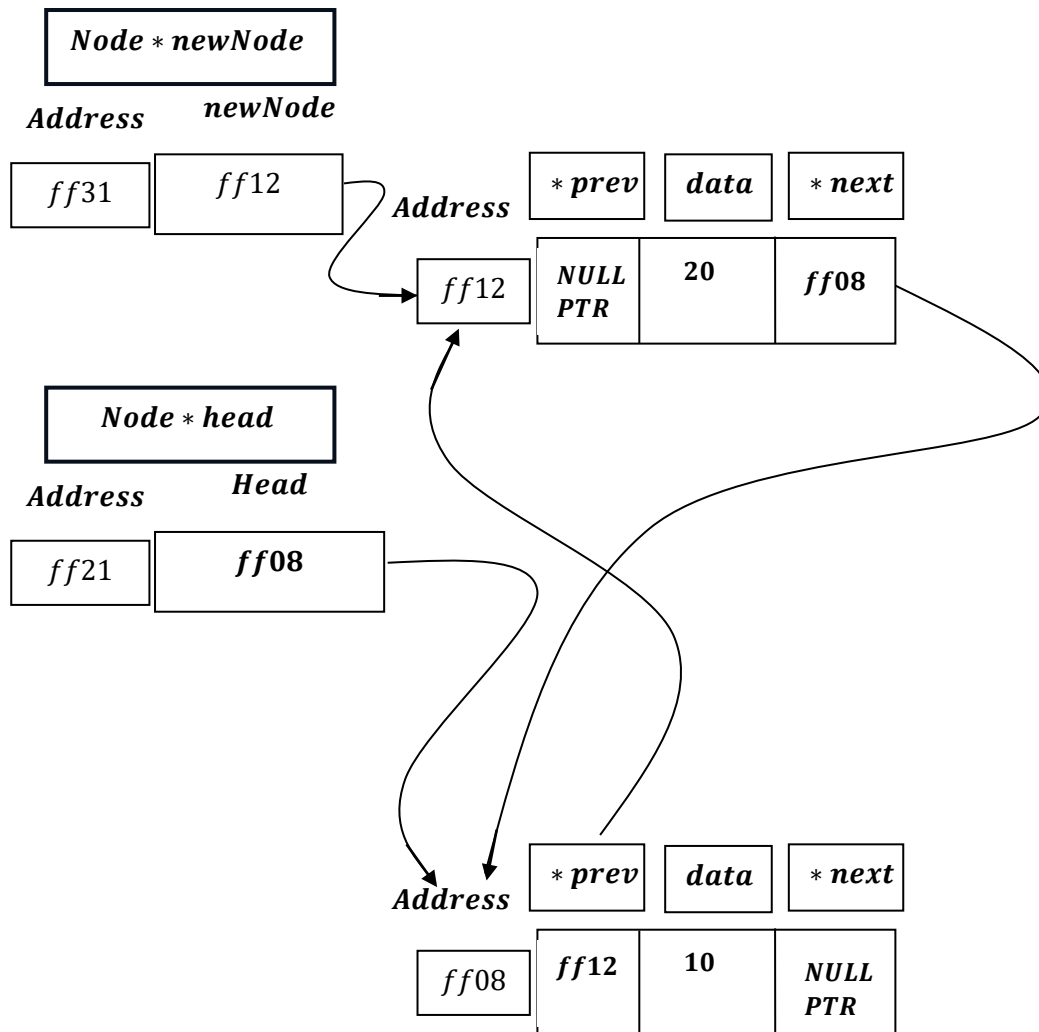
Now,

```
head->prev = newNode;
```

i.e., $head \rightarrow prev = ff12$;



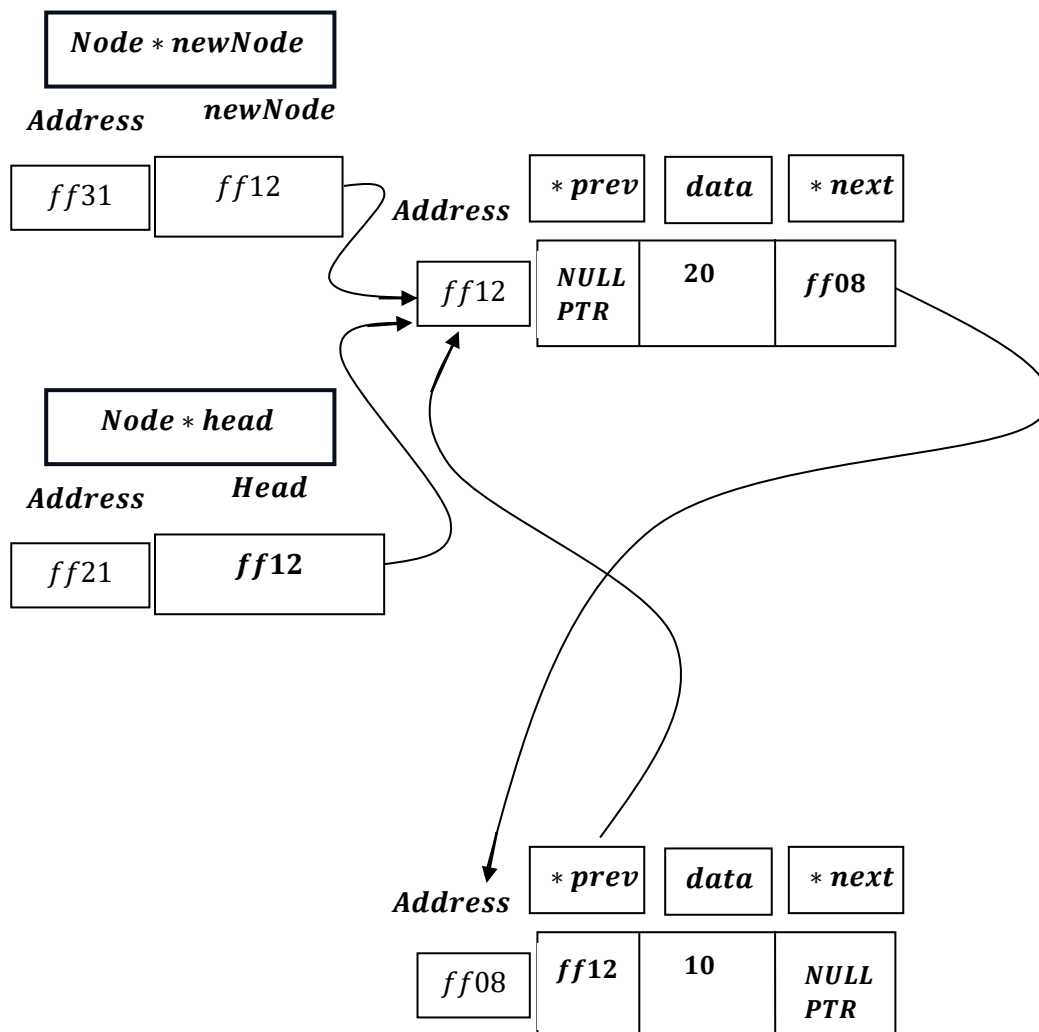
Hence, previous pointer of previous node will now point to current new node.



Next,

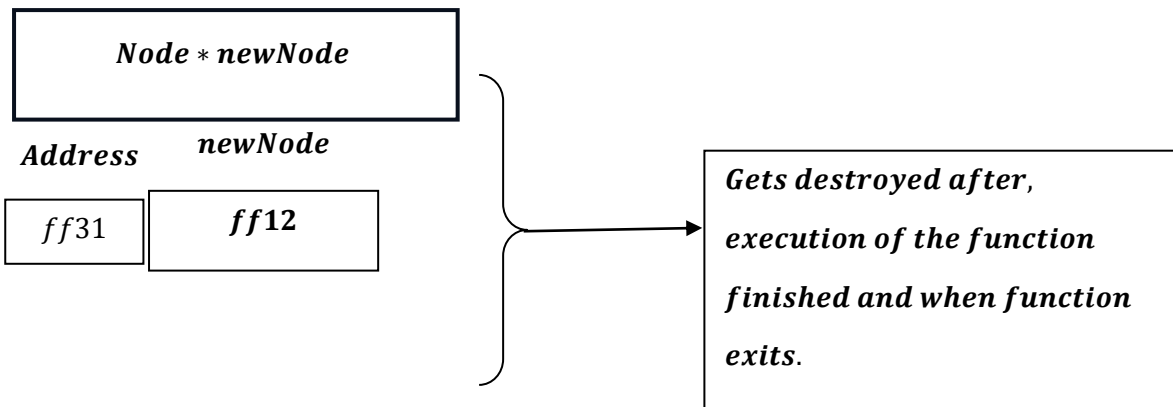
```
head = newNode;
```

i.e., `head = ff12`.

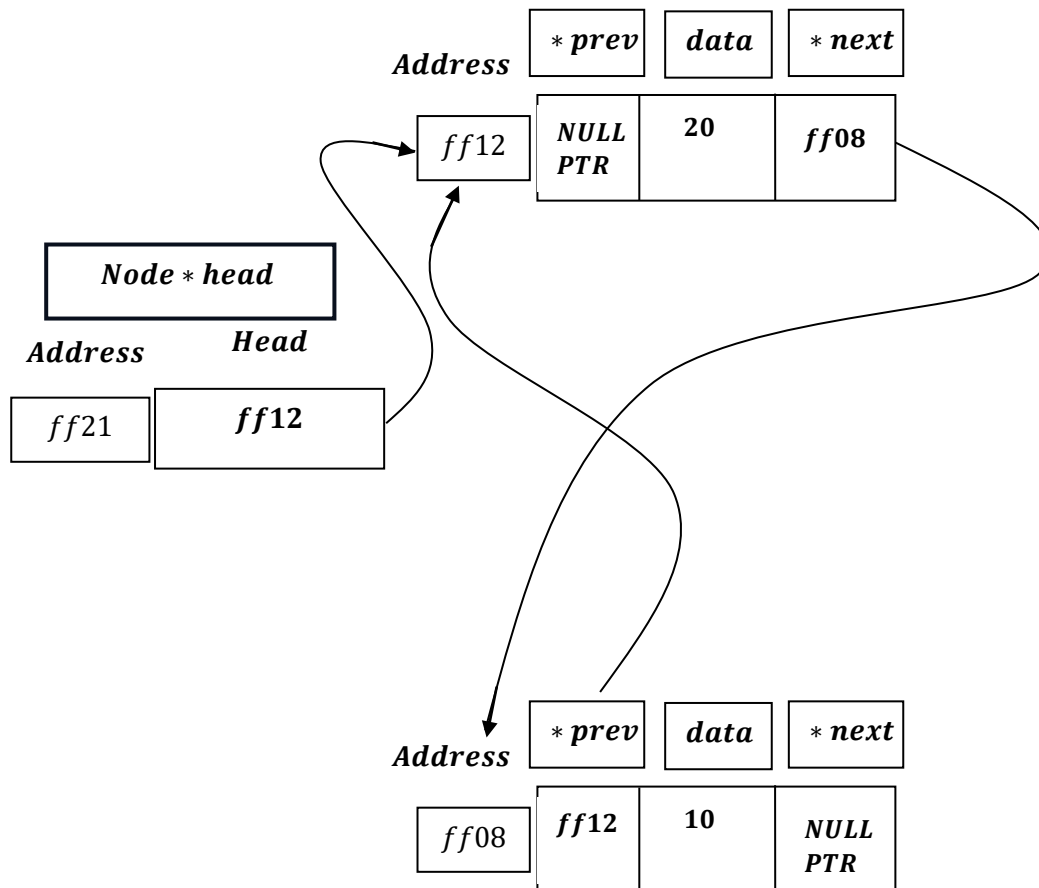


*Now, Node * newNode is temporary variable, therefore gets destroyed after function execution get finished .*

i. e.



And we are left with:



We can re – write the program as :

```
void insertAtBeginning(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    newNode->prev = nullptr;
    newNode->next = head;
    if (head != nullptr)
    {
        head->prev = newNode;
    }
    head = newNode;
    cout << data << " inserted at the beginning." << endl;
}
```

Note :

```
if (newNode == nullptr)
{
    cout << "Error: Memory allocation failed." << endl;
    return;
}
```

This is actually for understanding , hence for optimization , it is removed.

When head == NULL/nullptr , then :

```
Node *newNode = (Node *)malloc(sizeof(Node));
newNode->data = data;
newNode->prev = nullptr;
newNode->next = head;
head = newNode;
}
```

This will run and newNode → next = head = nullptr or NULL .

Except this every condition remain same , when head = NULL or nullptr.

When head ≠ NULL/nullptr , then :

```
void insertAtBeginning(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    newNode->prev = nullptr;
    newNode->next = head;
    if (head != nullptr)
    {
        head->prev = newNode;
    }
    head = newNode;
}
```

This will run exactly as the else part, of earlier code.

Note, newNode → next = head gets adjusted when head ≠ NULL /nullptr or head = NULL /nullptr, but the process remain same.

Hence , this is more optimized code . than earlier one .

And this process continues as we continue to insert data at bengining in Linked List.

Doubly Linked List - Insert At Beginning [Working Summary]:

→ **If the head pointer is null (i.e., the list is empty)**, the newly created node's `prev` and `next` pointers are both set to `nullptr`. The `head` pointer is then assigned the address of this new node, making it the first and only node in the list.

→ **If the head pointer is not null (i.e., the list is not empty)**, the newly created node's `next` pointer is set to point to the current `head(Node1/first node)` node. The current `head(Node1/first node)` node's `prev` pointer is then updated to point back to the newly created node. Finally, the main `head` pointer is updated to point to the newly created node, officially making it the new head of the list.

Time Complexity of the program

```
void insertAtBeginning(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));    →  $O(1)$ 
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;    →  $O(1)$ 
        return;    →  $O(1)$ 
    }

    newNode->data = data;    →  $O(1)$ 

    if (head == nullptr)
    {
        newNode->prev = nullptr;    →  $O(1)$ 
        newNode->next = nullptr;    →  $O(1)$ 

        head = newNode;    →  $O(1)$ 

        cout << data << " inserted at the beginning." << endl;    →  $O(1)$ 
    }

    else
    {
        newNode->prev = nullptr;    →  $O(1)$ 

        newNode->next = head;    →  $O(1)$ 

        head->prev = newNode;    →  $O(1)$ 

        head = newNode;    →  $O(1)$ 
        cout << data << " inserted at the beginning." << endl;    →  $O(1)$ 
    }
}
```

→ The function allocates memory for a new node using malloc. This operation takes constant time, typically around $O(1)$ on most systems.

→ The function checks if the new node pointer is null, indicating memory allocation failed. This comparison is a simple operation that takes constant time, $O(1)$. Including output statement of if and return statement takes constant time $O(1)$.

→ When input integer data value assigned to integer structure member variable `data` through newNode pointer of structure named Node takes constant time $O(1)$.

→ Checking pointer head is null takes constant amount of time i. e. $O(1)$, which indicates the list is empty, if empty then :

- prev pointer of newNode assigned to Null Pointer takes constant time $O(1)$.
- next pointer of newNode assigned to Null Pointer takes constant time $O(1)$.
- head pointer's value assigned to newNode pointer's value , makes head pointer points to new node which takes constant amount of time $O(1)$.

→ Checking pointer head is null takes constant amount of time i. e. $O(1)$, if not empty then else part:

- prev pointer of newNode assigned to Null Pointer takes constant time $O(1)$.
- next pointer of newNode assigned to head pointer's value i. e.

now next pointer will point to the address held by head pointer takes constant time $O(1)$.

- head pointer's value assigned to newNode pointer's value , makes head pointer points to new node which takes constant amount of time $O(1)$.

Hence if we sum up i. e.

$$O(1) + (O(1) + (O(1) + O(1))) + (O(1) + (O(1) + O(1) + O(1) + O(1))) = O(1)$$

i. e. Time complexity remains constant .

<i>Doubly Linked List</i>	<i>Time Complexity</i>
<i>InsertAtBeginning</i>	<i>$O(1)$</i>

Doubly Linked List – Insert At Beginning –Space Complexity and Algorithm.

Program :

```
void insertAtBeginning(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }

    newNode->data = data;

    if (head == nullptr)
    {
        newNode->prev = nullptr;
        newNode->next = nullptr;
        head = newNode;
        cout << data << " inserted at the beginning." <<
endl;
    }

    else
    {
        newNode->prev = nullptr;
        newNode->next = head;
        head->prev = newNode;
        head = newNode;
        cout << data << " inserted at the beginning." <<
endl;
    }
}
```

```
..... •  
case 1:  
    cout << "Enter data to insert: ";  
    cin >> data;  
    insertAtBeginning(data);  
    break;
```

Space Complexity

Auxiliary Space

Auxiliary space is the extra or temporary space used by an algorithm, excluding the space taken up by the inputs.

For the `insertAtBeginning` function, the only new memory allocation is for `newNode`:

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

The size of `newNode` is `sizeof(Node)`, which is a **constant** value. It does not depend on the number of nodes already in the list. Therefore, the auxiliary space complexity is:

- **Auxiliary Space:** $O(1)$
-

Input Space

Input space is the amount of memory used to store the input values. The definition of "input" can vary, leading to different analyses.

1. Parameter-only Definition

This common definition considers only the space taken by the function's parameters. The function signature is `void insertAtBeginning(int data)`.

- The input is a single integer, `data`.
- An `int` occupies a fixed, constant amount of memory.

Therefore, using this definition, the input space complexity is:

- **Input Space (Parameter-only):** $O(1)$

2. Full-Data Definition

This broader definition considers the space taken by the entire data structure the function operates on. The function manipulates a doubly linked list pointed to by a global or class member `head`.

- If the linked list contains **n** nodes, the total memory occupied by the list is proportional to **n**.

Therefore, using this definition, the input space complexity is:

- **Input Space (Full-Data):** $O(n)$
-

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity (O(1))** is the expected answer.

Algorithm: Insert at Beginning (Doubly Linked List)

DINSATBEG(START, ITEM)

*(Node *)malloc(sizeof(Node)) is availability of space for insertion of Data,
hence, let's name it AVAIL.*

1. [ALLOCATE NEW NODE]

Set NEW := AVAIL.

2. [MEMORY ALLOCATED?]

If NEW = NULL, then:

Write: "Error: Memory allocation failed." and EXIT.

[End of If structure.]

3. [COPY DATA]

Set INFO[NEW] := ITEM.

4. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

a. Set PREVLINK[NEW] := NULL.

b. Set NEXTLINK[NEW] := NULL.

c. Set START := NEW.

d. Write: ITEM, "inserted at the beginning."

ELSE: [List is not empty]

a. Set PREVLINK[NEW] := NULL.

b. Set NEXTLINK[NEW] := START.

c. Set PREV[START] := NEW [Point the original first node's PREV back to the new node].

d. Set START := NEW.

e. Write: ITEM, "inserted at the beginning."

[END OF IF STRUCTURE]

5. Exit

Doubly Linked List – Insert At End And Time Complexity.

Program :

```
void insertAtEnd(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = nullptr;

    if (head == nullptr)
    {
        newNode->prev = nullptr;
        head = newNode;
        cout << data << " inserted at the end." << endl;
        return;
    }
    Node *temp = head;
    while (temp->next != nullptr)
    {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;
    cout << data << " inserted at the end." << endl;
}

...

case 2:
    cout << "Enter data to insert: ";
    cin >> data;
    insertAtEnd(data);
    break;
```

Program Analysis:

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of *newNode* will be equal to the size of a *Node* structure and *newNode* is declared as a pointer to *Node* structure.

→ The size of a pointer (*newNode*) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ The size of the *Node* structure itself is determined by the sum of the sizes of its members:

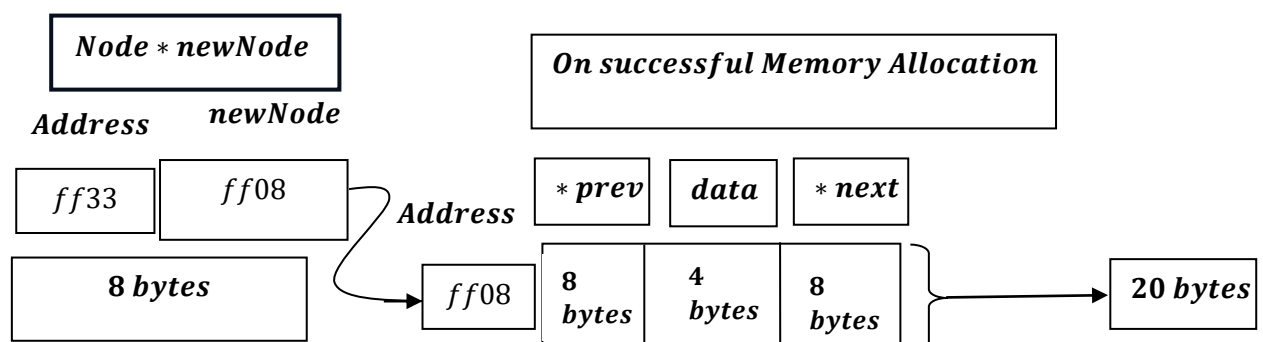
→ *data*(an integer) is typically 4 bytes.

→ *next*(a pointer to another *Node*) is typically 8 bytes.

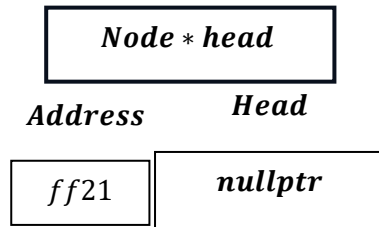
→ *prev*(a pointer to previous *Node*) is typically 8 bytes.

→ Therefore ,the size of the *Node* structure is typically 20 bytes(4 bytes for *data* + 8 bytes for *next* + 8 bytes for *previous*) .

Say,



1st condition : HEAD POINTER points NULLPTR(LIST IS EMPTY)



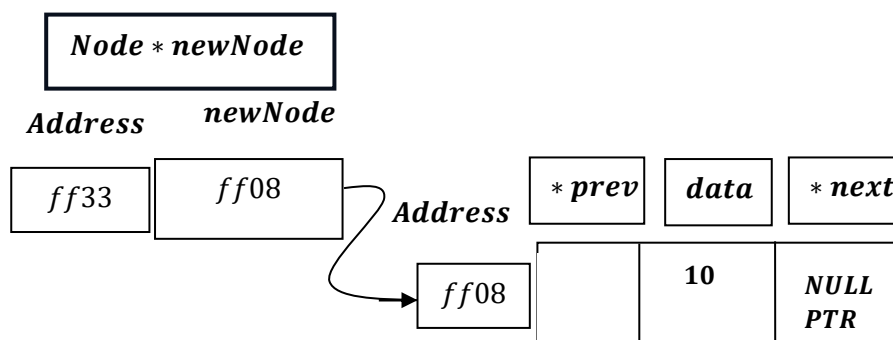
Then:

```
case 2:
    cout << "Enter data to insert: ";
    cin >> data;
    insertAtEnd(data);
    break;
```

And, say data is : 10.

```
Node *newNode = (Node *)malloc(sizeof(Node));
newNode->data = data;
newNode->next = nullptr;
```

i. e.,

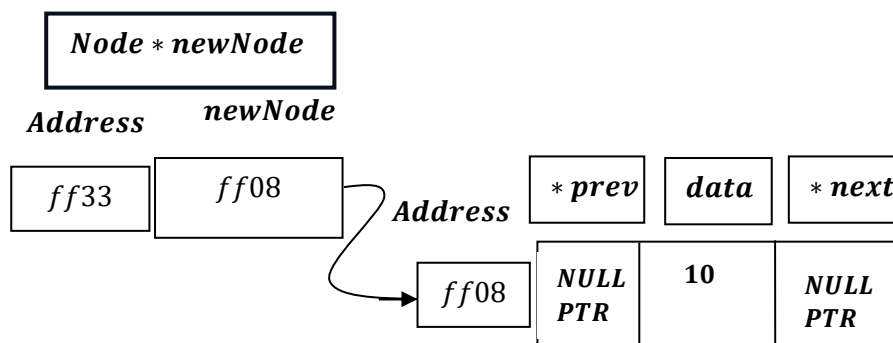


Now,

```
if (head == nullptr)
{
    newNode->prev = nullptr;
    head = newNode;
    cout << data << " inserted at the end." << endl;
    return;
}
```

i.e.,

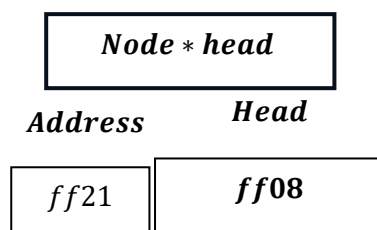
```
newNode->prev = nullptr;
```



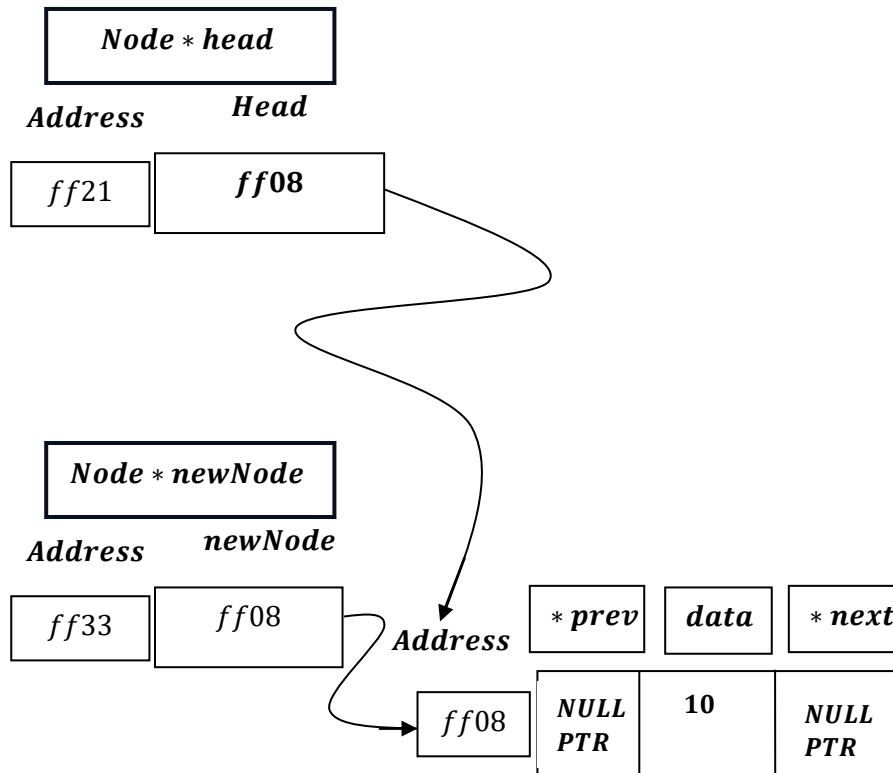
Next,

```
head = newNode;
```

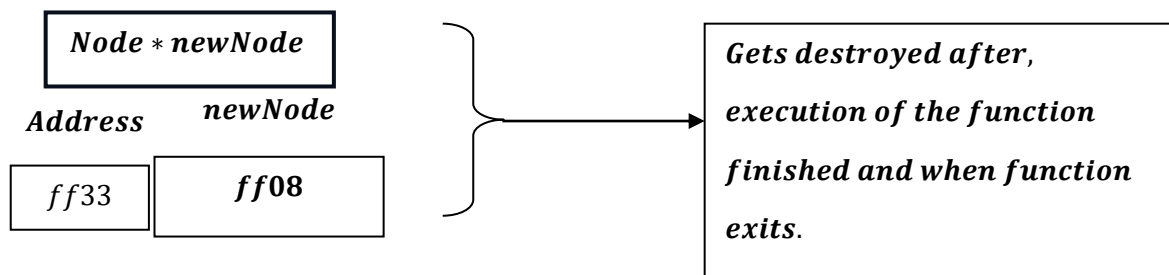
i.e, `head = ff08`



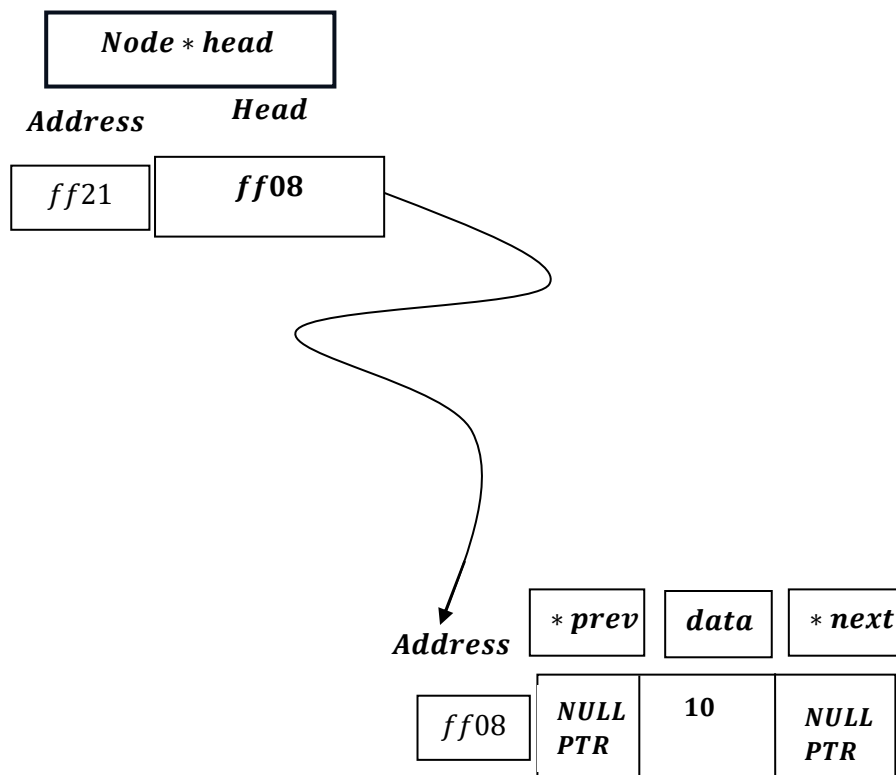
Hence ,head pointer will now point to address : ff08.



Now, **Node * newNode** temporary variable, therefore gets destroyed after function execution get finished .

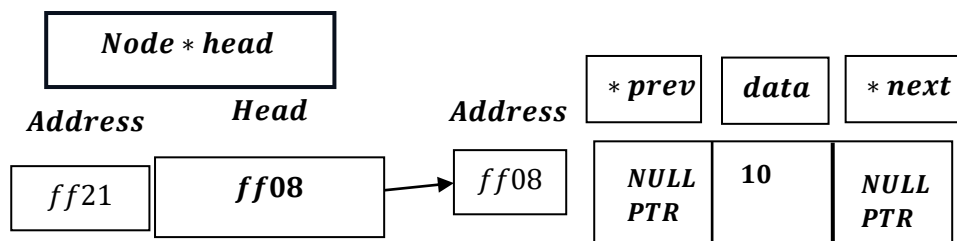


Hence ,we are left with :



2nd condition
: HEAD POINTER not points to NULLPTR (LIST IS NOT EMPTY)

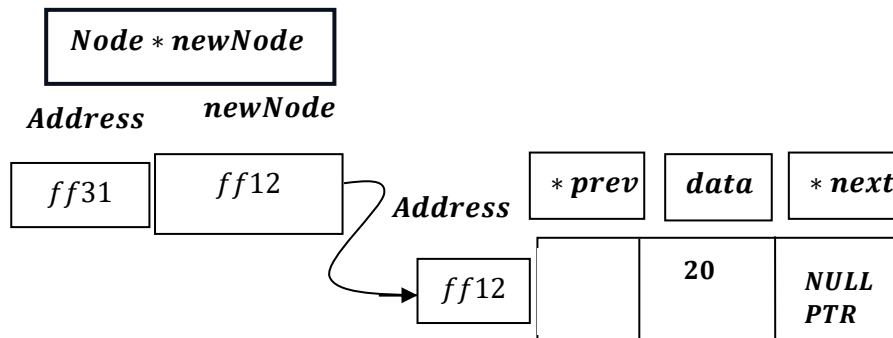
We have:



Lets insert another data , say 20 .

```
Node *newNode = (Node *)malloc(sizeof(Node));
newNode->data = data;
newNode->next = nullptr;
```

i. e., 1. Creation of newNode pointer and 2. assigning data and next pointer in memory allocation, pointed by newNode pointer.



```
if (head == nullptr)
```

here , Head $\neq$ NULLPTR , hence false , therefore exit if .

```
Node *temp = head;
```

*can be broken as, Node * temp ;*

```
temp = head;
```

The Breakdown

1. Node *temp = head; (Declaration with Initialization)

This single line performs two actions at once:

- **Declaration:** `Node *temp` tells the compiler to create a new variable named `temp` that is a pointer to a `Node` object.
- **Initialization:** `= head` immediately assigns the memory address currently stored in the `head` pointer to the new `temp` pointer.

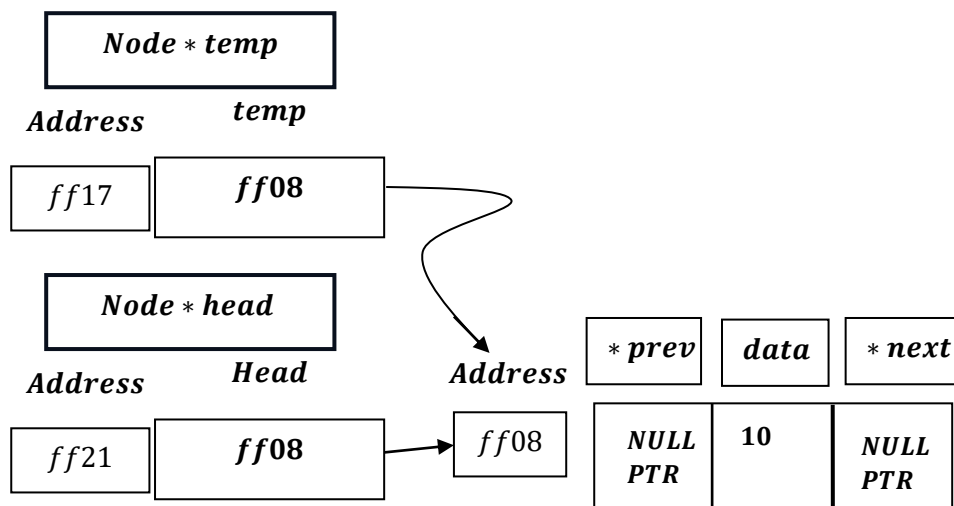
This is the most common and preferred way to write it because it's concise and ensures the pointer holds a valid (or `nullptr`) value from the moment it's created.

2. `Node *temp;` followed by `temp = head;` (Separate Declaration and Assignment)

This is the two-step version of the same process:

- **Declaration:** `Node *temp;` creates the pointer variable `temp`. At this exact moment, `temp` is **uninitialized** and holds a garbage memory address.
- **Assignment:** `temp = head;` then copies the memory address from `head` into the `temp` pointer.

i. e., `temp = head = ff08`. hence `temp` will now point to address $\rightarrow$ `ff08`.



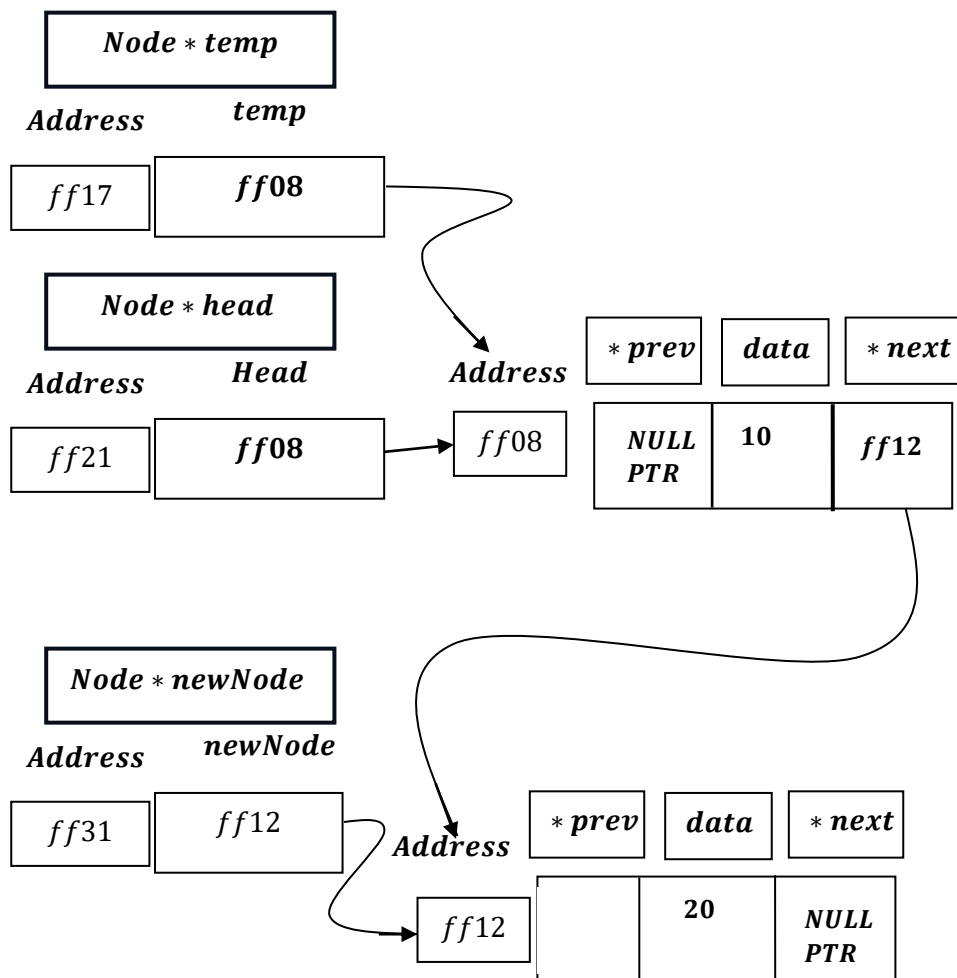
Next,

```
while (temp->next != nullptr)
{
    temp = temp->next;
}
```

Here, `temp  $\rightarrow$  next = nullptr`, hence while loop will not execute.

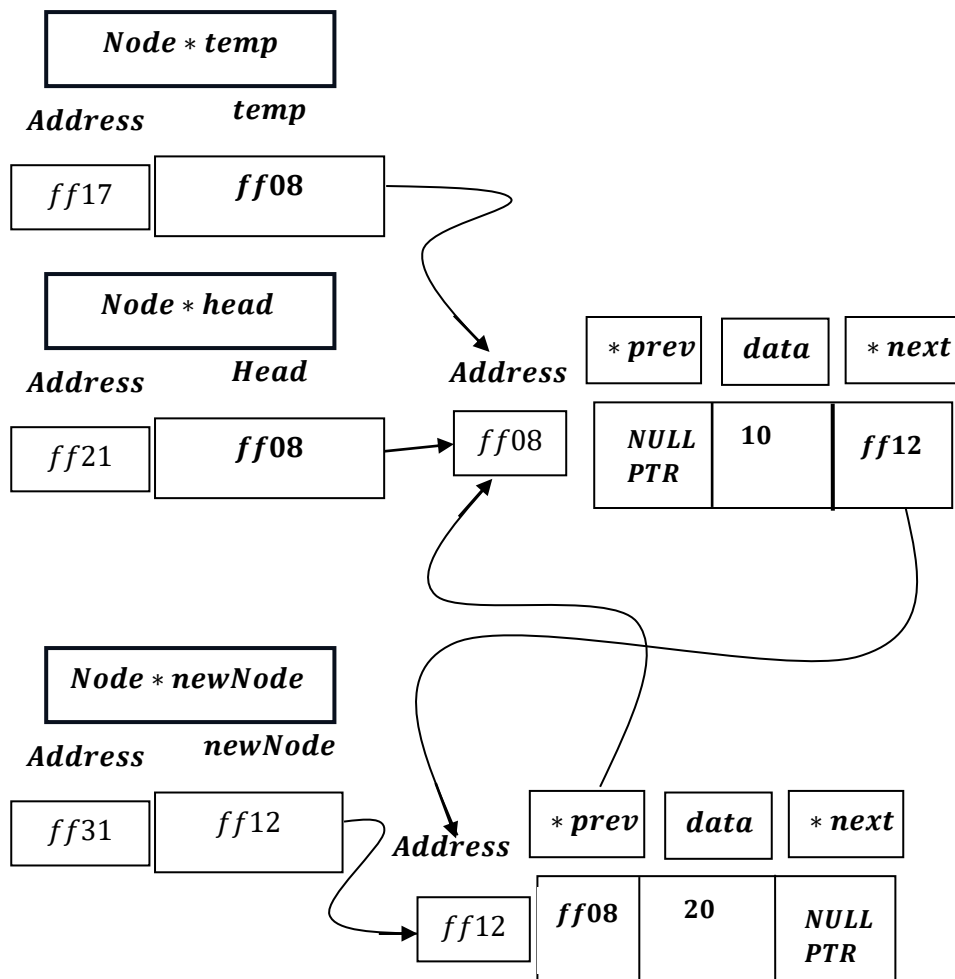
```
temp->next = newNode;
```

i. e. $temp \rightarrow next = newNode = ff12$.

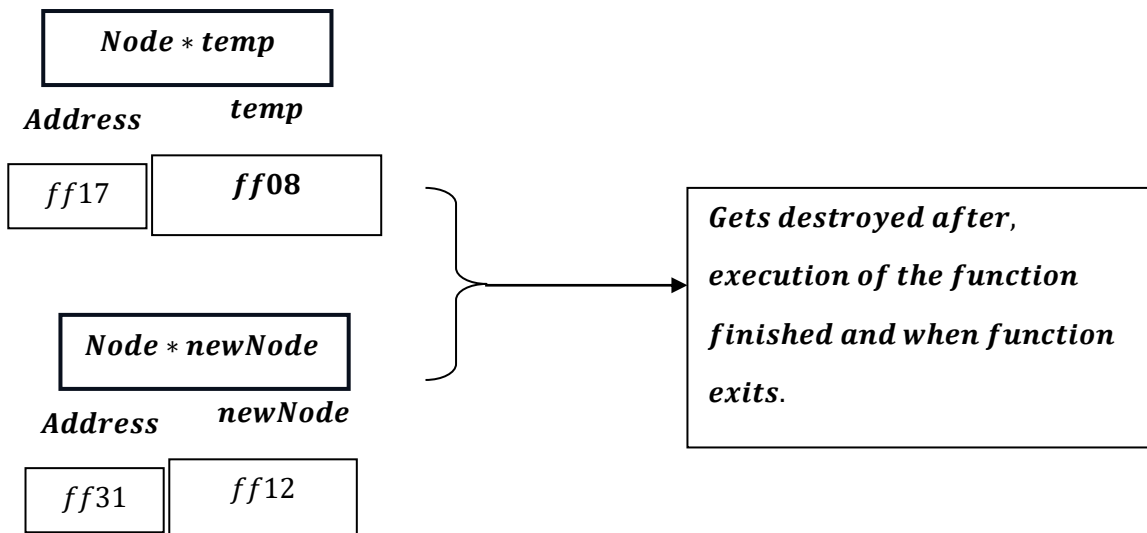


```
newNode->prev = temp;
```

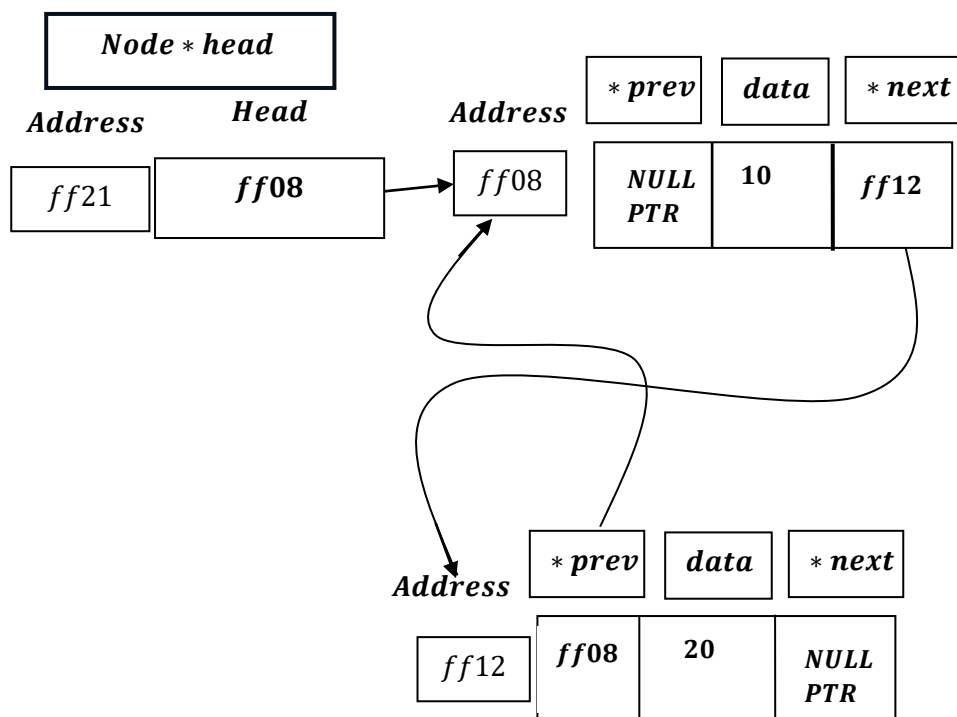
i. e., $newNode \rightarrow prev = temp = ff08$.



Now, `Node * newNode`, `Node * temp` temporary variables, therefore gets destroyed after function execution get finished .



Hence ,we are left with :



Doubly Linked List - Insert At End [Working Summary]:

First, a new node is created, its data is set, and its `next` pointer is assigned `nullptr` since it will always be the new last node.

→ **If the head pointer is null (i.e., the list is empty)**, the new node's `prev` pointer is also set to `nullptr`. The `head` pointer is then updated to point to this new node, making it the only element in the list.

→ **If the list is not empty**, a temporary pointer (`temp`) traverses the list from the `head` until it reaches the current last node (the node whose `next` pointer is `nullptr`). Once at the end, the original last node's (`temp`) `next` pointer is set to point to the `newNode`, and the `newNode`'s `prev` pointer is set to point back to `temp`, officially attaching the new node at the end of the list.

Time Complexity of the program

```
void insertAtEnd(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node)); → O(1)

    newNode->data = data; → O(1)

    newNode->next = nullptr; → O(1)

    if (head == nullptr) → O(1)
    {
        newNode->prev = nullptr; → O(1)

        head = newNode; → O(1)

        cout << data << " inserted at the end." << endl; → O(1)

        return; → O(1)
    }

    Node *temp = head; → O(1)

    while (temp->next != nullptr) runs`n - 1` times ,when head ≠ NULL
    {
        temp = temp->next; statement also runs`n - 1` times .
    }

    temp->next = newNode; → O(1)

    newNode->prev = temp; → O(1)

    cout << data << " inserted at the end." << endl; → O(1)
}
```

Best Case: $O(1)$, when head is Null $\rightarrow$ Absolute Best Case

$\rightarrow$ The function begins by allocating memory for a new node using malloc. This operation takes constant time, typically $O(1)$, regardless of the size of the list.

$\rightarrow$ The memory allocated for integer data pointed by pointer `newNode` is assigned to data input by user takes constant time, typically $O(1)$.

$\rightarrow$ The memory allocated for pointer variable `next` pointed by pointer `newNode` is assigned to `nullptr` takes constant time, typically $O(1)$.

$\rightarrow$ The function checks if the head of the list is null, indicating an empty list. If the list is empty, the new node becomes the head. This step takes constant time, $O(1)$.

- `prev` pointer of allocated memory pointed by `newNode` pointer set to `nullptr`, takes constant amount of time $O(1)$.***
- `newNode` pointer's value assigned to head pointer, takes constant amount of time $O(1)$.***
- Output to user, the integer `data` input with "inserted at the end" statement, which is a string, takes constant amount of time $O(1)$.***
- Then, return statement runs, takes constant amount of time $O(1)$ and exit from the function.***

Hence Total Time Complexity: $O(1) + O(1) + O(1) + (O(1) + (O(1) + O(1) + O(1))) = O(1)$

Best Case $\rightarrow O(1)$, when 2nd node is inserted i.e. if $n = 1$, where n is node and another node is Inserted . $\rightarrow$ Not the absolute best case.

$\rightarrow$ The function begins by allocating memory for a new node using malloc. This operation takes constant time, typically $O(1)$, regardless of the size of the list.

$\rightarrow$ The memory allocated for integer data pointed by pointer `newNode` is assigned to data input by user takes constant time, typically $O(1)$.

$\rightarrow$ The memory allocated for pointer variable `next` pointed by pointer `newNode` is assigned to `nullptr` takes constant time, typically $O(1)$.

$\rightarrow$ The function checks if the head of the list is null, now list is having a single node, whose `prev` pointer is `NULLPTR`, `next` pointer is also `NULLPTR`, and with a single integer data, hence the check returns false, hence, takes constant time, typically $O(1)$.

$\rightarrow$ Now `temp` pointer of structure `Node` assigned to the value of `head` pointer, makes `temp` pointer points to the address that `head` pointer points to, this process takes constant time, typically $O(1)$.

$\rightarrow$ while (`temp  $\rightarrow$  next  $\neq$  NULLPTR`), but `temp  $\rightarrow$  next = NULLPTR`, while loop condition fails here and hence fails to run, this condition checking takes takes constant amount of time $O(1)$.

$\rightarrow$ `next` pointer pointed by `temp` pointer assigned to value of `newNode` pointer of `Node` structure, hence `next` pointer will now point to the address to which `newNode` pointer pointing to, this process takes constant time, typically $O(1)$.

→ *prev pointer pointed by newNode pointer of Node structure assigned to the value of temp pointer of Node structure , hence prev pointer will now point to the address to which temp pointer pointing to , this process takes constant time, typically $O(1)$.*

→ *Output to user , the integer `data` input with "inserted at the end" statement , which is a string , takes constant amount of time $O(1)$.*

*Hence Total Time Complexity: $O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1)$
= $O(1)$.*

Why not Absolute Best Case ?

⇒ *When $n = 1$, loop runs 0 times. [maintaining , $n - 1$ times]*

⇒ *When $n = 2$, loop runs 1 times. [maintaining , $n - 1$ times]*

⇒ *When $n = 3$, loop runs 2 times. [maintaining , $n - 1$ times]*

·

·

·

⇒ *When $n = n$, loop runs $n - 1$ times. [maintaining , $n - 1$ times]*

Hence , when $n \neq 0$ i.e. List is not empty , it maintains $n - 1$ times running of loop , therefore , when $n = 1$, loop doesnot runs , hence function remains at constant time complexity.

When $n > 1$, Loop runs , we cannot take this scenario as function remaining at constant time , therefore when $n = 1$, as function remains at constant time , we can take this as best case but not the `Absolute Best Case`. The `Absolute Best Case` is when List is Empty.

Worst Case: $O(n)$, when $n > 1$

→ The function begins by allocating memory for a new node using malloc. This operation takes constant time, typically $O(1)$, regardless of the size of the list.

→ The memory allocated for integer data pointed by pointer newNode is assigned to data input by user takes constant time, typically $O(1)$.

→ The memory allocated for pointer variable next pointed by pointer newNode is assigned to nullptr takes constant time, typically $O(1)$.

→ The function checks if the head of the list is null, now list is having a single node, whose prev pointer is NULLPTR, next pointer is also NULLPTR, and with a single integer data, hence the check returns false, hence, takes constant time, typically $O(1)$.

→ Now temp pointer of structure Node assigned to the value of head pointer, makes temp pointer points to the address that head pointer points to, this process takes constant time, typically $O(1)$.

→ while ($temp \rightarrow next \neq NULLPTR$), and now, $temp \rightarrow next \neq NULLPTR$, meets the condition of while loop, hence body of the while loop i. e.

temp pointer of Node structure gets assigned to the value (Address) held by next pointer of allocated memory pointed by temp pointer, i. e. temp will now point to next node and traverse to $n - 1$ node of list containing 'n' nodes.

–Which makes time complexity here is not constant i. e. $O(n - 1) = O(n)$.

→ next pointer pointed by temp pointer assigned to value of newNode pointer of Node structure , hence next pointer will now point to the address to which newNode pointer pointing to , this process takes constant time, typically $O(1)$.

→ prev pointer pointed by newNode pointer of Node structure assigned to the value of temp pointer of Node structure , hence prev pointer will now point to the address to which temp pointer pointing to , this process takes constant time, typically $O(1)$.

→ Output to user , the integer `data` input with "inserted at the end" statement , which is a string , takes constant amount of time $O(1)$.

Hence Total Time Complexity: $O(1) + O(1) + O(1) + O(1) + O(1) + O(n) + O(1) + O(1) + O(1) = O(n)$.

<i>Doubly Linked List</i>	<i>Condition</i>	<i>Time Complexity</i>
<i>InsertAtEnd</i>	<i>1. List is Empty</i>	<i>$O(1)$</i>
	<i>2. List is not Empty</i>	<i>$O(n)$</i>

We can say , if List is Empty , we get our best case scenario, and when list is not empty , we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine where the insertion to be made , hence we can say average case is our worst case here i. e. always the entire list is traversed i. e. $O(n) = \Theta(n)$.

In the case of insertAtEnd, the insertion always happens at the end, regardless of the current state of the list. This means the execution time is solely dependent on the length of the list. If the list is empty, the execution time is constant (best case). If the list is not empty, the execution time is proportional to the length of the list (worst case). Since the insertion position is predetermined, there's no element of randomness or varying input data to consider. Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here. Hence insertAtEnd doesn't have a true average case.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$$

<i>Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>InsertAt End</i>	<i>1. Best Case</i>	$\Omega(1)$
	<i>2. Worst Case</i>	$O(n)$
	<i>3. Average Case</i>	$\Theta(n)$

Doubly Linked List – Insert At End –Space Complexity and Algorithm.

Program :

```
void insertAtEnd(int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = nullptr;

    if (head == nullptr)
    {
        newNode->prev = nullptr;
        head = newNode;
        cout << data << " inserted at the end." << endl;
        return;
    }
    Node *temp = head;
    while (temp->next != nullptr)
    {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;
    cout << data << " inserted at the end." << endl;
}

...

case 2:
    cout << "Enter data to insert: ";
    cin >> data;
    insertAtEnd(data);
    break;
```

Space Complexity

Auxiliary Space

Auxiliary space is the extra memory used by the function, not including the space taken by the inputs.

In the `insertAtEnd` function, the new memory usage is:

1. `Node *newNode`: One new node is allocated. The size of this node (`sizeof(Node)`) is **constant**.
2. `Node *temp`: One temporary pointer is created to traverse the list. The size of a pointer is also **constant**.

Since the amount of extra memory used does not change with the size of the linked list, the auxiliary space complexity is $O(1)$.

- **Auxiliary Space: $O(1)$**
-

Input Space

Input space is the memory used to store the inputs. We can analyze this from two perspectives.

1. Parameter-only Definition

This view considers only the space taken by the function's direct parameters. The function is `void insertAtEnd(int data)`.

- The input is a single integer, `data`, which occupies a fixed, constant amount of memory.

Under this definition, the input space is constant.

- **Input Space (Parameter-only): $O(1)$**

2. Full-Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list itself.

- If the linked list contains **n** nodes, the total memory it occupies is proportional to **n**.

Under this definition, the input space depends on the size of the list.

- **Input Space (Full-Data): $O(n)$**

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity ($O(1)$)** is the expected answer.

The traversal loop (`while (temp->next != nullptr)`) affects the function's time complexity ($O(n)$) but not its space complexity.

Algorithm: Insert at End (Doubly Linked List)

DINSEND(START, ITEM)

((Node)malloc(sizeof(Node)) is availability of space for insertion of Data,
hence, let's name it AVAIL.)*

1. *[ALLOCATE NEW NODE]*

Set NEW := AVAIL.

2. *[COPY DATA & SET FORWARD POINTER]*

a. Set INFO[NEW] := ITEM.

b. Set NEXTLINK[NEW] := NULL [Sets the forward link of the new node to NULL, as it will be the last node].

3. *[CHECK IF LIST IS EMPTY]*

If START = NULL, then:

a. Set PREVLINK[NEW] := NULL.

b. Set START := NEW.

c. Write: ITEM "inserted at the end." and EXIT.

[END OF IF STRUCTURE]

[List is not empty?]:

a. Set PTR := START [PTR is a temporary pointer assigned to the start of the list].

b. Repeat while NEXTLINK[PTR] ≠ NULL: [Traverses the list to find the last node].

Set PTR := NEXTLINK[PTR] [Updates PTR to point to the next node].

[END of Loop]

c. Set NEXTLINK[PTR] := NEW [Sets the NEXT pointer of the last node (PTR) to the new node].

d. Set PREVLINK[NEW] := PTR [Sets the PREV pointer of the new node back to the former last node (PTR)].

e. Write: ITEM, "inserted at the end."

4. *Exit*

[END OF ALGORITHM]

Doubly Linked List – Insert At Position And Time Complexity.

Program :

```
void insertAtPosition(int data, int pos)
{
    if (pos < 1)
    {
        cout << "Invalid position!" << endl;
        return;
    }
    if (pos == 1)
    {
        insertAtBeginning(data);
        return;
    }

    Node *temp = head;
    for (int i = 1; i < pos - 1 && temp != nullptr; i++)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }

    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;

    newNode->next = temp->next;
    newNode->prev = temp;
    temp->next = newNode;
    if (newNode->next != nullptr)
```

```

    {
        newNode->next->prev = newNode;
    }
    cout << data << " inserted at position " << pos << "."
<< endl;
}

... .

    case 3:
        cout << "Enter data to insert: ";
        cin >> data;
        cout << "Enter position: ";
        cin >> pos;
        insertAtPosition(data, pos);
        break;

```

Program Analysis:

```

if (pos < 1)
{
    cout << "Invalid position!" << endl;
    return;
}

```

If pos or input position by user less than 1 ,then it will give output ``Invalid position`` and return void and exit from function.

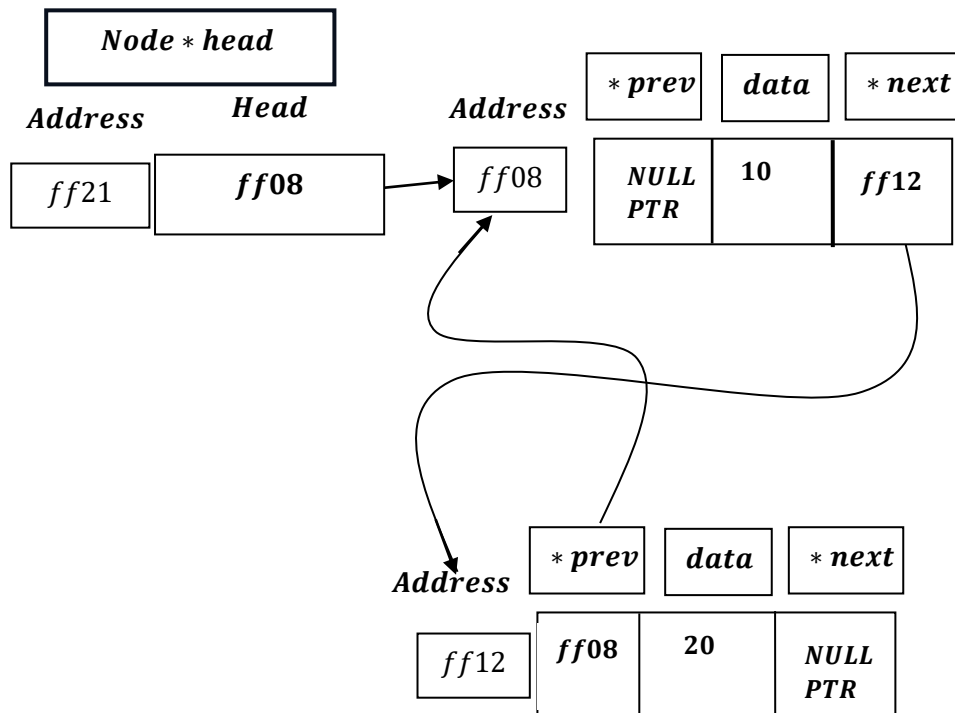
```

if (pos == 1)
{
    insertAtBeginning(data);
    return;
}

```

If pos or input position by user equal to 1 ,then it run func: *inserAtBeginning* , passing integer user input ``data`` i.e. *insertAtBengining(data)* to insert a node with data at beginning .

Now ,say we have a list:

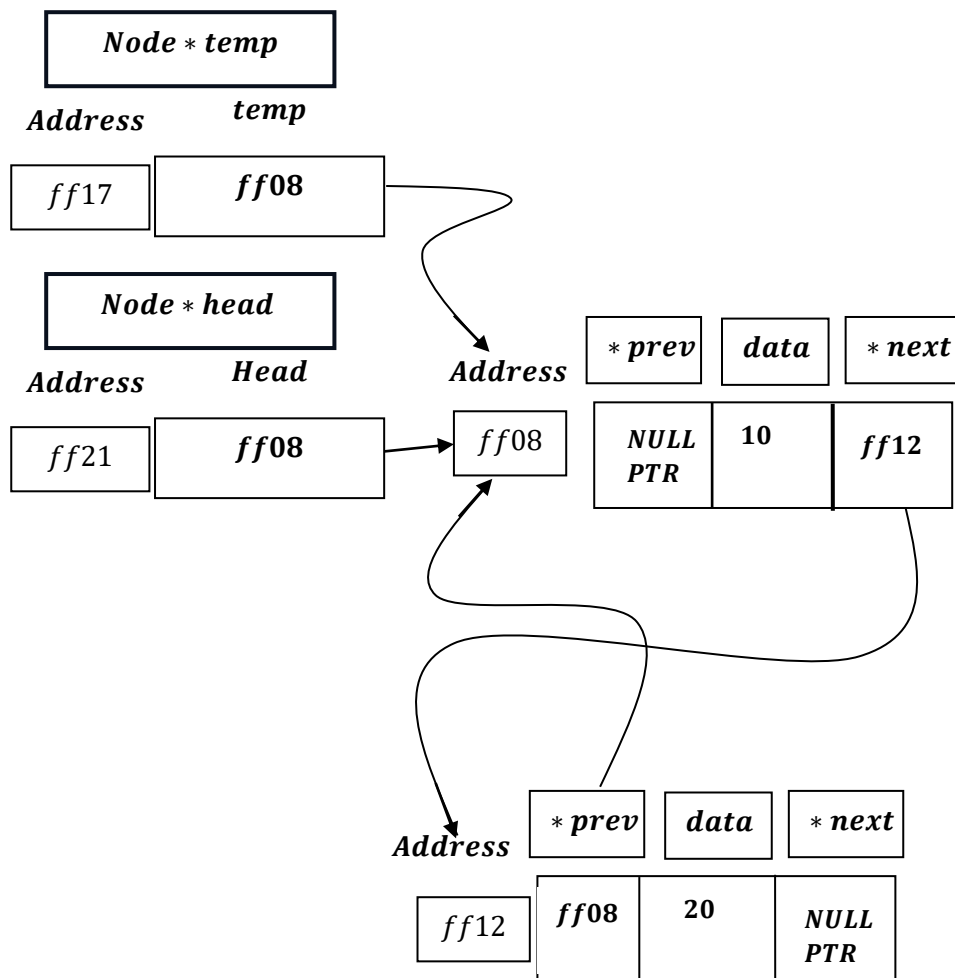


Now, we want to insert data 15 at pos: 2.

Next,

```
Node *temp = head;
```

```
Node * temp = head = ff08;
```



Next,

```
for (int i = 1; i < pos - 1 && temp != nullptr; i++)
{
    temp = temp->next;
}
```

So, pos is: 2 and $i = 1$ and $i < 2 - 1$ i.e. $i < 1$, which implies $i = 1$ and $i < 1$ is contradictory and condition of for loop fails though $temp \neq NULLPTR$ is True. Hence, as condition fails, it exits from for loop.


```
if (temp == nullptr)
{
    cout << "Position out of bounds!" << endl;
    return;
}
```

temp = ff08 and is not nullptr, hence, condition check fails or becomes false.

Next,

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.

→ The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ The size of the Node structure itself is determined by the sum of the sizes of its members:

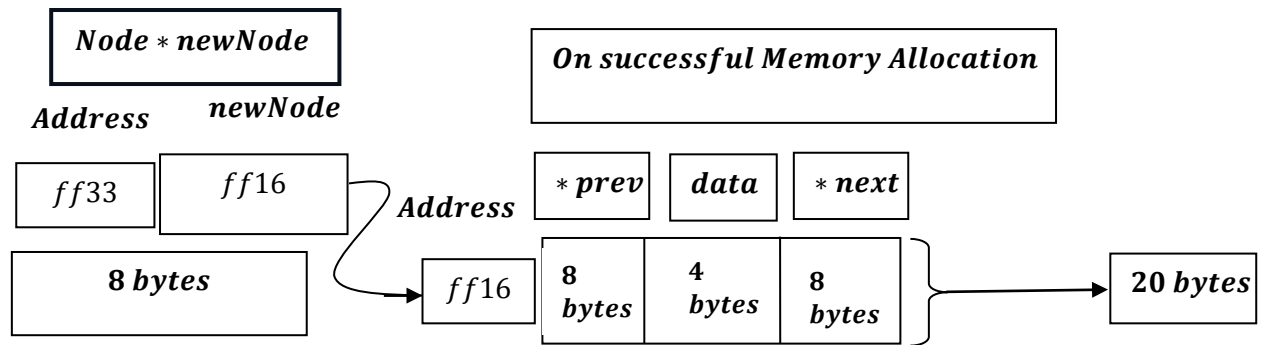
→ data(an integer) is typically 4 bytes.

→ next(a pointer to another Node) is typically 8 bytes.

→ prev(a pointer to previous Node) is typically 8 bytes.

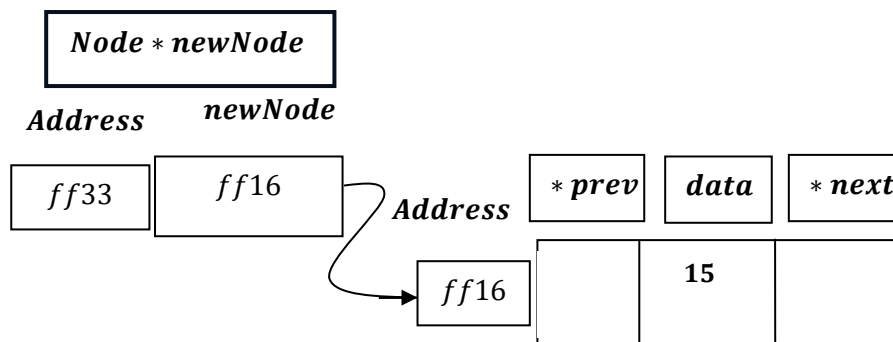
→ Therefore, the size of the Node structure is typically 20 bytes(4 bytes for data + 8 bytes for next + 8 bytes for previous) .

Say,



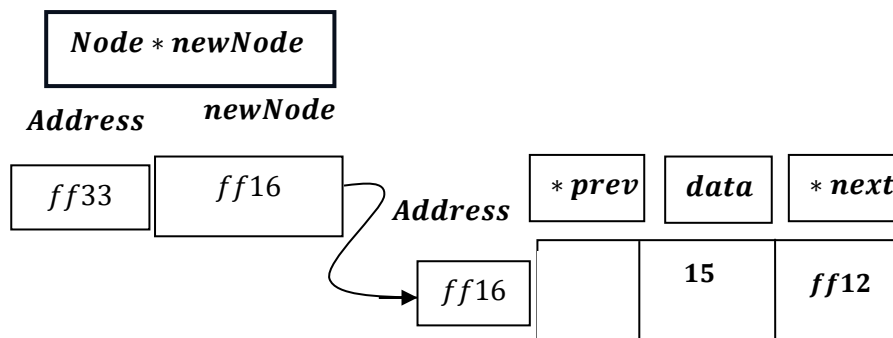
```
newNode->data = data;
```

newNode → *data* = *data* = 15;

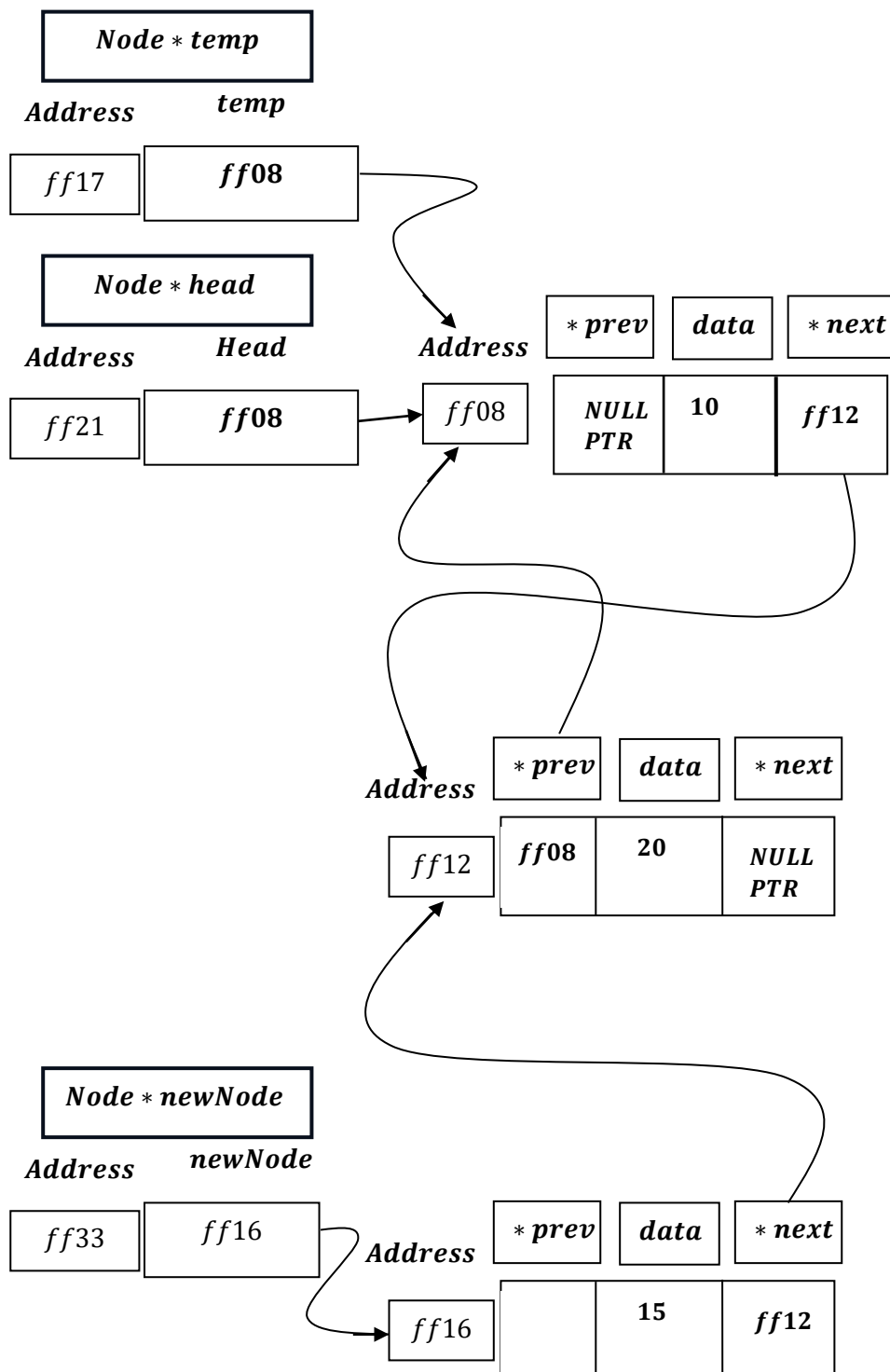


```
newNode->next = temp->next;
```

newNode → *next* = *temp* → *next* = ff12.

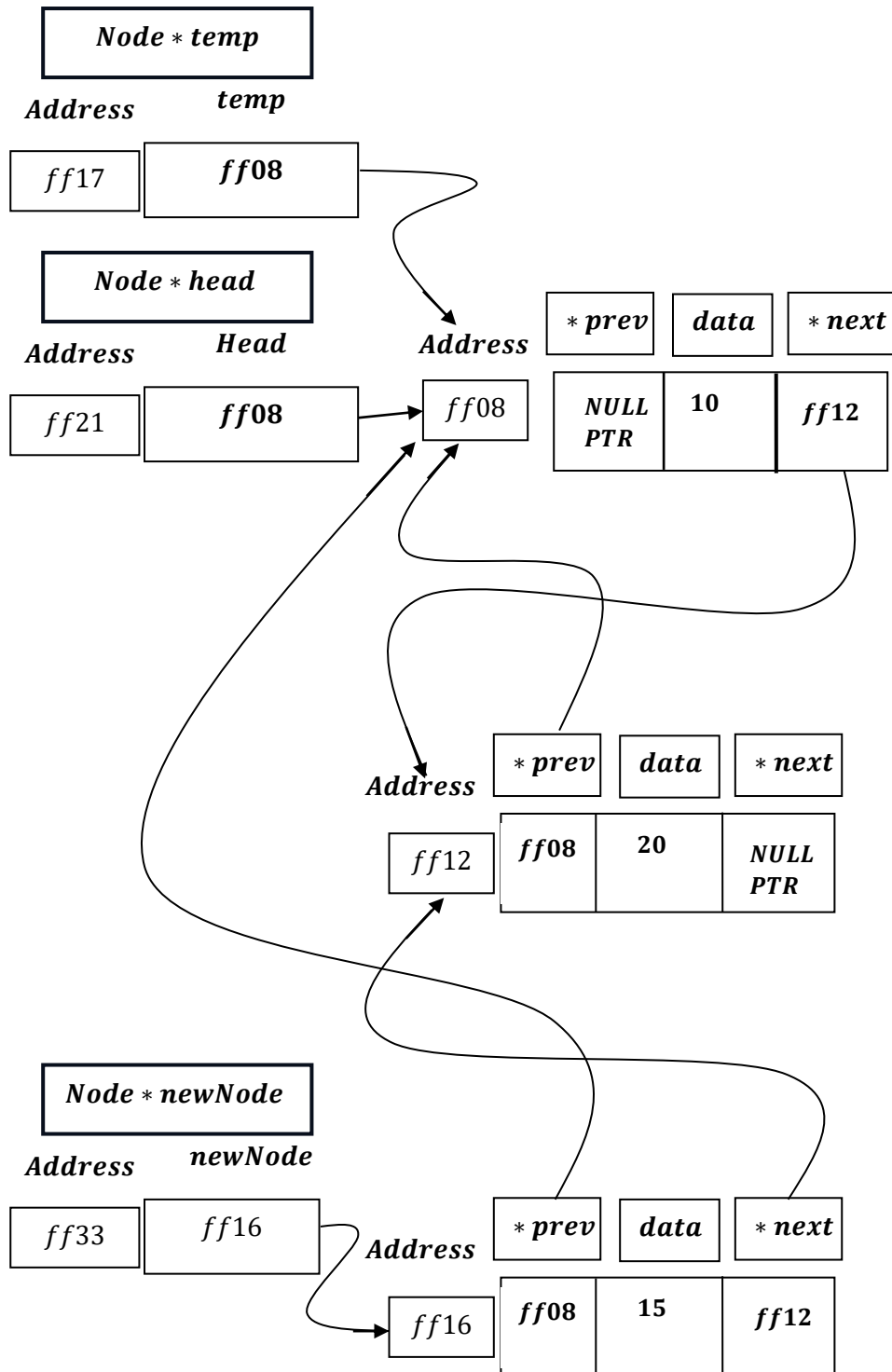


And now, next pointer will point to ff12 address.



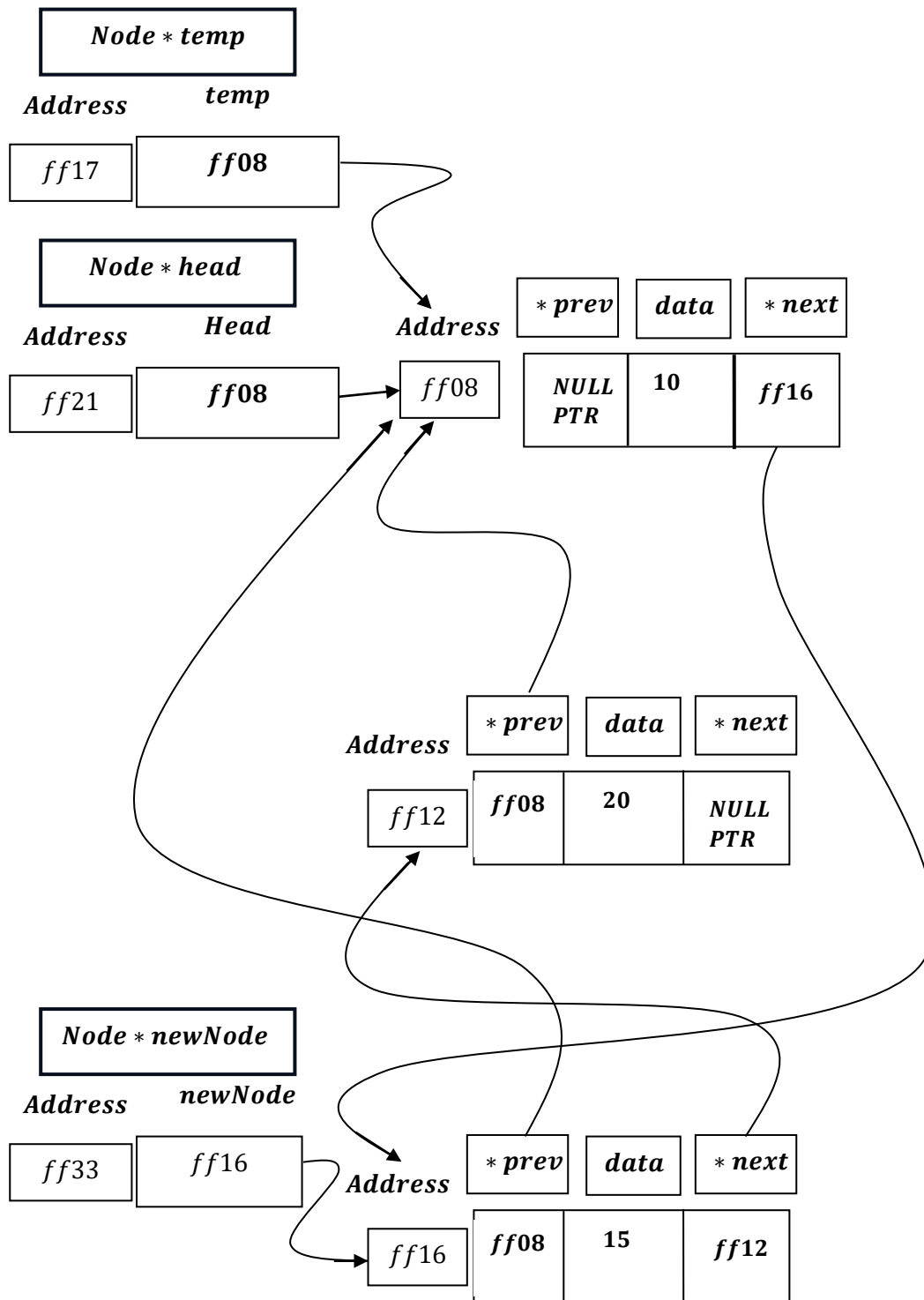
```
newNode->prev = temp;
```

newNode → *prev* = *temp* = *ff08*



```
temp->next = newNode;
```

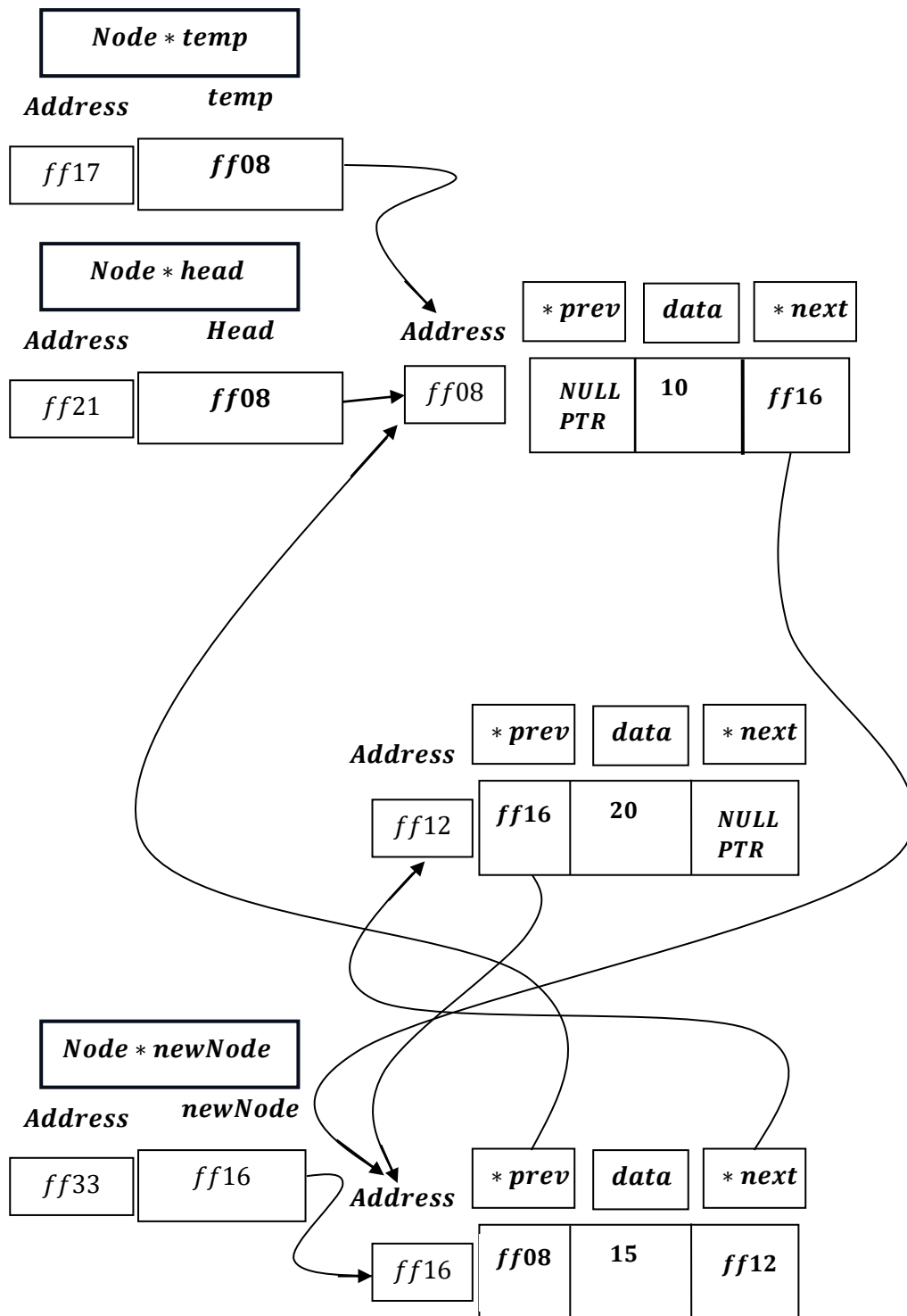
$temp \rightarrow next = newNode = ff16$



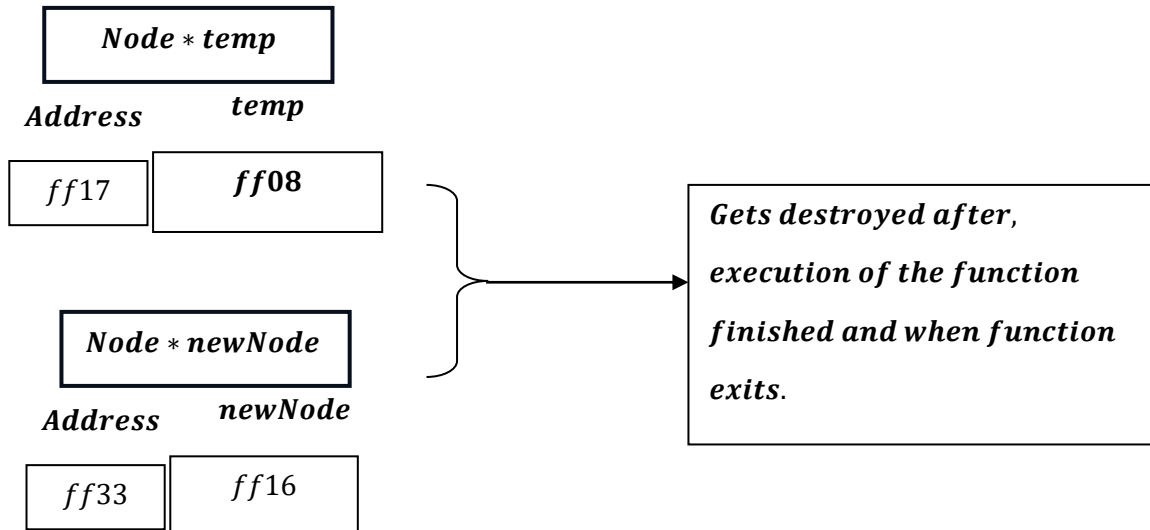
```
if (newNode->next != nullptr)
{
    newNode->next->prev = newNode;
}
```

if $newNode \rightarrow next \neq nullptr$, as $newNode \rightarrow next = ff12$, then:

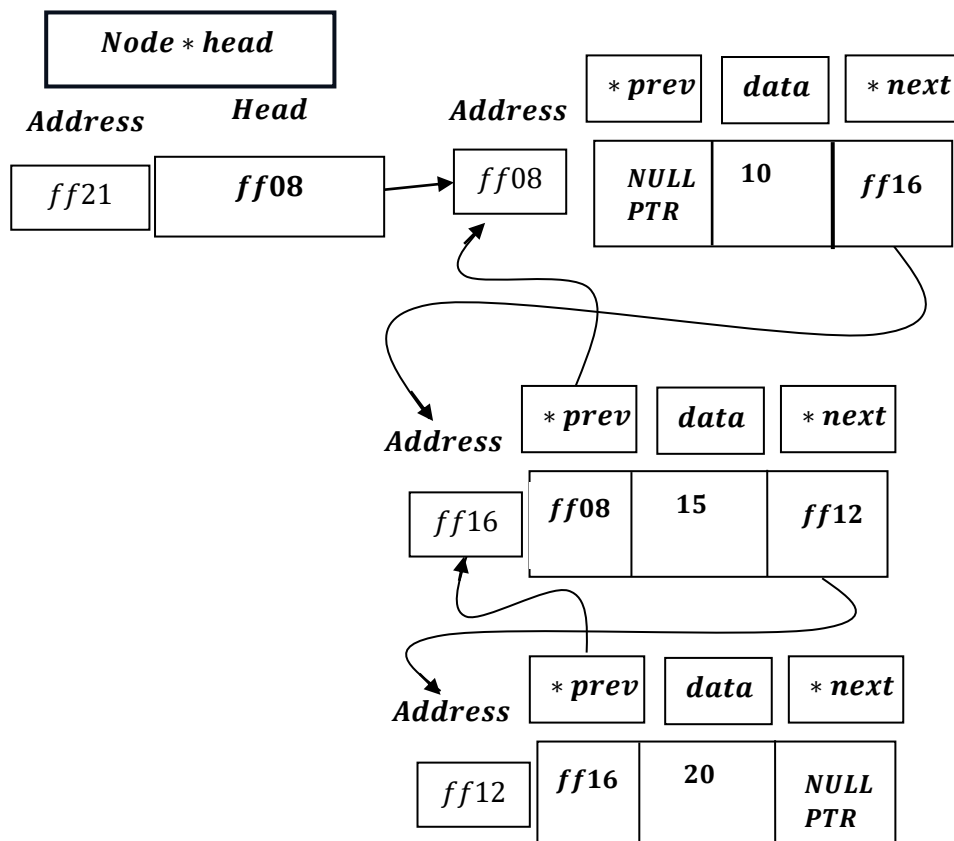
- *$newNode \rightarrow next \rightarrow prev = newNode = ff16$.*



Now, `Node * newNode`, `Node * temp` temporary variables, therefore gets destroyed after function execution get finished .

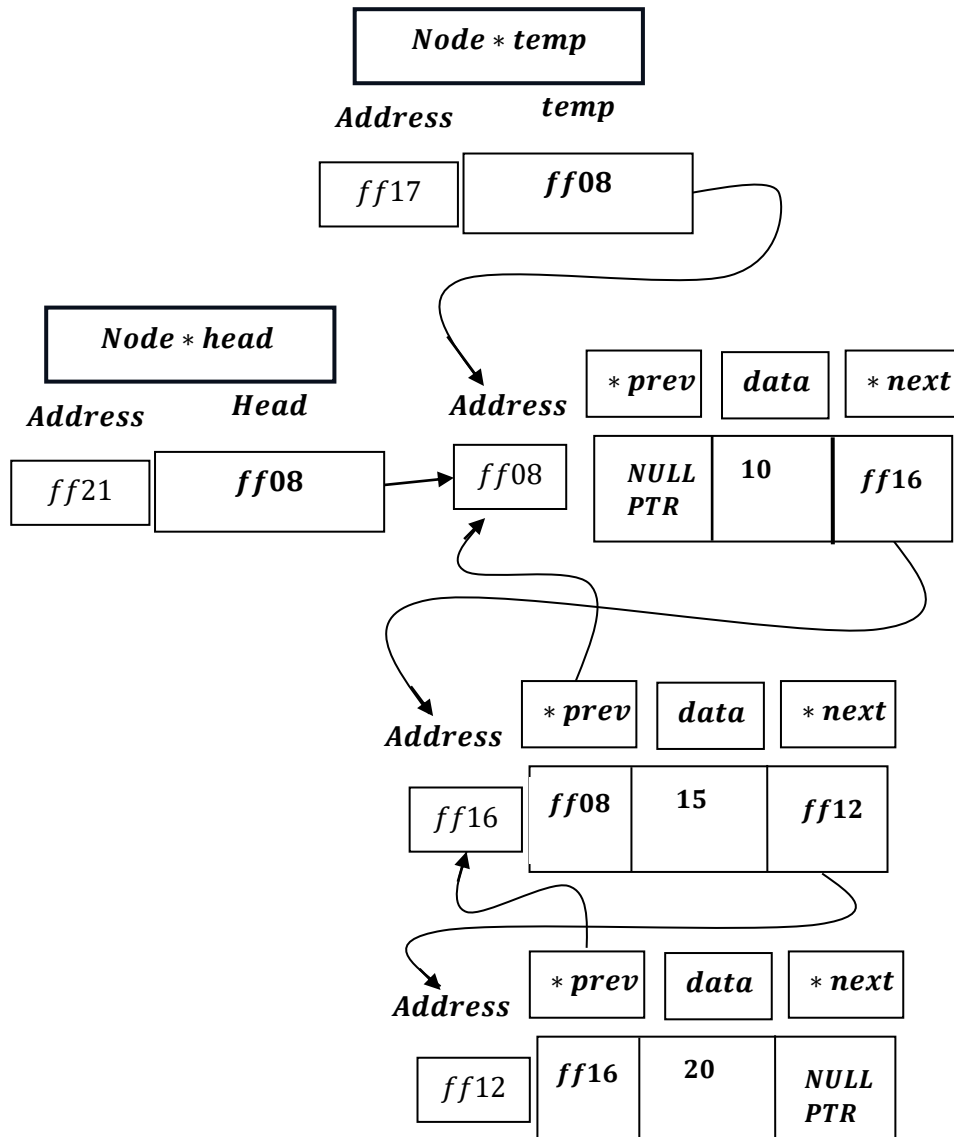


Hence ,we are left with :



When $temp = nullptr$?

Suppose, we have list:

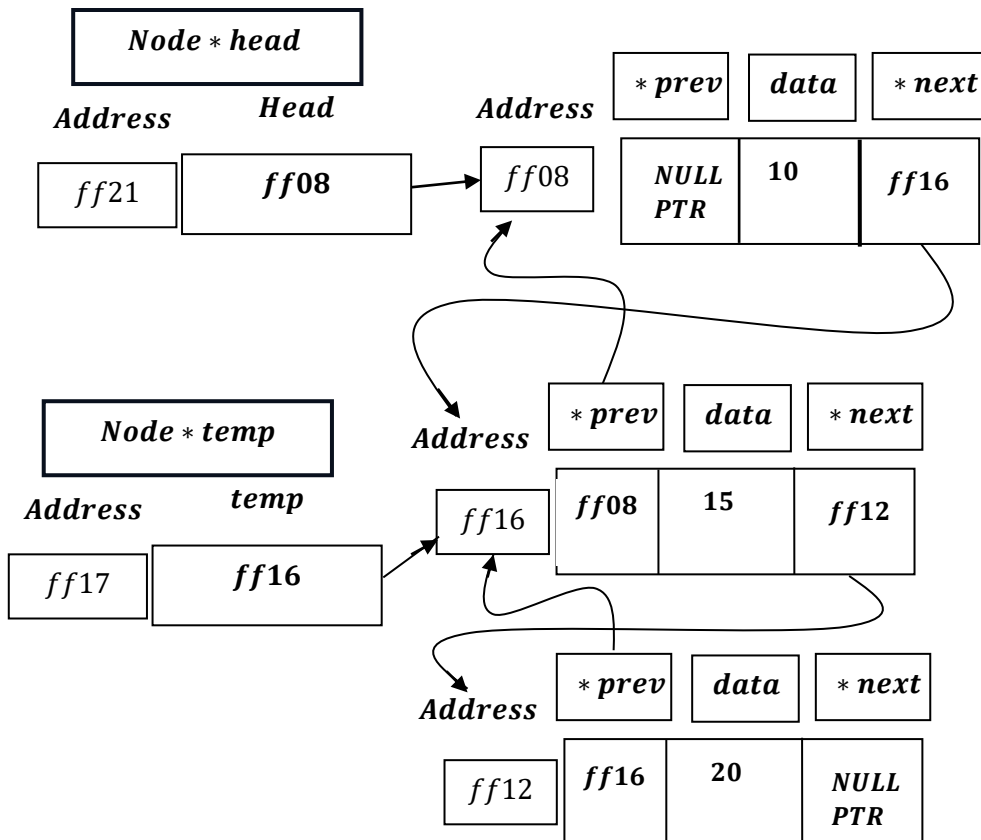


And ,pos input by user 5 ,as input 4 will be the last node, thats why condition is :
 $i = 1$ and $i < pos - 1$. As When we will insert the last $(n + 1^{th})$ node ,
 $temp$ pointer will traverse to $\rightarrow n^{th}$ node .

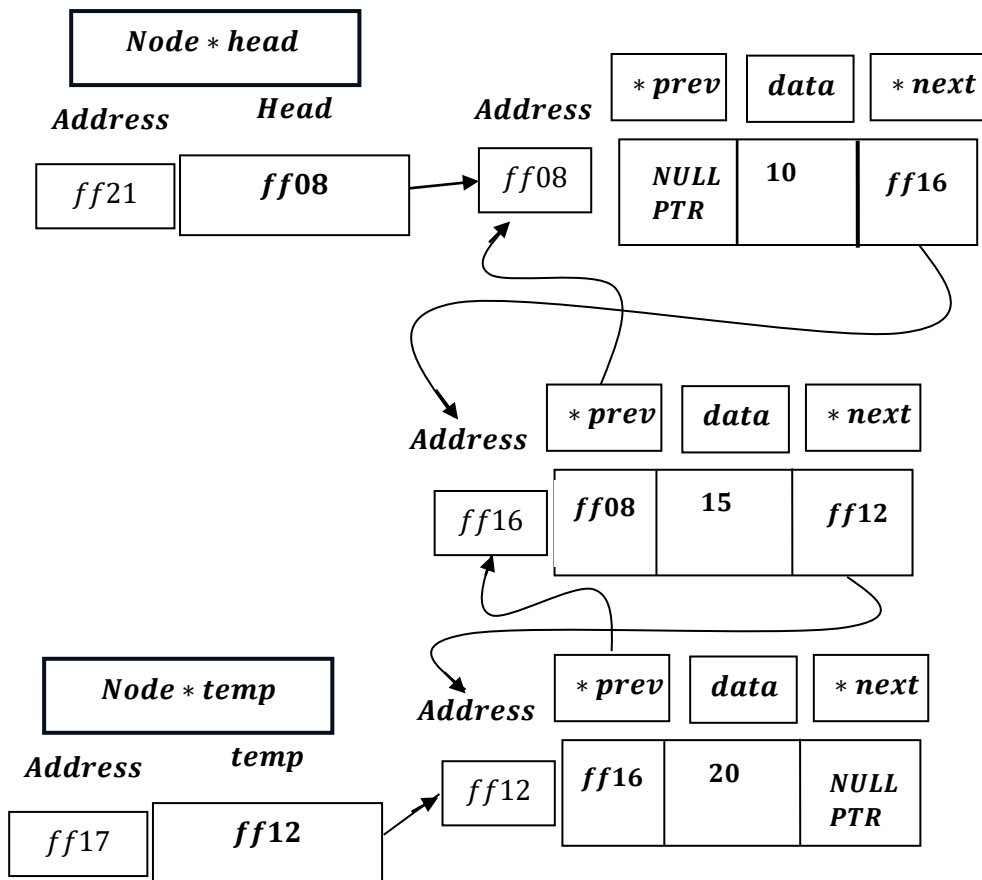
As ,pos input 4 , `i` will be first 1 , i.e. first traversal and then

$i < (4 - 1) = i < 3$ i.e. $i = 2$ i.e. second traversal , implies traversal needed 2 times to point to n^{th} node.

$pos = 4, i = 1$

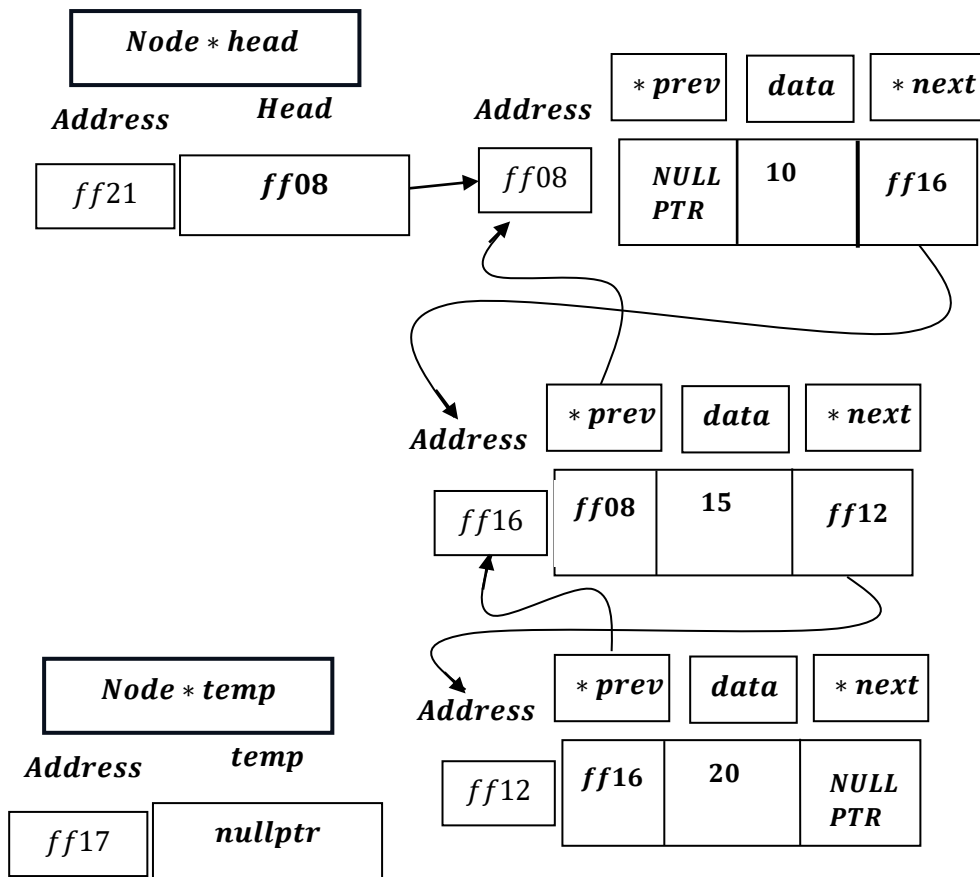


$pos = 4, i = 2[i < (4 - 1 = 3)]$



But if $n = 5$, then:

$pos = 5, i = 3[i < (5 - 1 = 4)]$



As, there is no such node hence temp pointer will be assigned to ``nullptr``.

```
for (int i = 1; i < pos - 1 && temp != nullptr; i++)
{
    temp = temp->next;
}
```

For condition fails as temp = nullptr and loop exits.

```
if (temp == nullptr)
{
    cout << "Position out of bounds!" << endl;
    return;
}
```

Now, temp = NULLPTR, hence, it gives output : "Position out of bounds!"

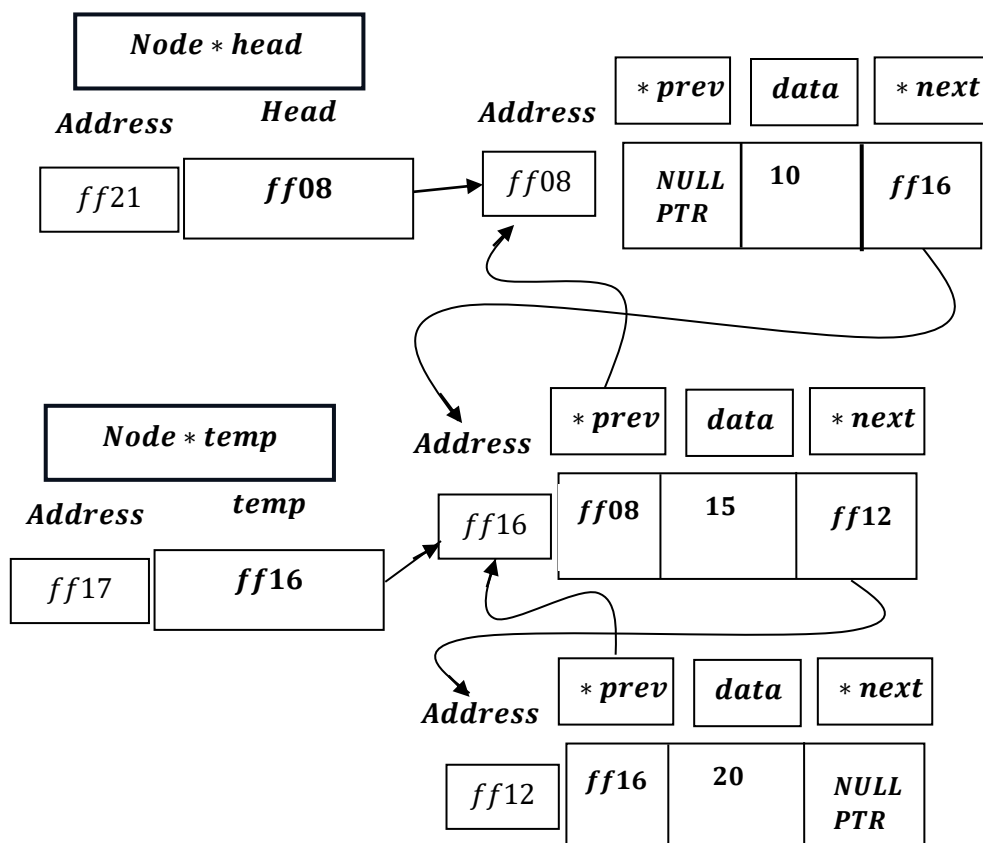
And exits from function.

When newNode → next will be NULLPTR according to function here?

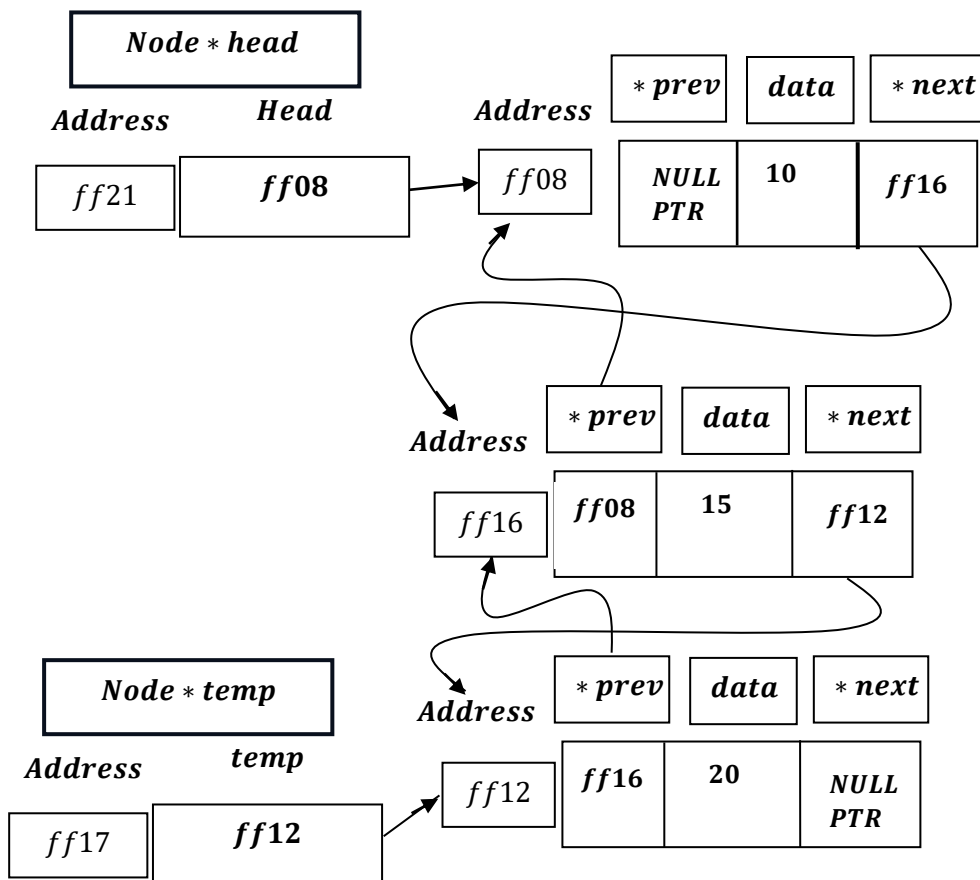
Answer is just simple when we are inserting node at the end and

temp → next = NULLPTR.

pos = 4, i = 1



$pos = 4, i = 2[i < (4 - 1 = 3)]$



And for loop exists ,

```
newNode->next = temp->next;
```

$newNode \rightarrow next = temp \rightarrow next = NULLPTR.$

Hence,

```
if (newNode->next != nullptr)
{
    newNode->next->prev = newNode;
}
```

If condition is false here and statement will not run when we try to insert node at end due to `newNode → next → prev` doesn't exist and `newNode → next = NULLPTR`.

Doubly Linked List - Insert At Position [Working Summary]:

This function inserts a new node at a specific numerical position in the list.

→ **First, it handles edge cases:**

- If the position (`pos`) is less than 1, it's considered invalid, and the function exits.
- If the position is 1, it calls the `insertAtBeginning` function to handle the insertion and then exits.

→ **For any other position, it traverses the list.** A temporary pointer, `temp`, starts at the `head` and moves forward `pos - 2` times. The goal is to make `temp` point to the node at position `pos - 1` (the node that will be *just before* the new node).

→ **After the traversal, it performs a check.** If `temp` is `nullptr`, it means the requested position was out of bounds (the list is shorter than the position), so it prints an error and exits.

→ **Finally, it performs the insertion.** A new node is created. The function then re-wires four pointers to fit the new node between the node at `pos - 1` (`temp`) and the node at the original position `pos` (`temp->next`).

1. The **new node's** `next` points to the original node at `pos`.
 2. The **new node's** `prev` points back to the node at `pos - 1` (`temp`).
 3. The **next pointer of the node at `pos - 1` (`temp`)** is updated to point to the new node.
 4. The **prev pointer of the original node at `pos`** is updated to point back to the new node (this step is skipped if inserting at the very end of the list).
-

Time Complexity of Insert At Position Function Module

```
void insertAtPosition(int data, int pos)
{
    if (pos < 1) → O(1)
    {
        cout << "Invalid position!" << endl; → O(1)
        return; → O(1)
    }
    if (pos == 1) → O(1)
    {
        insertAtBeginning(data); → O(1)
        return; → O(1)
    }

    Node *temp = head; → O(1)
    for (int i = 1; i < pos - 1 && temp != nullptr; i++)
    {
        temp = temp->next;
    }
    → 1 + 1 + ... to pos - 2 times = O(pos - 2) or, O(pos) ≈ O(n).
    → runs 'pos - 2' times at worst condition.

    if (temp == nullptr) → O(1)
    {
        cout << "Position out of bounds!" << endl; → O(1)
        return; → O(1)
    }

    Node *newNode = (Node *)malloc(sizeof(Node)); → O(1)
    newNode->data = data; → O(1)

    newNode->next = temp->next; → O(1)

    newNode->prev = temp; → O(1)

    temp->next = newNode; → O(1)
}
```



```

if (newNode->next != nullptr) → O(1)
{
    newNode->next->prev = newNode; → O(1)
}
cout << data << " inserted at position " << pos << "."
<< endl; → O(1)
}

```

There are three case scenarios:

- 1. *Position is less than 1 or 1 and 2 , i.e. we want to insert data at first or second . (Best Case)*
- 2. *Position is $N + 1$, i.e. we want to insert data position at end . (Worst Case)*
- 3. *Inserted position from 3 to $N + 1$, for loop runs from 1 to $N - 1$. While if position is 1 and 2 , it takes constant amount of time : $O(1)$. And when the position is uniformly distributed from 1 to $(N + 1)$. (Average Case i.e. Undetermined Position). That is we donot whether the position will be 1 or 2 or $N + 1$, where Node is to be inserted.*

1. When Position is less than 1 or 1 and 2 (Best Case)

1. A. When Position is less than 1 or equal to 1 [Absolute Best Case]:

1. *If position `pos` less than 1, it outputs ``Invalid Position`` and return void , which exits from the function, and these process takes constant amount of time , typically $O(1)$.*

2. *if position `pos` is equal to 1 , then it runs insertAtBeginning function , we know this function takes constant amount of time , typically $O(1)$ with retun void statement ,*

(return statement) which makes function to exit, process takes constant amount of time $O(1)$.

1. B. When Position is equal to 2 [Best Case but not the Absolute Best Case]

1. If position `pos` less than 1, which is false, this check takes constant time, $O(1)$.

2.. if position `pos` is equal to 1, which is false, this check takes constant time, $O(1)$.

3. Temp pointer of Node named Structure, its value assigned to head pointer's value i.e. the address to which head pointer points to, making Temp pointer points to that address. This operation takes constant amount of time i.e. $O(1)$.

4. Now for loop, $i = 1$ and $i < (pos - 1)$ i.e. $i < (2 - 1)$ i.e., $i < 1$, the condition contradicts itself, hence for loop does not run, the condition checking takes constant amount of time i.e. $O(1)$.

5. newNode pointer of Node named structure points and allocates a memory for the Node structure, allocating for prev pointer, integer data, and next pointer, this pointing as well as allocating through `malloc` takes constant amount of time i.e., $O(1)$.

6. integer data in the memory allocated pointed by newNode pointer of Node structure set to data by user input, takes constant amount of time i.e., $O(1)$.

7. next pointer of allocated memory pointed by newNode pointer of Node structure set to the value(address) obtained by next pointer of allocated memory pointed by temp pointer of Node structure and it makes next pointer pointed by newNode to point to that address, which is pointed by next pointer pointed by temp pointer, the process takes constant amount of time i.e., $O(1)$.

8. *prev pointer of allocated memory pointed by newNode pointer of Node structure set to the value(address)obtained by temp pointer of Node structure and it makes prev pointer pointed by newNode pointer point to that address ,which is pointed by temp pointer ,the process takes constant amount of time i.e., $O(1)$.*

9. *next pointer of allocated memory pointed by temp pointer of Node structure set to the value(address) obtained by newNode pointer of Node structure and it makes next pointer pointed by temp pointer point to that address ,which is pointed by newNode pointer ,the process takes constant amount of time i.e., $O(1)$.*

10. *Now ,If condition,checks next pointer pointed by newNode pointer of Node structure not equal to `nullptr` ,which is true then prev pointer pointed by next pointer pointed by newNode pointer of Node structure set to the value i.e.address obtained by newNode pointer ,making prev pointer pointed to address ,which is being pointed by newNode. ,the process takes constant amount of time i.e., $O(1)$.*

11. *Now we output a statement,user input data inserted at positon ,the process takes constant amount of time i.e., $O(1)$.*

Hence ,total time complexity it takes : $O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(1)$,i.e.constant time complexity.

Why not Absolute Best Case ?

⇒ When pos = 1, loop runs 0 times. [maintaining ,pos – 2 times]

⇒ When pos = 2, loop runs 0 times. [maintaining ,pos – 2 times]

⇒ When pos = 3, loop runs 1 times. [maintaining ,pos – 2 times]

.

.

.

⇒ When n = n, loop runs `pos – 2` times. [maintaining ,pos – 2 times]

⇒ When n = n + 1, loop runs `pos – 2` times. [maintaining ,pos – 2 times]

Loop should run `pos – 2` times as if we want to enter the last node

in the list, i.e., $n + 1$ node then, temp pointer must point to the current last node i.e. $(n)^{th}$ node.

Why $pos - 2$ times is needed because temp must point to Head node or, the node pointed by head at first, then traversal take place, as it already point a node, therefore traversal will be $pos - 2$ times.

If we notice minutely then we will observe that, the loop runs 1 to $n - 1$ times in worst case scenario, being n is the size of the list.

Such as if n is 3 then if we want to insert at $n + 1$ node i.e. $pos = 4$, then, it will take loop to run 2 times i.e. $i < (n + 1) - 1$ or, $i < n$, hence $n - 1$ times.

Therefore when, $pos = 2$, as function remains at constant time, we can take this as best case but not the 'Absolute Best Case'. The 'Absolute Best Case' is when List is Empty.

3. When Position is $N + 1$, Worst Case Scenario

1. If position pos less than 1, which is false, this check takes constant time, $O(1)$.

2.. if position pos is equal to 1, which is false, this check takes constant time, $O(1)$.

3. Temp pointer of Node named Structure, its value assigned to head pointer's value i.e. the address to which head pointer points to, making Temp pointer points to that address. This operation takes constant amount of time i.e. $O(1)$.

4. Now for loop, $i = 1$ and $i < (pos - 1)$ i.e. $i < (pos - 1)$ i.e., $i = pos - 2$, the condition assert itself as true, then:

$temp = temp \rightarrow next$, runs $pos - 2$ times till temp points to current n^{th} node. i.e. statement runs: $1 + 1 + 1 + 1 + \dots (pos - 2)$ times = $O(pos - 2) = O(pos)$ and if we substitute pos with n , statement runs: $O(n)$ times.

Alternatively ,

*if n size = 1 , inserting ' $n + 1$ ' node make loop runs 0 times i.e. $i < (n + 1) - 1$ or,
 $i < n$ i.e. $n - 1$ times .*

*if n size = 2 , inserting ' $n + 1$ ' node make loop runs 1 times i.e. $i < (n + 1) - 1$ or,
 $i < n$ i.e. $n - 1$ times .*

*if n size = 3 , inserting ' $n + 1$ ' node make loop runs 2 times i.e. $i < (n + 1) - 1$ or,
 $i < n$ i.e. $n - 1$ times .*

*.
.*

*if n size = n , inserting ' $n + 1$ ' node make loop runs i.e. $i < (n + 1) - 1$ or,
 $i < n$ i.e. $n - 1$ times .*

*Alternatively, ' $temp = temp \rightarrow next$ ', runs ' $n - 1$ ' times till $temp$ points to
current ' n^{th} ' node i.e. statement runs : $1 + 1 + \dots (n - 1)$ times = $O(n - 1) = O(n)$.*

*5. newNode pointer of Node named structure points and allocates a memory
for the Node structure , allocating for prev pointer , integer data, and next pointer
, this pointing as well as allocating through ' $malloc$ ' takes constant amount of
time i.e., $O(1)$.*

*6. integer data in the memory allocated pointed by newNode pointer of Node structure
set to data by user input , takes constant amount of time i.e., $O(1)$.*

*7. next pointer of allocated memory pointed by newNode pointer of Node structure
set to the value(address) obtained by next pointer of allocated memory pointed by
temp pointer of Node structure and it makes next pointer pointed by newNode to
point to that address , which is pointed by next pointer pointed by temp pointer ,*

the process takes constant amount of time i.e., $O(1)$.

8. prev pointer of allocated memory pointed by newNode pointer of Node structure set to the value(address) obtained by temp pointer of Node structure and it makes prev pointer pointed by newNode pointer point to that address ,which is pointed by temp pointer ,the process takes constant amount of time i.e., $O(1)$.

9. next pointer of allocated memory pointed by temp pointer of Node structure set to the value(address) obtained by newNode pointer of Node structure and it makes next pointer pointed by temp pointer point to that address ,which is pointed by newNode pointer ,the process takes constant amount of time i.e., $O(1)$.

10. Now ,If condition, checks next pointer pointed by newNode pointer of Node structure not equal to `nullptr` ,which is true then prev pointer pointed by next pointer pointed by newNode pointer of Node structure set to the value i.e. address obtained by newNode pointer ,making prev pointer pointed to address ,which is being pointed by newNode. ,the process takes constant amount of time i.e., $O(1)$.

11. Now we output a statement, user input data inserted at position ,the process takes constant amount of time i.e., $O(1)$.

Hence ,total time complexity it takes : $O(1) + O(1) + O(1) + O(n) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(n)$, i.e. constant time complexity.

3. Inserted position from 1 to $N + 1$, while loop runs from 1 to N .

And when the position is uniformly distributed from 1 to $(N + 1)$:

(Average Case i. e. Undetermined Position)

→ **If the position is 1, the loop's inner statement runs 0 time.**

→ **If the position is 2, the loop's inner statement runs 0 time.**

→ **If the position is 3, the loop's inner statement runs 1 times.**

→ **If the position is 4, the loop's inner statement runs 2 times.**

.

.

→ **If the position is $(N + 1)$, the loop's inner statement runs : $N - 1$ times ,**

As when $i = N < N$ becomes false , hence no execution of loop's inner statements.

$$\text{Hence, } \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}$$

Now, as we take all the possibilites then , we take the best case also i. e.

$O(1)$, when position = 1 and 2. Considering ' $T(P)$ ' is time complexity of position ' P ' .

$$T(P) = \begin{cases} 1 & , \text{when } P = 1. \\ 1 & , \text{when } P = 2. \\ P - 2 \text{ or } N - 1 & , \text{when } P = 3, 4, \dots, N + 1. \end{cases}$$

Now, If there is ' N ' no. of nodes there we can insert data at $N + 1$ positions.

For position 1, the probability of insertion is $\frac{1}{N+1}$.

For position 2, the probability of insertion is $\frac{1}{N+1}$.

.

.

For position $N+1$, the probability of insertion is $\frac{1}{N+1}$.

Hence probability of insertion will always remain $\frac{1}{N+1}$.

$\therefore$ Average Time Complexity is : probability of insertion $\times$ (number of possible iterations + constant time complexities for left over positions) .

$$= \left(\frac{1}{N+1} \times 1 + \frac{1}{N+1} \times 2 + \dots + \frac{1}{N+1} \times N - 1 \right) + \frac{1}{N+1} \times 1 + \frac{1}{N+1} \times 1$$

$$= \frac{1}{N+1} \times \{(1 + 2 + 3 + \dots + N - 1) + 1 + 1\}$$

$$= \frac{1}{N+1} \times \left\{ \sum_{i=1}^{N-1} i + 2 \right\}$$

$$= \frac{1}{N+1} \times \left(\frac{N(N-1)}{2} + 2 \right)$$

$$= \frac{N(N-1)}{2(N+1)} + \frac{2}{N+1}$$

$$= \frac{N^2 - N}{2N+2} + \frac{2}{N+1}$$

Appplying Big Theta in the equation:

$$= \Theta\left(\frac{N^2 - N}{2N + 2}\right) + \Theta\left(\frac{2}{N + 1}\right)$$

$$= \Theta\left(\frac{N^2 - N}{2N + 2}\right) + \Theta\left(\frac{2}{N + 1}\right)$$

Now, taking out the dominant term of numerator and denominator:

- ***In the numerator, $N^2 - N$, the term N^2 grows much faster than N .***

So, for large N , the dominant term is N^2 .

- ***In the denominator, $2N + 2$, the term $2N$ grows much faster than the constant 2. The dominant term is $2N$.***

$$= \Theta\left(\frac{N^2}{2N}\right) + \Theta\left(\frac{2}{N}\right)$$

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{2}{N}\right)$$

$$= \Theta\left(\frac{N}{2}\right) + 2 \times \Theta\left(\frac{1}{N}\right)$$

Now removing the constant `2` we get:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

Now removing the constant $\frac{1}{2}$, we get:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

When combining two Big Theta Θ terms, largest growth rate is the result:

$\rightarrow \Theta(N)$ grows linearly with N .

$\rightarrow \Theta\left(\frac{1}{N}\right)$ decreases to a constant, i.e. when N grows larger, $\frac{1}{N}$ approaches to 0.

This we can prove by limit and continuity also:

$$\lim_{n \rightarrow \infty} \left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0 \left[\frac{c}{\infty} = 0, \text{ where } c \text{ is constant} \right].$$

As $\Theta(N)$'s growth rate is higher, therefore, average case is $\Theta(N)$.

Hence, Average Time Complexity is : $\Theta(N)$.

Note, by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N(N-1)}{2(N+1)} + \frac{2}{N+1} \leq c_2 \times N$$

Average Case Time Complexity[Most Simplest Alternative Approach]:
(Simplest approach)

For position 1, the probability of insertion is $\frac{1}{N+1}$.

For position 2, the probability of insertion is $\frac{1}{N+1}$.

.

.

For position $N+1$, the probability of insertion is $\frac{1}{N+1}$.

**Hence probability of insertion will always remain $\frac{1}{N+1}$,
for position 1 to $N+1$.**

Average case complexity : Probability of insertion $\times$ $\left(\begin{array}{c} \text{Summation} \\ \text{of number of} \\ \text{Position.} \end{array} \right)$

$$= \frac{1}{N+1} \times \left(\sum_{i=1}^{N+1} i \right)$$

$$= \frac{1}{N+1} \times \left(\frac{1}{2} (N+1) ((N+1) + 1) \right) \left[\text{As, } \sum_{k=1}^n k = \frac{1}{2} (n)(n+1) \right]$$

$$= \frac{1}{N+1} \times \left(\frac{(N+1)(N+2)}{2} \right)$$

$$= \frac{N+2}{2}$$

$$= \frac{N}{2} + \frac{2}{2}$$

$$= \frac{N}{2} + 1$$

Applying Big Theta Θ we get:

$$= \Theta\left(\frac{N}{2}\right) + \Theta(1)$$

$$= \frac{1}{2} \times \Theta(N)$$

$$= \Theta(N)$$

Hence, Average Time Complexity is : $\Theta\left(\frac{N}{2} + 1\right)$ or $\Theta(N)$.

Now,

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N}{2} + 1 \leq c_2 \times N$$

<i>Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>InsertAt Position</i>	<i>1. Best Case</i>	$\Omega(1)$
	<i>2. Worst Case</i>	$O(n)$
	<i>3. Average Case</i>	$\Theta(n)$

Doubly Linked List – Insert At End –Space Complexity and Algorithm.

Program :

```
void insertAtPosition(int data, int pos)
{
    if (pos < 1)
    {
        cout << "Invalid position!" << endl;
        return;
    }
    if (pos == 1)
    {
        insertAtBeginning(data);
        return;
    }

    Node *temp = head;
    for (int i = 1; i < pos - 1 && temp != nullptr; i++)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }

    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;

    newNode->next = temp->next;
    newNode->prev = temp;
    temp->next = newNode;
```

```

    if (newNode->next != nullptr)
    {
        newNode->next->prev = newNode;
    }
    cout << data << " inserted at position " << pos << "."
<< endl;
}

... .

    case 3:
        cout << "Enter data to insert: ";
        cin >> data;
        cout << "Enter position: ";
        cin >> pos;
        insertAtPosition(data, pos);
        break;

```

Space Complexity

Auxiliary Space

Auxiliary space is the extra memory used by an algorithm, excluding the space taken by the inputs.

In the `insertAtPosition` function, the new memory usage is:

- `Node *newNode`: A single new node is allocated. The size of this node is **constant**.
- `Node *temp`: A temporary pointer is created to traverse the list. The size of a pointer is also **constant**.
- `int i`: An integer variable for the `for` loop counter, which occupies a **constant** amount of space.

Since the amount of extra memory does not grow with the size of the linked list, the auxiliary space complexity is $O(1)$.

- **Auxiliary Space: $O(1)$.**

Input Space

Input space is the memory used to store the function's inputs. This can be viewed in two ways.

1. Parameter-only Definition

This perspective considers only the space taken by the function's parameters: `void insertAtPosition(int data, int pos)`.

- The inputs are two integers, `data` and `pos`, each of which occupies a fixed amount of memory.

Under this common definition, the input space is constant.

- **Input Space (Parameter-only): $O(1)$.**

2. Full-Data Definition

This broader view includes the space taken by the entire data structure the function operates on—in this case, the doubly linked list.

- If the linked list contains ***n*** nodes, the total memory it occupies is proportional to ***n***.

Under this definition, the input space is linear with respect to the size of the list.

- **Input Space (Full-Data): $O(n)$.**

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In typical analysis, the "space complexity" of a function refers to its auxiliary space. Therefore, the most common and expected answer for this function is $O(1)$. The for loop traversal affects the function's time complexity ($O(n)$ in the worst case), but not its space complexity.

Algorithm: Insert at a Given Position (Doubly Linked List)

DINSPOS(START, ITEM, POS)

*((Node *)malloc(sizeof(Node)) is availability of space for insertion of Data,
hence, let's name it AVAIL.*)

1. [CHECK FOR INVALID POSITION]

If POS < 1, then:

Write: "Invalid position!" and EXIT.

[End of If structure.]

2. [CHECK FOR INSERTION AT BEGINNING]

If POS = 1, then:

Call DINSATBEG(START, ITEM) [Use the beginning insertion algorithm].

Translating DINSBEG(START, ITEM):

DINSATBEG(START, ITEM)

*((Node *)malloc(sizeof(Node)) is availability of space for insertion of Data,
hence, let's name it AVAIL.*)

1. [ALLOCATE NEW NODE]

Set NEW := AVAIL.

2. [MEMORY ALLOCATED?]

If NEW = NULL, then:

Write: "Error: Memory allocation failed." and EXIT.

[End of If structure.]

3. [COPY DATA]

Set INFO[NEW] := ITEM.

4. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

a. Set PREVLINK[NEW] := NULL.

b. Set NEXTLINK[NEW] := NULL.

c. Set START := NEW.

d. Write: ITEM, "inserted at the beginning."

ELSE: [List is not empty]

a. Set PREVLINK[NEW] := NULL.

b. Set NEXTLINK[NEW] := START.

c. Set PREV[START] := NEW [Point the original first node's PREV back to the new node].

d. Set START := NEW.

e. Write: ITEM, "inserted at the beginning."

[END OF IF STRUCTURE]

5. *Exit*

EXIT.

[End of If structure.]

3. [TRAVERSE TO THE NODE BEFORE THE TARGET POSITION]

Set PTR := START [PTR is a pointer initialized to the start of the list].

FOR(*SET COUNT := 1 ; COUNT < (POS - 1) AND PTR ≠ NULL ; SET COUNT := COUNT + 1*)**DO** :

Set PTR := NEXTLINK[PTR] [Updates PTR to the next node].

[END of Loop]

4. [CHECK IF POSITION IS OUT OF BOUNDS]

If PTR = NULL, then:

Write: "Position out of bounds!" and EXIT.

[End of If structure.]

5. [ALLOCATE NEW NODE]

Set NEW := AVAIL.

6. [MEMORY ALLOCATED?]

If NEW = NULL, then:

Write: "Error: Memory allocation failed." and EXIT.

[End of If structure.]

7. [COPY DATA INTO NEW NODE]

Set INFO[NEW] := ITEM.

8. [LINK THE NEW NODE INTO THE LIST]

a. Set NEXTLINK[NEW] := NEXTLINK[PTR] [New node's NEXT points to what was after PTR].

b. Set PREVLINK[NEW] := PTR [New node's PREV points back to PTR].

c. Set NEXTLINK[PTR] := NEW [PTR now points forward to the NEW node].

d. If NEXTLINK[NEW] ≠ NULL, then: [This handles cases where the node is not inserted at the very end].

Set PREVLINK[NEXTLINK[NEW]] := NEW $\left(\begin{array}{l} \text{The node that was originally after PTR} \\ \text{must now point its PREVLINK back to the NEW node} \end{array} \right)$.

[End of If structure]

9. [OUTPUT THE INSERTED ITEM AT POSITION]

Write: ITEM, "inserted at", POS.

10. Exit

Doubly Linked List – Insert After Element And Time Complexity.

Program :

```
void insertAfterElement(int data, int element)
{
    Node *temp = head;
    while (temp != nullptr && temp->data != element)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Element " << element << " not found in the
list." << endl;
        return;
    }

    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;

    newNode->next = temp->next;
    newNode->prev = temp;
    temp->next = newNode;
    if (newNode->next != nullptr)
    {
        newNode->next->prev = newNode;
    }
    cout << data << " inserted after " << element << "." <<
endl;
}

..... •
```

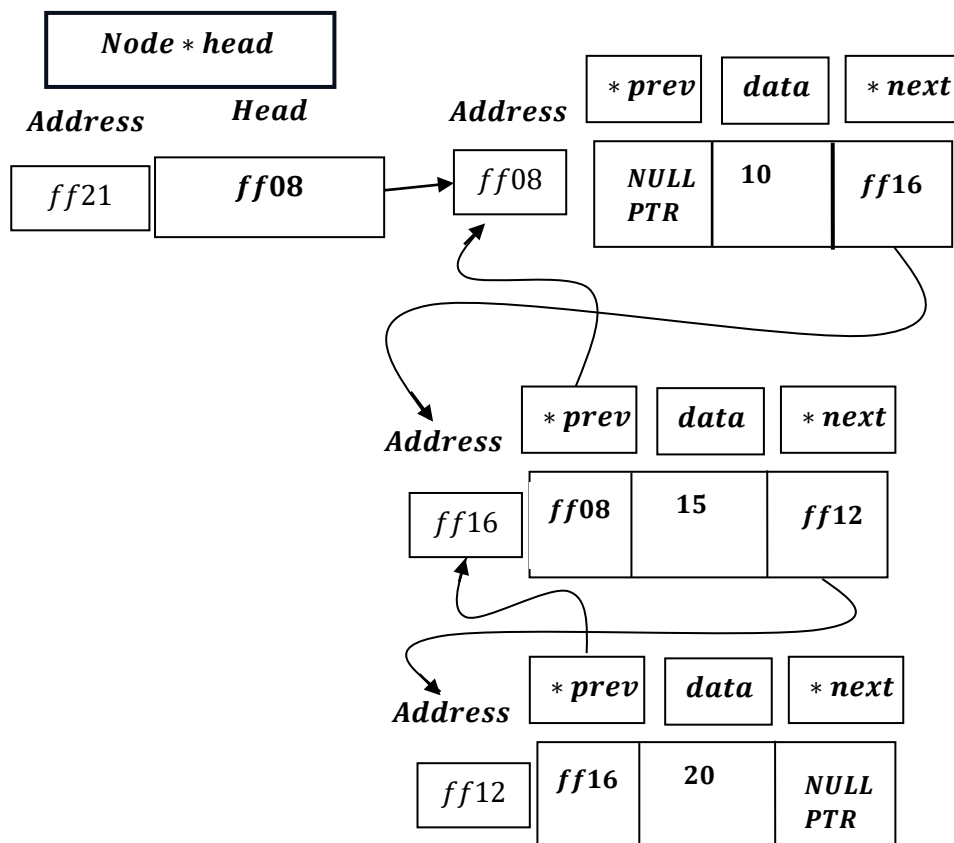
```

case 4:
    cout << "Enter data to insert: ";
    cin >> data;
    cout << "Enter the element to insert after: ";
    cin >> element;
    insertAfterElement(data, element);
    break;

```

Program Analysis:

Suppose we have a list:



```

void insertAfterElement(int data, int element)
{.....}

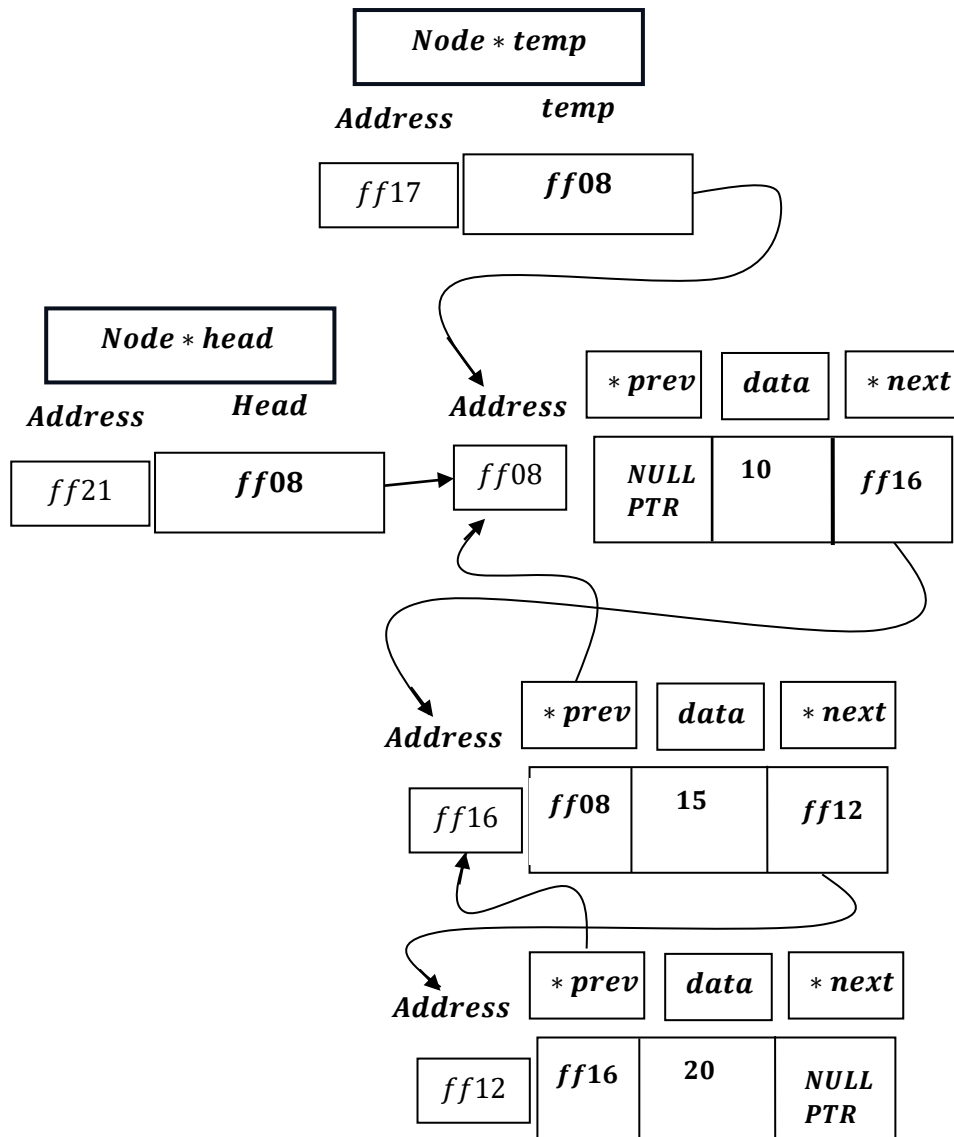
```

Here data → is for new user input.

element → The data is to be input , after the element , present in the list.

Next,

```
Node *temp = head;
```



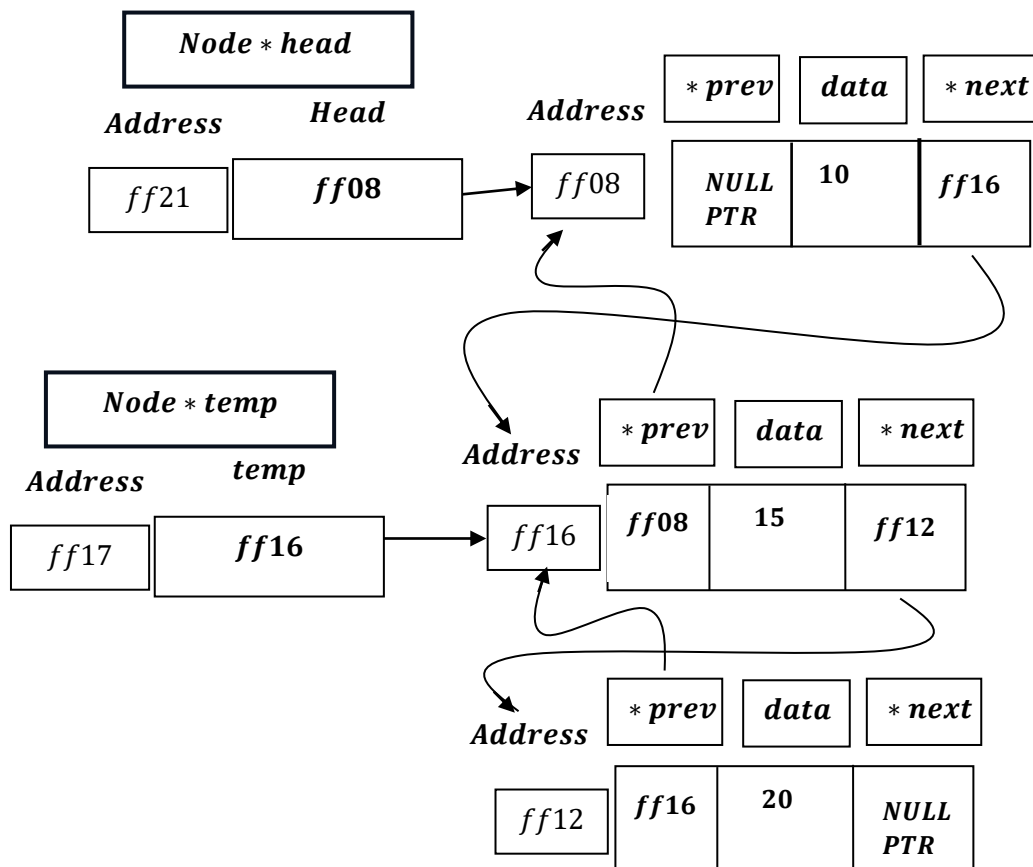
Lets input for the worst case, lets enter data = 25 after, Element = 20, hence condition is : $temp \rightarrow data \neq element$ i.e. $temp \rightarrow data \neq 20$, which is true and $temp \neq nullptr$ i.e. temp will point to the last node , i.e.

current n^{th} node.

```
while (temp != nullptr && temp->data != element)
{
    temp = temp->next;
}
```

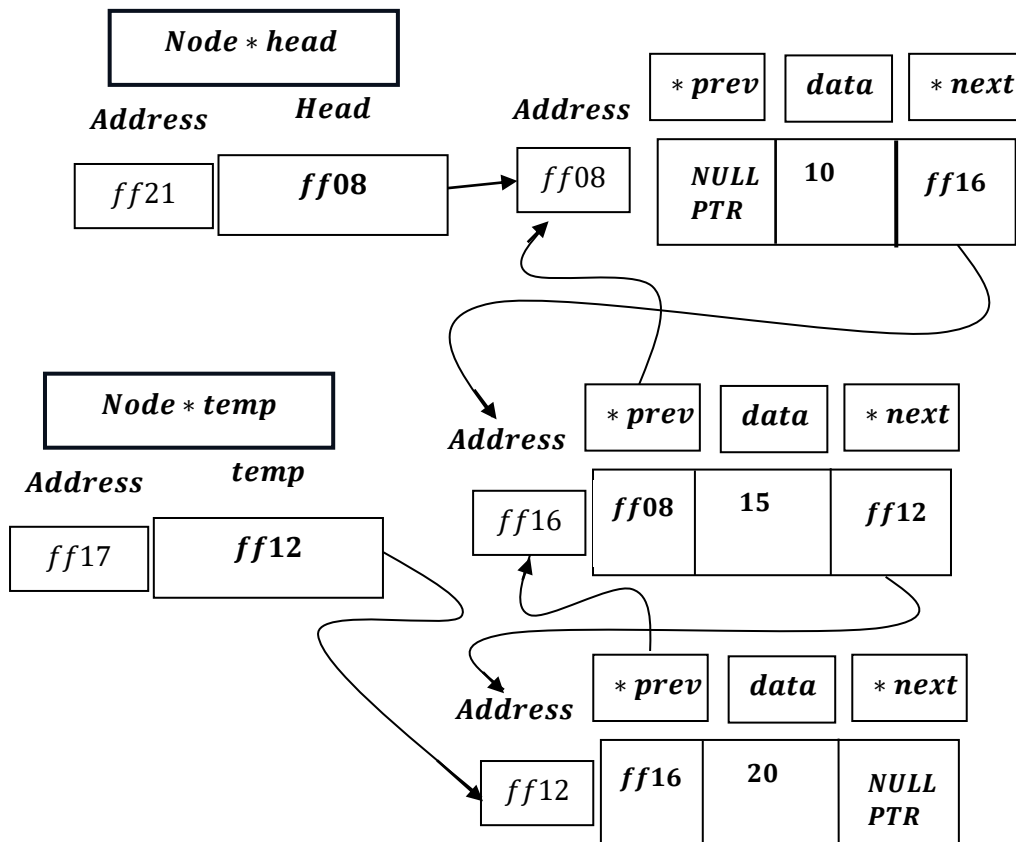
CONDITION: $TEMP \neq NULLPTR$ AND $TEMP \rightarrow DATA \neq ELEMENT \Rightarrow$

$TEMP \rightarrow DATA = 10 \neq ELEMENT = 20$, THEN , $TEMP = TEMP \rightarrow NEXT = ff16$.



CONDITION: $TEMP \neq NULLPTR$ AND $TEMP \rightarrow DATA \neq ELEMENT \Rightarrow$

$TEMP \rightarrow DATA = 15 \neq ELEMENT = 20$, THEN , $TEMP = TEMP \rightarrow NEXT = ff12$.



CONDITION: $TEMP \neq NULLPTR$ AND $TEMP \rightarrow DATA \neq ELEMENT \Rightarrow$
 $TEMP \rightarrow DATA = 20 \neq ELEMENT = 20$, Condition becomes false , hence ,
we get out of loop.

Next , it will check $temp = nullptr$, which is false , it exit if {...} block.

```
if (temp == nullptr){....}
```

As $temp = nullptr$, it will print , the user input Element is not found in the list.

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of *newNode* will be equal to the size of a Node structure and *newNode* is declared as a pointer to Node structure.

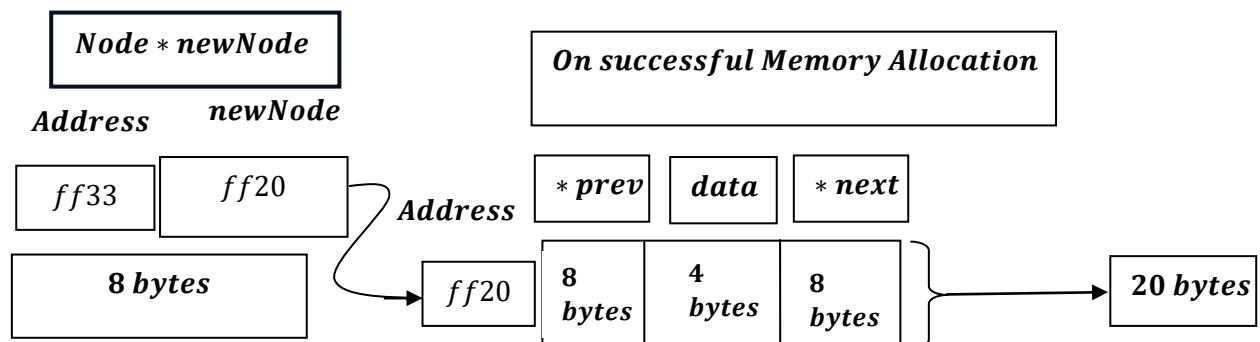
→ The size of a pointer (*newNode*) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ The size of the Node structure itself is determined by the sum of the sizes of its members:

- *data*(an integer) is typically 4 bytes.
- *next*(a pointer to another Node) is typically 8 bytes.
- *prev*(a pointer to previous Node) is typically 8 bytes.

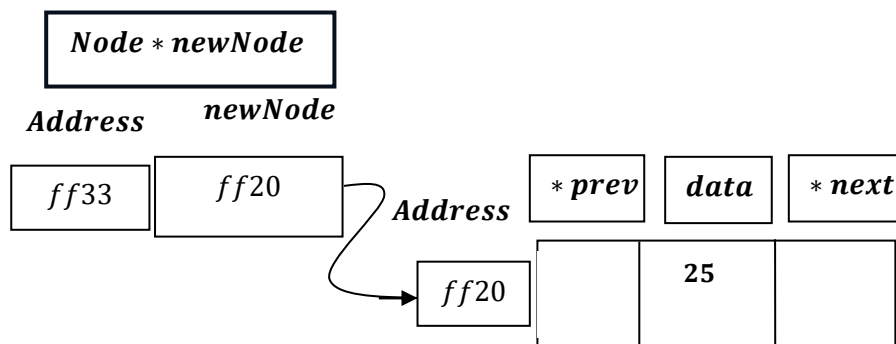
→ Therefore , the size of the Node structure is typically 20 bytes(4 bytes for data + 8 bytes for next + 8 bytes for previous) .

Say,



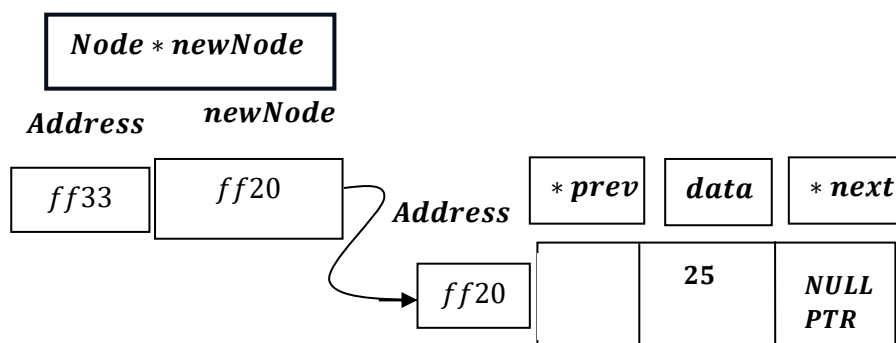
```
newNode->data = data;
```

newNode → *data* = 25,



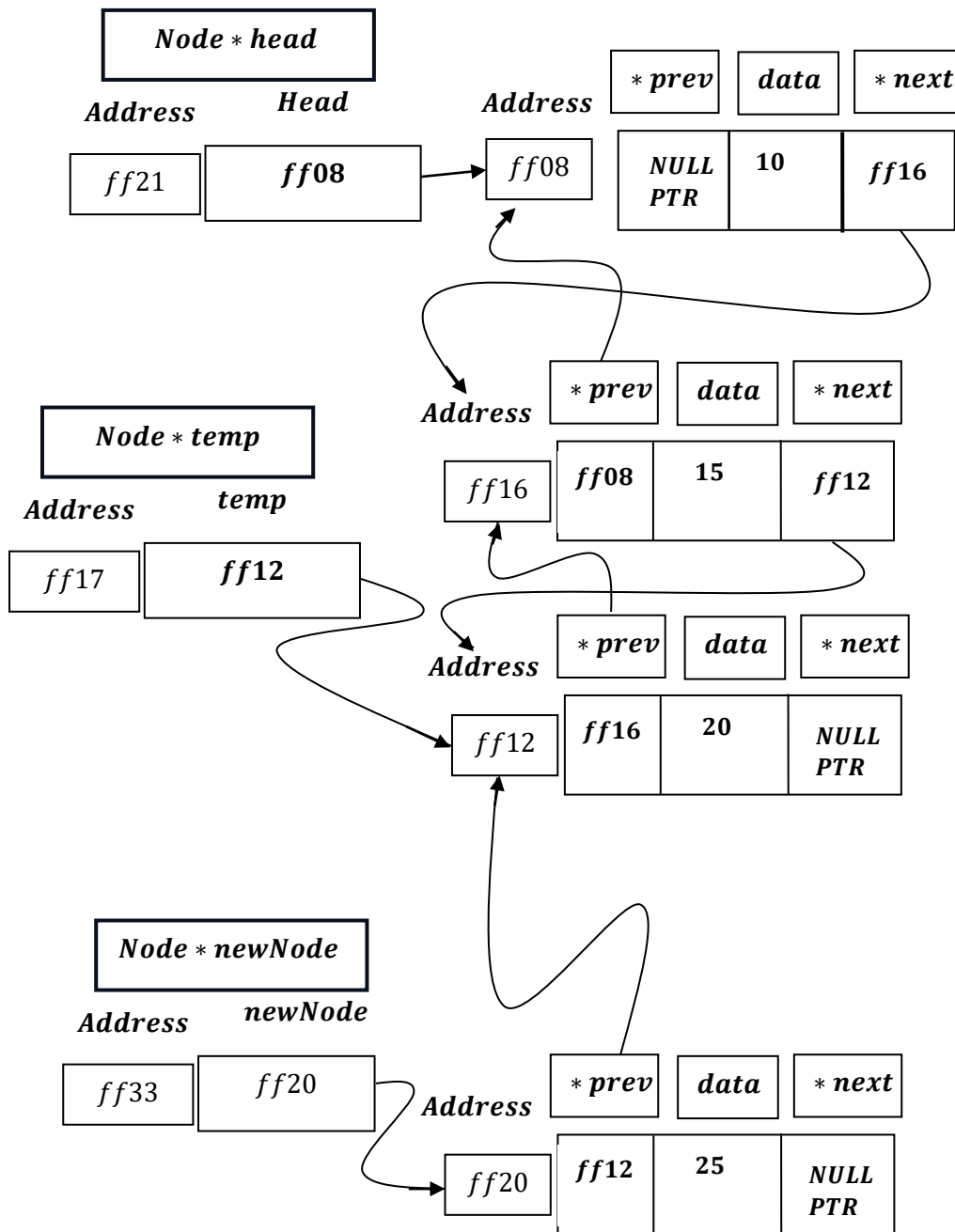
```
newNode->next = temp->next;
```

$newNode \rightarrow next = temp \rightarrow next = nullptr.$



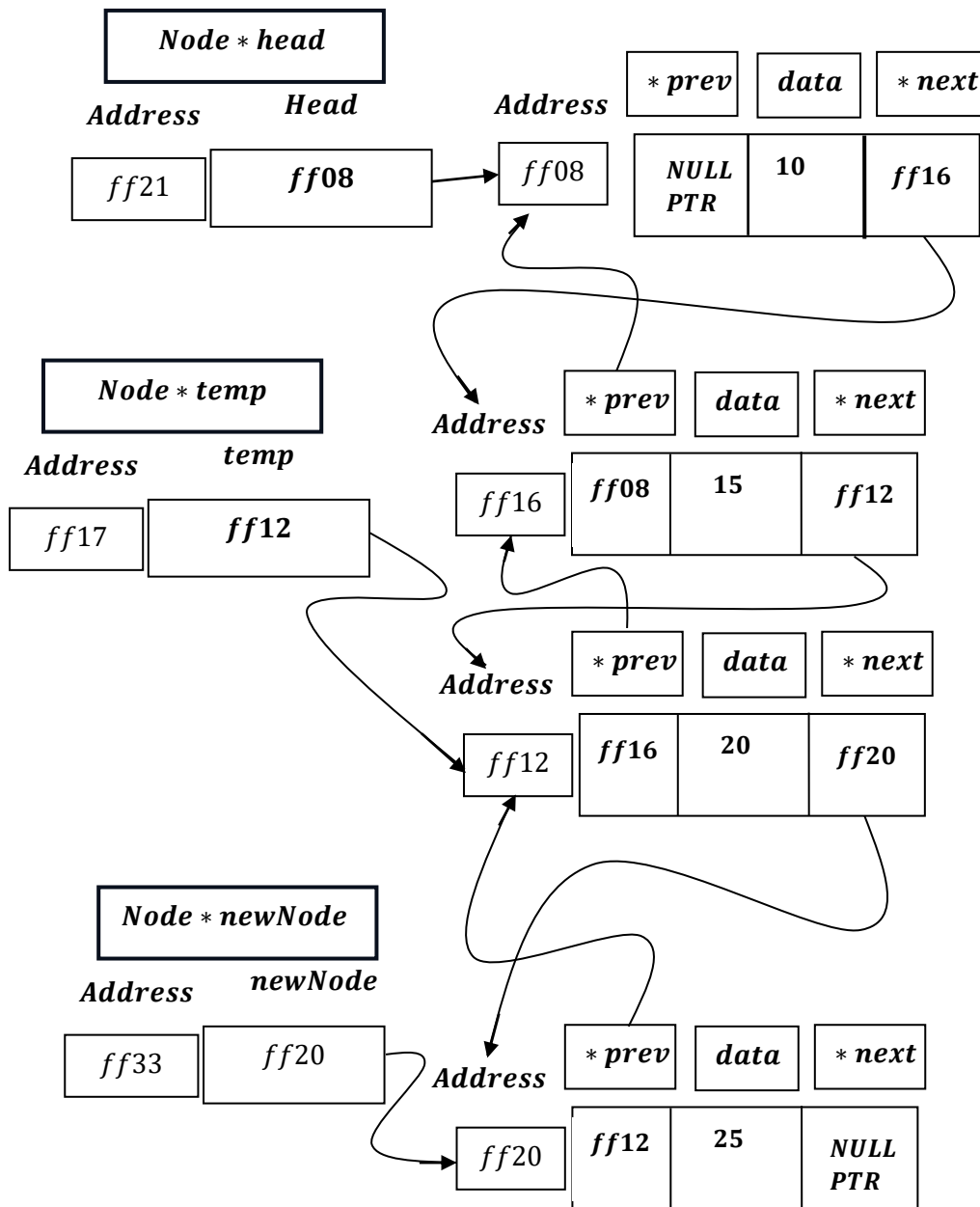
```
newNode->prev = temp;
```

$newNode \rightarrow prev = temp = ff12$

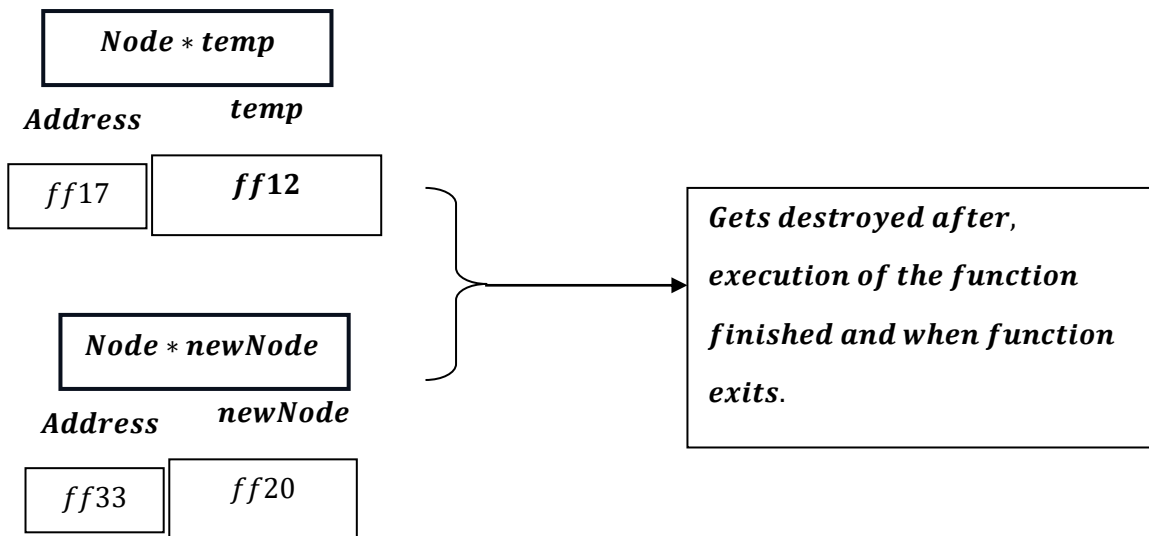


```
temp->next = newNode;
```

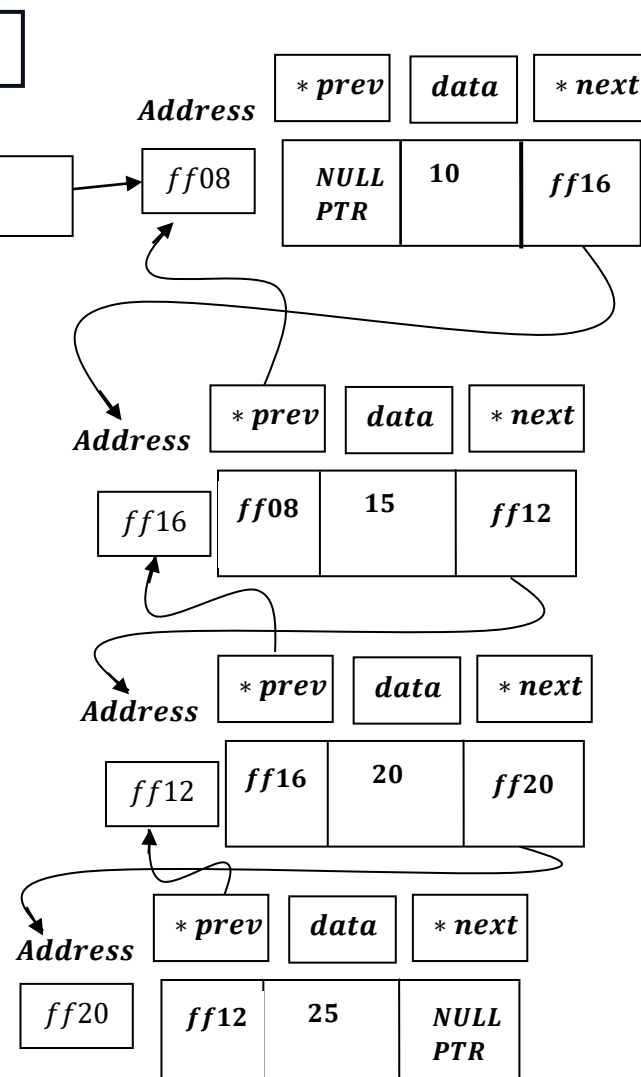
$temp \rightarrow next = newNode = ff20.$



Now, **Node * newNode** , **Node * temp** temporary variables, therefore gets destroyed after function execution get finished .



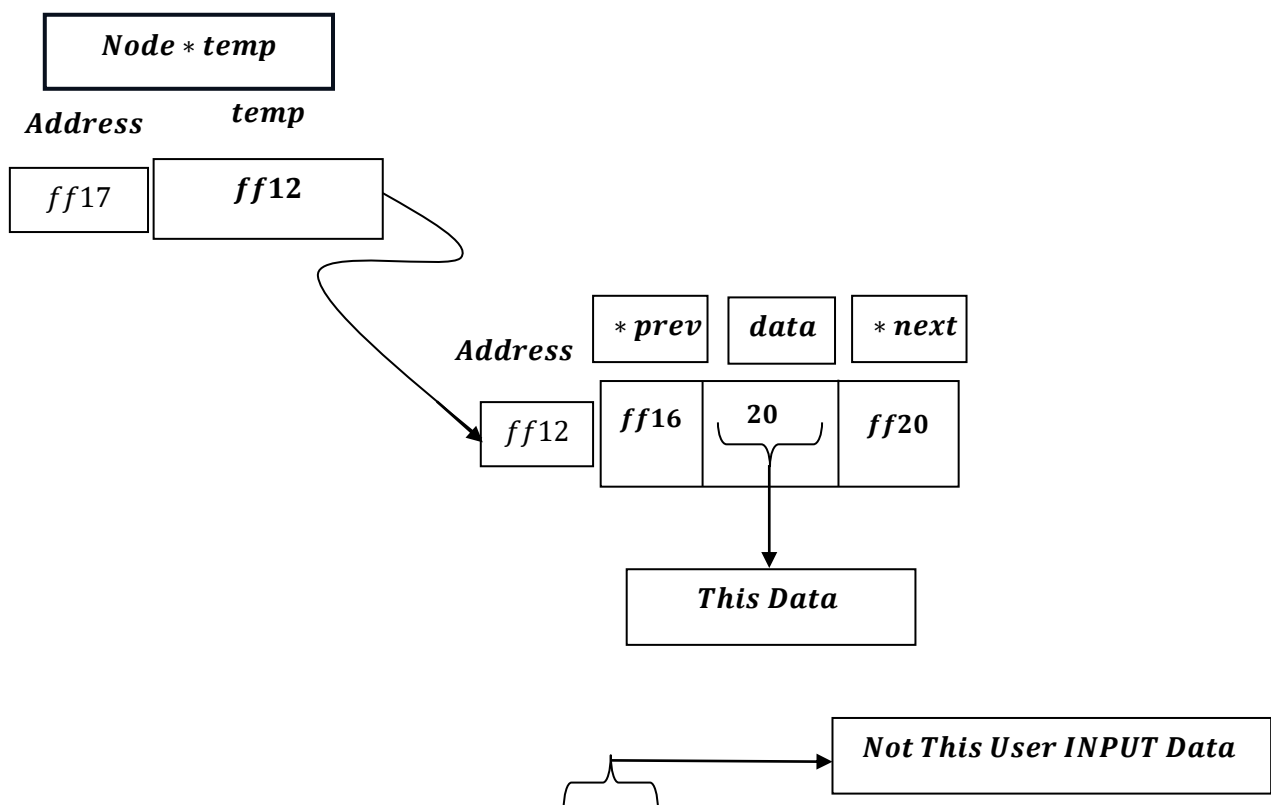
Hence , we are left with :



Temp = NULLPTR or Temp ≠ NULLPTR and
TEMP → DATA = ELEMENT or TEMP → DATA ≠ ELEMENT

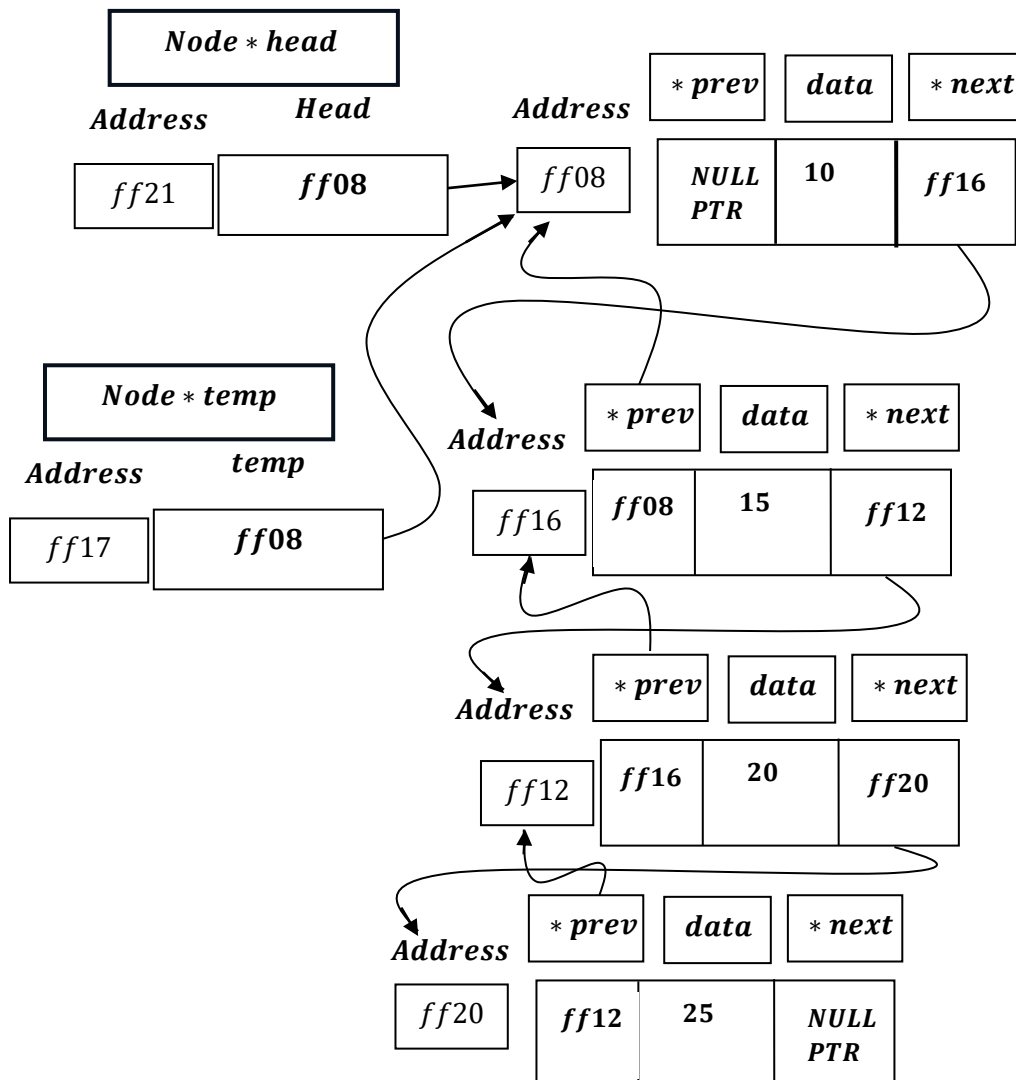
```
while (temp != nullptr && temp->data != element)
{
    temp = temp->next;
}
```

TEMP → DATA is not the USER INPUT DATA but TEMP pointed DATA already stored in NODE.



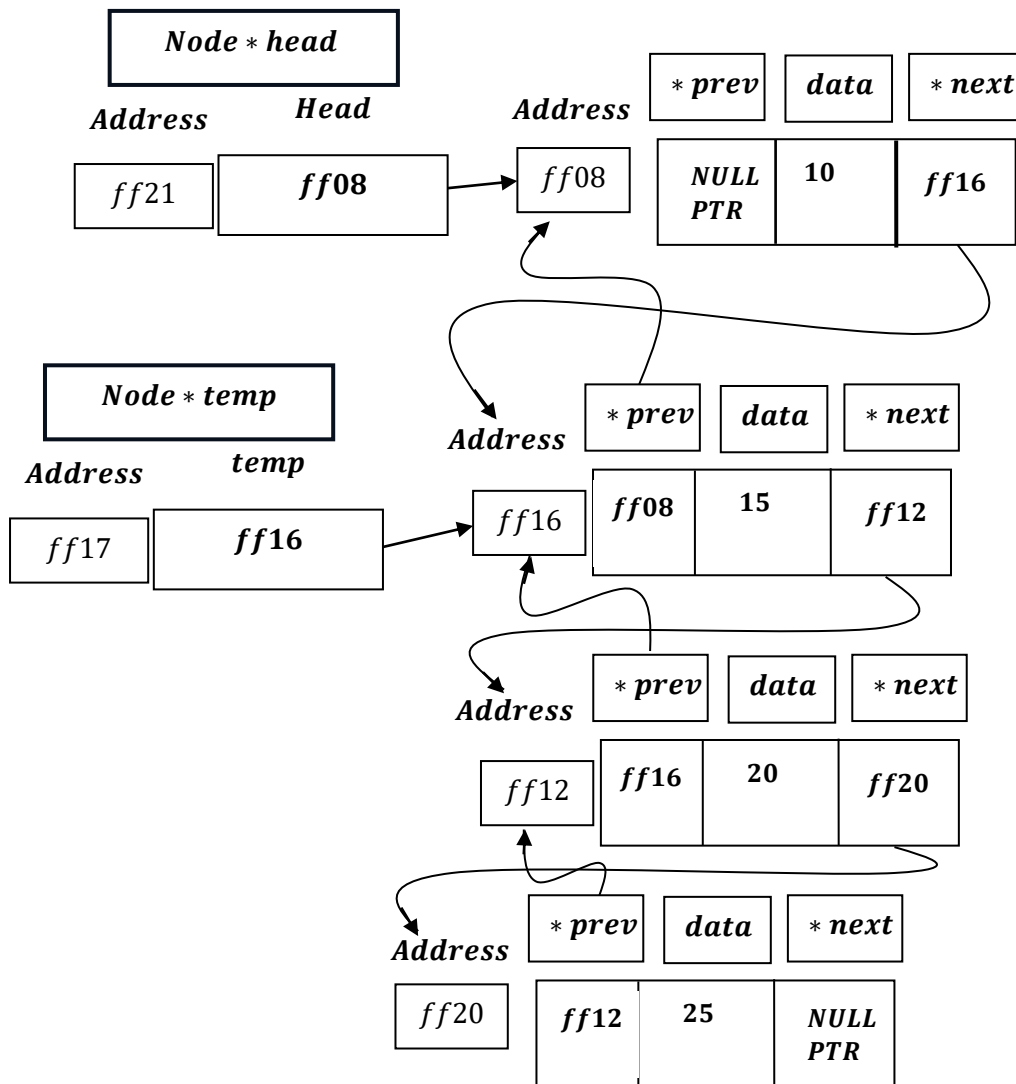
```
void insertAfterElement(int data, int element)
{.....}
```

Now say, we have a list:



And we want to enter data : 23 after element : 20.

*Now ,user input `element : 20` ,and $TEMP \rightarrow DATA \neq Element \neq 20$,hence ,
 $TEMP \rightarrow DATA$ will be 15 and $TEMP = TEMP \rightarrow NEXT = ff16$ and according to
condition of the loop: $TEMP \neq NULLPTR$ is true.*



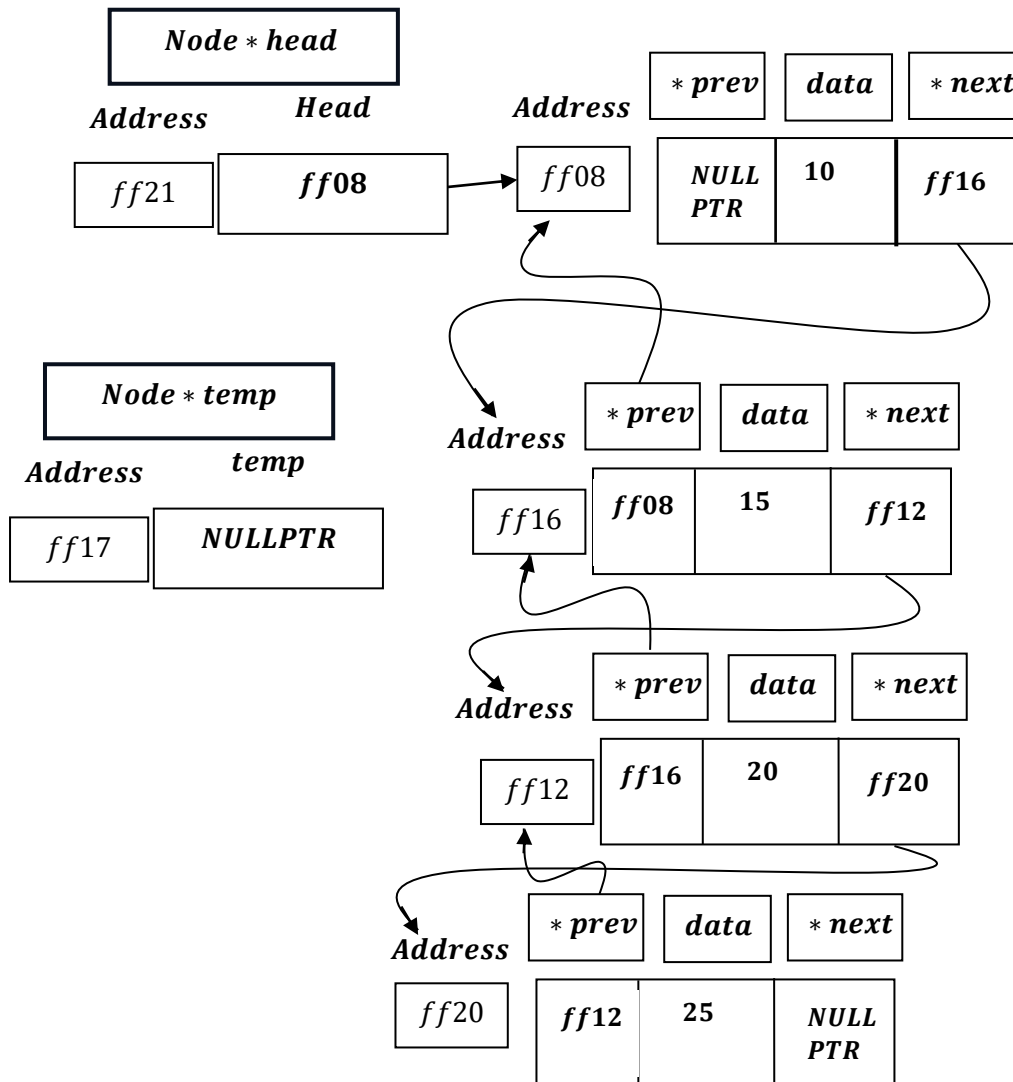
Hence , it is proved , that till `TEMP → DATA = ELEMENT` , TEMP will not be assigned to NULLPTR.

Conditions that will make TEMP pointer will point to NULLPTR :

→ HEAD points to NULLPTR.

→ ELEMENT input by USER is different and not present in any DATA section of NODES of the LIST.

Such we want to enter data : 26 after element : 24 ,but 24 is not present in the LIST.



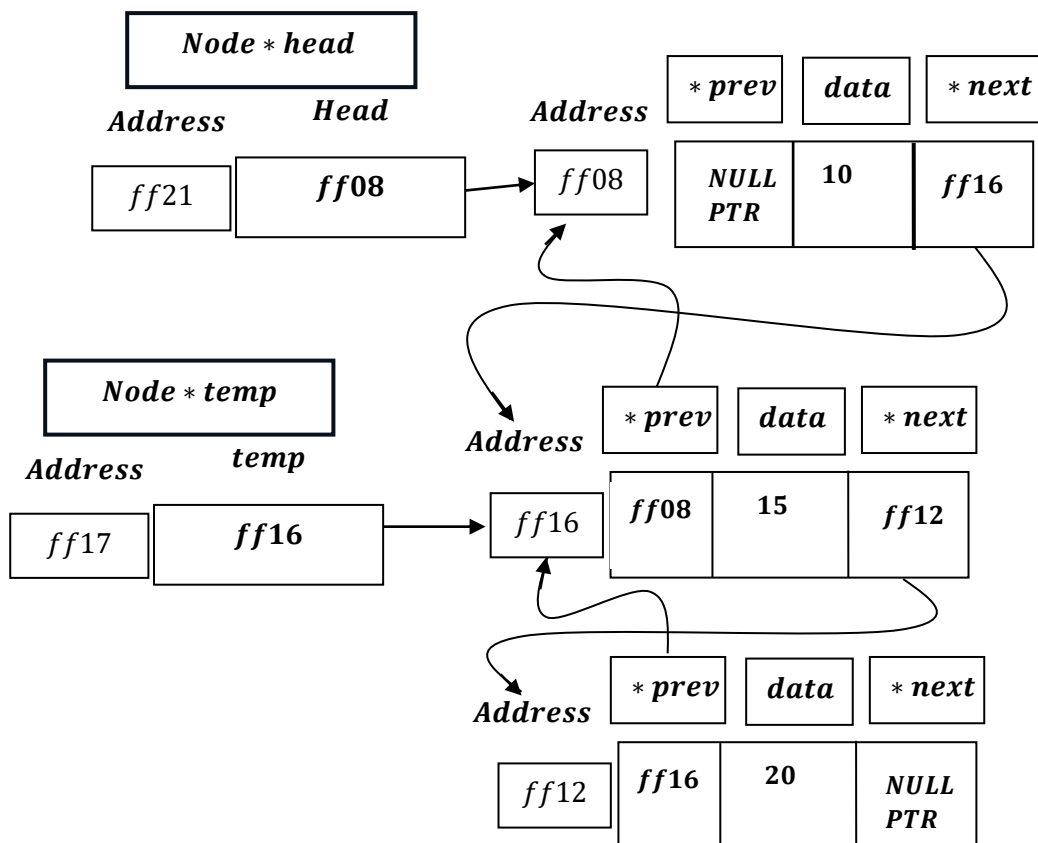
Hence,

```
if (temp == nullptr)
{
    cout << "Element " << element << " not found in the
list." << endl;
    return;
}
```

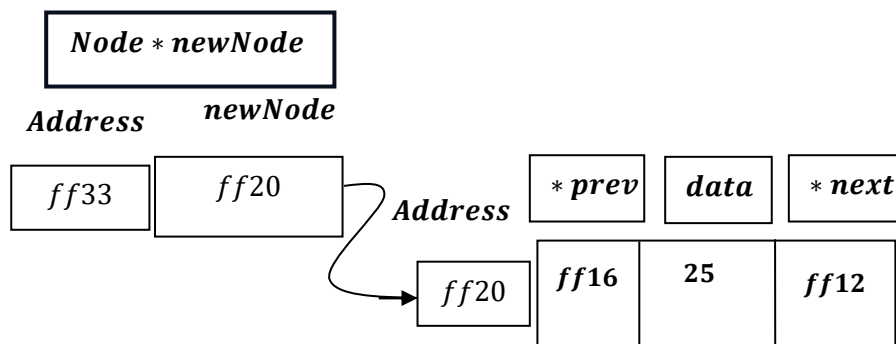

It prints output Element 24 not found in the list. And return statement ends the function or exists from the function.

```
if (newNode->next != nullptr)
{
    newNode->next->prev = newNode;
}
```

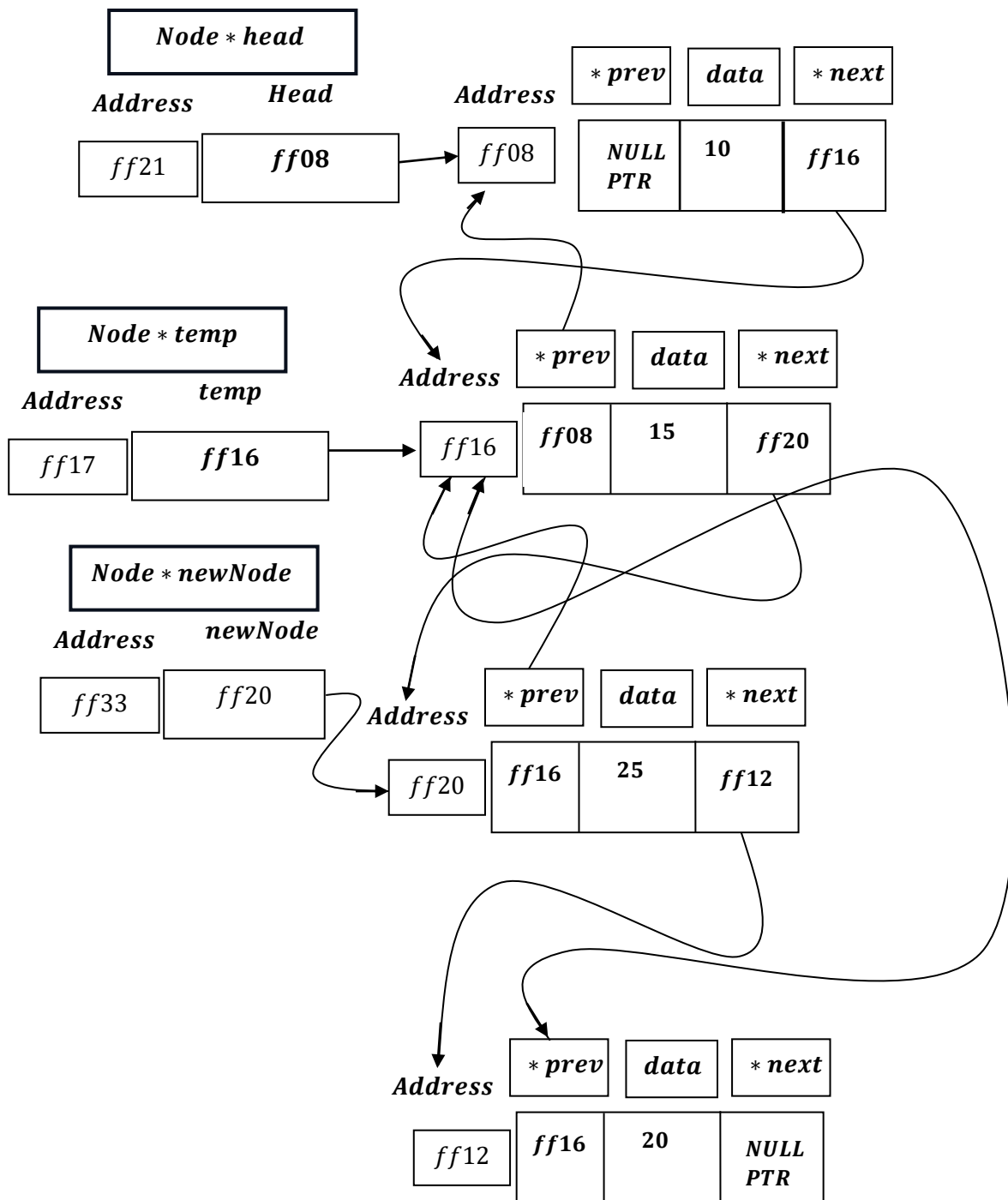
Lets we have a List:



Next we have newNode:

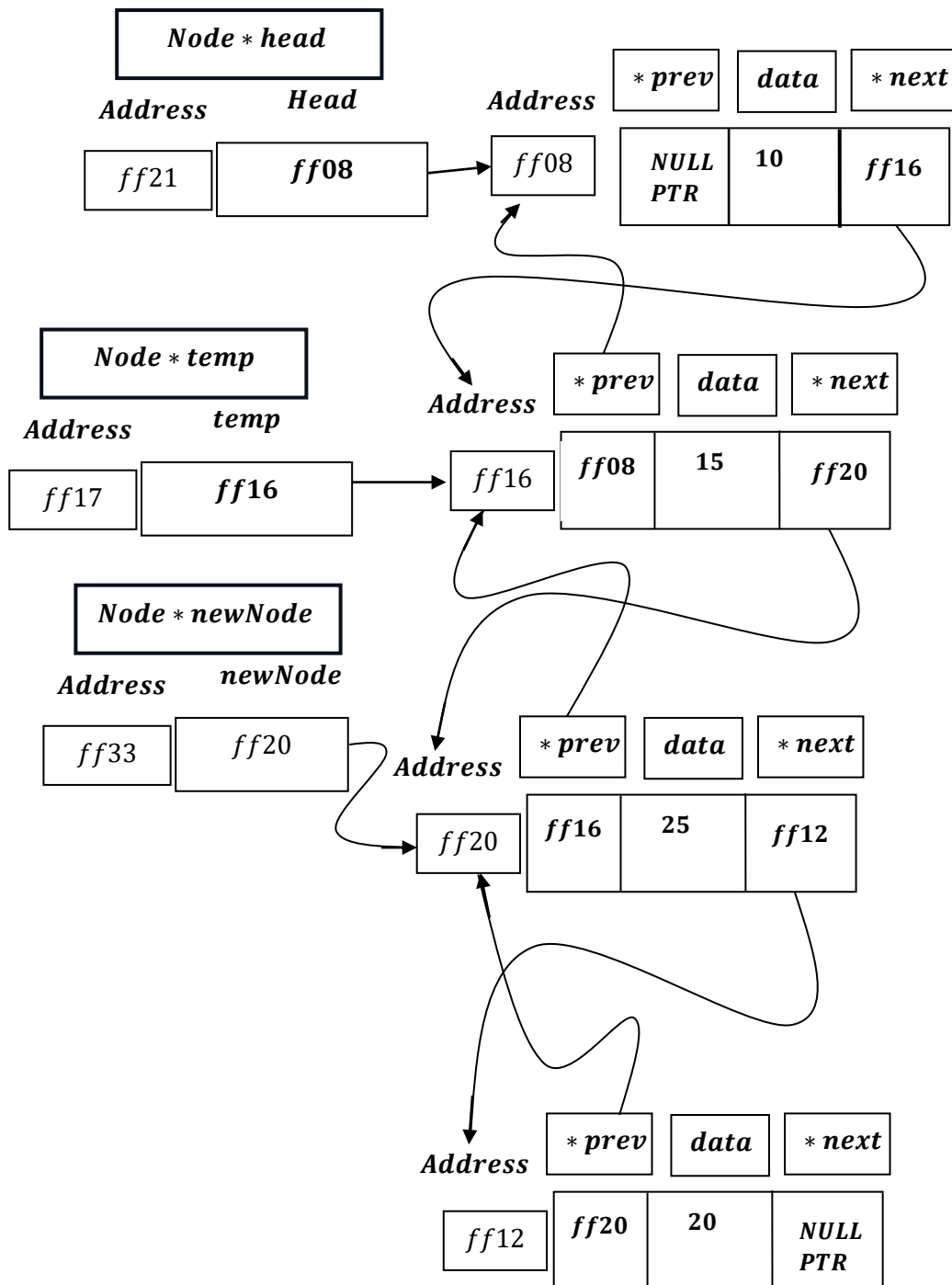


*Therefore ,adjusting newNode in the List,making $TEMP \rightarrow NEXT = newNode$
 $= ff20$.*

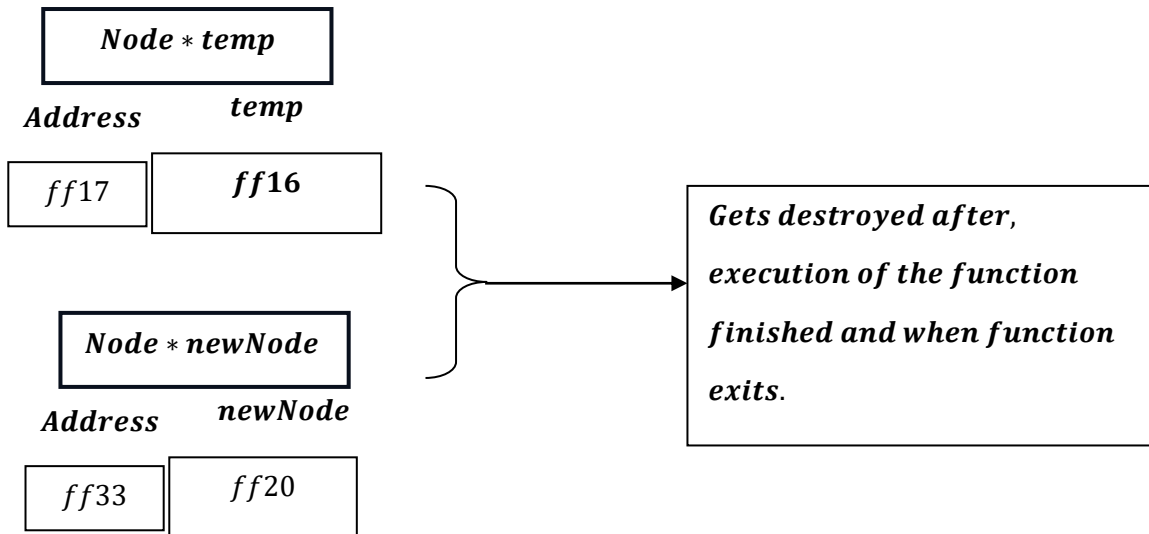


Now, we need, $\text{newNode} \rightarrow \text{next} \rightarrow \text{prev} = \text{newNode} = \text{ff20}$.

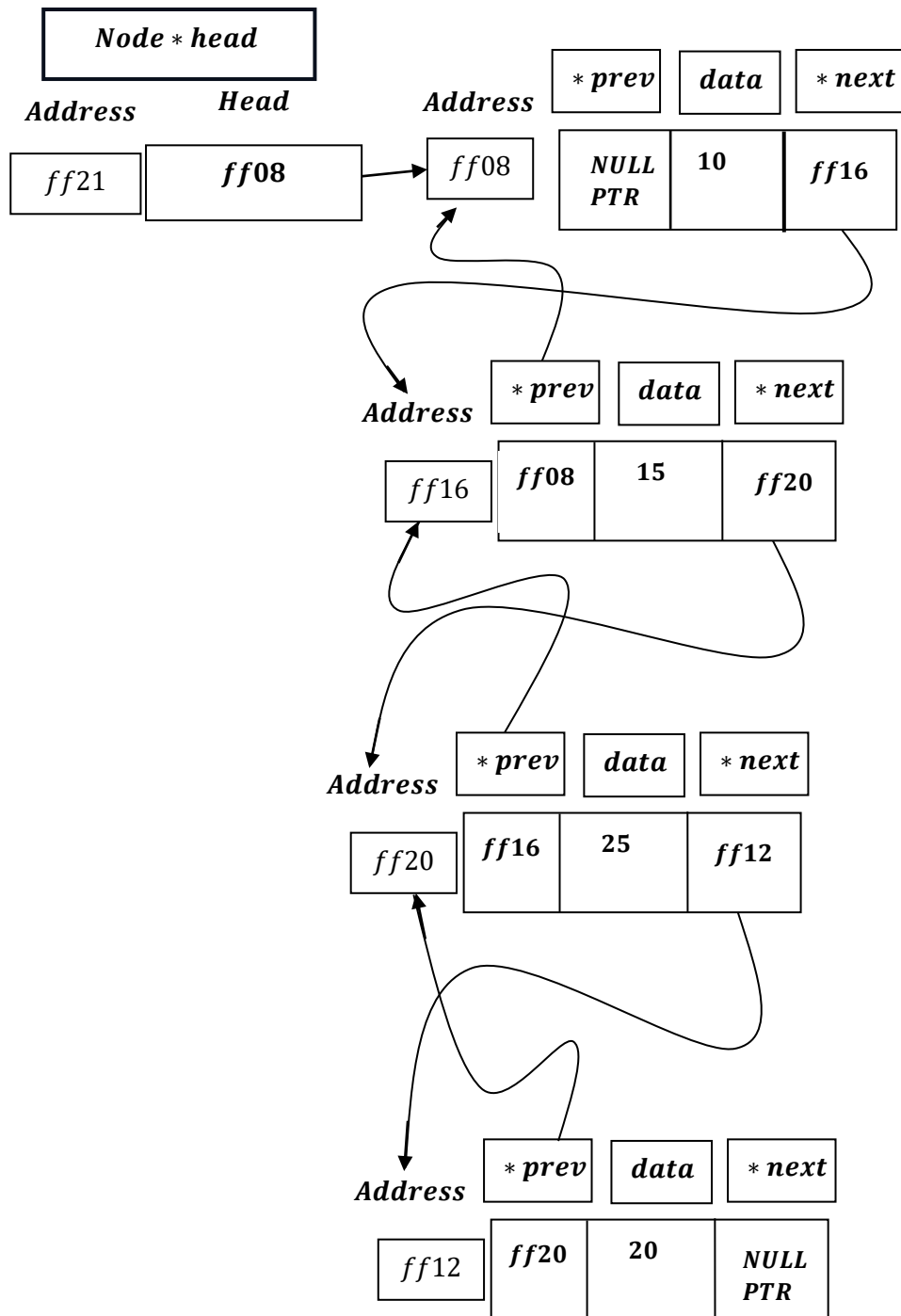
```
if (newNode->next != nullptr)
{
    newNode->next->prev = newNode;
}
```



Now, **Node * newNode** , **Node * temp** temporary variables, therefore gets destroyed after function execution get finished .



Hence ,we are left with:



Doubly Linked List - Insert After Element [Working Summary]:

This function inserts a new node with a specified `data` value immediately after the first node containing a target `element` in a doubly linked list.

Working Summary

1. **Search for the Element:** The function starts at the `head` of the list and traverses it using a temporary pointer, `temp`. It continues moving `temp` to the next node until it either finds the node where `temp->data` matches the `element` or it reaches the end of the list (`temp` becomes `nullptr`).
2. **Handle "Not Found" Case:** If the loop finishes because `temp` is `nullptr`, it means the `element` was not in the list. The function prints an error message and stops.
3. **Create the New Node:** If the `element` is found, the function allocates memory for a `newNode` and sets its `data` field to the new `data` value.
4. **Perform the Insertion:** The core logic involves re-wiring the pointers to fit the `newNode` into the list. This is a four-step process:
 - `newNode->next = temp->next;` — The new node's `next` pointer is set to point to the node that was originally after `temp`.
 - `newNode->prev = temp;` — The new node's `prev` pointer is set to point back to `temp`.
 - `temp->next = newNode;` — The `temp` node's `next` pointer is updated to point to the new node.
 - `newNode->next->prev = newNode;` — The `prev` pointer of the node *following* the new node is updated to point back to the `newNode`. This step is crucial and is only done if the new node is not being inserted at the very end of the list.

After these steps, the `newNode` is correctly spliced into the list, and a success message is printed.

Time Complexity of Insert After Element Function Module

```
void insertAfterElement(int data, int element)
{
    Node *temp = head;  $\rightarrow O(1)$ 

    while (temp != nullptr && temp->data != element)
    {
        temp = temp->next;  $\rightarrow O(n-1) = O(n)$ 
    }

    if (temp == nullptr)
    {
        cout << "Element " << element << " not found in the list." << endl;
        return;  $\rightarrow O(1)$ 
    }

    Node *newNode = (Node *)malloc(sizeof(Node));  $\rightarrow O(1)$ 
    newNode->data = data;  $\rightarrow O(1)$ 
    newNode->next = temp->next;  $\rightarrow O(1)$ 
    newNode->prev = temp;  $\rightarrow O(1)$ 
    temp->next = newNode;  $\rightarrow O(1)$ 

    if (newNode->next != nullptr)
    {
        newNode->next->prev = newNode;  $\rightarrow O(1)$ 
    }
    cout << data << " inserted after " << element << "." << endl;  $\rightarrow O(1)$ 
}
```


There are three case scenarios:

→ 1. Data should be inserted after the first element in the List . (Best Case)

→ 2. Position is $N + 1$, i. e. we want to insert data after n^{th} Element . (Worst Case)

→ 3. Inserted position from 2 to $N + 1$, for loop runs from 1 to $N - 1$.

While if Element is entered after first element, complexity is : $O(1)$.

And when the position is uniformly distributed from 1 to $(N + 1)$.

(Average Case i. e. Undetermined Position). That is we donot whether the position will be 1 or 2 or $N + 1$, where Node is to be inserted.

Best Case[Considered Absolute Best Case]

1. We create a Temp pointer of Node structure assigned to address stored as value in Head pointer of Node structure which implies Temp pointer will now point to address pointed by Head pointer , this process takes constant amount of time $O(1)$.

2. Now we come to loop condition , temp points to head's address , hence temp is not `nullptr` until and unless head points to nullptr but as data to be input after the first element , hence $temp \rightarrow data = element$, therefore according to condition $temp \rightarrow data \neq element$ fails , hence loop doesn't runs.

The condition verification process takes constant amount of time $O(1)$.

3. As `if condition  $temp = NULLPTR$ ` fails , hence if block doesn't execute therefore , the condition verification process takes constant amount of time $O(1)$.

4. newNode pointer of Node named structure points and allocates a memory for the Node structure , allocating for prev pointer , integer data, and next pointer , this pointing as well as allocating through `malloc` takes constant amount of time i. e. , $O(1)$.

5. integer data in the memory allocated pointed by newNode pointer of Node structure set to data by user input , takes constant amount of time i.e., $O(1)$.

6. next pointer of allocated memory pointed by newNode pointer of Node structure set to the value(address) obtained by next pointer of allocated memory pointed by temp pointer of Node structure and it makes next pointer pointed by newNode to point to that address , which is pointed by next pointer pointed by temp pointer , the process takes constant amount of time i.e., $O(1)$.

7. prev pointer of allocated memory pointed by newNode pointer of Node structure set to the value(address) obtained by temp pointer of Node structure and it makes prev pointer pointed by newNode pointer point to that address , which is pointed by temp pointer , the process takes constant amount of time i.e., $O(1)$.

8. next pointer of allocated memory pointed by temp pointer of Node structure set to the value(address) obtained by newNode pointer of Node structure and it makes next pointer pointed by temp pointer point to that address , which is pointed by newNode pointer , the process takes constant amount of time i.e., $O(1)$.

9. Now , If condition, checks next pointer pointed by newNode pointer of Node structure not equal to `nullptr` , which is true then prev pointer pointed by next pointer pointed by newNode pointer of Node structure set to the value i.e. address obtained by newNode pointer , making prev pointer pointed to address , which is being pointed by newNode. , the process takes constant amount of time i.e., $O(1)$.

10. Now we output a statement, user input data inserted at position , the process takes constant amount of time i.e., $O(1)$.

Hence , total time complexity it takes : $O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(1)$, i.e. constant time complexity.

2. Best Case[When Head points to `NULLPTR`]

1. We create a Temp pointer of Node structure assigned to address stored as value in Head pointer of Node structure which implies Temp pointer will now point to address pointed by Head pointer , this process takes constant amount of time $O(1)$.

2. Now we come to loop condition , temp points to head's address , hence temp is not `nullptr` until and unless head points to nullptr , here head points to `nullptr` , hence temp also points to `nullptr` , the loop condition fails $O(1)$.

3. As `if condition temp = NULLPTR` is true , hence if block executes and if block statements runs along with return statement exits the function , This process takes constant amount of time $O(1)$.

Hence , total time complexity it takes : $O(1) + O(1) + O(1) = O(1)$, i.e. constant time complexity.

3. When Element entered at Position $\Rightarrow N + 1$, Worst Case Scenario

1. We create a Temp pointer of Node structure assigned to address stored as value in Head pointer of Node structure which implies Temp pointer will now point to address pointed by Head pointer , this process takes constant amount of time $O(1)$.

2. Now we come to loop condition , temp points to head's address , hence temp is not `nullptr` until and unless head points to nullptr but as data to be input after the element at n^{th} node , hence $\text{temp} \rightarrow \text{data} = \text{element}$, therefore according to condition $\text{temp} \rightarrow \text{data} \neq \text{element}$ fails , when $\text{temp} = \text{temp} \rightarrow \text{next}$, the statement inside the loop gets excuted, makes temp pointer to point at n^{th} node .

Hence the inner statement of the loop runs $n - 1$ times as, it was already pointed at Head node or First node.

$$\therefore O(n - 1) = O(n).$$

3. As `if condition temp = NULLPTR` fails, hence if block doesn't execute therefore, the condition verification process takes constant amount of time $O(1)$.

4. newNode pointer of Node named structure points and allocates a memory for the Node structure, allocating for prev pointer, integer data, and next pointer, this pointing as well as allocating through `malloc` takes constant amount of time i.e., $O(1)$.

5. integer data in the memory allocated pointed by newNode pointer of Node structure set to data by user input, takes constant amount of time i.e., $O(1)$.

6. next pointer of allocated memory pointed by newNode pointer of Node structure set to the value(address) obtained by next pointer of allocated memory pointed by temp pointer of Node structure and it makes next pointer pointed by newNode to point to that address, which is pointed by next pointer pointed by temp pointer, the process takes constant amount of time i.e., $O(1)$.

7. prev pointer of allocated memory pointed by newNode pointer of Node structure set to the value(address) obtained by temp pointer of Node structure and it makes prev pointer pointed by newNode pointer point to that address, which is pointed by temp pointer, the process takes constant amount of time i.e., $O(1)$.

8. next pointer of allocated memory pointed by temp pointer of Node structure set to the value(address) obtained by newNode pointer of Node structure and it makes next pointer pointed by temp pointer point to that address, which is pointed by newNode pointer, the process takes constant amount of time i.e., $O(1)$.

9. Now ,If condition, checks next pointer pointed by newNode pointer of Node structure not equal to `nullptr` , which is true then prev pointer pointed by next pointer pointed by newNode pointer of Node structure set to the value i. e. address obtained by newNode pointer , making prev pointer pointed to address , which is being pointed by newNode. ,the process takes constant amount of time i. e., $O(1)$.

10. Now we output a statement, user input data inserted at position , the process takes constant amount of time i. e., $O(1)$.

Hence ,total time complexity it takes : $O(1) + O(n) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(n)$, i. e. constant time complexity.

4. Inserted position from 1 to $N + 1$, while loop runs from 1 to $N - 1$.

And when the position is uniformly distributed from 1 to $(N + 1)$:

(Average Case i. e. Undetermined Position)

→ If the Element is inserted after 1st node, the loop's inner statement runs 0 time.

→ If the Element is inserted after 2nd node, the loop's inner statement runs 1 time.

→ If the Element is inserted at 3rd node, the loop's inner statement runs 2 time.

→ If the Element is inserted at 4th node, the loop's inner statement runs 3 time.

.

.

→ If the Element is inserted at $(n + 1)^{th}$ node, the loop's inner statement runs $N - 1$ time.

As when $Temp \rightarrow Data = Element$, the condition becomes false , hence no execution of loop's inner statements.

$$\therefore, \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}$$

Now, as we take all the possibilities then, we take the best case also i.e.

$O(1)$, when Element entered after first node = 1.

Considering ' $T(P)$ ' is time complexity of position ' P '.

$$T(P) = \begin{cases} 1 & , \text{when } P = 1. \\ N - 1 & , \text{when } P = 2, 3, 4, \dots, N + 1. \end{cases}$$

Now, If there is ' N ' no. of nodes there we can insert data at $N + 1$ positions.

For position 1, the probability of insertion is $\frac{1}{N+1}$.

For position 2, the probability of insertion is $\frac{1}{N+1}$.

.

.

For position $N + 1$, the probability of insertion is $\frac{1}{N+1}$.

Hence probability of insertion will always remain $\frac{1}{N+1}$.

$\therefore$ Average Time Complexity is : probability of insertion $\times$ (number of possible iterations + constant time complexities for left over positions) .

$$= \left(\frac{1}{N+1} \times 1 + \frac{1}{N+1} \times 2 + \dots + \frac{1}{N+1} \times N - 1 \right) + \frac{1}{N+1} \times 1$$

$$= \frac{1}{N+1} \times \{(1+2+3+\dots+N-1)+1\}$$

$$= \frac{1}{N+1} \times \left\{ \sum_{i=1}^{N-1} i + 1 \right\}$$

$$= \frac{1}{N+1} \times \left(\frac{N(N-1)}{2} + 1 \right)$$

$$= \frac{N(N-1)}{2(N+1)} + \frac{1}{N+1}$$

$$= \frac{N^2 - N}{2N+2} + \frac{1}{N+1}$$

Applying Big Theta in the equation:

$$= \Theta\left(\frac{N^2 - N}{2N+2}\right) + \Theta\left(\frac{1}{N+1}\right)$$

$$= \Theta\left(\frac{N^2 - N}{2N+2}\right) + \Theta\left(\frac{1}{N+1}\right)$$

Now, taking out the dominant term of numerator and denominator:

- ***In the numerator, $N^2 - N$, the term N^2 grows much faster than N .***

So, for large N , the dominant term is N^2 .

- ***In the denominator, $2N + 2$, the term $2N$ grows much faster***

than the constant 2. The dominant term is $2N$.

$$= \Theta\left(\frac{N^2}{2N}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

Now removing the constant $\frac{1}{2}$, we get:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

When combining two Big Theta Θ terms, largest growth rate is the result:

$\rightarrow \Theta(N)$ grows linearly with N .

$\rightarrow \Theta\left(\frac{1}{N}\right)$ decreases to a constant, i. e. when N grows larger, $\frac{1}{N}$ approaches to 0.

This we can prove by limit and continuity also:

$$\lim_{n \rightarrow \infty} \left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0 \left[\frac{c}{\infty} = 0, \text{ where } c \text{ is constant}\right].$$

As $\Theta(N)$'s growth rate is higher, therefore, average case is $\Theta(N)$.

Hence, Average Time Complexity is : $\Theta(N)$.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N(N-1)}{2(N+1)} + \frac{1}{N+1} \leq c_2 \times N$$

Average Case Time Complexity[Most Simplest Alternative Approach]:

(Simplest approach)

**(If the elements in the linked list are uniformly distributed
and the key is equally likely to be any element in the list)**

**Note: –The checking that Element or Key is found or not , done in the
condition of while loop, and the condition checks `n` times .**

–if key is found at 1st Node , Key/Element searched for 1 time .

–if key is found at 2nd Node , Key/Element searched for 2 time .

–if key is found at 3rd Node, Key/Element searched for 3 time .

.

.

–if key is found at Nth Node , Key/Element searched for N time .

As , the key is equally likely to be any element in the list,

Probability of finding 1st element (p_1), if key equals to 1st element = $\frac{1}{N}$.

Probability of finding 2nd element (p_2), if key equals to 2nd element = $\frac{1}{N}$.

.

.

Probability of finding N^{th} element (p_2), if key equals to N^{th} element $= \frac{1}{N}$.

Therefore,

$$\text{Probability}(P) = p_1 = p_2 = \dots = p_n = \frac{1}{N}.$$

Therefore, Average Case = Number of times the Key is searched $\times$ probability of finding elements.

That is the average number of executions is the sum of all possible executions weighted by their probabilities. This is given by:

$$\begin{aligned}\therefore \text{Average Case} &= \frac{1}{N} \times \left(\sum_{i=1}^N i \right) \\ &= \frac{1}{N} \times \frac{N(N+1)}{2} \left[\sum_{i=1}^N i = \frac{N(N+1)}{2} \right] \\ &= \frac{N+1}{2} \\ &= \frac{N}{2} + \frac{1}{2}\end{aligned}$$

Applying , Big Theta Θ we get :

$$\begin{aligned}&= \frac{1}{2} \times (\Theta(N) + \Theta(1)) \\ &= \Theta(N) + \Theta(1)\end{aligned}$$

$$= \Theta(N).$$

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N$$

<i>Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Insert After Element</i>	<i>1. Best Case</i>	<i>$\Omega(1)$</i>
	<i>2. Worst Case</i>	<i>$O(n)$</i>
	<i>3. Average Case</i>	<i>$\Theta(n)$</i>

***Doubly Linked List – Insert After Element
–Space Complexity and Algorithm***

Program :

```
void insertAfterElement(int data, int element)
{
    Node *temp = head;
    while (temp != nullptr && temp->data != element)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Element " << element << " not found in the
list." << endl;
        return;
    }

    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;

    newNode->next = temp->next;
    newNode->prev = temp;
    temp->next = newNode;
    if (newNode->next != nullptr)
    {
        newNode->next->prev = newNode;
    }
    cout << data << " inserted after " << element << "." <<
endl;
}

..... •
```

```

case 4:
    cout << "Enter data to insert: ";
    cin >> data;
    cout << "Enter the element to insert after: ";
    cin >> element;
    insertAfterElement(data, element);
    break;

```

Space Complexity

Auxiliary Space

Auxiliary space is the extra or temporary space used by an algorithm, excluding the space taken up by the inputs.

In the `insertAfterElement` function, the memory usage is:

Node *temp: A single temporary pointer is created to find the element. The size of a pointer is constant.

Node *newNode: One new node is allocated to hold the new data. The size of this node (`sizeof(Node)`) is also constant.

Since the amount of extra memory used does not change with the size of the Linked List, the auxiliary space complexity is $O(1)$. The while loop affects the function's time complexity but not its space complexity.

Auxiliary Space: $O(1)$

Input Space

Input space is the memory used to store the function's inputs. This can be analyzed from two perspectives.

1. Parameter-only Definition

This view considers only the space taken by the function's direct parameters: `void insertAfterElement(int data, int element)`.

The inputs are two integers, `data` and `element`, each of which occupies a fixed, constant amount of memory.

Under this common definition, the input space is constant.

Input Space (Parameter-only): $O(1)$

2. Full-Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list itself.

If the Linked List contains n nodes, the total memory it occupies is proportional to n .

Under this definition, the input space is linear with respect to the size of the list.

Input Space (Full-Data): $O(n)$

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, when asked for the "space complexity" of a function, the expected answer is the auxiliary space complexity, which is $O(1)$.

Algorithm: Insert After Element (Doubly Linked List)

DINSAFTER(START, ITEM, ELEMENT)

*((Node *)malloc(sizeof(Node)) is availability of space for insertion of Data,
hence, let's name it AVAIL.*

1. *[TRAVERSE LIST TO FIND ELEMENT]*

Set PTR := START [PTR is a pointer initialized to the start of the list].

Repeat while PTR ≠ NULL AND INFO[PTR] ≠ ELEMENT:

Set PTR := NEXTLINK[PTR] [Updates PTR to the next node].

[END of Loop]

2. *[CHECK IF ELEMENT WAS FOUND]*

If PTR = NULL, then:

Write: "Element not found in the list." and EXIT.

[End of If structure.]

3. *[ALLOCATE NEW NODE]*

Set NEW := AVAIL.

4. *[COPY DATA INTO NEW NODE]*

Set INFO[NEW] := ITEM.

5. *[LINK THE NEW NODE INTO THE LIST]*

a. Set NEXTLINK[NEW] := NEXTLINK[PTR] [New node's NEXT points to what was after PTR].

b. Set PREVLINK[NEW] := PTR [New node's PREV points back to PTR].

c. Set NEXTLINK[PTR] := NEW [PTR now points forward to the NEW node].

d. If NEXTLINK[NEW] ≠ NULL, then: [This handles cases where the node is not inserted at the very end].

Set PREVLINK[NEXTLINK[NEW]] := NEW $\left(\begin{array}{l} \text{The node that was originally after PTR must} \\ \text{now point its PREV back to the NEW node} \end{array} \right)$.

[End of If structure]

6. *[OUTPUT THE INSERTED ITEM AFTER ELEMENT]*

Write: ITEM, "inserted after", ELEMENT.

7. *Exit*

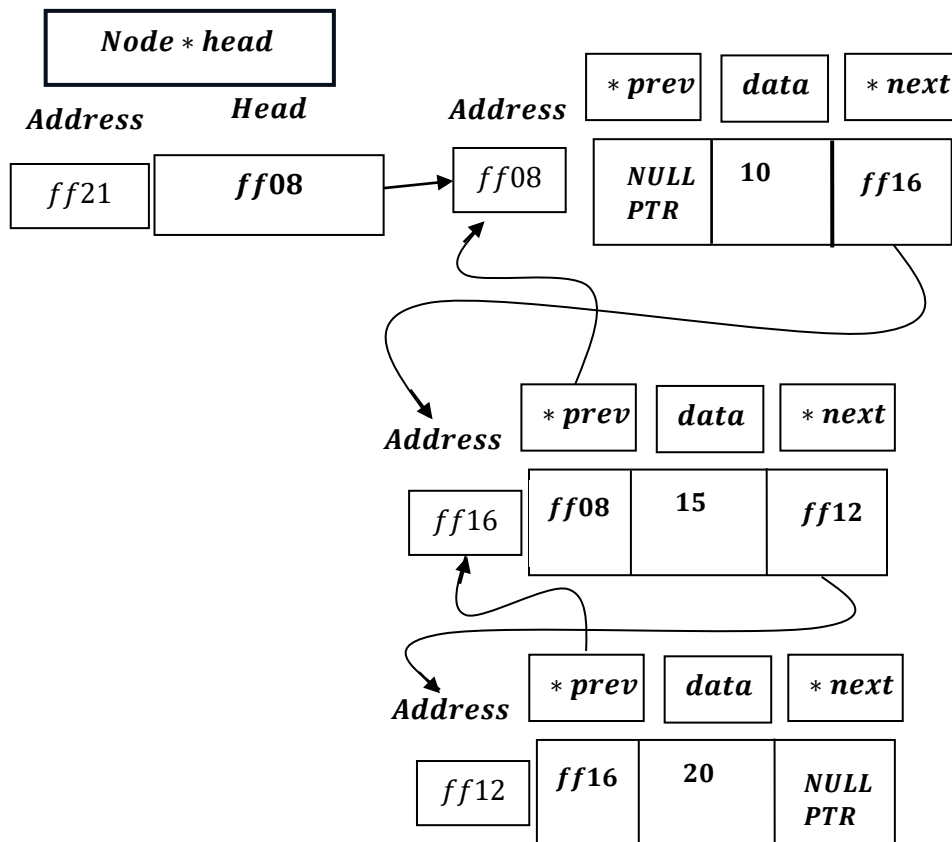
Doubly Linked List – Delete From Beginning And Time Complexity.

Program :

```
void deleteFromBeginning()
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
    Node *temp = head;
    head = head->next;
    if (head != nullptr)
    {
        head->prev = nullptr;
    }
    cout << "Deleted " << temp->data << " from the
beginning." << endl;
    temp->next = nullptr;
    free(temp);
    temp = nullptr;
}
... .
case 5:
    deleteFromBeginning();
    break;
```

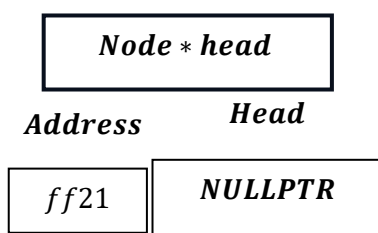
Program Analysis:

Lets consider a Doubly Linked List :



Now, first Line of Code:

```
if (head == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}
```

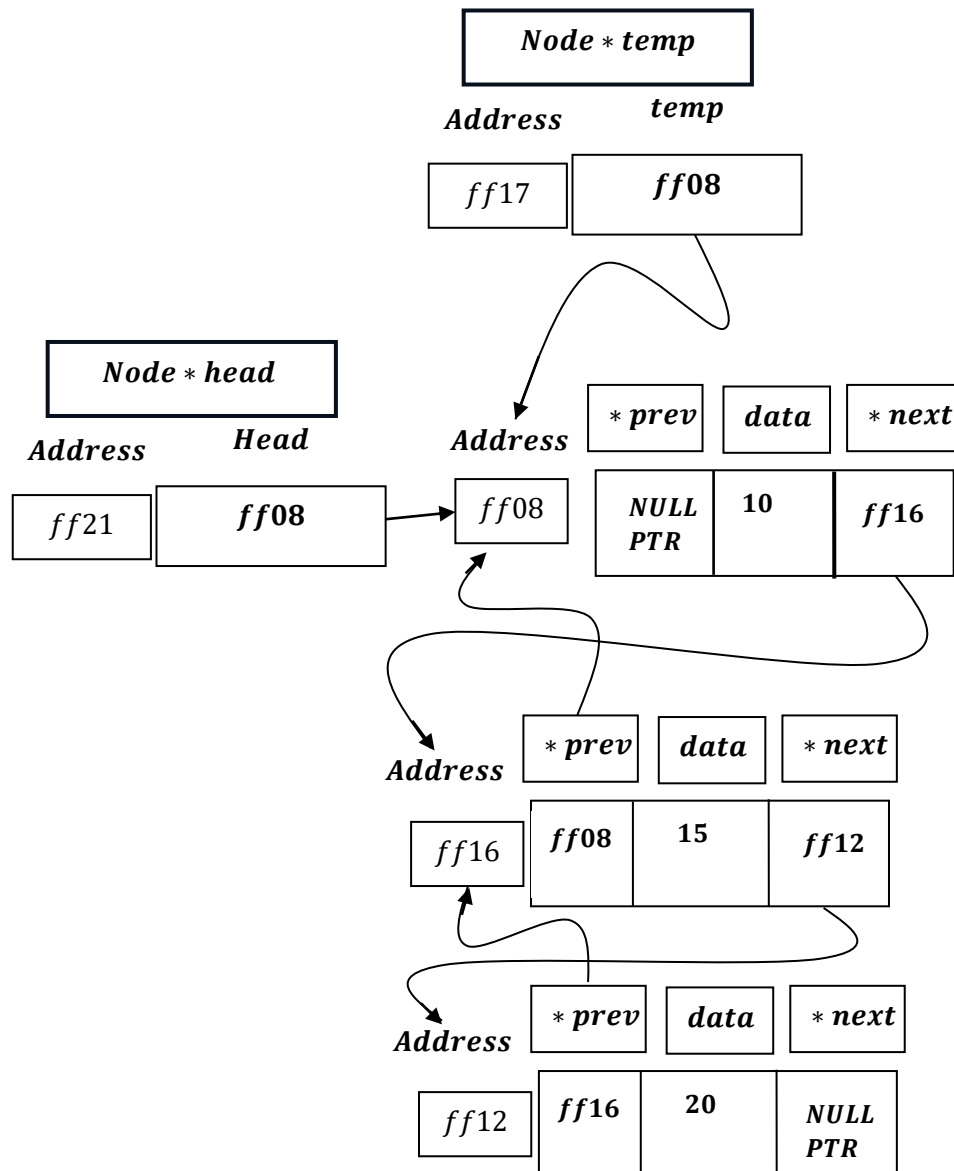


Then, it will print statement : `` List is empty. Nothing to delete.``

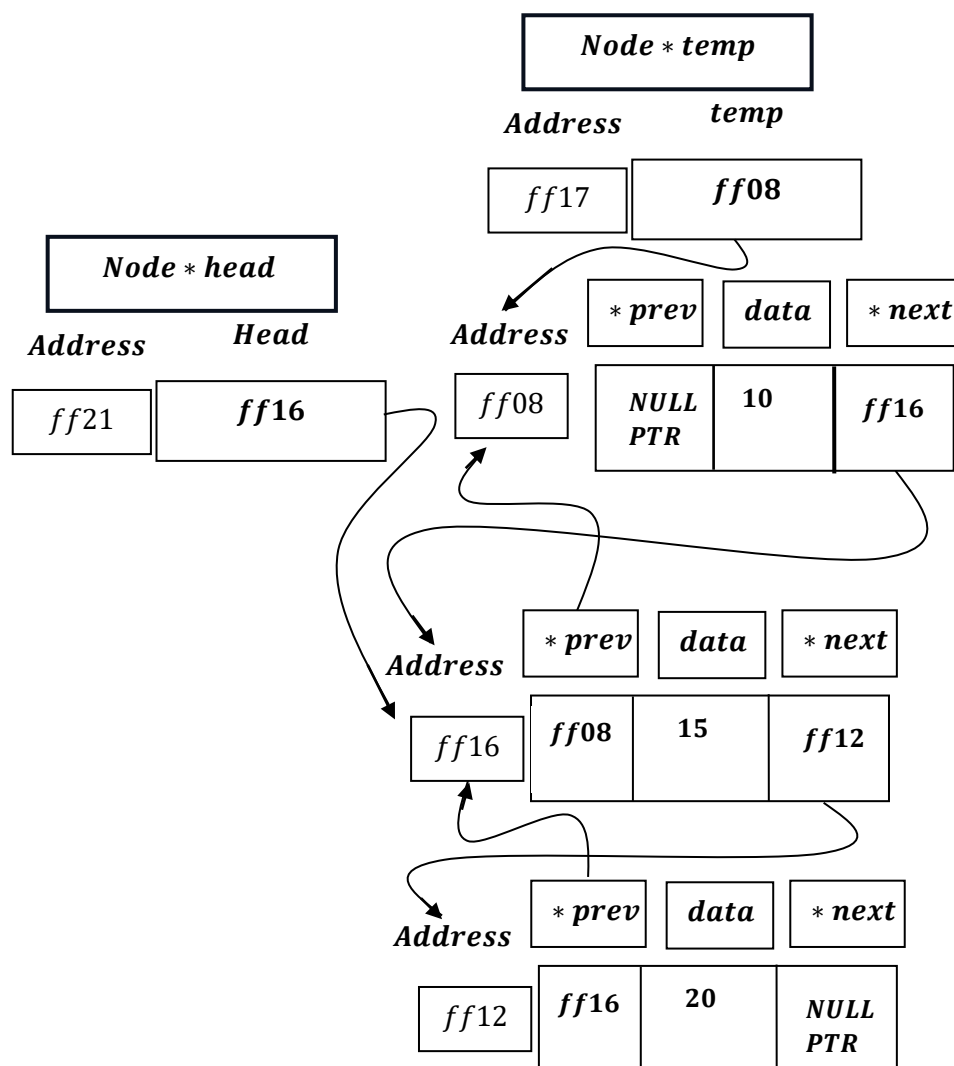
And, return statement will exit from the function.

Next, Line of Code:

```
Node *temp = head;
```



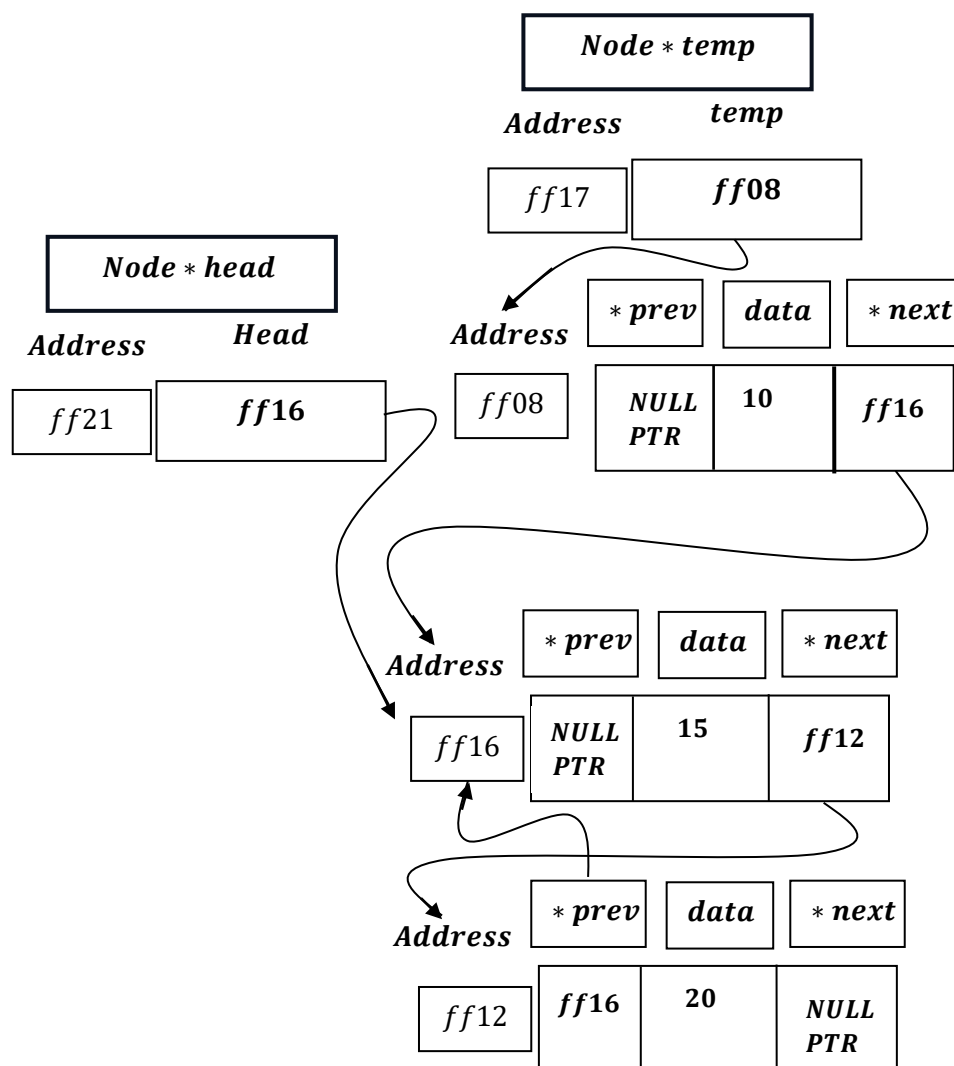
```
head = head->next;
```



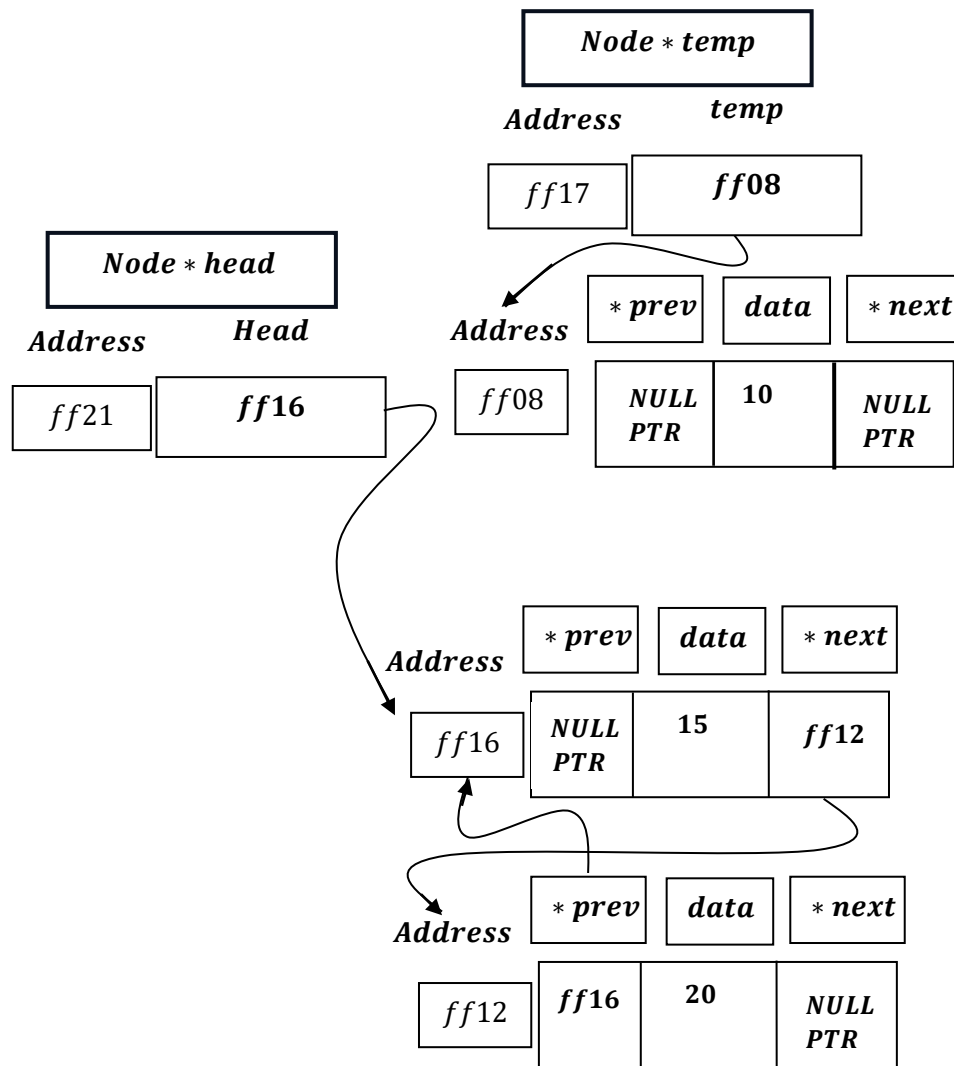
Next, Line of Code:

```
if (head != nullptr)
{
    head->prev = nullptr;
}
```

At this present state , head is not pointing to NULLPTR ,Then:

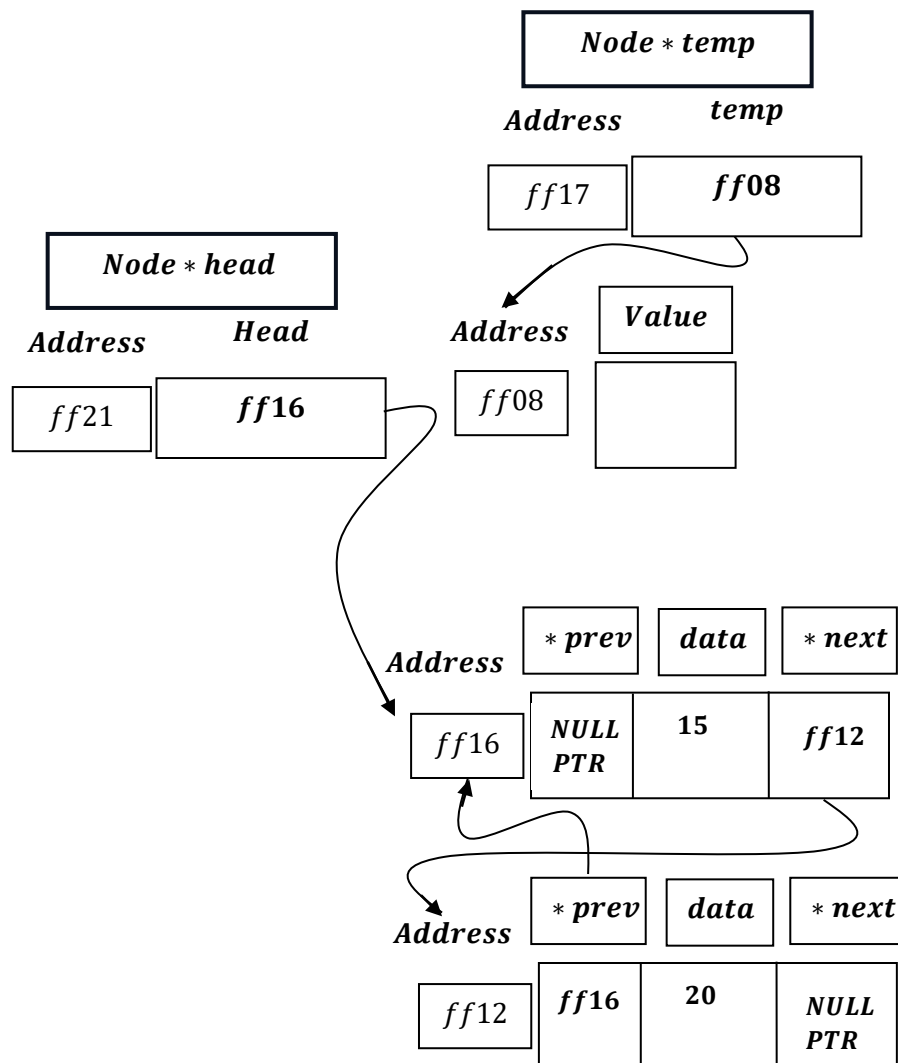


```
temp->next = nullptr;
```



```
free(temp);
```

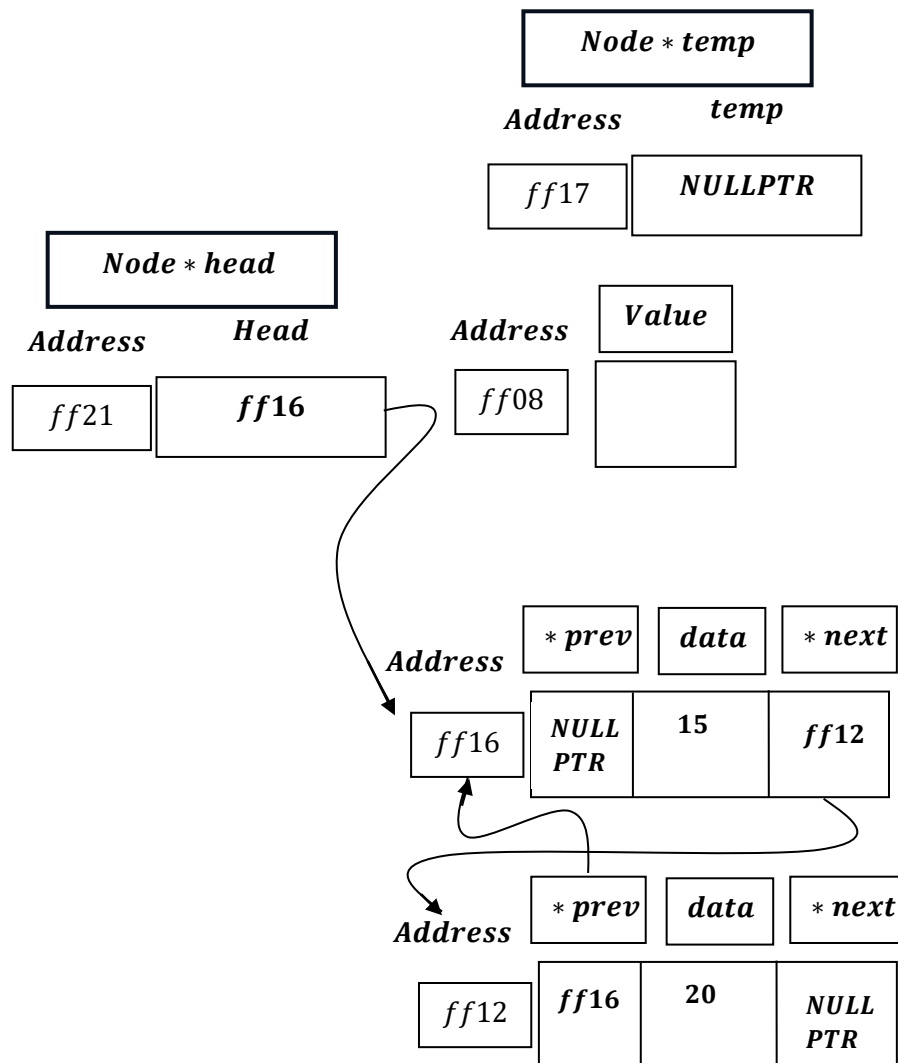
This will free the memory location that temp pointed to:



Now, temp is a dangling pointer. Now, the memory location after freeing , is not a valid memory location , hence the pointer `temp` is now a dangling pointer.

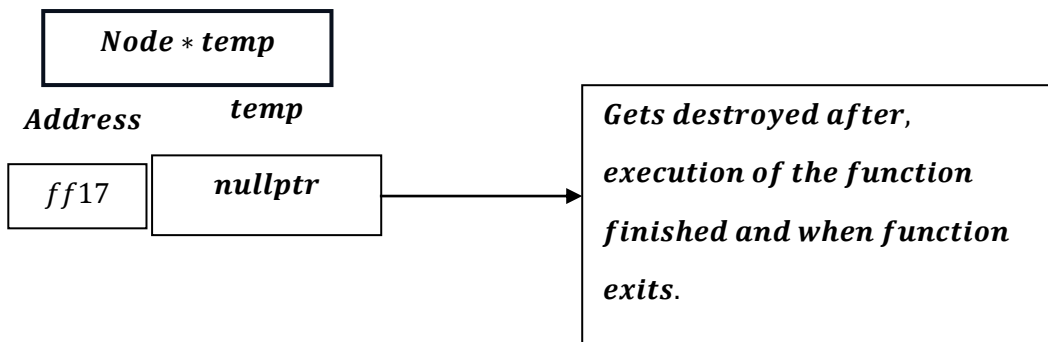
A dangling pointer is a pointer that continues to exist even after the memory it points to has been deallocated or freed. It's called dangling because it is no longer points to a valid object.

```
temp = nullptr;
```

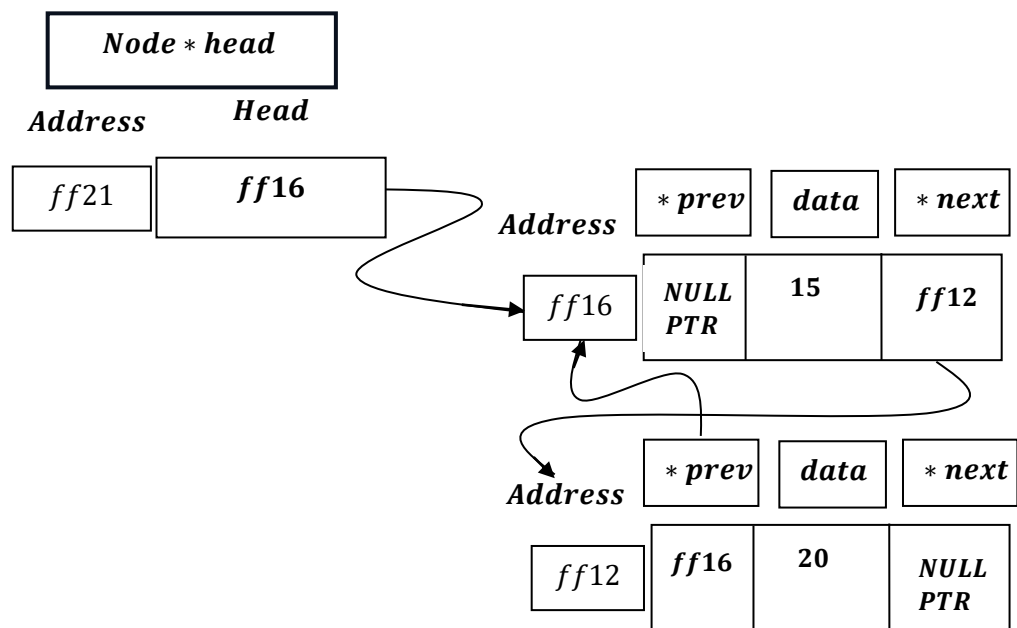


temp = nullptr; → This ensures that any future checks on the pointer (e.g., if (temp != NULLPTR)) will correctly prevent the program from trying to use the invalid memory location.

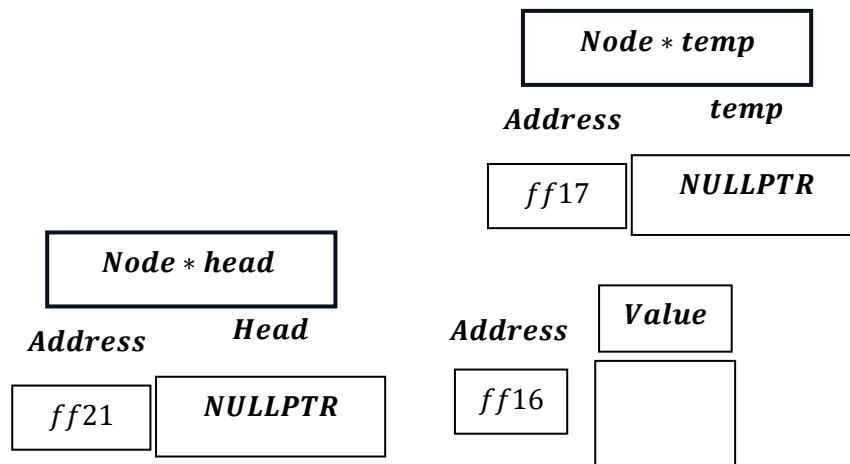
Node * temp temporary variable, therefore gets destroyed after function execution get finished.



Hence ,we are left with:



Note: –After ,deleting from beginning, List can become Empty.



Then again :

```
if (head == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}
```

will run and return statement will make exit from the function.

Doubly Linked List – Delete From Beginning [Working Summary]:

1. **Handles Empty List:** It first checks if the list is empty (`head == nullptr`). If so, it prints a message and stops.
2. **Saves the Head Node:** It creates a temporary pointer, `temp`, to keep track of the current head node, which is the one to be deleted.
3. **Advances the Head:** The list's official `head` pointer is moved to the next node in the sequence (`head = head->next`). This second node is now the new front of the list.
4. **Updates New Head's Link:** It checks if the new `head` exists (i.e., the list had more than one node). If it does, its `prev` (previous) pointer is set to `nullptr`, correctly marking it as the first node.
5. **The next pointer of the old node is nullified as a good practice.**
6. **Deallocates Memory:** The function then frees the memory occupied by the original head node (stored in `temp`).
7. **Prevents Dangling Pointer:** As a final safety measure, it sets the `temp` pointer to `nullptr` to prevent it from pointing to the now-released memory.

Time Complexity of Delete From Beginning Function Module



Best Case = Worst Case = Average Case

1. Checking if the list is empty:

if (head == nullptr)

- *This is a simple pointer comparison.*
- *Time Complexity: $O(1)$ (constant time)*
- *Explanation: Regardless of the size of the list, this operation always takes the same.*

2. If the list is empty:

```
cout << "Error: List is empty." << endl;  
return;
```

- *Printing an error message and returning.*
- *Time Complexity: $O(1)$ (constant time).*
- *Explanation: Printing a fixed string and returning are both constant time operations.*

3. If the list is not empty:

```
Node * temp = head;
```

- *Assigning the head pointer to a temporary variable.*
- *Time Complexity: $O(1)$ (constant time).*
- *Explanation: This is a simple pointer assignment, which always takes the same amount of time.*

4. Updating the head pointer:

```
head = head → next;
```

- *Moving the head pointer to the next node.*
- *Time Complexity: $O(1)$ (constant time)*
- *Explanation: This is another pointer assignment, which is a constant time operation.*

5. Checking List is not Empty

```
if (head != nullptr)
```

- *This is a simple pointer comparison.*
- *Time Complexity: $O(1)$ (constant time)*
- *Explanation: Regardless of the size of the list, this operation always takes the same.*

6. If the list is not empty:

`head → prev = nullptr;`

- Sets prev pointer of memory pointed by head to NULLPTR .**
- Time Complexity: $O(1)$ (constant time)**
- Explanation: Regardless of the size of the list, this operation always takes the same.**

5. Printing the deleted node's data:

`cout << "Deleted " << temp → data << "from the beginning" << endl;`

- Outputting the data of the node being deleted.**
- Time Complexity: $O(1)$ (constant time)**
- Explanation: Printing a single value is a constant time operation, regardless of the size of the list.**

6. Clearing the next pointer of the deleted node:

`temp→ next = nullptr;`

- Setting the next pointer of the deleted node to null.**
- Time Complexity: $O(1)$ (constant time).**
- Explanation: This is a simple pointer assignment, which is always constant time.**

7. Freeing the memory:

`free(temp);`

- Deallocating the memory of the deleted node.**
- Time Complexity: $O(1)$ (constant time)**
- Explanation: Freeing a single node is a constant time operation in most memory management systems.**

8. Nullifying the temporary pointer:

temp = nullptr;

- *Setting the temporary pointer to null.*
- *Time Complexity: $O(1)$ (constant time)*
- *Explanation: This is a simple pointer assignment, which is always constant time.*

Overall Time Complexity:

$$O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(1).$$

- *Every operation in this function is $O(1)$.*
- *When we combine constant time operations, the result is still constant time.*
- *Therefore, the overall time complexity of `deleteFromBeginning` is $O(1)$.*

Conclusion:

The `deleteFromBeginning` function has a time complexity of $O(1)$, which means it performs in constant time regardless of the size of the linked list. This is because it only operates on the head of the list and does not need to traverse the entire list.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq 1 \leq c_2 \times 1$$

<i>Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Deletion From Beginning</i>	<i>Worst Case =</i> <i>Best Case</i> <i>Hence no true Average Case[As we delete only from first and hence no random selection].</i> <i>Hence , we can say,</i> <i>Worst Case = Best Case = Average Case.</i>	$\Omega(1) = \Theta(1) = O(1).$

1. Best Case: The list is empty. The function checks if the head is `nullptr` and returns immediately. This is an $O(1)$ operation.

2. Worst Case: The list is not empty. The function deletes the first node and updates the head pointer. This involves a few pointer manipulations and is also an $O(1)$ operation.

Since the function's behavior does not depend on the size of the list and always performs a constant amount of work, there is no variation in its time complexity. Therefore, there is no distinct average case; the time complexity remains $O(1)$ in all scenarios.

Doubly Linked List – Delete From Beginning And Time Complexity.

Program :

```
void deleteFromBeginning()
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
    Node *temp = head;
    head = head->next;
    if (head != nullptr)
    {
        head->prev = nullptr;
    }
    cout << "Deleted " << temp->data << " from the
beginning." << endl;
    temp->next = nullptr;
    free(temp);
    temp = nullptr;
}
... .
case 5:
    deleteFromBeginning();
    break;
```

Space Complexity

Auxiliary Space

Auxiliary space is the extra memory used by a function, excluding the space taken by the inputs themselves.

In the deleteFromBeginning function, the only new variable created is:

*Node *temp: A single temporary pointer used to hold the address of the node being deleted. The size of this pointer is constant.*

The function's main memory operation is free(temp), which deallocates memory, not allocates it. Since the amount of extra memory used does not grow with the size of the linked list, the auxiliary space complexity is $O(1)$.

Auxiliary Space: $O(1)$

Input Space

Input space is the memory used to store the function's inputs. This can be analyzed from two perspectives.

1. Parameter-only Definition

This view considers only the space taken by the function's direct parameters. The function signature is void deleteFromBeginning().

The function takes no parameters.

Under this definition, the input space is constant.

Input Space (Parameter-only): $O(1)$

2. Full-Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list.

If the linked list contains n nodes before the deletion, the total memory it occupies is proportional to n .

Under this definition, the input space is linear with respect to the size of the list.

Input Space (Full-Data): $O(n)$

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, the "space complexity" of a function refers to its auxiliary space. Therefore, the most common and expected answer for this function is $O(1)$.

Algorithm: Delete from Beginning (Doubly Linked List)

DDELBEG(START)

1. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

Write: "List is empty. Nothing to delete." and EXIT.

[End of If structure.]

2. [STORE ADDRESS OF FIRST NODE]

Set PTR := START [PTR is a pointer used to hold the node to be deleted].

3. [ADVANCE START TO THE SECOND NODE]

Set START := NEXTLINK[START].

4. [UPDATE NEW FIRST NODE, IF IT EXISTS]

If START ≠ NULL, then:

Set PREVLINK[START] := NULL (The PREV pointer of the new first node is set to NULL to unlink from previous first node).

[End of If structure.]

5. [OUTPUT THE DELETED NODE]

Write: "DELETED", INFO[PTR], "from the beginning."

6. [DEALLOCATE THE ORIGINAL FIRST NODE]

Set NEXTLINK[PTR] := NULL (The NEXT pointer of the previous first node de – links from the current and new first node)

free(PTR) [Releases the memory of the node pointed to by PTR].

7. [AVOID DANGLING POINTER]

Set PTR := NULL.

8. *Exit*
-

Doubly Linked List – Delete From End And Time Complexity.

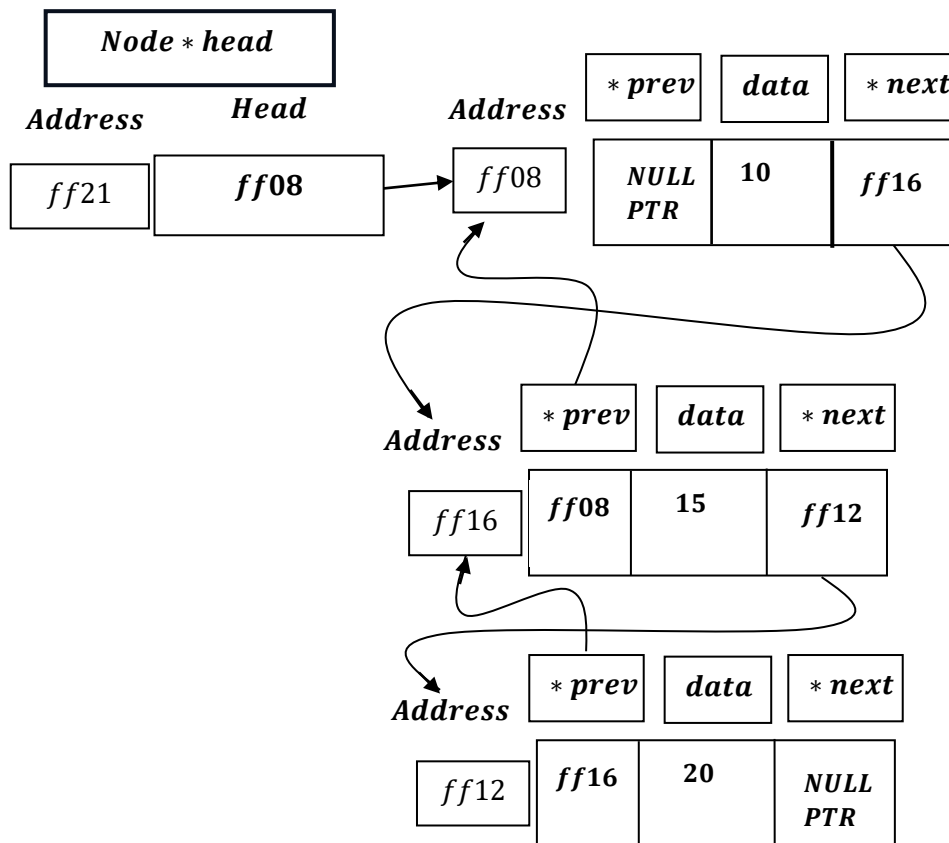
Program :

```
void deleteFromEnd()
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
    Node *temp = head;
    if (temp->next == nullptr)
    { // Only one node in the list
        head = nullptr;
    }
    else
    {
        while (temp->next != nullptr)
        {
            temp = temp->next; // Traverse to the last node
        }
        temp->prev->next = nullptr;
        temp->prev = nullptr;

        cout << "Deleted " << temp->data << " from the end." <<
endl;
        free(temp);
        temp = nullptr;
    }
    ... .
case 6:
    deleteFromEnd();
    break;
```

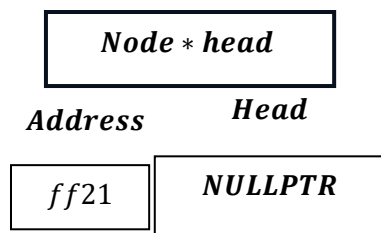
Program Analysis:

Lets consider a Doubly Linked List :



Now, first Line of Code:

```
if (head == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}
```

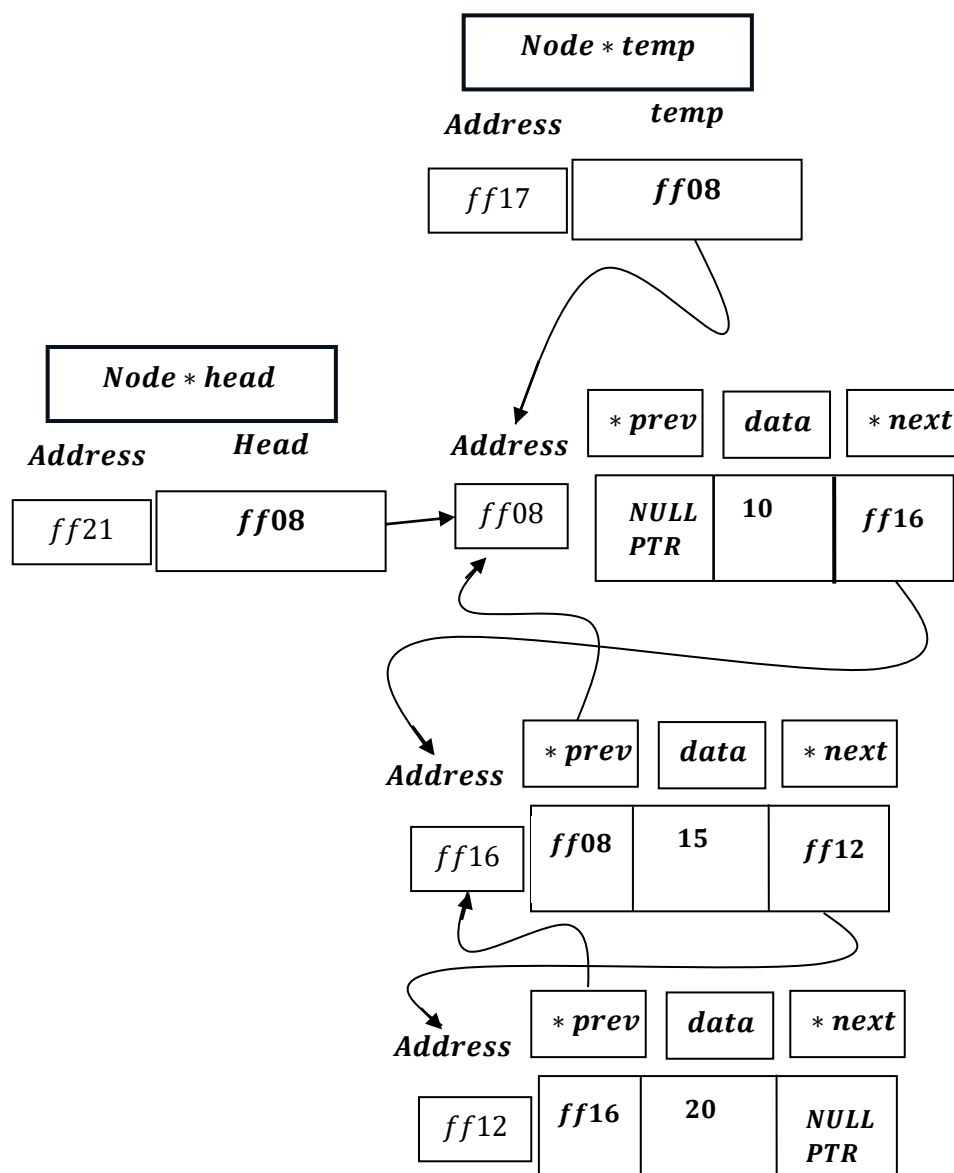


Then, it will print statement : `` List is empty. Nothing to delete.``

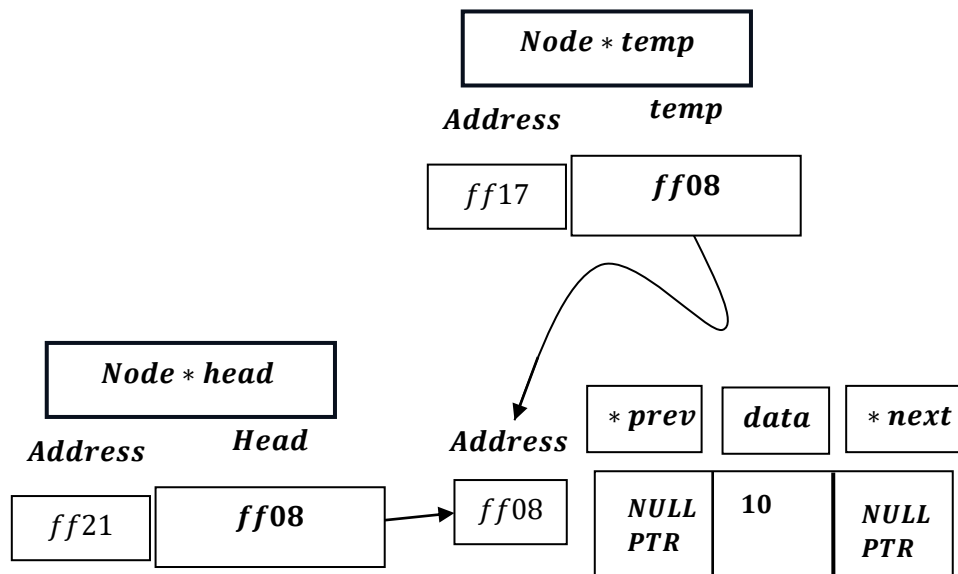
And, return statement will exit from the function.

Next, Line of Code:

```
Node *temp = head;
```

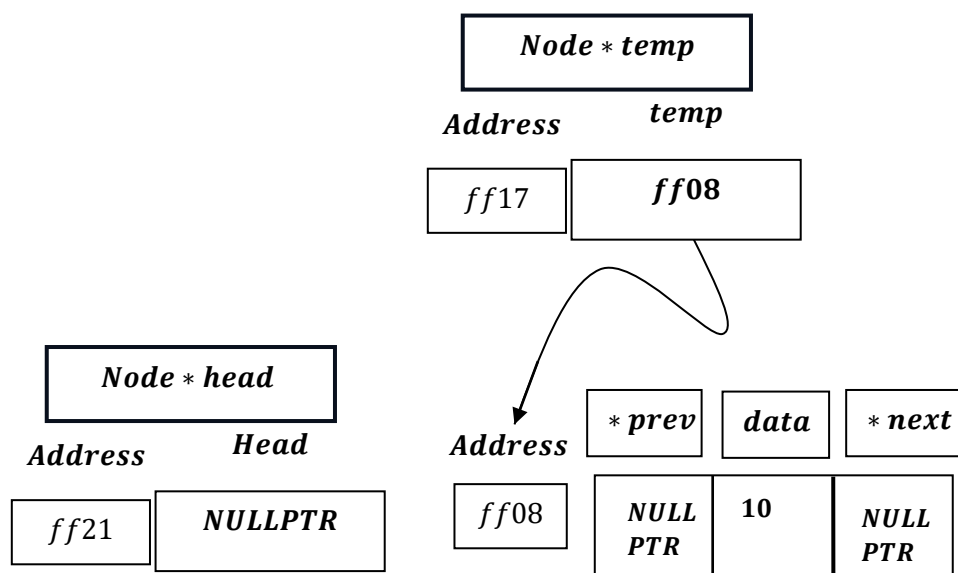


1. If there is single node :



```
if (temp->next == nullptr)
{
    head = nullptr;
}
```

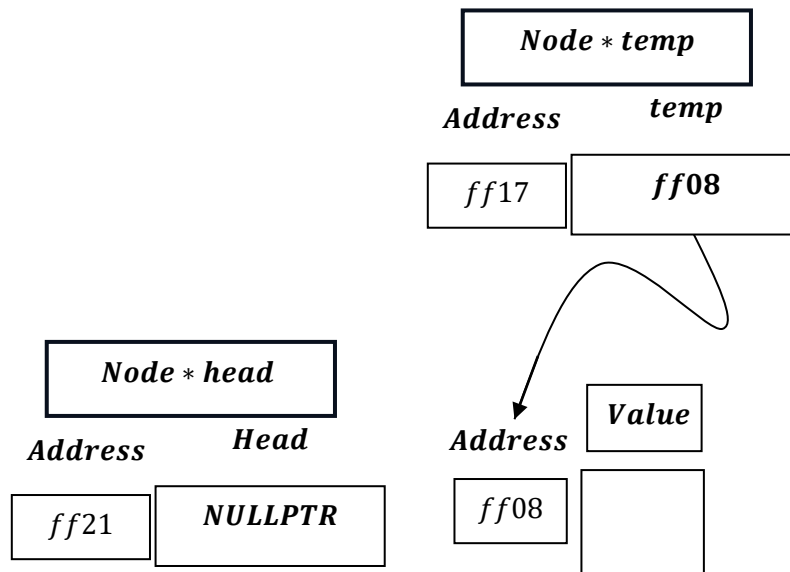
The condition: $temp \rightarrow next = nullptr$ is true , then if block's statement gets executed.



Next,

```
free(temp);
```

This will free the memory location that temp pointed to:

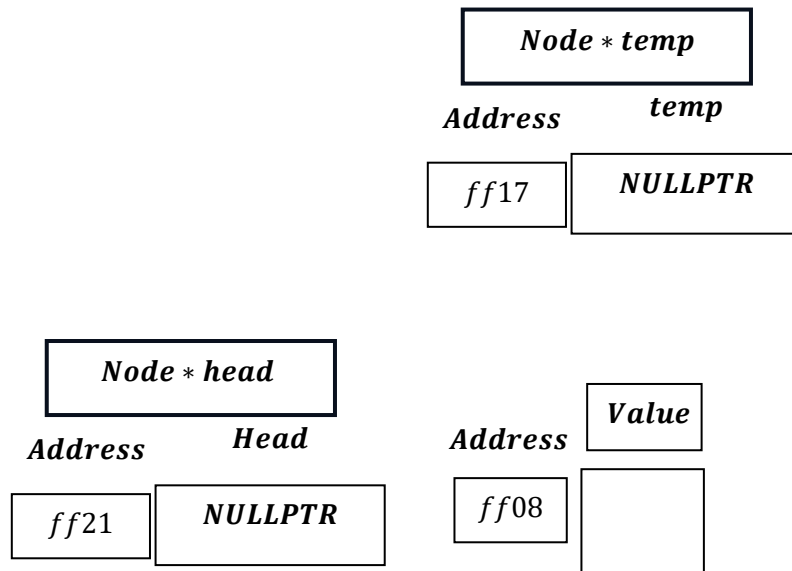


Now, temp is a dangling pointer. Now, the memory location after freeing , is not a valid memory location , hence the pointer `temp` is now a dangling pointer.

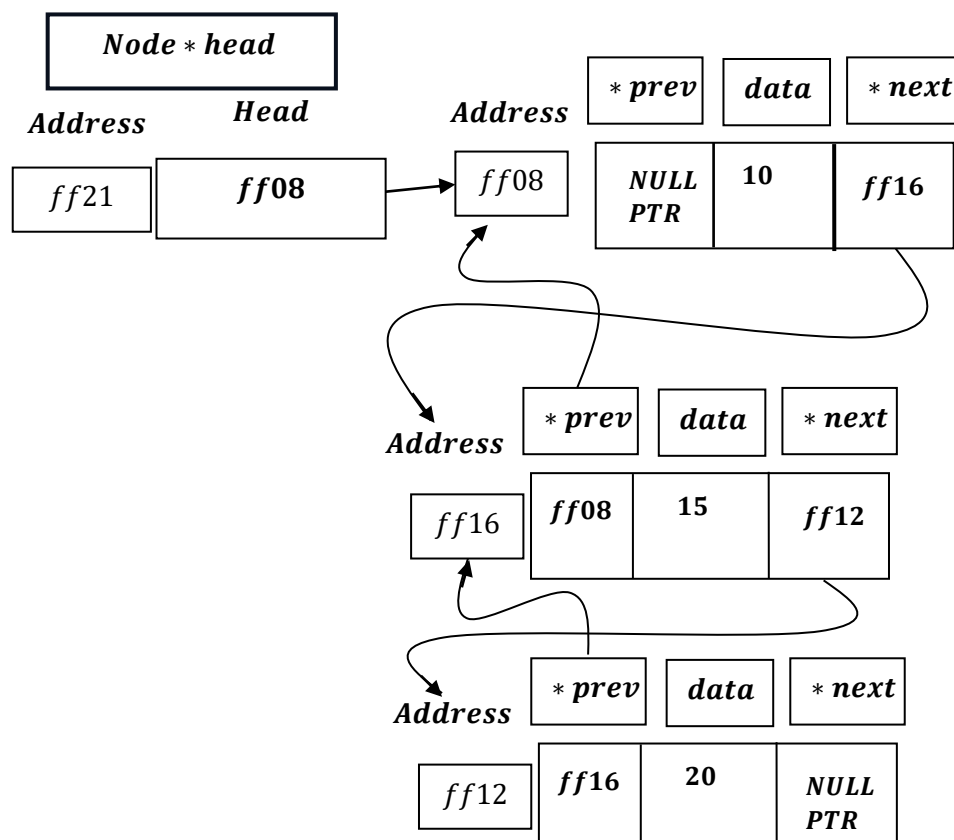
A dangling pointer is a pointer that continues to exist even after the memory it points to has been deallocated or freed. It's called dangling because it is no longer points to a valid object.

Hence,

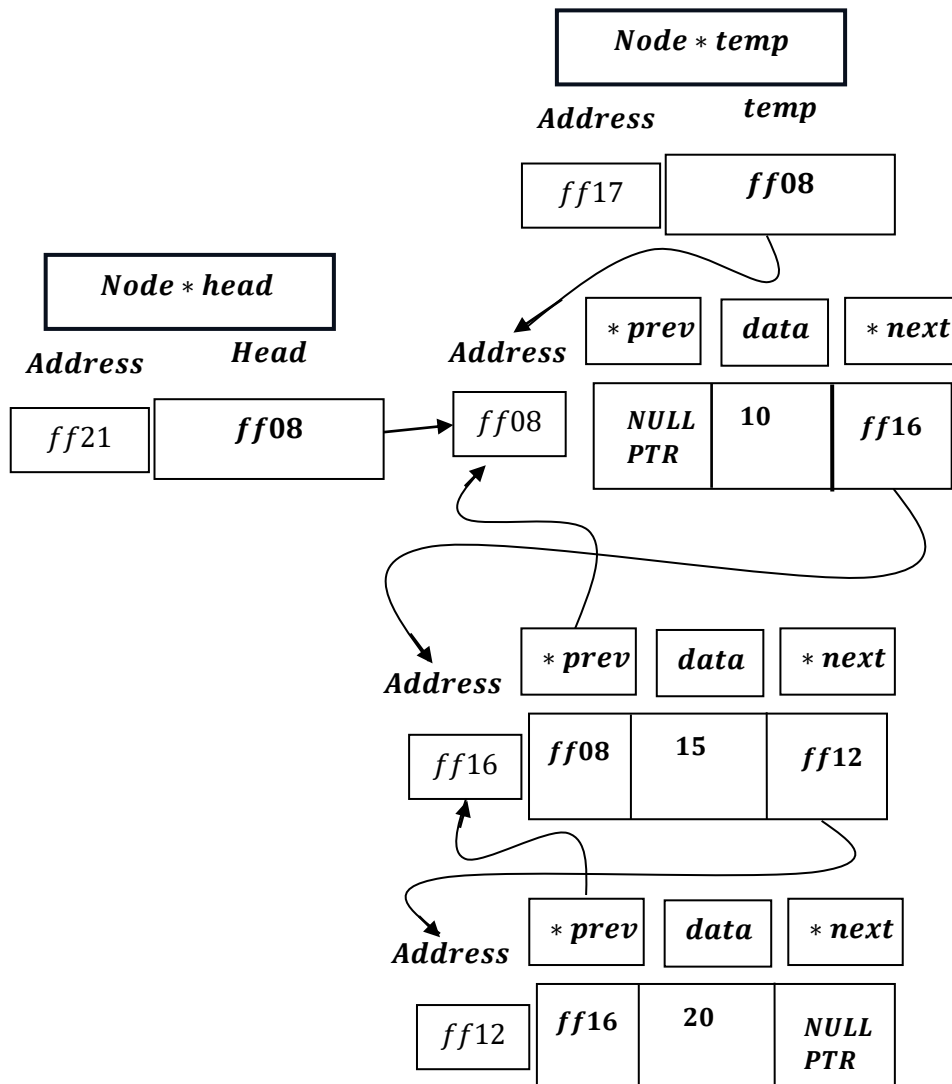
```
temp = nullptr;
```



2. If there is 'n' no. of nodes:

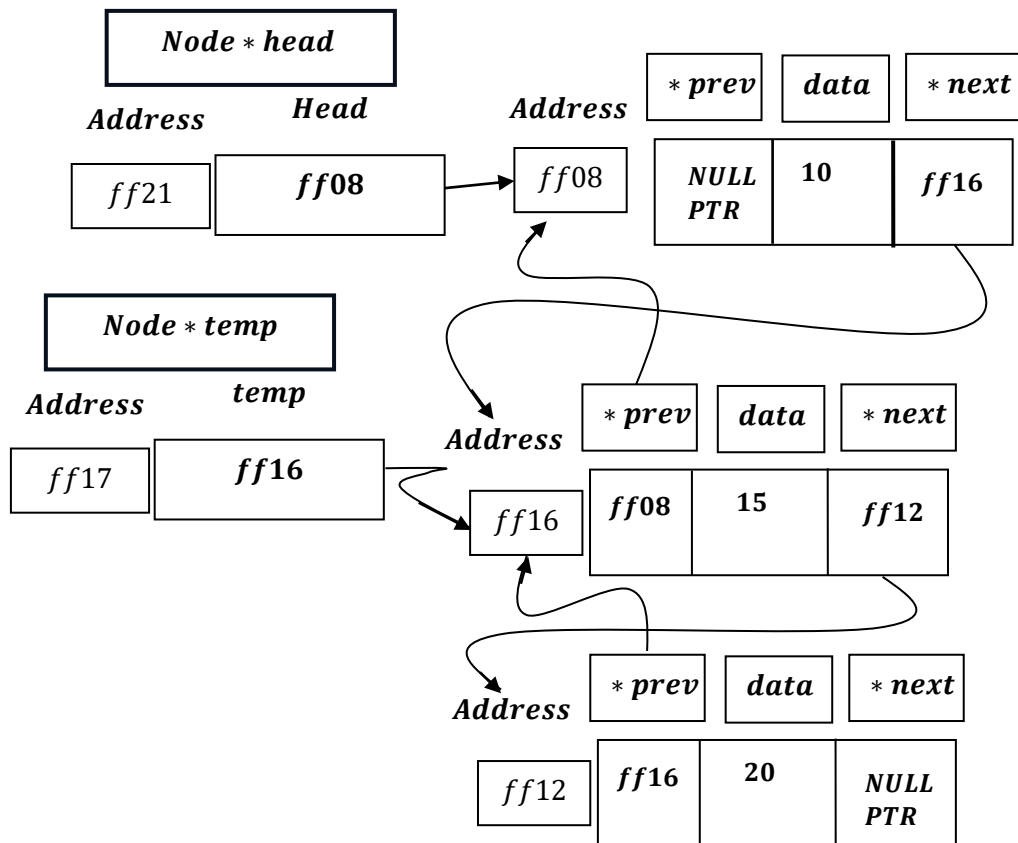



```
Node *temp = head;
```

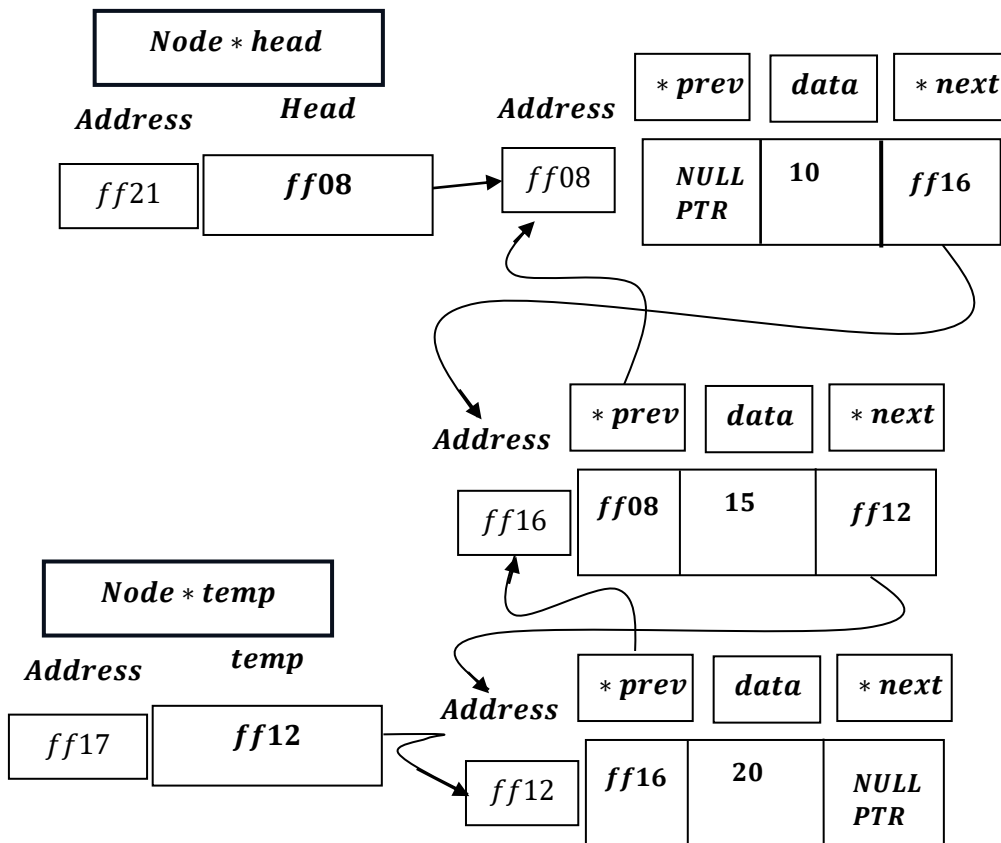


```
if (temp->next == nullptr){... }
else
{
    while (temp->next != nullptr)
    {
        temp = temp->next; // Traverse to the last node
    }
    .....
}
```

temp → *next* ≠ *nullptr* is true , then: *temp* = *temp* → *next* = ff16.



temp → *next* ≠ nullptr is true , then: *temp* = *temp* → *next* = ff12.

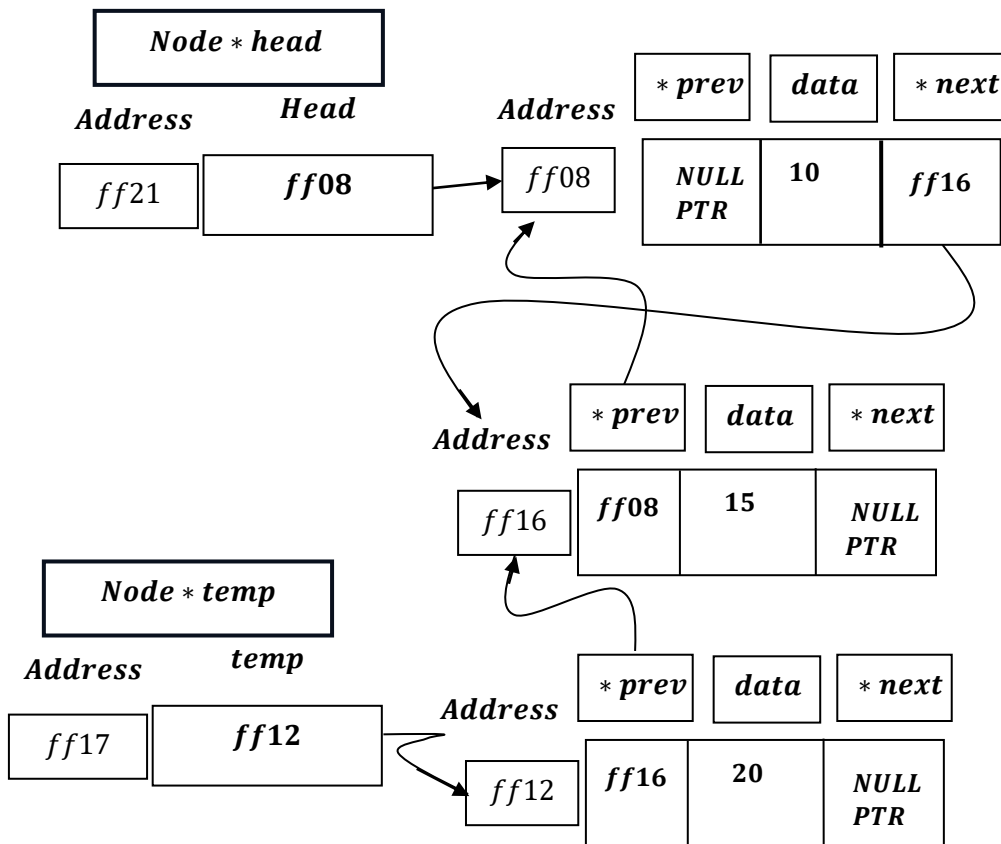


temp → *next* ≠ *nullptr* is false , hence loop condition becomes false , and loop exits.

```
if (temp->next == nullptr){... }
else
{
    while (temp->next != nullptr){ .....}
    temp->prev->next = nullptr;
    .....
}
```

i.e.,

```
temp->prev->next = nullptr;
```



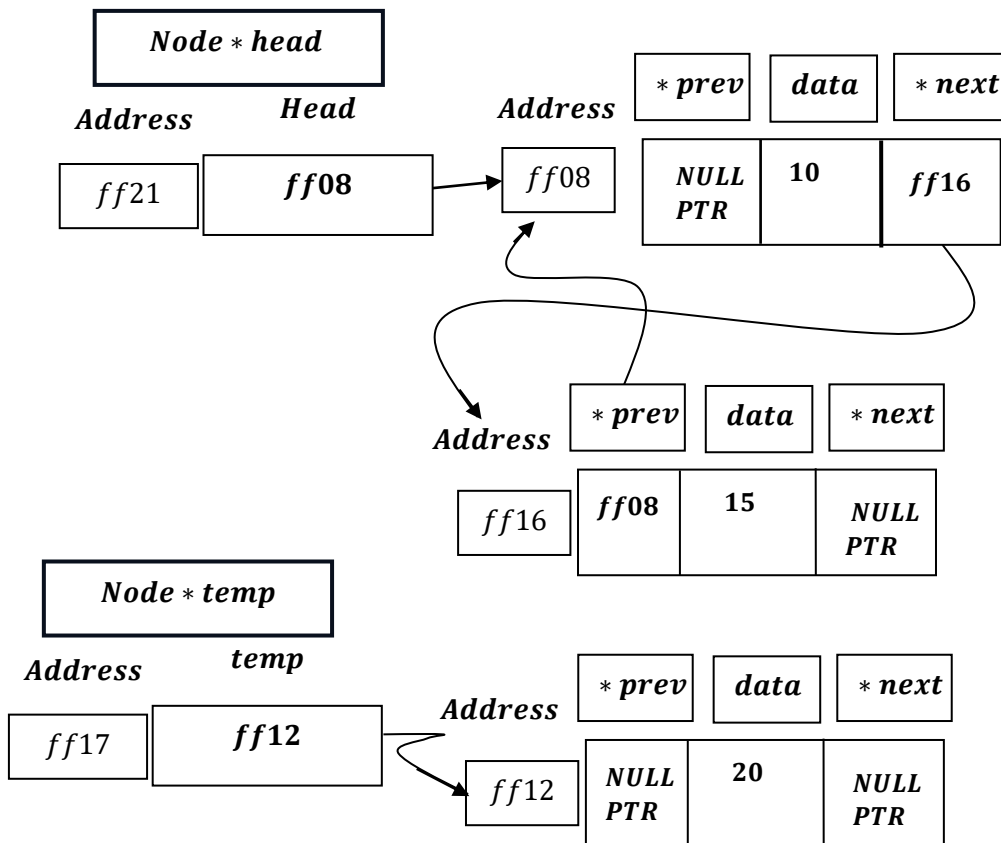
```

if (temp->next == nullptr){... }
else
{
    while (temp->next != nullptr){ .....}
    temp->prev->next = nullptr;
    temp->prev = nullptr;
}

```

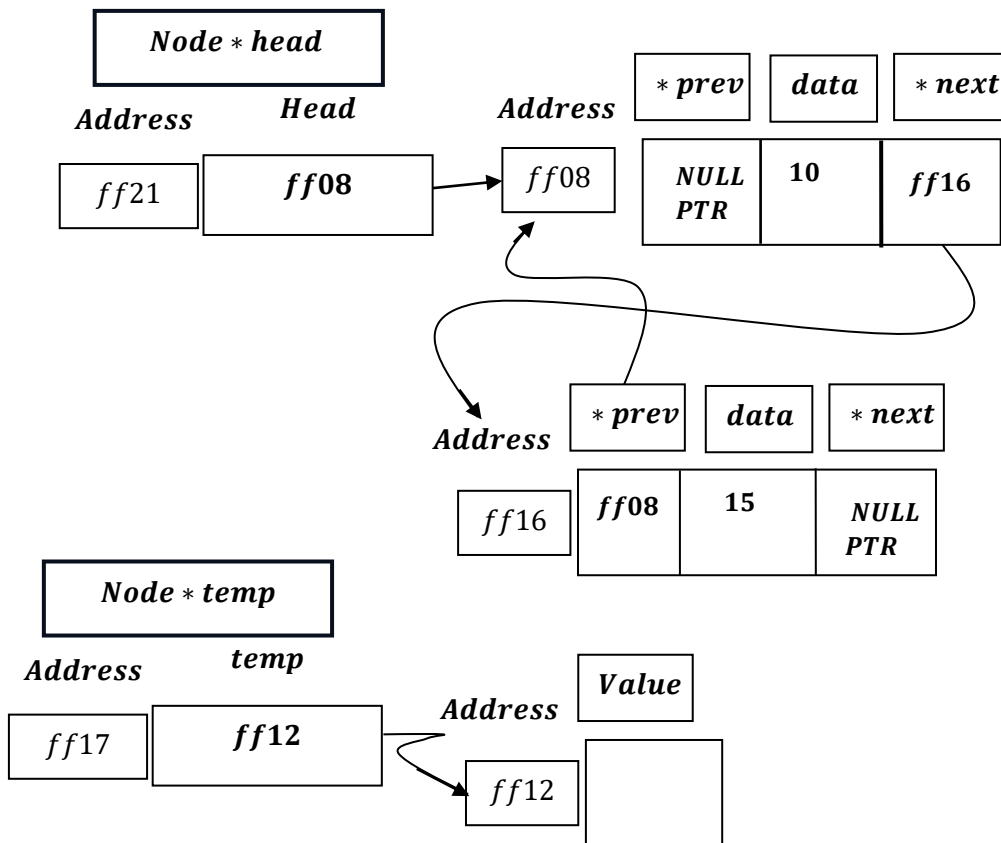
i.e.,

```
temp->prev = nullptr;
```



Next,

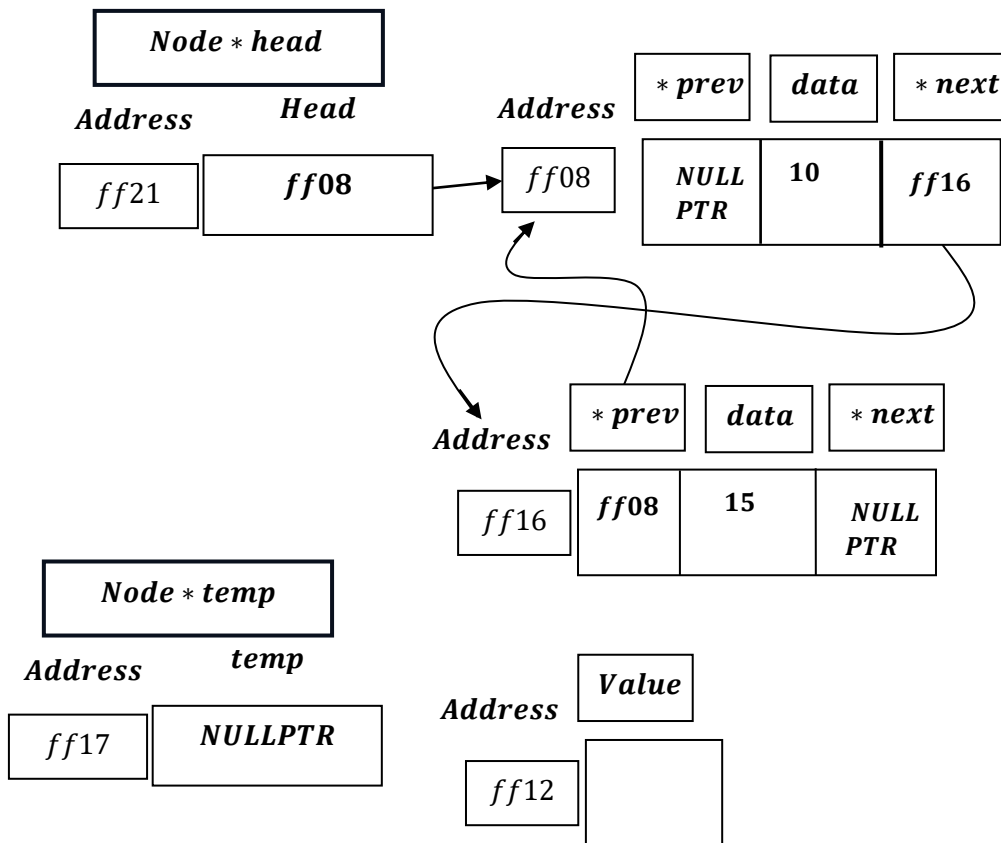
```
free(temp);
```



Now, temp is a dangling pointer. Now, the memory location after freeing, is not a valid memory location, hence the pointer `temp` is now a dangling pointer.

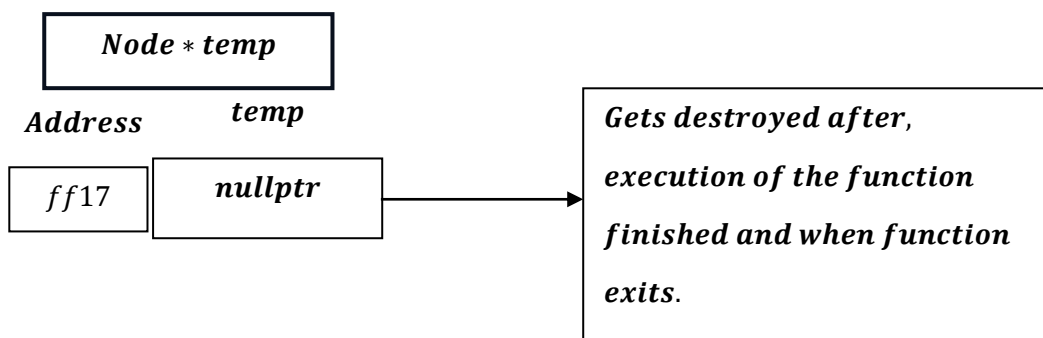
A dangling pointer is a pointer that continues to exist even after the memory it points to has been deallocated or freed. It's called dangling because it is no longer points to a valid object.

```
temp = nullptr;
```

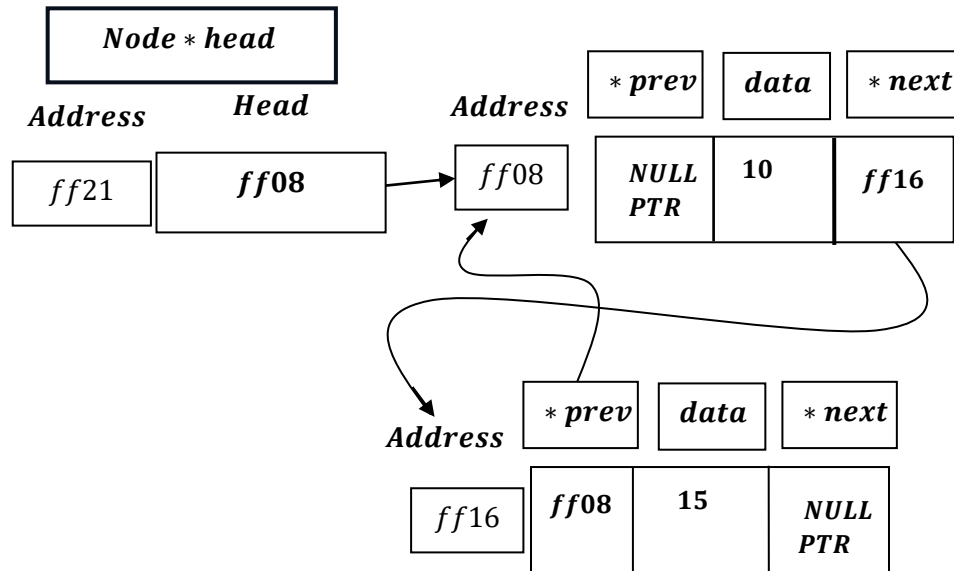


temp = nullptr; → This ensures that any future checks on the pointer (e.g., if (temp != NULLPTR)) will correctly prevent the program from trying to use the invalid memory location.

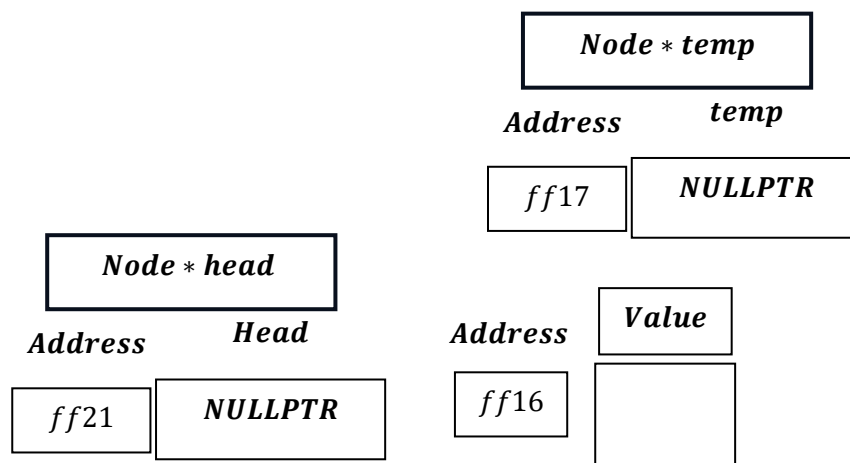
*Node * temp temporary variable, therefore gets destroyed after function execution get finished.*



Hence ,we are left with:



Note: –After ,deleting from beginning,List can become Empty.



Then again :


```
if (head == nullptr)
{
    cout << "List is empty. Nothing to delete." << endl;
    return;
}
```

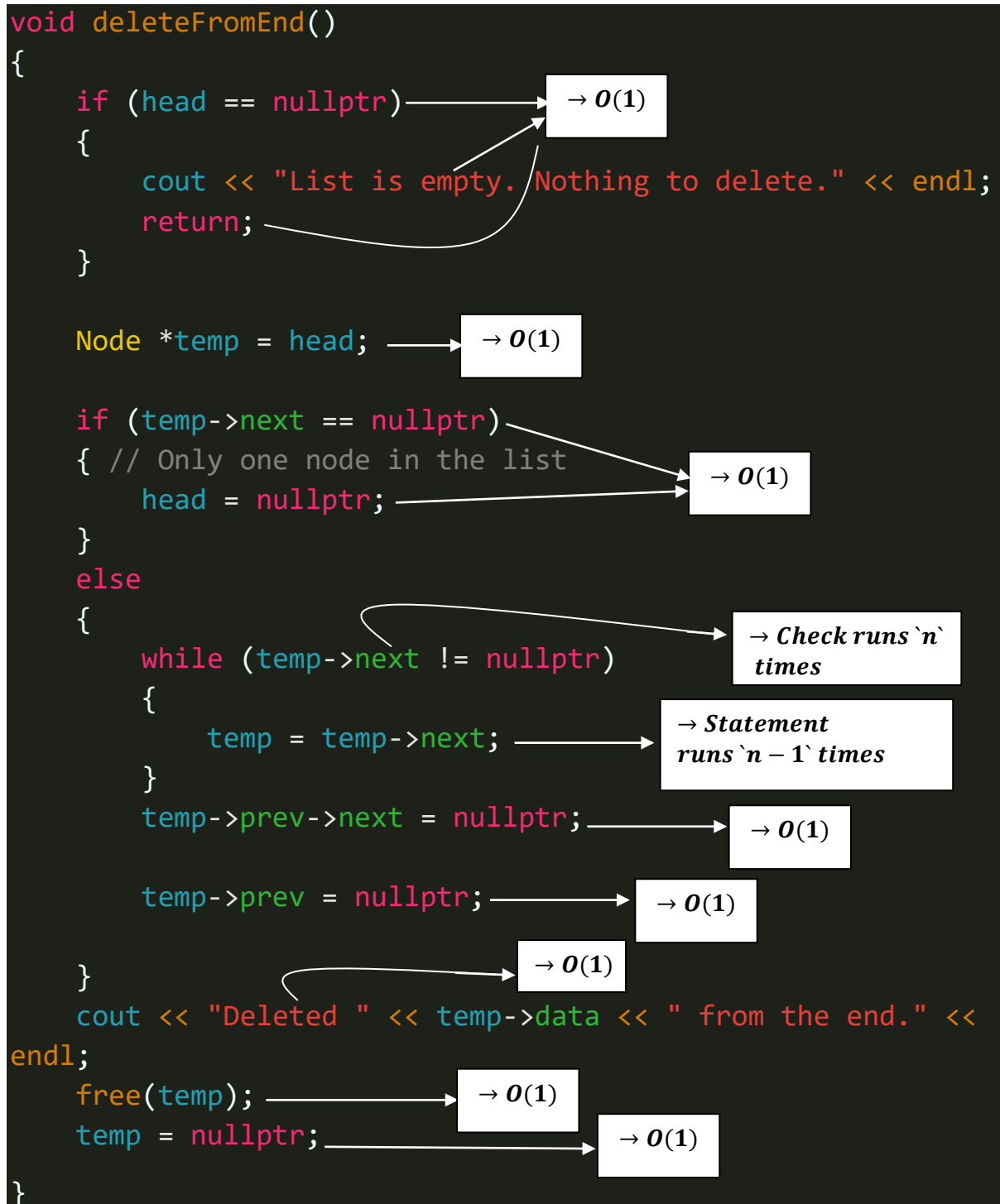
will run and return statement will make exit from the function.

Doubly Linked List – Delete From End [Working Summary]:

This function removes the last node from a doubly linked list.

1. **Handles Empty List:** It first checks if the list is empty (`head == nullptr`). If it is, the function prints a message and exits.
 2. **Handles Single-Node List:** It checks if the first node is also the last node (`temp->next == nullptr`). If so, it sets the `head` to `nullptr`, effectively emptying the list.
 3. **Finds the Last Node:** If the list has multiple nodes, it uses a `while` loop to traverse the list until `temp` points to the very last node (`temp->next` is `nullptr`).
 4. **Detaches the Last Node:**
 - `temp->prev->next = nullptr;` This is the key step. It accesses the second-to-last node (`temp->prev`) and sets its `next` pointer to `nullptr`, officially making it the new end of the list.
 - `temp->prev = nullptr;` The `prev` pointer of the node to be deleted is also nullified.
 5. **Deallocates Memory:** Finally, it frees the memory that was used by the last node (pointed to by `temp`) and then sets the `temp` pointer itself to `nullptr` to prevent it from becoming a dangling pointer.
-

Time Complexity of Delete From End Function Module



1. Best Case ,when there is single head node ,where head pointer pointing to the single head node or List is Empty.

1. When List is Empty[Absolute Best Case]:

→ If `head = nullptr`, this checking takes constant time. The `cout` and `return` statements take constant time. Hence overall process takes, $O(1)$ time.

2. When List has a single Node[Best Case but not Absolute]:

→ If `head = nullptr` which is false, this checking takes constant time. Hence overall process takes, $O(1)$ time.

→ Temp pointer is assigned to Head Node, which makes Temp pointer to point at memory location to which Head pointer is pointing to, this process takes constant amount of time i.e. $O(1)$.

→ If condition checks next pointer in memory location pointed by temp pointer of Node named structure whether equals to `NULLPTR`, if equals to `NULLPTR`, then, it is approved that List has a single node, thereafter, if block statement runs i.e. Head pointer gets assigned to `NULLPTR` making LIST empty. this process takes constant amount of time i.e. $O(1)$.

→ Next, it prints out, data which is to be deleted, this process takes constant amount of time i.e. $O(1)$.

→ `free(temp)` makes allocated memory pointed by temp reusable, this takes constant amount of time i.e. $O(1)$.

→ Then to avoid dangling pointers, temp pointer of Node named structure set to `nullptr`, this takes constant amount of time i.e. $O(1)$.

Total Time Complexity: $O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(1)$

Why not Absolute Best Case?

**Ans: – Since here for single node, it needs more `Line of Codes` to execute ,
than for Empty List , therefore more operation to be executed , hence ,
For single node, its not the absolute best case.**

2. Worst Case , when there present multiple nodes in the linked list.

→ If head = nullptr which is false , this checking takes constant time.

Hence overall process takes, $O(1)$ time.

**→ Temp pointer is assigned to Head Node , which makes Temp pointer to point at
memory location to which Head pointer pointing to , this process takes constant
amount of time i. e. $O(1)$.**

**→ If condition checks next pointer in memory location pointed by temp pointer
of Node named structure whether equals to NULLPTR, the check is false , hence,
this process takes constant amount of time i. e. $O(1)$.**

→ As if block is false , hence else block gets executed : –

**→ So, while condition checks $temp \rightarrow next \neq NULLPTR$
check runs `n` times as last check will be false hence ,
inner statement i. e. traversal: $temp = temp \rightarrow next$
gets executed `n – 1` time , we consider loop statement
execution here , hence $O(n - 1) = O(n)$.**

→ Now Temp pointer pointed to Memory Allocation's previous pointer pointed to previous Memory Allocation or previous node's Next pointer sets to NULLPTR, this detach Next pointer of Previous node pointed to allocated memory pointed by Temp pointer. This process takes constant amount of time i.e. $O(1)$.

→ Now we make Temp pointer pointed to Memory Allocation's previous pointer set to NULL, which again again detaches previous pointer from pointing to previous Node. This process takes constant amount of time i.e. $O(1)$.

→ Next, it prints out, data which is to be deleted, this process takes constant amount of time i.e. $O(1)$.

→ `free(temp)` makes allocated memory pointed by temp reusable, this takes constant amount of time i.e. $O(1)$.

→ Then to avoid dangling pointers, temp pointer of Node named structure set to 'nullptr', this takes constant amount of time i.e. $O(1)$.

Total Time Complexity: $O(1) + O(1) + (O(1) + (O(n) + O(1) + O(1))) + O(1) + O(1) = O(n)$.

We can say, if List is Empty, or we have a single head node connected to Linked List, we get our best case scenario, and when list is not empty and there present multiple nodes, we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine where the deletion to be made, hence we can say average case is our worst case here i.e. always the entire list is traversed i.e. $O(n) = \Theta(n)$.

In the case of DeleteAtEnd, the deletion always happens at the end, regardless of the current state of the list. This means the execution time is solely dependent on the length of the list. If the list is empty, the execution time is constant (best case). If the list is not empty, the execution time is proportional to the length of the list (worst case). Since the deletion position is predetermined, there's no element of randomness or varying input data to consider. Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here. Hence DeleteAtEnd doesn't have a true average case.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$$

<i>Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>DeleteAt End</i>	<i>1. Best Case</i>	$\Omega(1)$
	<i>2. Worst Case</i>	$O(n)$
	<i>3. Average Case</i>	$\Theta(n)$

Doubly Linked List – Delete From End – Space Complexity and Algorithm.

Program :

```
void deleteFromEnd()
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
    Node *temp = head;
    if (temp->next == nullptr)
    { // Only one node in the list
        head = nullptr;
    }
    else
    {
        while (temp->next != nullptr)
        {
            temp = temp->next; // Traverse to the last node
        }
        temp->prev->next = nullptr;
        temp->prev = nullptr;

        cout << "Deleted " << temp->data << " from the end." <<
endl;
        free(temp);
        temp = nullptr;
    }
    ... .
case 6:
    deleteFromEnd();
    break;
```

Space Complexity

Auxiliary Space

Auxiliary space is the extra memory used by a function, excluding the space taken by the inputs themselves.

In the deleteFromEnd function, the only new variable created is:

*Node *temp: A single temporary pointer used to traverse the list and hold the address of the node being deleted. The size of this pointer is constant.*

The function's main memory operation is free(temp), which deallocates memory. The while loop affects the function's time complexity but not its space complexity. Since the extra memory used does not grow with the list's size, the auxiliary space is $O(1)$.

Auxiliary Space: $O(1)$

Input Space

Input space is the memory used to store the function's inputs. This can be analyzed from two perspectives.

1. Parameter – only Definition

This view considers only the space taken by the function's direct parameters. The function signature is void deleteFromEnd().

The function takes no parameters.

Under this definition, the input space is constant.

Input Space (Parameter-only): $O(1)$

2. Full – Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list.

If the linked list contains n nodes before the deletion, the total memory it occupies is proportional to n .

Under this definition, the input space is linear with respect to the size of the list.

Input Space (Full-Data): $O(n)$

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, the "space complexity" of a function refers to its auxiliary space. Therefore, the most common and expected answer for this function is $O(1)$.

Algorithm: Delete from End (Doubly Linked List)

DDELEND(START)

1. **[CHECK IF LIST IS EMPTY]**

If START = NULL, then:

Write: "List is empty. Nothing to delete." and EXIT.

[End of If structure.]

2. **[INITIALIZE POINTER]**

Set PTR := START [PTR is a pointer that will find the last node].

3. **[CHECK FOR SINGLE NODE vs MULTI – NODE LIST]**

If NEXTLINK[PTR] = NULL, then: [List has only one node]

Set START := NULL [The list becomes empty].

ELSE: [List has more than one node]

a. Repeat while NEXTLINK[PTR] ≠ NULL: [Traverses the list to find the last node].

Set PTR := NEXTLINK[PTR] [Updates PTR to the next node].

[END of Loop]

b. Set NEXTLINK[PREVLINK[PTR]] := NULL (The second – to – last node's NEXT pointer is set to NULL, making it the new last node).

c. Set PREVLINK[PTR] := NULL [UNLINK PREV pointer of LAST NODE from 2nd LAST NODE]

4. **[DEALLOCATE THE ORIGINAL LAST NODE]**

Write: "Deleted", INFO[PTR], " from the end."

free(PTR) [Releases the memory of the node pointed to by PTR].

5. **[AVOID DANGLING POINTER]**

SET PTR := NULL.

6. **Exit**
-

Doubly Linked List – Delete From End And Time Complexity.

Program :

```
void deleteFromPosition(int pos)
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
    if (pos < 1)
    {
        cout << "Invalid position!" << endl;
        return;
    }
    if (pos == 1)
    {
        deleteFromBeginning();
        return;
    }

    Node *temp = head;
    for (int i = 1; i < pos && temp != nullptr; i++)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }
}
```

```

temp->prev->next = temp->next;
if (temp->next != nullptr)
{
    temp->next->prev = temp->prev;
    temp->next = nullptr;
}
temp->prev = nullptr;

cout << "Deleted " << temp->data << " from position " <<
pos << "." << endl;
free(temp);
temp = nullptr;
}

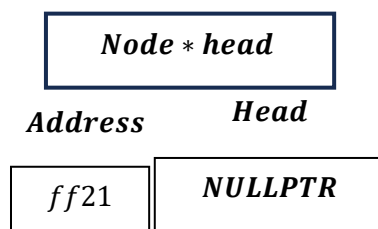
...

case 7:
    cout << "Enter position to delete: ";
    cin >> pos;
    deleteFromPosition(pos);
    break;

```

Program Analysis:

A. When LIST is Empty



Then, it will print statement : `` List is empty. Nothing to delete.``

And, return statement will exit from the function.

B. When $POS < 1$, As we consider POS starts from `1` – indexing

```
if (pos < 1)
{
    cout << "Invalid position!" << endl;
    return;
}
```

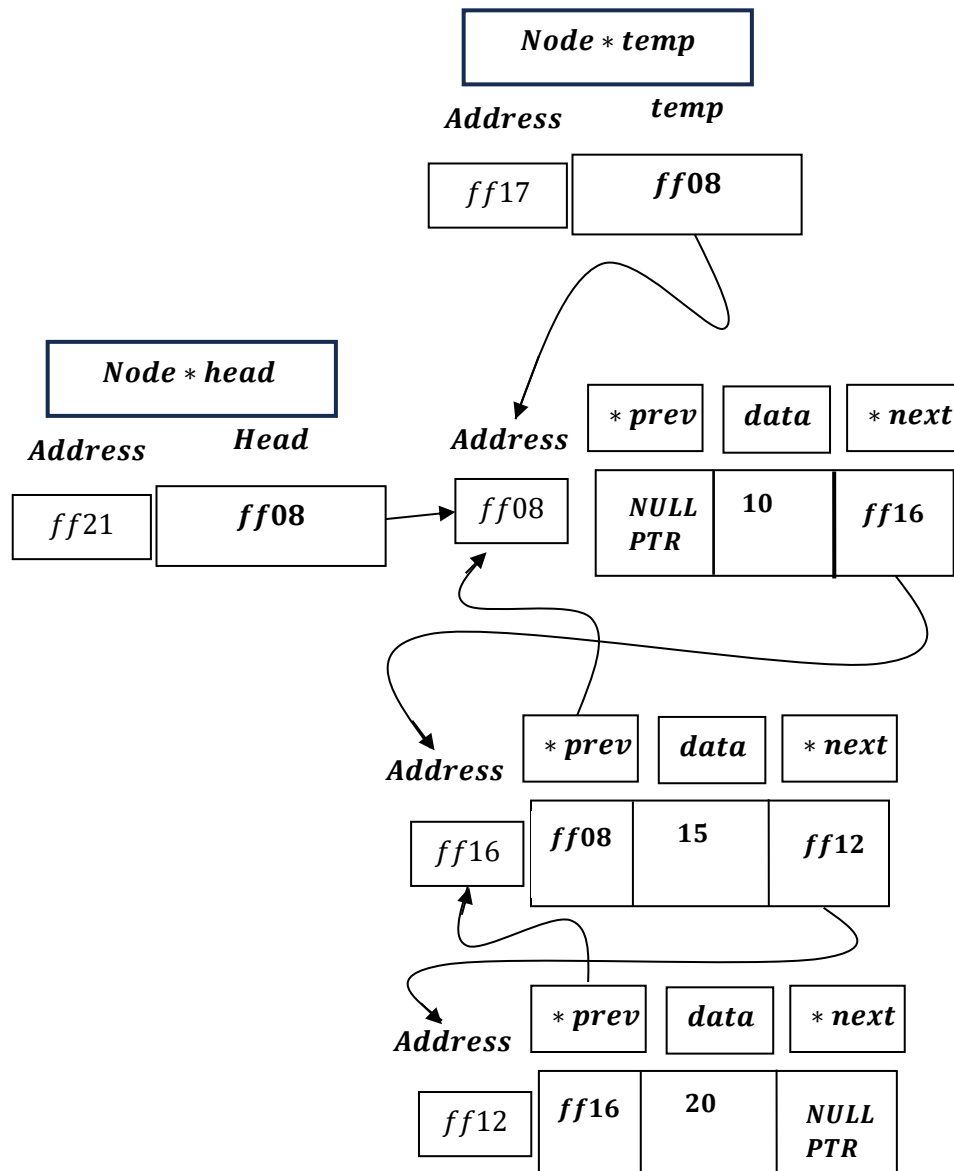
If $pos < 1$, then it prints Invalid Position and return statement, exits from function.

C. When $POS = 1$, then run delete from beginning function.

```
if (pos == 1)
{
    deleteFromBeginning();
    return;
}
```

As, $pos = 1$, deleteFromBeginning() function will run, the working of the function already discussed earlier, the return statement will exit from the function.

```
Node *temp = head;
```

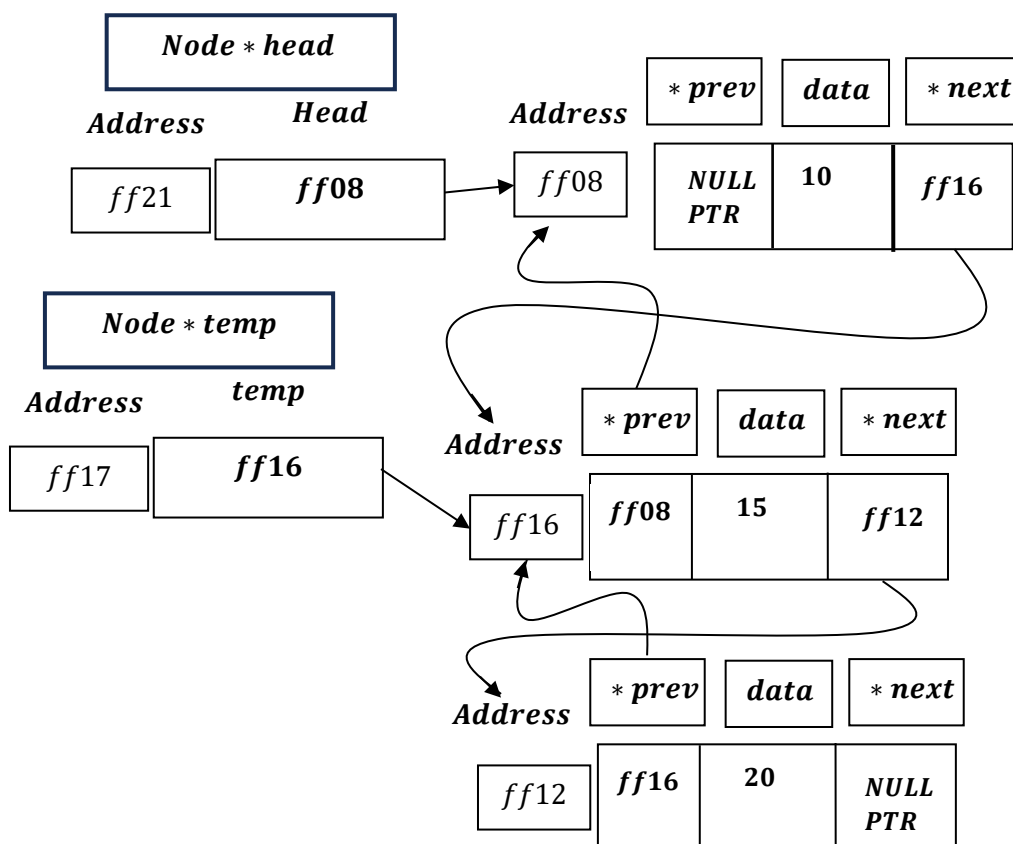


```
for (int i = 1; i < pos && temp != nullptr; i++)
{
    temp = temp->next;
}
```

Let pos = 2,

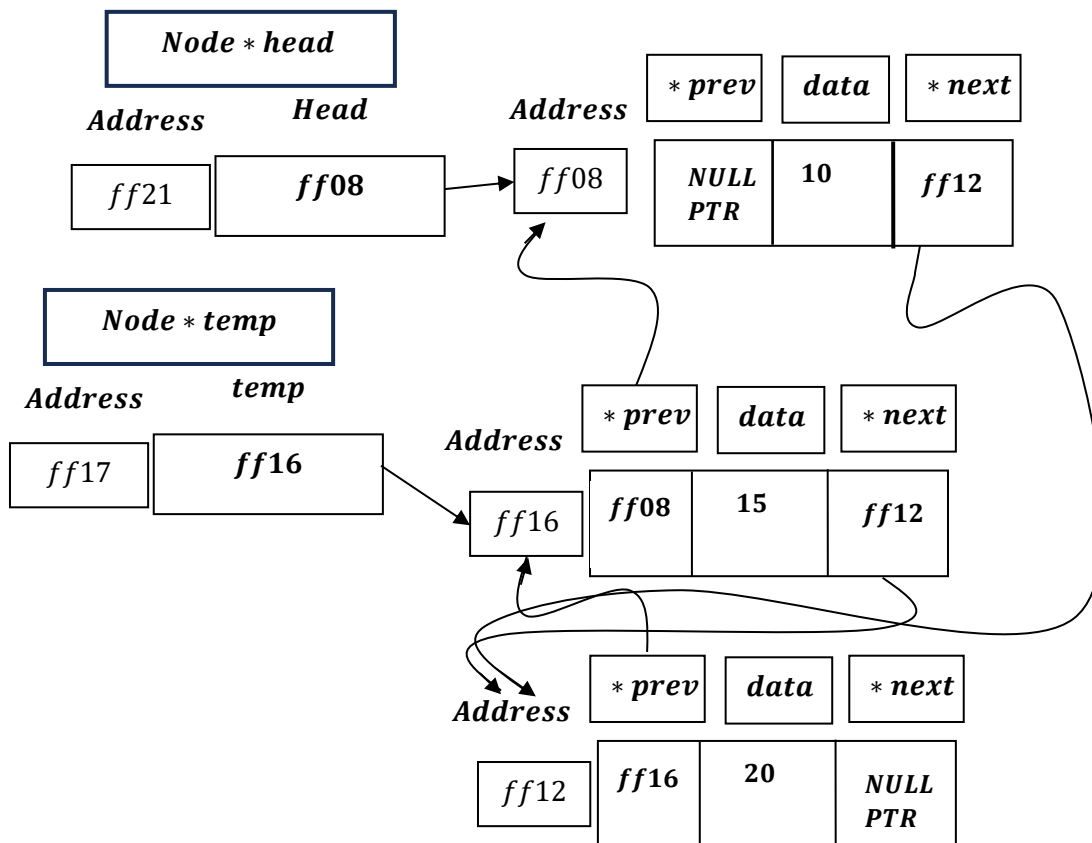
i = 1; i = 1 < pos = 2 is true and temp ≠ nullptr is true ,then :

temp = temp → next = ff16 ;



i = 2; i = 2 < pos = 2 is False and temp ≠ nullptr is true , ⇒ False Condition.

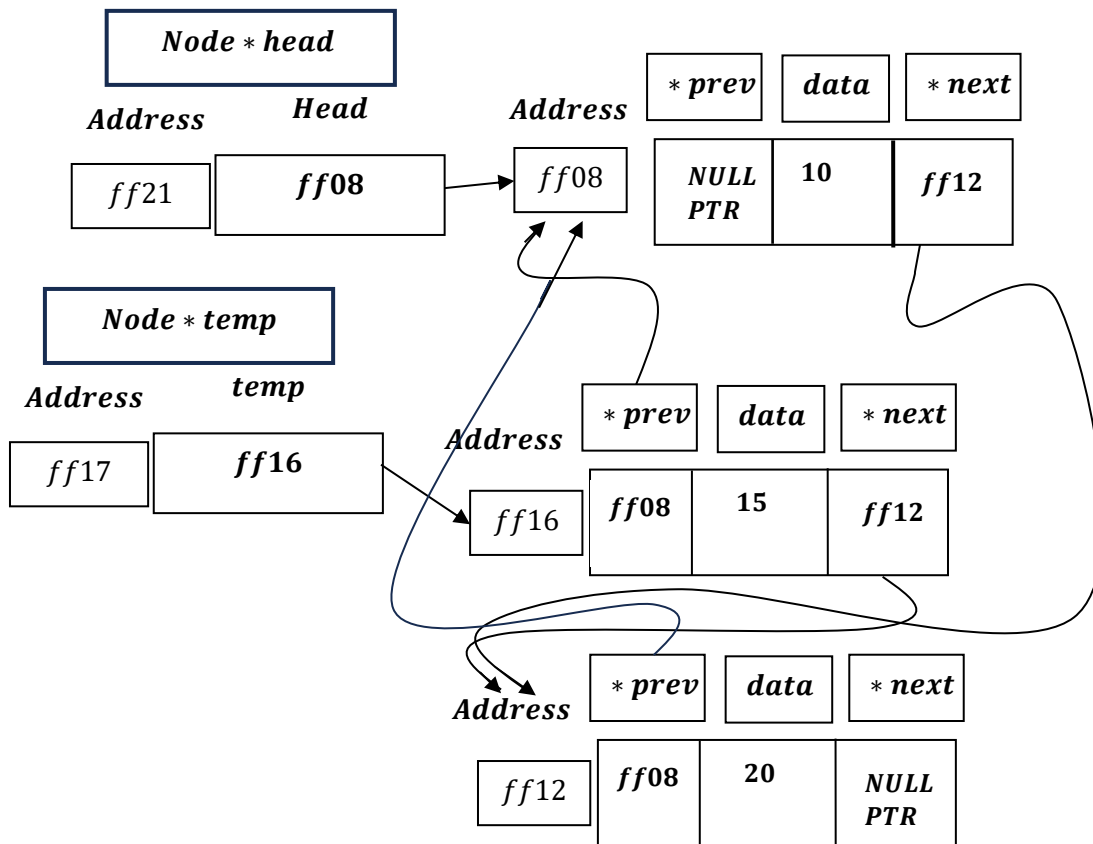
```
temp->prev->next = temp->next;
```



Next,

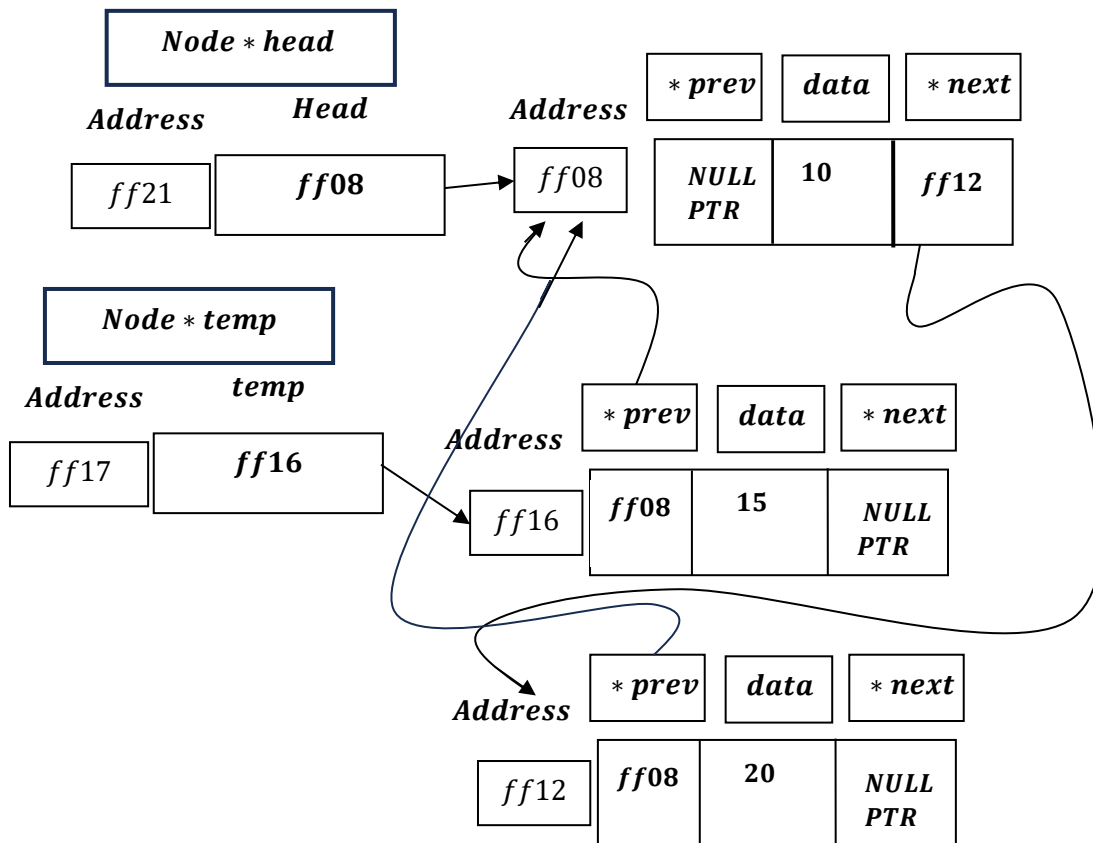
```
if (temp->next != nullptr)
{
    temp->next->prev = temp->prev;
    temp->next = nullptr;
}
```

i. e., $temp \rightarrow next \rightarrow prev = temp \rightarrow prev = ff08$;



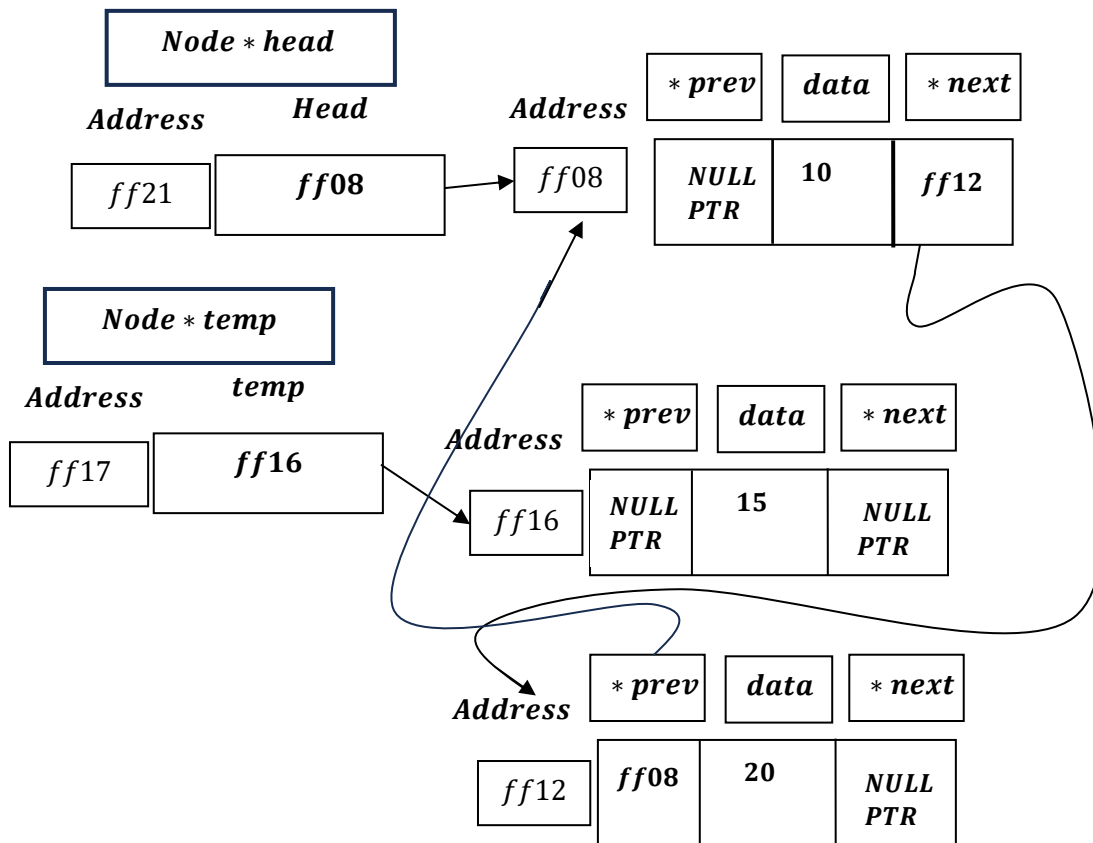
Next,

$temp \rightarrow next = nullptr;$



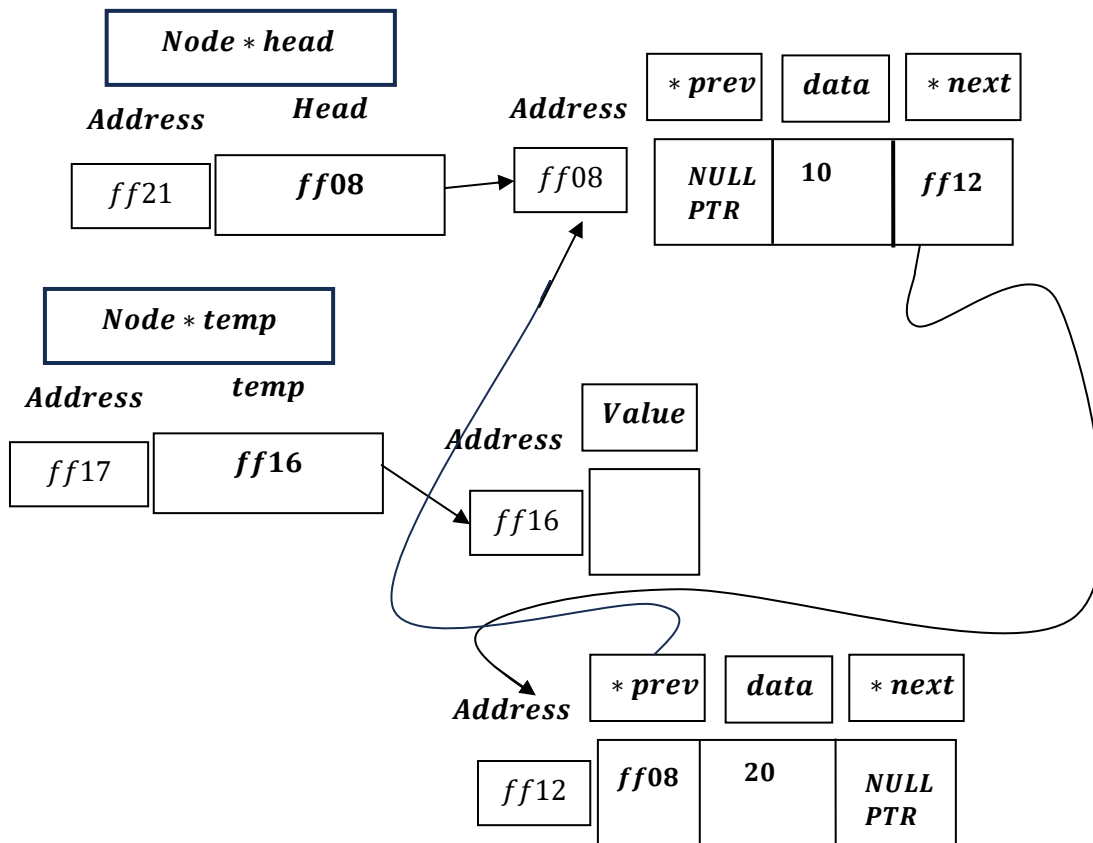
Next,

```
temp->prev = nullptr;
```



Next,

```
free(temp);
```

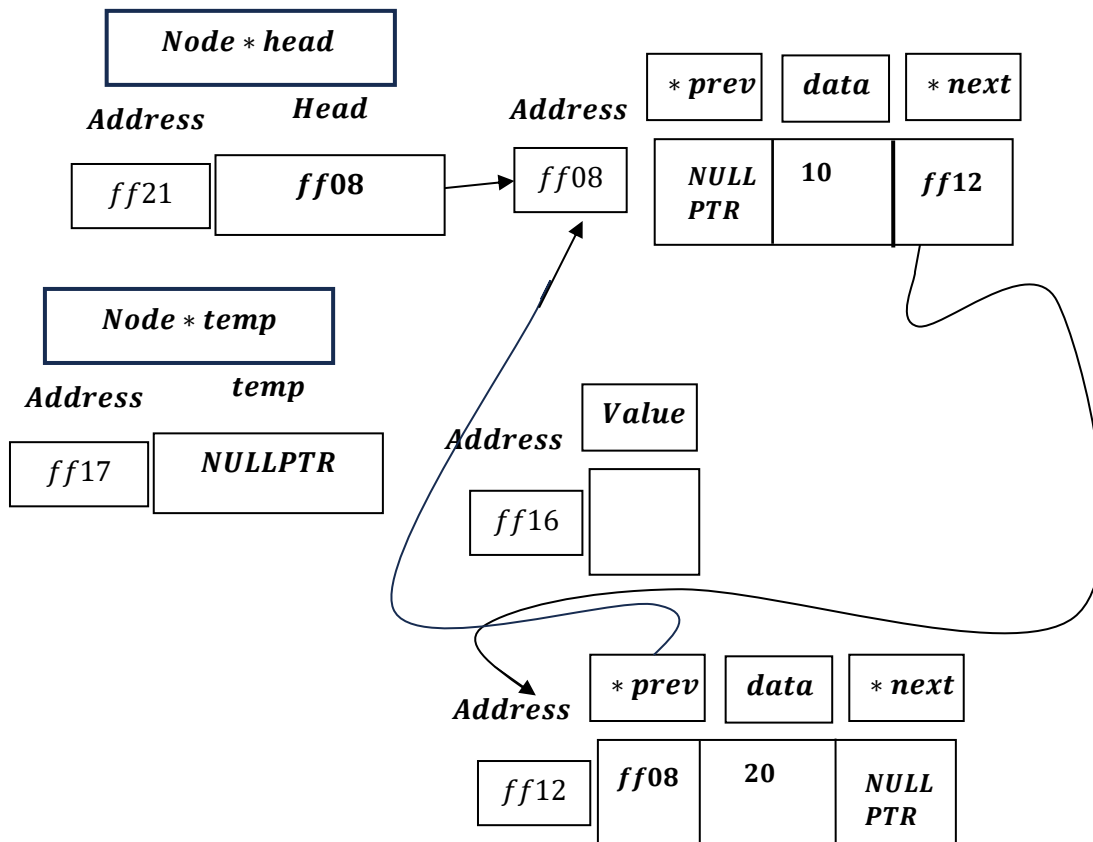


Now, temp is a dangling pointer. Now, the memory location after freeing , is not a valid memory location , hence the pointer `temp` is now a dangling pointer.

A dangling pointer is a pointer that continues to exist even after the memory it points to has been deallocated or freed. It's called dangling because it is no longer points to a valid object.

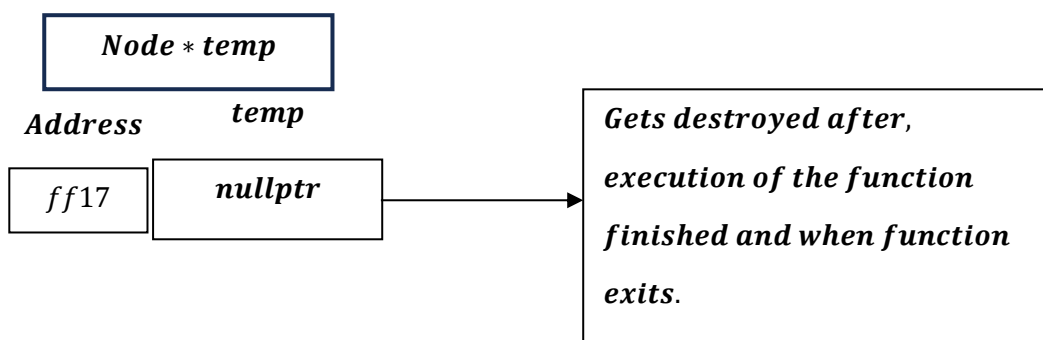
Next,

```
temp = nullptr;
```

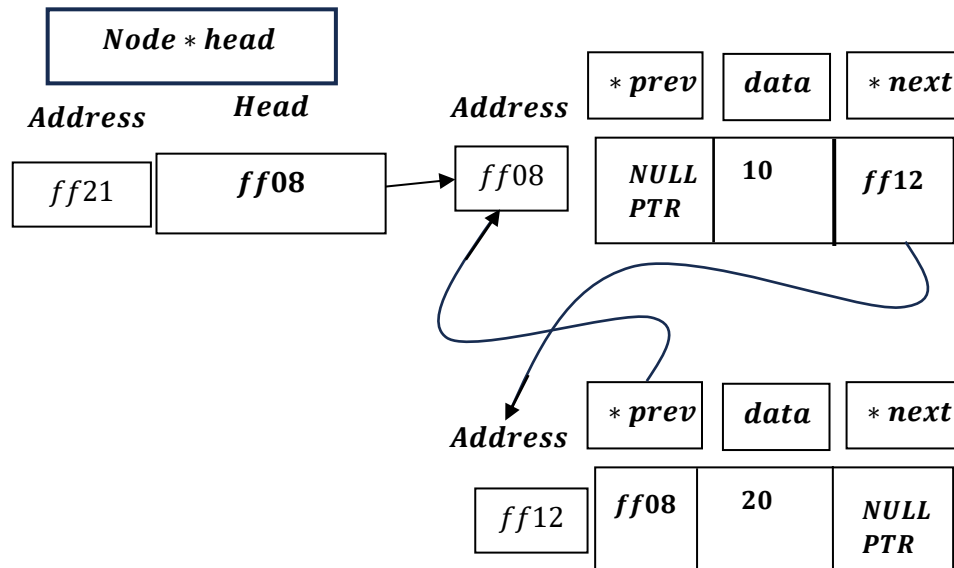


temp = nullptr; → This ensures that any future checks on the pointer (e.g., if (temp != NULLPTR)) will correctly prevent the program from trying to use the invalid memory location.

*Node * temp temporary variable, therefore gets destroyed after function execution get finished.*

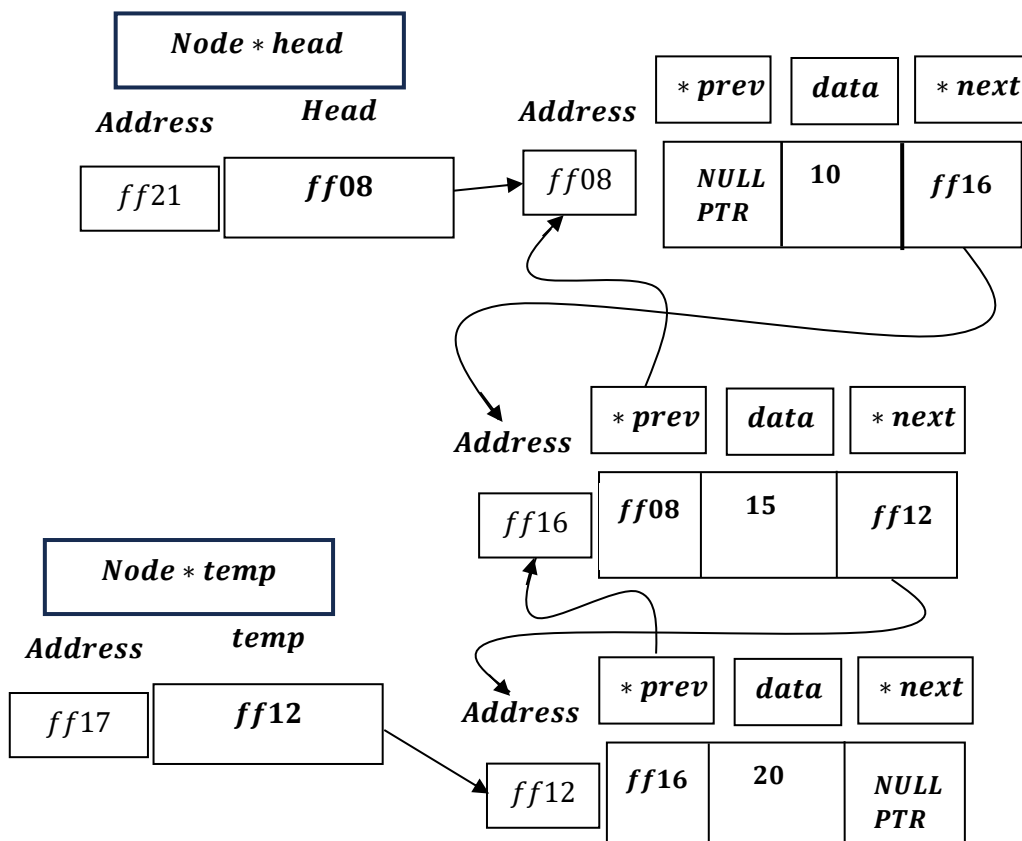


Hence , we are left with:



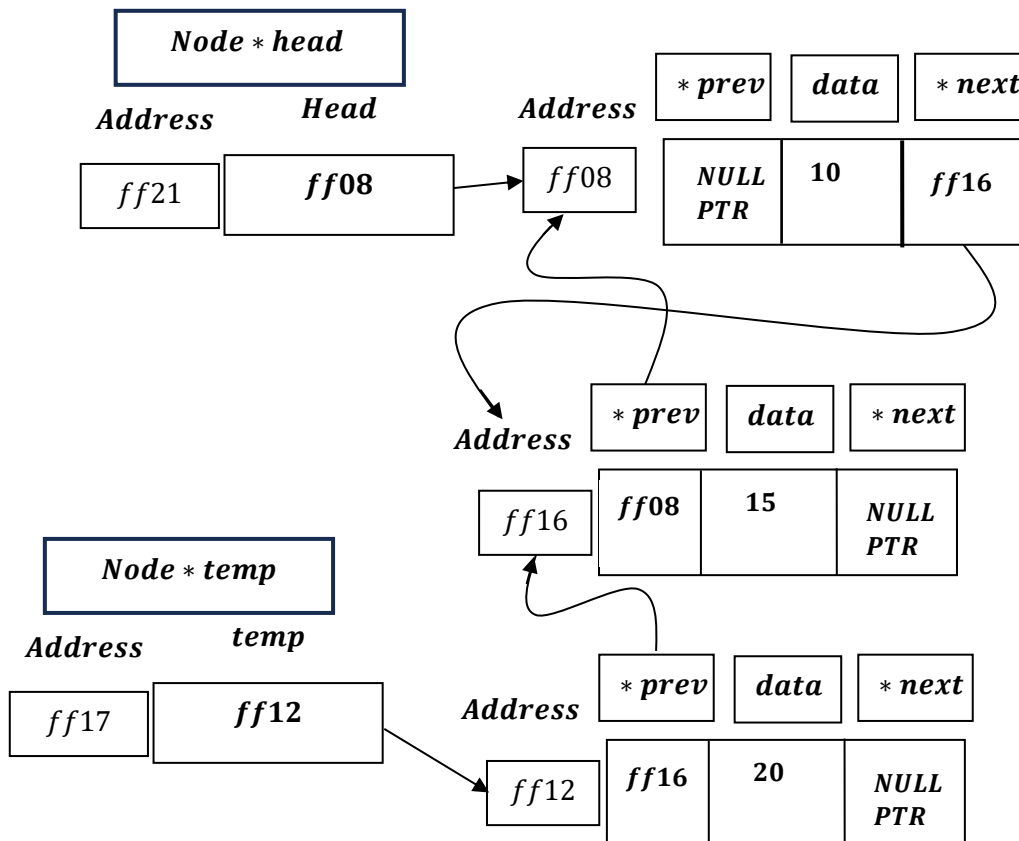
If we want to delete last node:

Let $pos = 3, i = 2; i < pos = 3$ and $temp \neq nullptr$ is true , then :



$i = 3 < pos = 3$ is false and $temp \neq nullptr$ is false, hence exit from function.

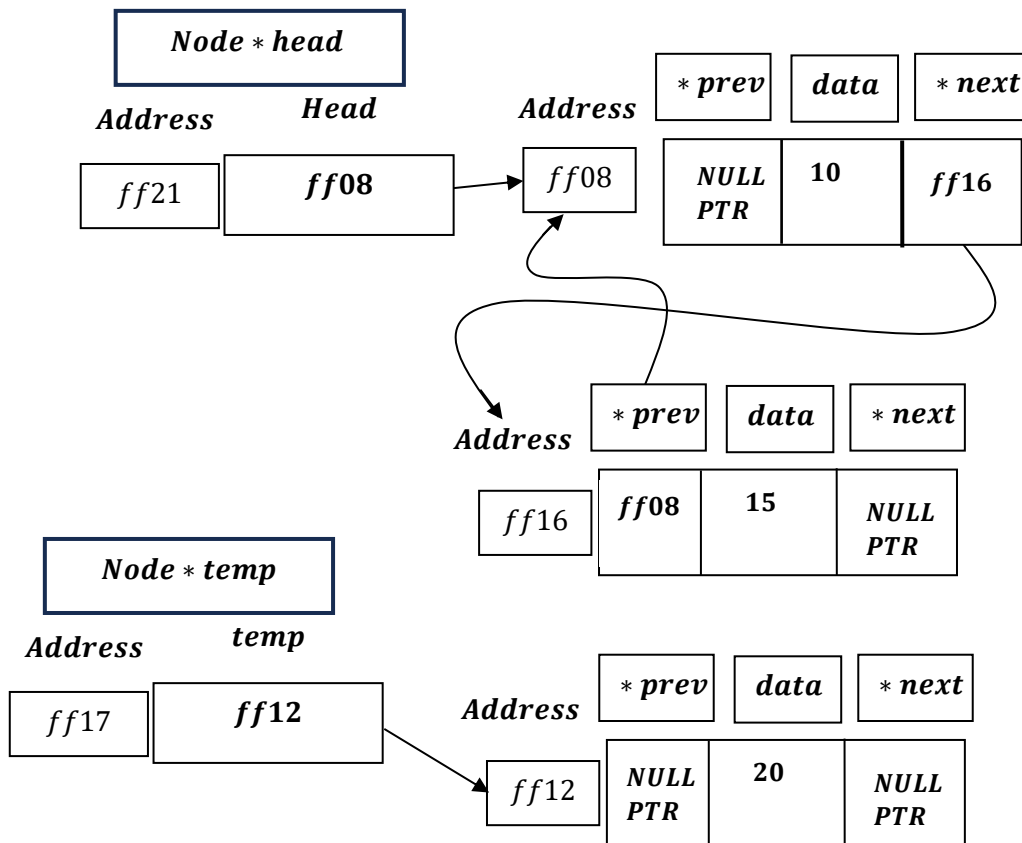
```
temp->prev->next = temp->next;
```



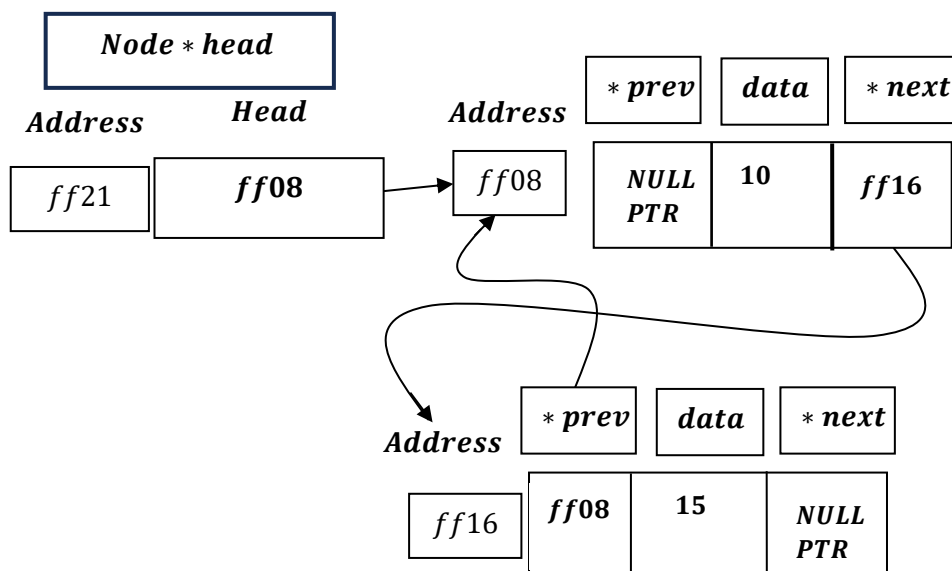
```
if (temp->next != nullptr){.....}
```

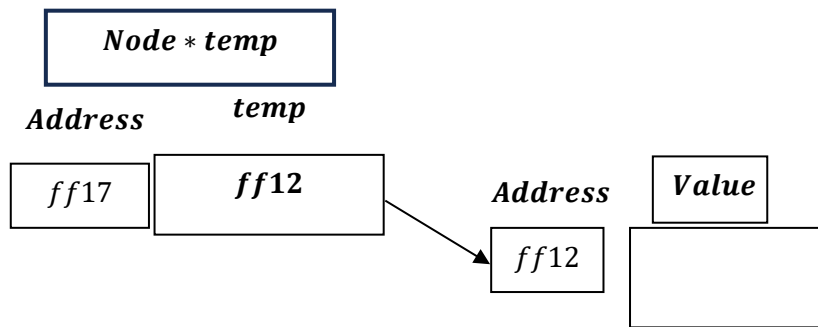
This section of if section will not run as ``temp → next != nullptr``; condition fails.

```
temp->prev = nullptr;
```

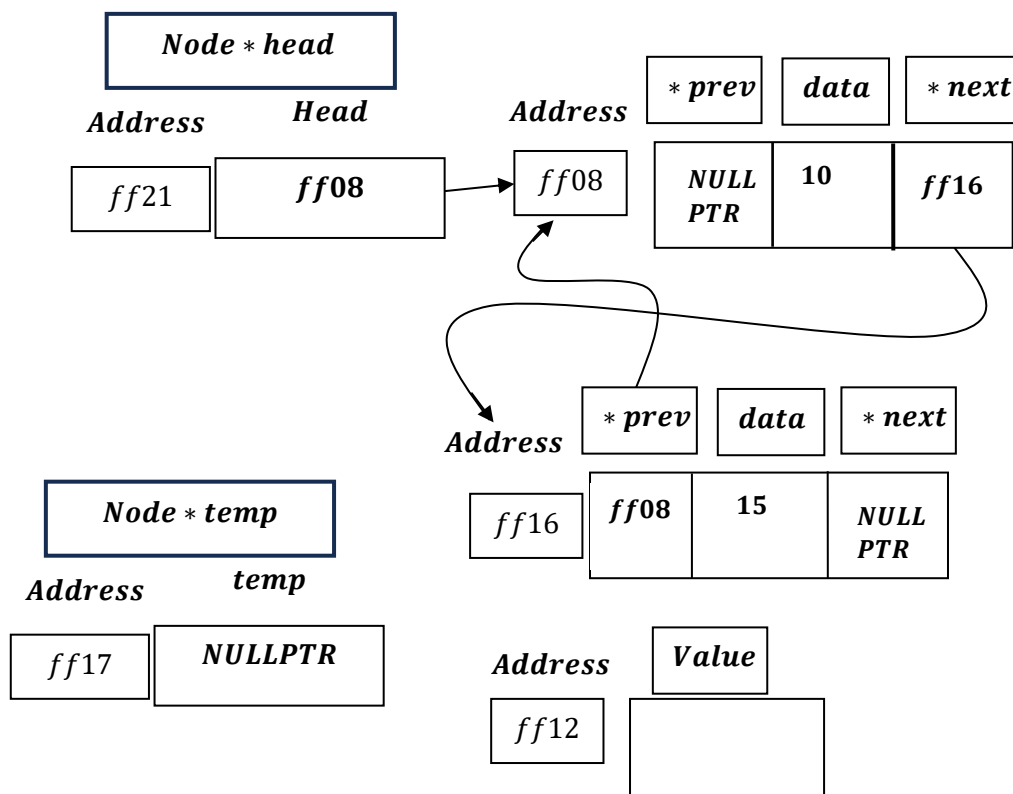


```
free(temp);
```

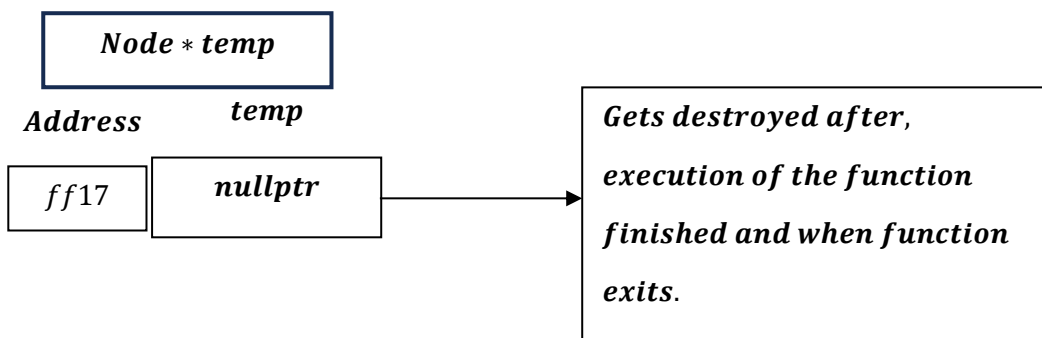




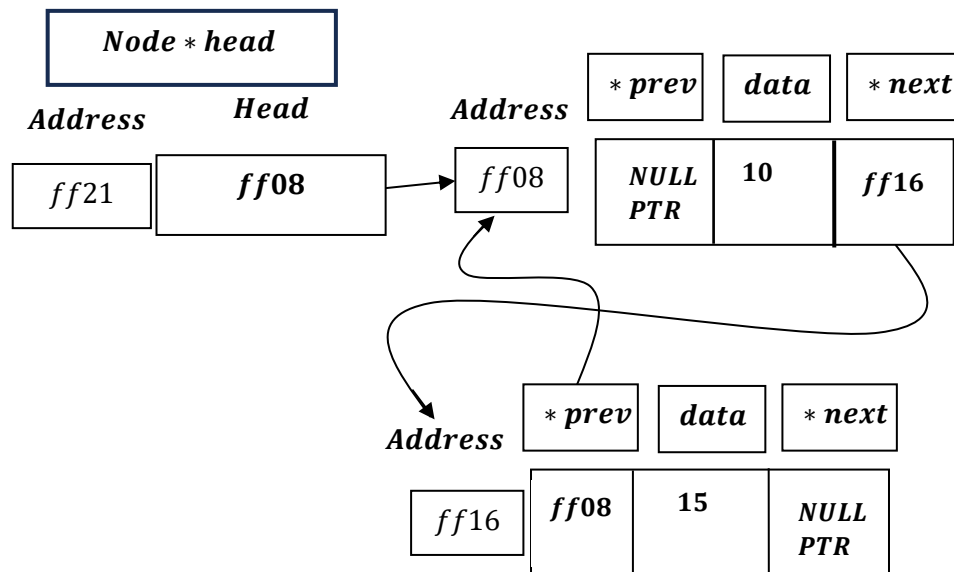
```
temp = nullptr;
```



*Node * temp temporary variable, therefore gets destroyed after function execution get finished.*



Hence ,we are left with:

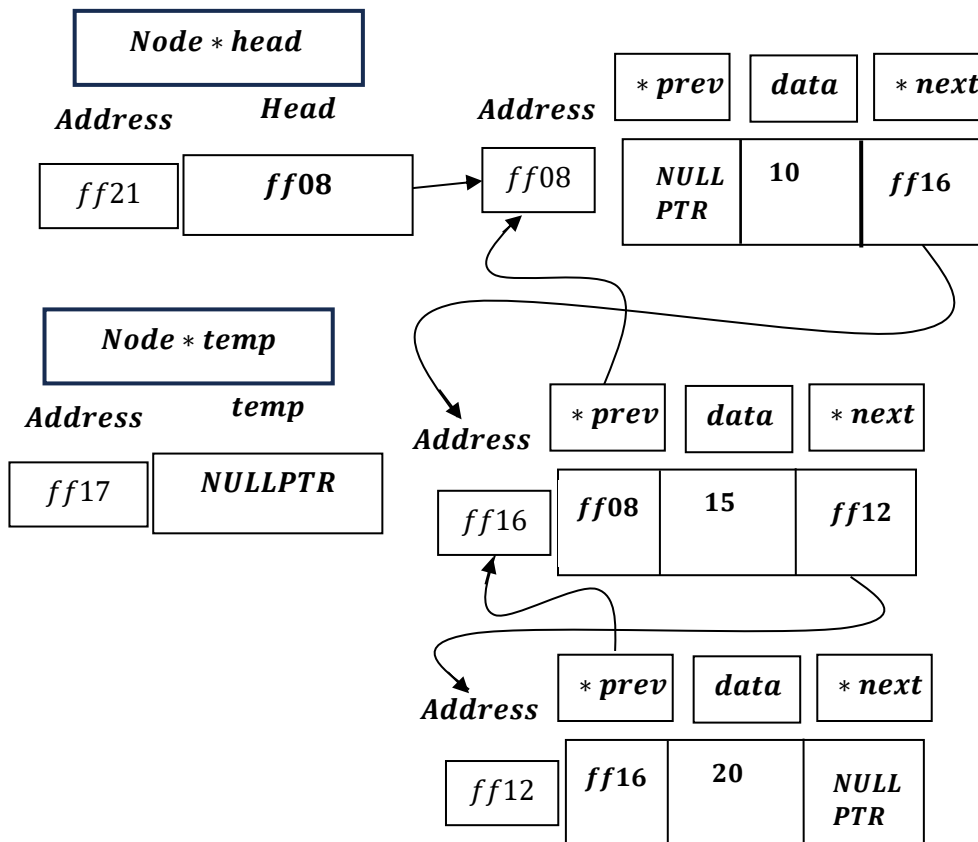


```
if (temp == nullptr)
{
    cout << "Position out of bounds!" << endl;
    return;
}
```

When this runs ?

```
for (int i = 1; i < pos && temp != nullptr; i++)
{
    temp = temp->next;
}
```

Lets say , $pos = 4$, $i = 3$; $i < pos = 4$ i.e. $i = 3$ and $temp \neq nullptr$ is true , then :



$i = 4 < pos = 4$ i.e. $i = 3$ is false and $temp \neq nullptr$ is false, hence exit from function.

And as position is given by user : ``Out of Bounds`` i.e.4.

```
if (temp == nullptr)
{
    cout << "Position out of bounds!" << endl;
    return;
}
```

The if block runs successfully and return statement exits from the function.

Doubly Linked List – Delete From Position [Working Summary]:

This function deletes a node from a specific position in a doubly linked list.

How it Works

1. Handles Edge Cases:

- It first checks if the **list is empty** (`head == nullptr`) or if the **position is invalid** (`pos < 1`).
- If the position is 1, it simply calls the `deleteFromBeginning()` function to handle the deletion and then stops.

2. Finds the Node:

- It starts a temporary pointer `temp` at the `head`.
- A `for` loop traverses the list, moving `temp` one step at a time until it reaches the desired `pos` or the end of the list.

3. Validates Position:

- After the loop, it checks if `temp` has become `nullptr`. If it has, it means the given `pos` was greater than the number of nodes in the list (out of bounds).

4. Detaches the Node:

- `temp->prev->next = temp->next`; This is the crucial first step. It tells the node *before* `temp` to skip over `temp` and point directly to the node *after* `temp`.
- `if (temp->next != nullptr)`: It checks if `temp` is not the last node.
- `temp->next->prev = temp->prev`; If it's not the last node, it tells the node *after* `temp` to point its `prev` link back to the node *before* `temp`, completing the "bypass."

5. Frees Memory:

- The links of the `temp` node (`prev` and `next`) are set to `nullptr` to fully isolate it.
 - `free(temp)` deallocates the memory for the detached node.
 - `temp = nullptr` sets the pointer to null to prevent it from becoming a dangling pointer.
-

Time Complexity of Delete From Position Function Module

```
void deleteFromPosition(int pos)
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
    if (pos < 1)
    {
        cout << "Invalid position!" << endl;
        return;
    }
    if (pos == 1)
    {
        deleteFromBeginning();
        return;
    }

    Node *temp = head;

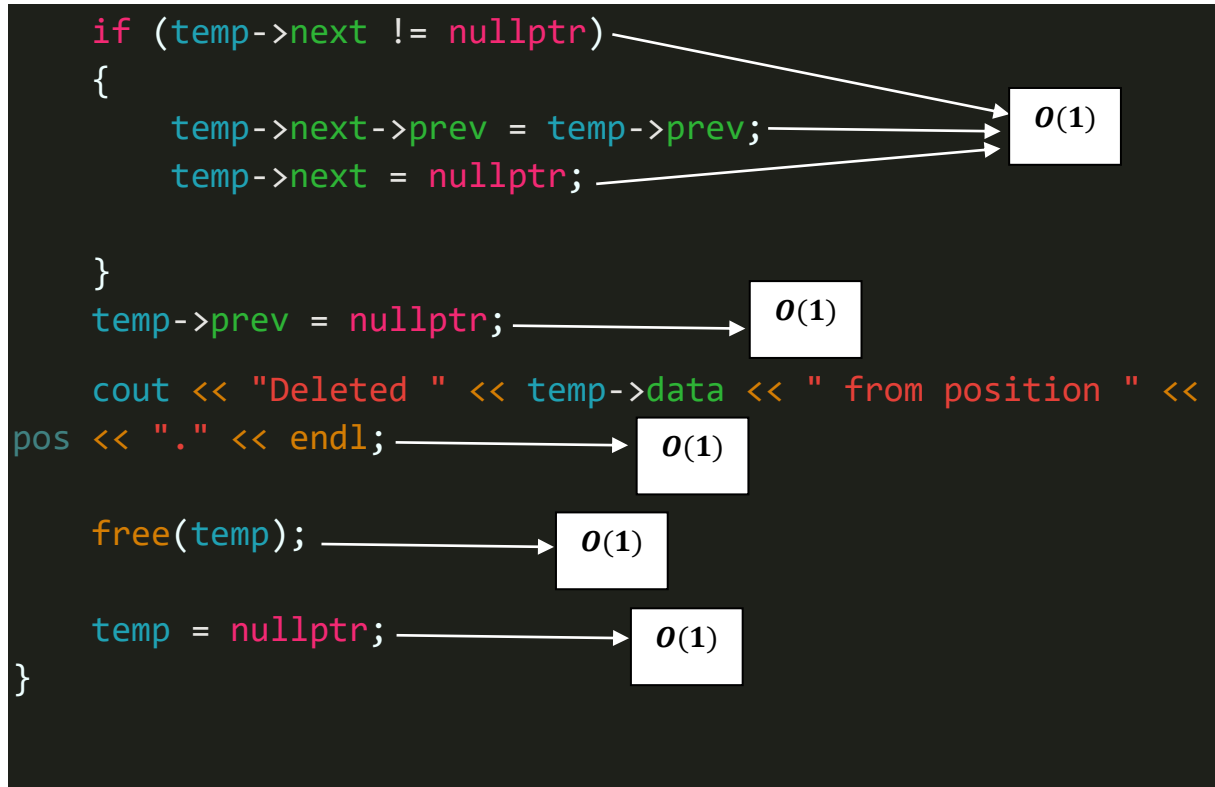
    for (int i = 1; i < pos && temp != nullptr; i++)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }

    temp->prev->next = temp->next;
}
```

Annotations for Time Complexity:

- `if (head == nullptr)` → $O(1)$
- `cout << "List is empty. Nothing to delete." << endl;` → $O(1)$
- `return;` → $O(1)$
- `if (pos < 1)` → $O(1)$
- `cout << "Invalid position!" << endl;` → $O(1)$
- `return;` → $O(1)$
- `if (pos == 1)` → $O(1)$
- `deleteFromBeginning();` → $O(1)$
- `return;` → $O(1)$
- `Node *temp = head;` → $O(1)$
- `for (int i = 1; i < pos && temp != nullptr; i++)` → $RUNS 'n - 1' times at Worst Case.$
- `temp = temp->next;` → $1 + 1 + 1 + \dots + 'n - 1' = O(n - 1) = O(n)$
- `if (temp == nullptr)` → $O(1)$
- `cout << "Position out of bounds!" << endl;` → $O(1)$
- `return;` → $O(1)$
- `temp->prev->next = temp->next;` → $O(1)$



Absolute Best Cases:

1. When List is Empty (head = NULLPTR)

→ If `head = nullptr`, this checking takes constant time. The `cout` and `return` statements take constant time. Hence overall process takes, $O(1)$ time.

2. When Pos < 1:

→ If `Pos < 1`, this checking takes constant time. The `cout` and `return` statements take constant time. Hence overall process takes, $O(1)$ time.

Not Absolute Best Case (When Pos = 1)

→ if $pos = 1$, it runs `deleteFromBeginning()` function, and function's time complexity is constant $\rightarrow O(1)$ and returns make exit from function, also takes constant $\rightarrow O(1)$, Hence total complexity is constant $\rightarrow O(1)$.

Why not Absolute Best Case?

Ans: As we know `deleteFromBeginning()` function have lots of line of codes needs to be processed, though constant but unlike $pos < 1$ and Empty List, which have less line of codes to get processed.

Worst Case

→ If $head = nullptr$ which is false, this checking takes constant time.
Hence overall process takes, $O(1)$ time.

→ If $pos < 1$ which is false, this checking takes constant time.
Hence overall process takes, $O(1)$ time.

→ if $pos = 1$ which is false, this checking takes constant time.
Hence overall process takes, $O(1)$ time.

→ Temp pointer is assigned to Head Node, which makes Temp pointer to point at memory location to which Head pointer pointing to, this process takes constant amount of time i.e. $O(1)$.

→ The loop runs from 1 and runs upto $(pos - 1)$, now if $pos = n$, then the loop runs $n - 1$ times, therefore loop statement: `temp = temp → next` runs $1 + 1 + \dots (n - 1)$ times = $O(n - 1)$ times = $O(n)$ time complexity.

→ if $temp = NULLPTR$, means during traversal it went Out of Bound and return statement makes function exit. This check takes constant amount of time i.e. $O(1)$.

→ Next pointer of Previous Node pointed by prev pointer of memory location pointed by temp pointer assigned to next pointer of memory location pointed by temp pointer i.e. assigned with `NULLPTR` making next pointer detaches to the memory location(last node), next pointer pointed to, also making previous node the last node. This assignment takes constant amount of time i.e. $O(1)$.

→ if checks `temp != NULLPTR` which is false as next pointer of last node carries `NULLPTR`, this check takes constant amount of time i.e. $O(1)$.

→ prev pointer pointed by temp pointer i.e. last node set to `NULLPTR`, hence prev pointer gets detached with previous node $(n - 1)^{th}$, this operation takes constant amount of time i.e. $O(1)$.

→ "Deleted data from position" is printed takes constant amount of time i.e. $O(1)$.

→ `free(temp)` frees the memory location that temp is pointing to, this process takes constant amount of time i.e. $O(1)$.

→ temp pointer is assigned to `nullptr` to avoid dangling pointer, this process takes constant amount of time i.e. $O(1)$.

Hence, total time complexity : $O(1) + O(1) + O(1) + O(1) + O(n) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(n)$ time complexity.

Deleted position from 2 to N, while loop runs from 1 to N – 1.

And when the position is uniformly distributed from 1 to N:

(Average Case i. e. Undetermined Position)

→ If the position is 1, the loop's inner statement runs 0 time.

→ If the position is 2, the loop's inner statement runs 1 time.

→ If the position is 3, the loop's inner statement runs 2 time.

.

.

→ If the position is (N), the loop's inner statement runs (N – 1) times.

As when $i = N < N$ becomes false , hence no execution of loop's inner statements.

$$\text{Hence, } \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}.$$

Now, as we take all the possibilites then , we take the best case also i. e.

$O(1)$, when position = 1 . Considering `T(P)` is time complexity of position `P` .

$$T(P) = \begin{cases} 1 & , \text{when } P = 1. \\ P - 1 & , \text{when } P = 2, 3, 4, \dots, N. \end{cases}$$

Now, If there is `N` no. of nodes there we can delete data at N positions.

For position 1, the probability of deletion is $\frac{1}{N}$.

For position 2, the probability of deletion is $\frac{1}{N}$.

.

.

For position N, the probability of deletion is $\frac{1}{N}$.

Hence probability of deletion will always remain $\frac{1}{N}$.

$\therefore$ Average Time Complexity is : probability of deletion $\times$ (number of possible iterations + constant time complexities for left over positions) .

$$= \left(\frac{1}{N} \times 1 + \frac{1}{N} \times 2 + \dots + \frac{1}{N} \times N - 1 \right) + \frac{1}{N} \times 1$$

$$= \frac{1}{N} \times \{(1 + 2 + 3 + \dots + N - 1) + 1\}$$

$$= \frac{1}{N} \times \left\{ \sum_{i=1}^{N-1} i + 1 \right\}$$

$$= \frac{1}{N} \times \left(\frac{N(N-1)}{2} + 1 \right)$$

$$= \frac{N-1}{2} + \frac{1}{N}$$

Applying Big Theta in the equation:

$$= \Theta\left(\frac{N-1}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

Now, taking out the dominant term of numerator and denominator:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

Now removing the constant $\frac{1}{2}$, we get:

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

When combining two Big Theta Θ terms, largest growth rate is the result:

$\rightarrow \Theta(N)$ grows linearly with N .

$\rightarrow \Theta\left(\frac{1}{N}\right)$ decreases to a constant, i.e. when N grows larger, $\frac{1}{N}$ approaches to 0.

This we can prove by limit and continuity also:

$$\lim_{n \rightarrow \infty} \left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0 \left[\frac{c}{\infty} = 0, \text{ where } c \text{ is constant} \right].$$

As $\Theta(N)$'s growth rate is higher, therefore, average case is $\Theta(N)$.

Hence, Average Time Complexity is : $\Theta(N)$.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N-1}{2} + \frac{1}{N} \leq c_2 \times N$$

Average Case Time Complexity[Most Simplest Alternative Approach]:

(Simplest approach)

For position 1, the probability of deletion is $\frac{1}{N}$.

For position 2, the probability of deletion is $\frac{1}{N}$.

.

.

For position N, the probability of deletion is $\frac{1}{N}$.

Hence probability of deletion will always remain $\frac{1}{N}$, for position 1 to N.

Average case complexity : Probability of deletion $\times$ $\left(\begin{array}{c} \text{Summation} \\ \text{of number of} \\ \text{Position.} \end{array} \right)$

$$= \frac{1}{N} \times \left(\sum_{i=1}^N i \right)$$

$$= \frac{1}{N} \times \left(\frac{N(N+1)}{2} \right)$$

$$= \frac{N+1}{2}$$

$$\text{or, } \frac{N}{2} + \frac{1}{2}$$

Appplying Big Theta in the equation:

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{2}\right)$$

Now removing the constant ' $\frac{1}{2}$ ', we get:

$$= \Theta(N) + \Theta(1)$$

$$= \Theta(N).$$

Hence, Average Time Complexity is : $\Theta\left(\frac{N+1}{2}\right)$ or $\Theta(N)$.

Now,

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N$$

<i>Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Delete At Position</i>	1. Best Case	$\Omega(1)$
	2. Worst Case	$O(n)$
	3. Average Case	$\Theta(n)$

Doubly Linked List – Delete From Position –Space Complexity and Algorithm

Program :

```
void deleteFromPosition(int pos)
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }
    if (pos < 1)
    {
        cout << "Invalid position!" << endl;
        return;
    }
    if (pos == 1)
    {
        deleteFromBeginning();
        return;
    }

    Node *temp = head;
    for (int i = 1; i < pos && temp != nullptr; i++)
    {
        temp = temp->next;
    }

    if (temp == nullptr)
    {
        cout << "Position out of bounds!" << endl;
        return;
    }

    temp->prev->next = temp->next;
```

```

    if (temp->next != nullptr)
    {
        temp->next->prev = temp->prev;
        temp->next = nullptr;
    }
    temp->prev = nullptr;

    cout << "Deleted " << temp->data << " from position " <<
pos << "." << endl;
    free(temp);
    temp = nullptr;
}

...

case 7:
    cout << "Enter position to delete: ";
    cin >> pos;
    deleteFromPosition(pos);
    break;

```

Space Complexity

Auxiliary Space

Auxiliary space refers to the extra or temporary memory used by an algorithm, not including the space taken by the inputs themselves.

In the deleteFromPosition function, the new memory usage is:

*Node *temp: A single temporary pointer is created to find and hold the address of the node to be deleted. The size of a pointer is constant.*

int i: An integer variable is used as a counter in the for loop. This also occupies a constant amount of space.

The function's main memory operation is free(temp), which deallocates memory. Since the amount of extra memory used does not grow with the size of the linked list, the auxiliary space complexity is $O(1)$.

Auxiliary Space: $O(1)$

Input Space

Input space is the memory used to store the function's inputs. This can be analyzed from two perspectives.

1. Parameter – only Definition

This view considers only the space taken by the function's direct parameters. The function signature is `void deleteFromPosition(int pos)`.

The input is a single integer, `pos`, which occupies a fixed, constant amount of memory.

Under this common definition, the input space is constant.

Input Space (Parameter-only): $O(1)$

2. Full – Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list.

If the linked list contains n nodes before the deletion, the total memory it occupies is proportional to n .

Under this definition, the input space is linear with respect to the size of the list.

Input Space (Full-Data): $O(n)$

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, the "space complexity" of a function refers to its auxiliary space. Therefore, the most common and expected answer for this function is $O(1)$. The for loop used for traversal affects the function's time complexity ($O(n)$ in the worst case), but not its space complexity.

Algorithm: Delete from a Given Position (Doubly Linked List)

DDELPOS(START, POS)

1. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

Write: "List is empty. Nothing to delete." and EXIT.

[End of If structure.]

2. [CHECK FOR INVALID POSITION]

If POS < 1, then:

Write: "Invalid position!" and EXIT.

[End of If structure.]

3. [CHECK FOR DELETION AT BEGINNING]

If POS = 1, then:

Call DDELBEG(START) [Use the beginning deletion algorithm].

Translate DDELBEG(START):

DDELBEG(START)

1. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

Write: "List is empty. Nothing to delete." and EXIT.

[End of If structure.]

2. [STORE ADDRESS OF FIRST NODE]

Set PTR := START [PTR is a pointer used to hold the node to be deleted].

3. [ADVANCE START TO THE SECOND NODE]

Set START := NEXTLINK[START].

4. [UPDATE NEW FIRST NODE, IF IT EXISTS]

If START ≠ NULL, then:

Set PREVLINK[START] := NULL (The PREV pointer of the new first node is set to NULL to unlink from previous first node).

[End of If structure.]

5. [OUTPUT THE DELETED NODE]

Write: "DELETED", INFO[PTR], "from the beginning."

6. [DEALLOCATE THE ORIGINAL FIRST NODE]

Set NEXTLINK[PTR] := NULL (The NEXT pointer of the previous first node de – links from the current and new first node)

free(PTR) [Releases the memory of the node pointed to by PTR].

7. [AVOID DANGLING POINTER]

Set PTR := NULL.

8. Exit

EXIT.

[End of If structure.]

4. [TRAVERSE TO THE NODE AT THE TARGET POSITION]

Set PTR := START [PTR is a pointer initialized to the start of the list].

FOR(Set COUNT := 1 ; COUNT < POS AND PTR ≠ NULL ; Set COUNT := COUNT + 1)DO:

SET PTR := NEXTLINK[PTR] .

[END of Loop]

5. *[CHECK IF POSITION IS OUT OF BOUNDS]*

If PTR = NULL, then:

Write: "Position out of bounds!" and EXIT.

[End of If structure.]

6. *[UNLINK THE NODE FROM THE LIST]*

a. Set NEXTLINK[PREVLINK[PTR]] := NEXTLINK[PTR] (The node before PTR now points to the node after PTR).

b. If NEXTLINK[PTR] ≠ NULL, then: (This handles cases where the node is not the last one in the list).

Set PREVLINK[NEXTLINK[PTR]] := PREVLINK[PTR] (The node after PTR now points its PREV back to the node before PTR).

[End of If structure]

7. *[DEALLOCATE THE TARGET NODE]*

Set PREVLINK[PTR] := NULL. [TARGET NODE's PREV pointer got UNLINKED]

Write : " DELETED" , INFO[PTR], "FROM POSITION", POS .

free(PTR) [Releases the memory of the node pointed to by PTR].

8. *[AVOID DANGLING POINTER]*

Set PTR := NULL .

9. *Exit*

Doubly Linked List – Search For Element And Time Complexity.

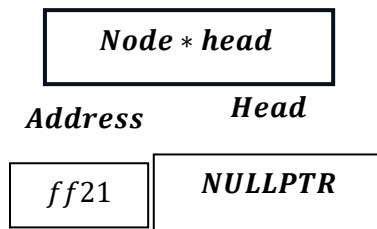
Program :

```
void searchForElement(int data)
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    Node *temp = head;
    int pos = 1;
    while (temp != nullptr)
    {
        if (temp->data == data)
        {
            cout << "Element " << data << " found at
position " << pos << "." << endl;
            return;
        }
        temp = temp->next;
        pos++;
    }
    cout << "Element " << data << " not found in the list."
<< endl;
}
```

Program Analysis

A. When LIST is Empty

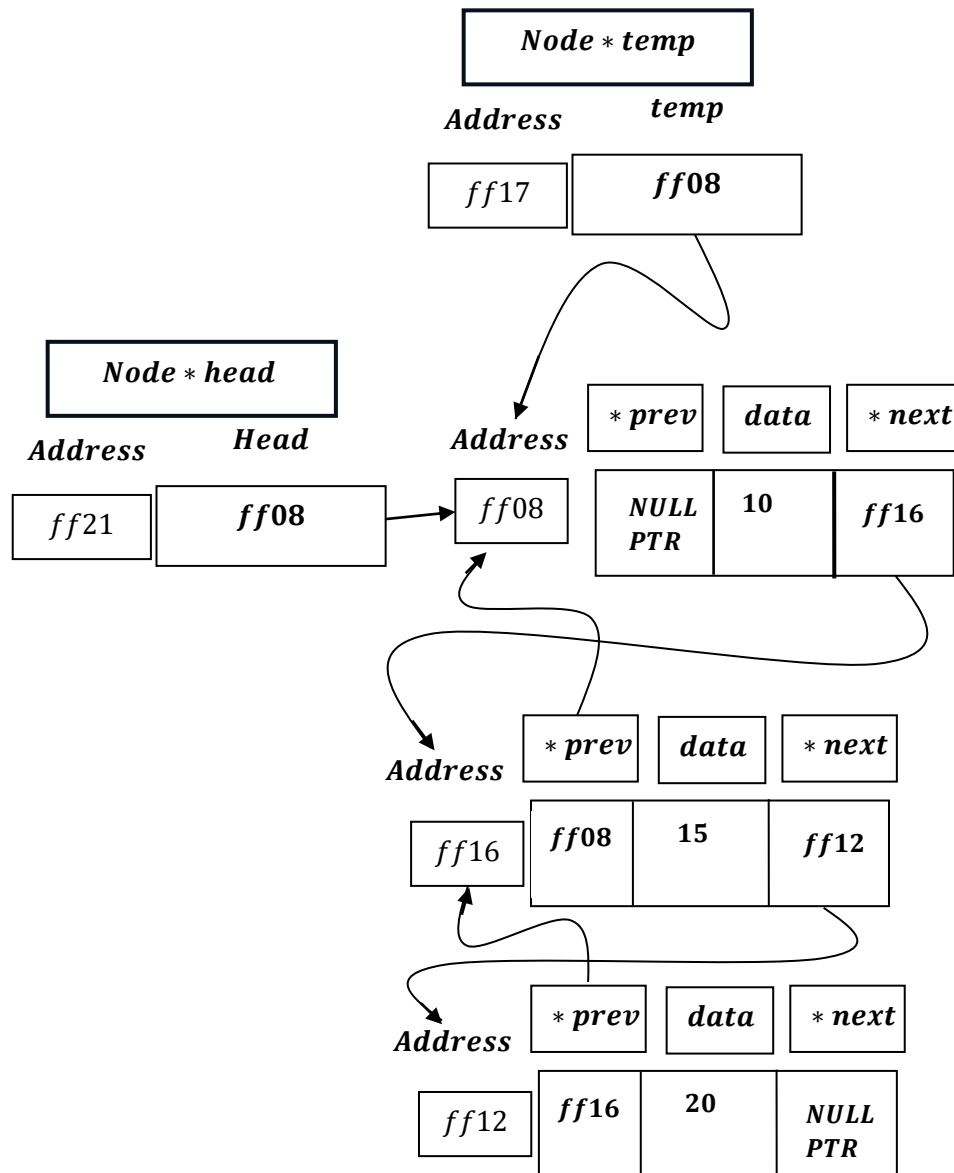
```
if (head == nullptr)
{
    cout << "List is empty." << endl;
    return;
}
```



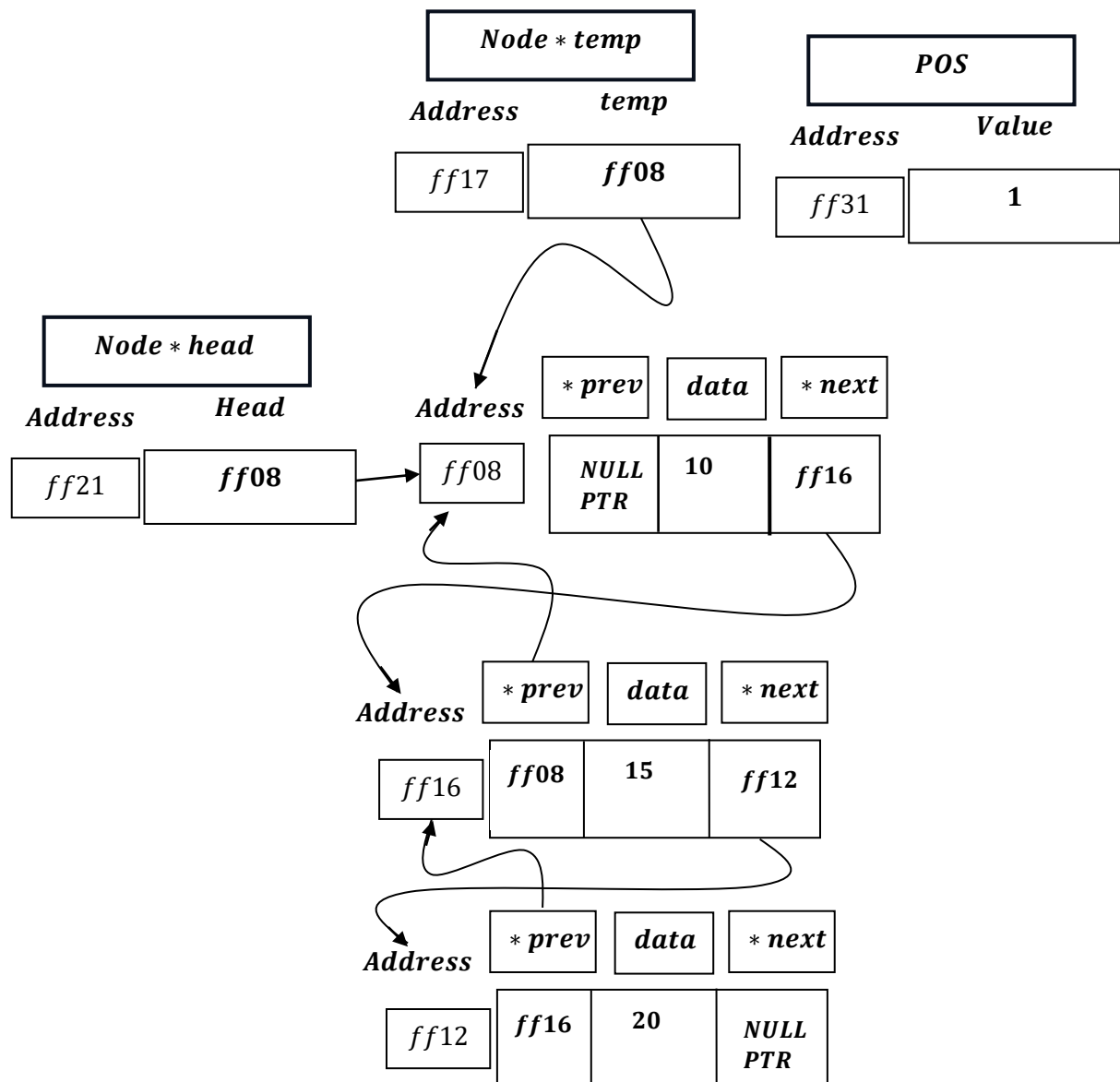
Then, it will print statement : `` List is empty.``

And, return statement will exit from the function.

```
Node *temp = head;
```



```
int pos = 1;
```



```

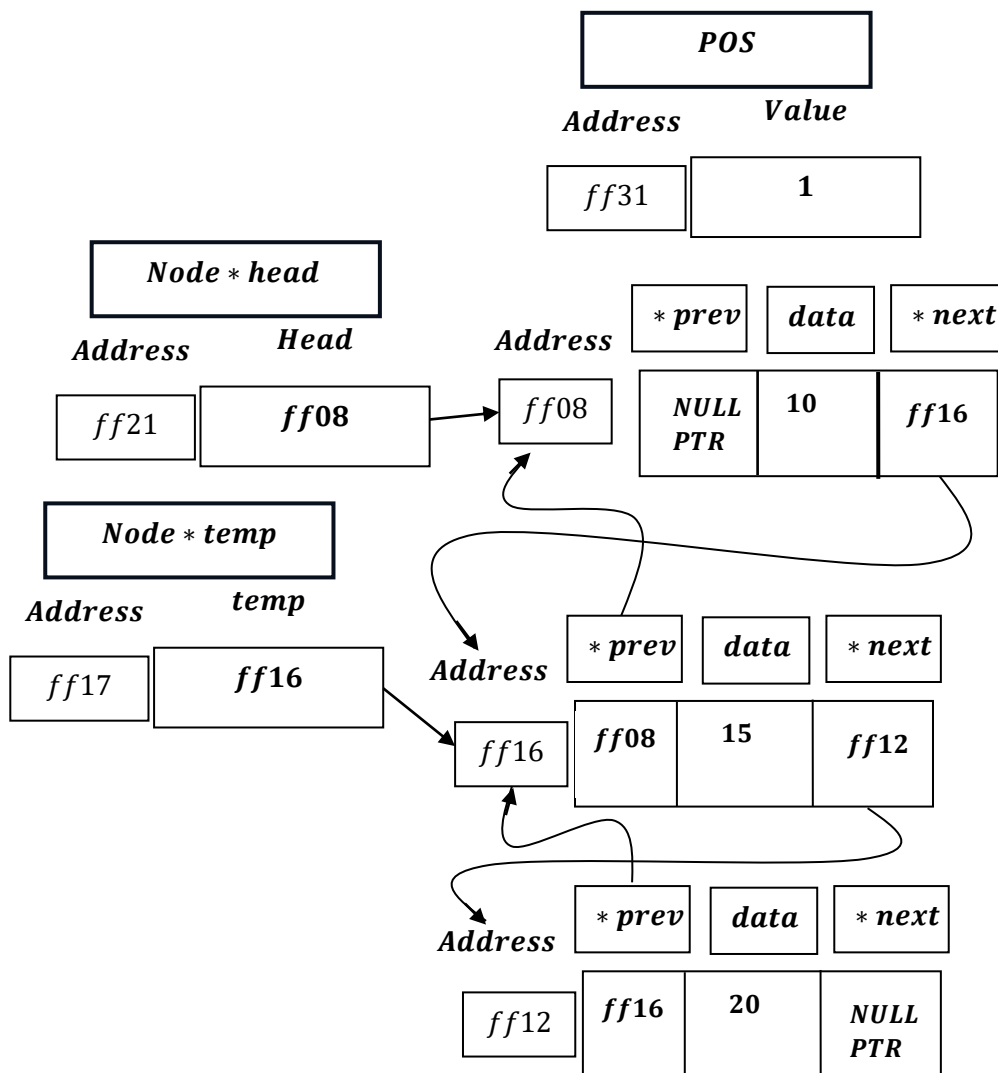
while (temp != nullptr)
{if (temp->data == data)
{
    cout << "Element " << data << " found at
position " << pos << "." << endl;
    return;
}
temp = temp->next;
pos++;
}

```


Let Data to find is : 20 ,Temp $\neq$ NULLPTR is true , hence:

$\rightarrow$ temp $\rightarrow$ data = 10 $\neq$ data = 20.

$\rightarrow$ hence, as it is false , temp = temp $\rightarrow$ next = ff16 .

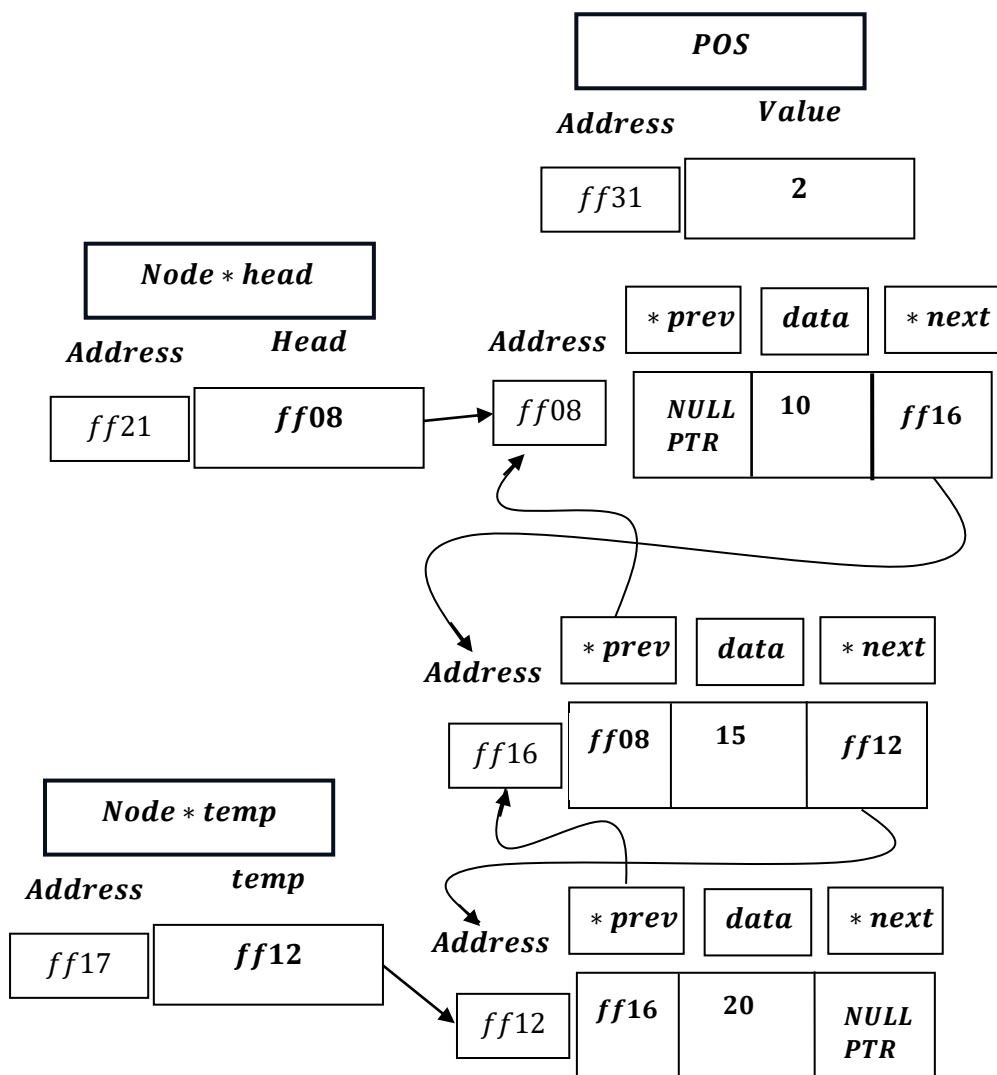


$$POS = POS + 1 = 1 + 1 = 2$$

Temp $\neq$ NULLPTR is true , hence:

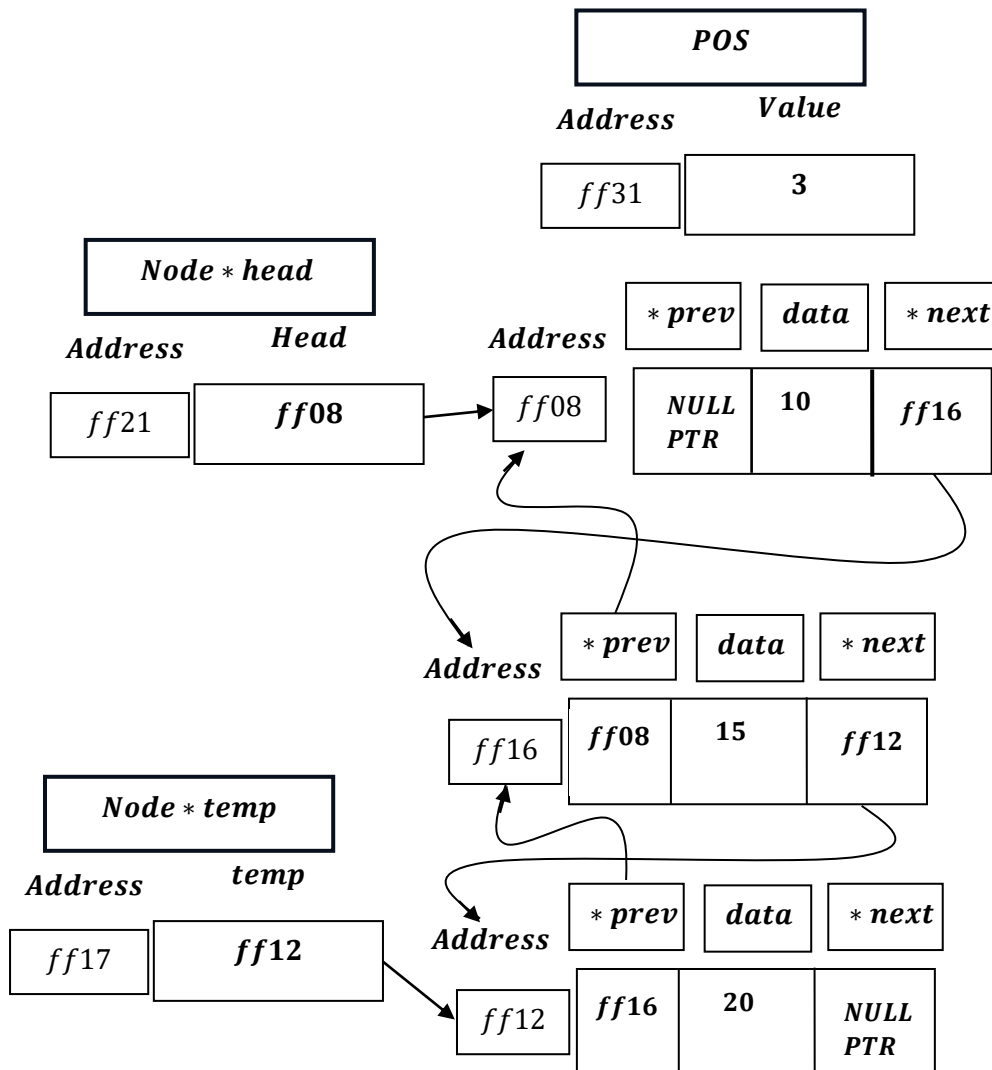
$\rightarrow$ temp $\rightarrow$ data = 15 $\neq$ data = 20.

$\rightarrow$ temp = temp $\rightarrow$ next = ff12



$$POS = POS + 1 = 2 + 1 = 3$$

Temp $\neq$ NULLPTR is true , hence:



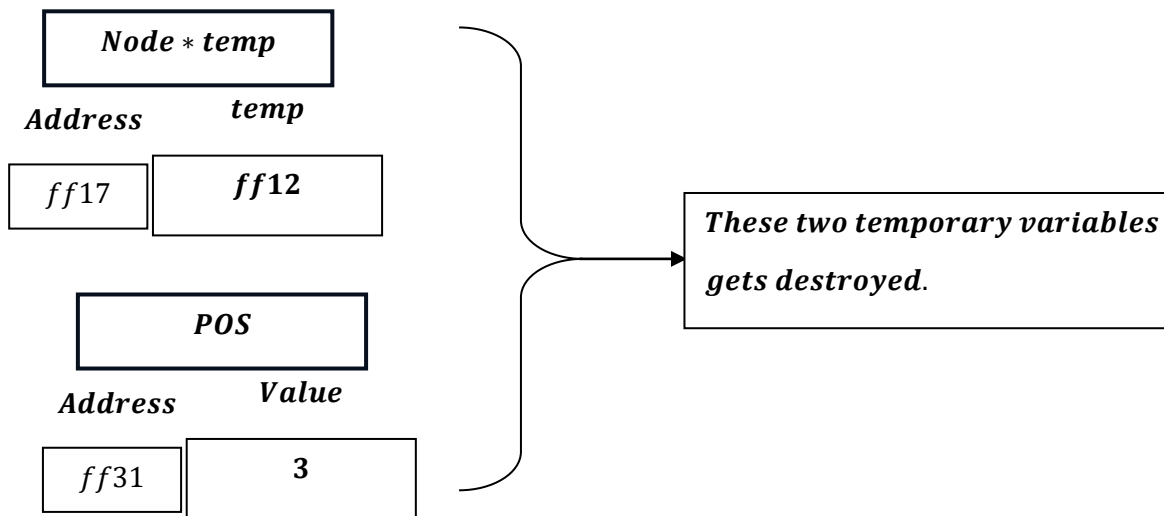
```
if (temp->data == data)
{
    cout << "Element " << data << " found at
position " << pos << "." << endl;
    return;
}
```

if temp → data = 20 = data = 20 is true:

→ It prints Element 20 found at position 3 .

→ Return statement makes exit from function.

As it exit from function Temp pointer along with pos variable gets destroyed:



Temp = NULLPTR ?

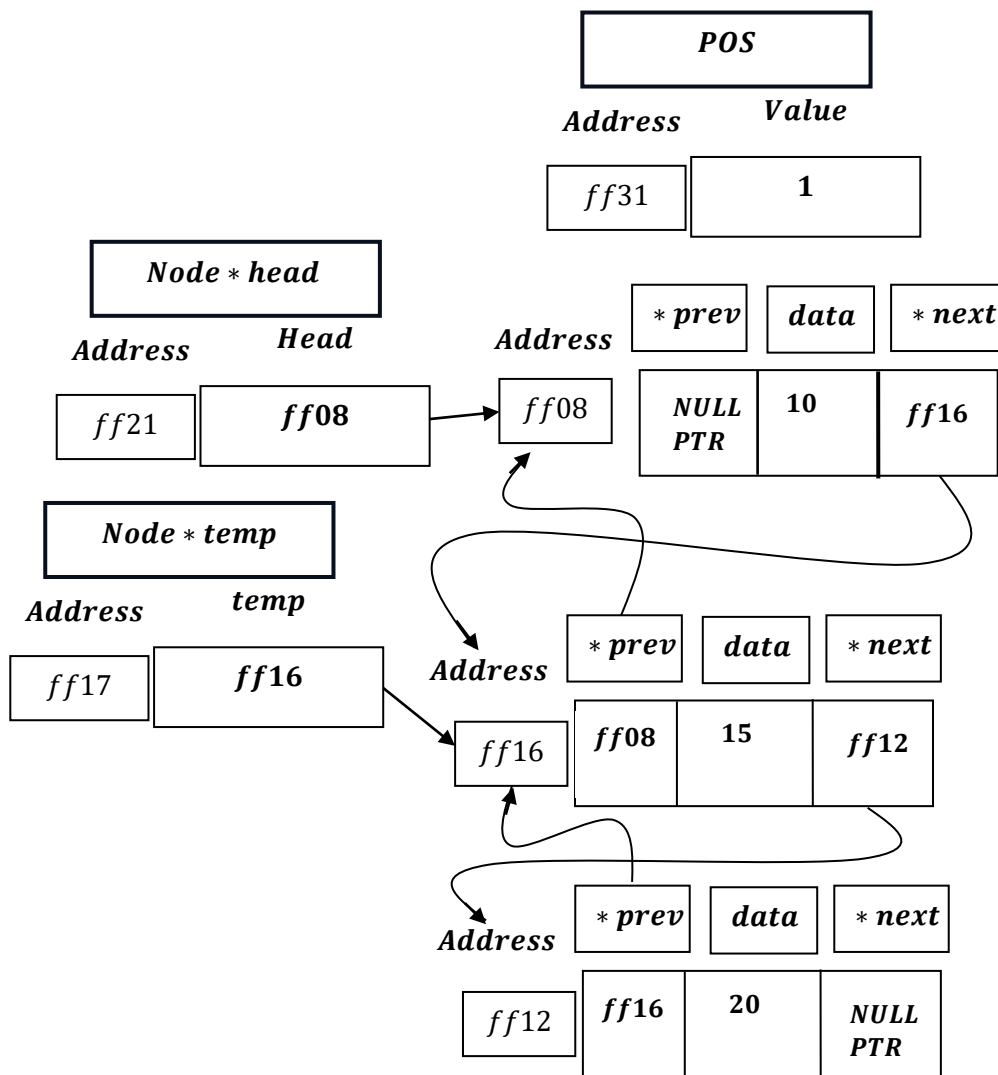
Q) When Temp = NULLPTR ? Ans: –When Data doesnot match with Data in nodes. Suppose data is 25 .

```
while (temp != nullptr)
{
    if (temp->data == data)
    {
        cout << "Element " << data << " found at
position " << pos << "." << endl;
        return;
    }
    temp = temp->next;
    pos++;
}
```

Let Data to find is : 25 ,Temp ≠ NULLPTR is true , hence:

→ temp → data = 10 ≠ data = 20.

→ hence, as it is false , temp = temp → next = ff16 .

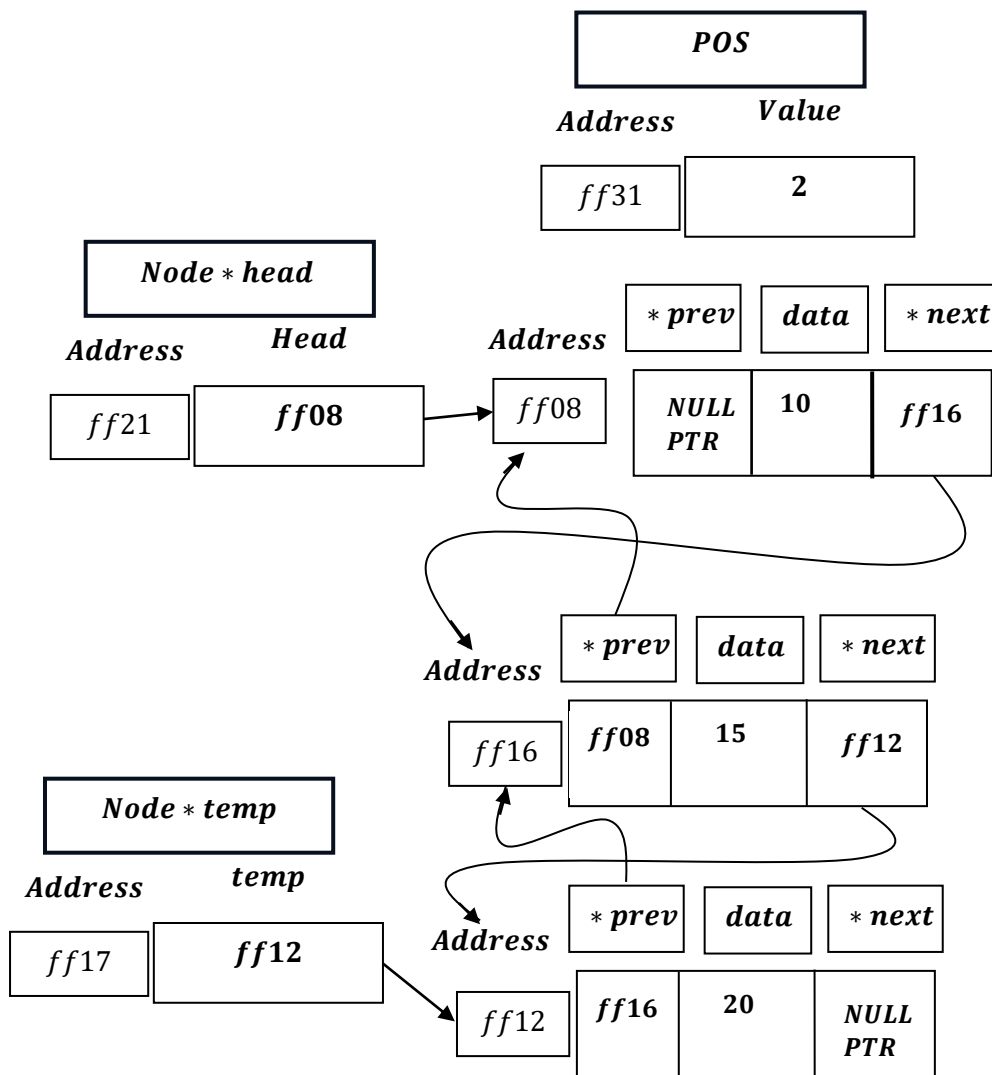


$$POS = POS + 1 = 1 + 1 = 2$$

Temp $\neq$ *NULLPTR* is true , hence:

$\rightarrow temp \rightarrow data = 15 \neq data = 25.$

$\rightarrow temp = temp \rightarrow next = ff12$

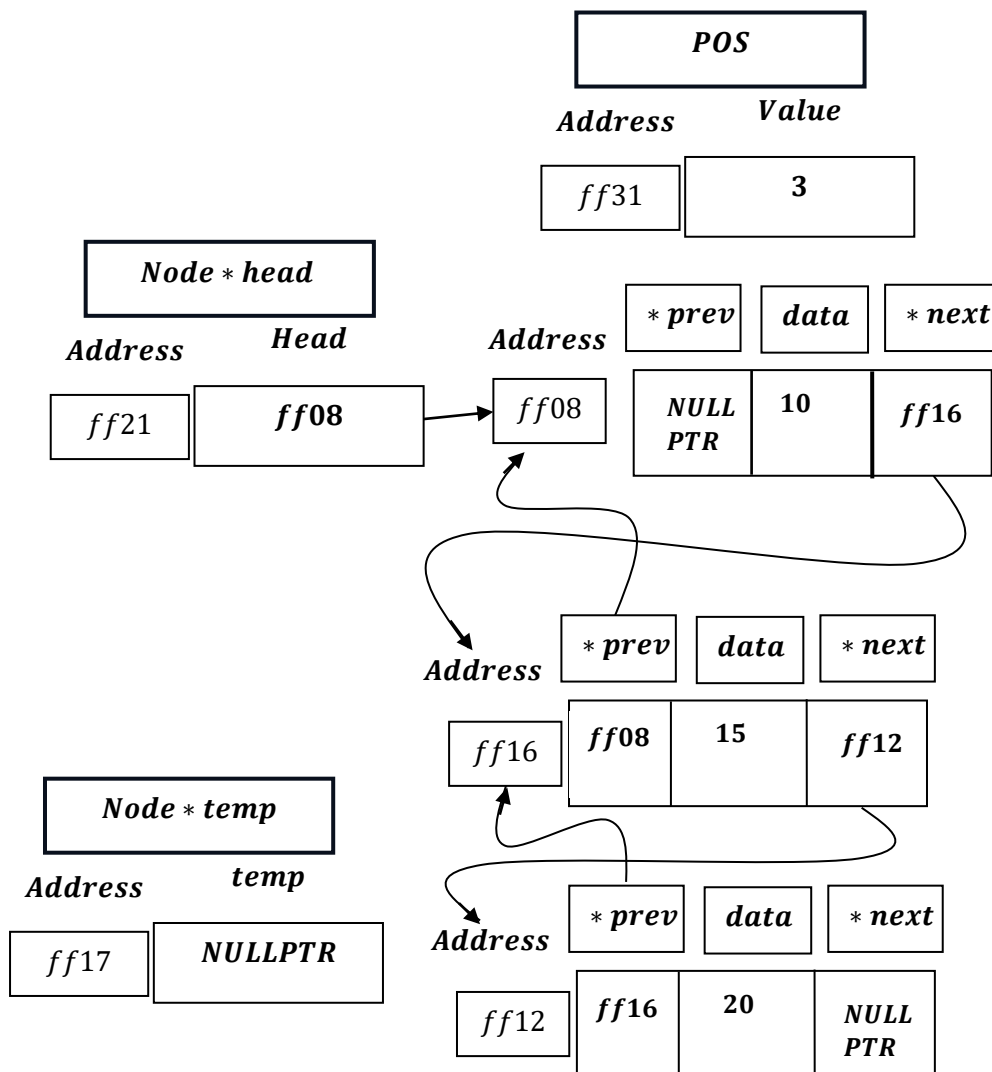


$$POS = POS + 1 = 2 + 1 = 3$$

Temp $\neq$ *NULLPTR* is true , hence:

$\rightarrow temp \rightarrow data = 20 \neq data = 25.$

$\rightarrow temp = temp \rightarrow next = NULLPTR$



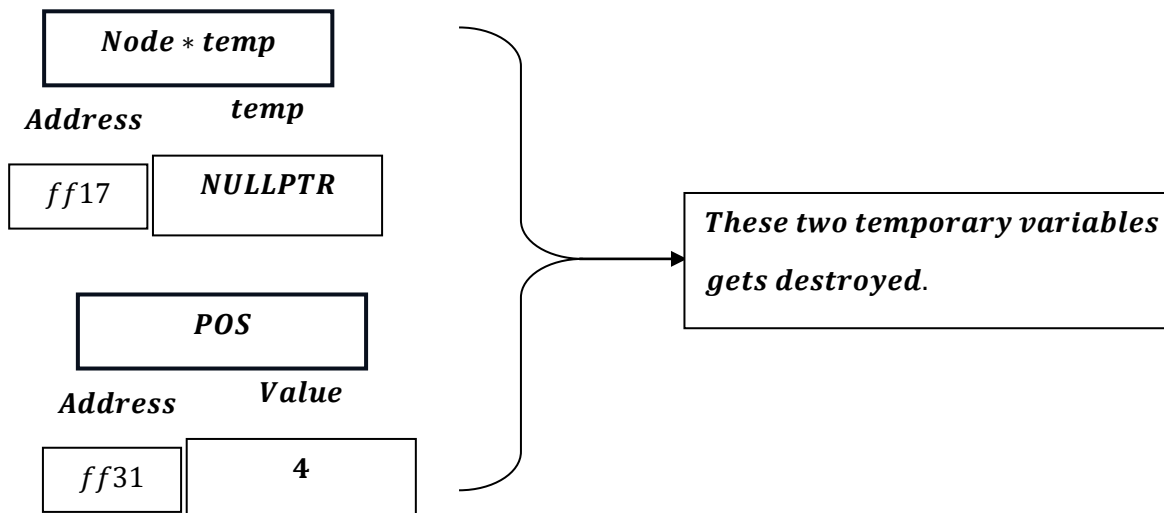
$POS = POS + 1 = 3 + 1 = 4.$

Now, $While(TEMP \neq NULLPTR)$ becomes false and it exits from loop. And, it prints:

```
cout << "Element " << data << " not found in the list."
<< endl;
```

i. e., Element 25 not found in the list.

As it exit from function Temp pointer along with pos variable gets destroyed:



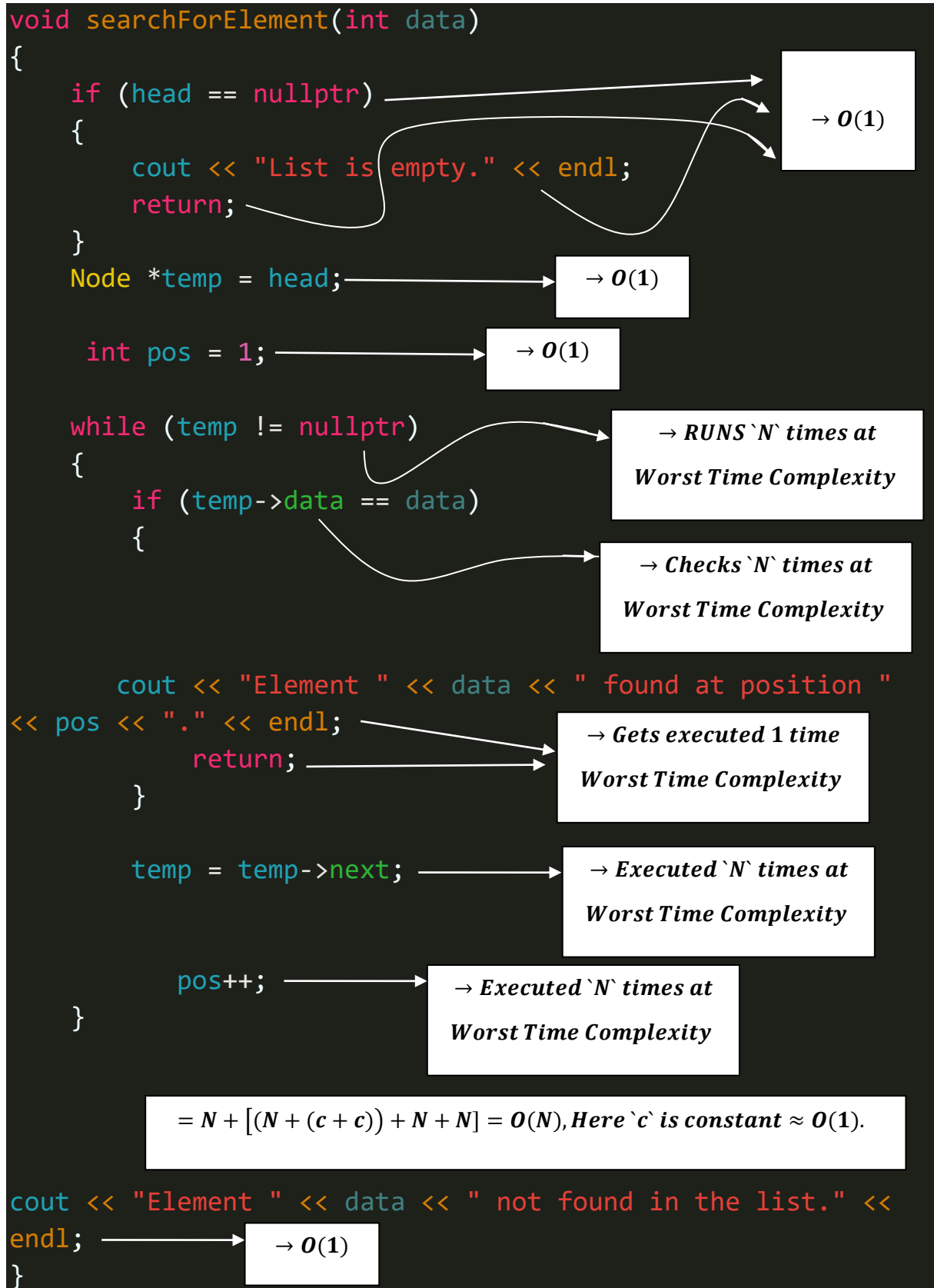
Doubly Linked List – Search For Element [Working Summary]:

This C++ function, `searchForElement`, is designed to find a specific integer within a singly linked list.

Working Summary

1. **Handles Empty List:** It first checks if the list is empty by seeing if the `head` pointer is `nullptr`. If it is, the function prints "List is empty." and exits.
 2. **Initializes Traversal:** If the list is not empty, it creates a temporary pointer `temp` that starts at the `head` of the list and initializes a position counter `pos` to 1.
 3. **Iterates Through the List:** The function enters a `while` loop that continues as long as `temp` is not `nullptr` (i.e., it hasn't reached the end of the list). In each iteration:
 - It **compares** the `data` in the current node (`temp->data`) with the `data` value being searched for.
 - **If a match is found**, it prints a success message with the element and its position, then immediately exits the function using `return`.
 - **If there's no match**, it moves to the next node (`temp = temp->next`) and increments the position counter (`pos++`).
 4. **Handles Element Not Found:** If the `while` loop finishes without finding the element (meaning `temp` becomes `nullptr`), the function prints a message indicating that the element was not found in the list.
-

Time Complexity of Search For Element Function Module



Absolute Best Case:

When List is Empty (head = NULLPTR)

→ If head = nullptr, this checking takes constant time. The cout and return statements takes constant time. Hence overall process takes, $O(1)$ time.

Worst Case (When Element Not Found)

→ If head = nullptr which is false, this checking takes constant time. Hence overall process takes, $O(1)$ time.

→ Temp pointer is assigned to Head Node, which makes Temp pointer to point at memory location to which Head pointer pointing to, this process takes constant amount of time i.e. $O(1)$.

→ integer pos variable assigned to 1. This process takes constant amount of time i.e. $O(1)$.

→ While loop condition : temp $\neq$ NULLPTR checks `N` times:

→ if condition checks data of the memory location pointed by temp pointer equals to user input data, this check takes `N` times. As condition is: ``Element not Found``, the condition never becomes true hence if block statement never runs.

→ Temp = Temp → next; runs N times.

→ POS ++ i.e. POS increment runs N times.

Hence total time complexity = $N + (N + N + N) = N + (3N) = O(N) + 3 O(N) = O(N)$.

→ Next, it prints `` Element [data] not found in the list. `` , this process takes constant amount of time i.e. $O(1)$.

Hence ,total time complexity : $O(1) + O(1) + O(1) + O(N) + O(1) = O(N)$.

Average Case

(If the elements in the linked list are uniformly distributed and the Element/key is equally likely to be any element in the list)

–if key is found at 1st Node , loop runs 1 time .

–if key is found at 2nd Node , loop runs 2 time .

.

.

–if key is found at Nth Node, loop runs N times.

As , the key is equally likely to be any element in the list,

probablity of finding 1st element (p_1), if key equals to 1st element = $\frac{1}{N}$.

probablity of finding 2nd element (p_2), if key equals to 2nd element = $\frac{1}{N}$.

.

.

probablity of finding Nth element (p_2), if key equals to Nth element = $\frac{1}{N}$.

Therefore,

$$\text{Probability}(P) = p_1 = p_2 = \dots = p_n = \frac{1}{N}.$$

Therefore, Average Case = Number of times the loops executes × probability of finding elements.

That is the average number of executions is the sum of all possible executions weighted by their probabilities. This is given by:

$$\begin{aligned}\therefore \text{Average Case} &= \frac{1}{N} \times \left(\sum_{i=1}^N i \right) \\ &= \frac{1}{N} \times \frac{N(N+1)}{2} \left[\sum_{i=1}^N i = \frac{N(N+1)}{2} \right] \\ &= \frac{N+1}{2} \\ &= \frac{N}{2} + \frac{1}{2}\end{aligned}$$

Applying , Big Theta Θ we get :

$$\begin{aligned}&= \frac{1}{2} \times (\Theta(N) + \Theta(1)) \\ &= \Theta(N) + \Theta(1) \\ &= \Theta(N).\end{aligned}$$

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N+1}{2} \leq c_2 \times N$$

<i>Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Search Element For Element</i>	<i>1. Best Case</i>	$\Omega(1)$
	<i>2. Worst Case</i>	$O(n)$
	<i>3. Average Case</i>	$\Theta(n)$

Doubly Linked List – Search For Element –Space Complexity and Algorithm

Program :

```
void searchForElement(int data)
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    Node *temp = head;
    int pos = 1;
    while (temp != nullptr)
    {
        if (temp->data == data)
        {
            cout << "Element " << data << " found at
position " << pos << "." << endl;
            return;
        }
        temp = temp->next;
        pos++;
    }
    cout << "Element " << data << " not found in the list."
<< endl;
}
```

Space Complexity

Auxiliary Space

Auxiliary space is the extra memory used by a function, excluding the space taken by the inputs themselves.

In the searchForElement function, the new variables created are:

Node *temp: A single temporary pointer to traverse the list. The size of a pointer is constant.

int pos: An integer variable to keep track of the current position. The size of an integer is also constant.

The function doesn't allocate any new nodes. Since the amount of extra memory used is fixed, the auxiliary space complexity is $O(1)$.

Auxiliary Space: $O(1)$

Input Space

Input space is the memory used to store the function's inputs. This can be analyzed from two perspectives.

1. Parameter – only Definition

This view considers only the space taken by the function's direct parameters. The function signature is `void searchForElement(int data)`.

The input is a single integer, data, which occupies a fixed, constant amount of memory.

Under this common definition, the input space is constant.

Input Space (Parameter-only): $O(1)$

2. Full – Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list.

If the linked list contains n nodes, the total memory it occupies is proportional to n .

Under this definition, the input space is linear with respect to the size of the list.

Input Space (Full-Data): $O(n)$

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, the "space complexity" of a function refers to its auxiliary space. Therefore, the most common and expected answer for this function is $O(1)$. The while loop traversal affects the function's time complexity ($O(n)$ in the worst case), but not its space complexity.

Algorithm: Search for an Element (Doubly Linked List)

DSEARCH(START, ITEM)

1. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

Write: "List is empty." and EXIT.

[End of If structure.]

2. [INITIALIZE POINTERS FOR TRAVERSAL]

a. Set PTR := START [PTR is a pointer used to traverse the list].

b. Set POS := 1 [POS is an integer to track the node position].

3. [TRAVERSE THE LIST TO FIND THE ITEM]

Repeat while PTR $\neq$ NULL:

a. If INFO[PTR] = ITEM, then: [Check if the current node contains the item].

Write: "Element ", ITEM, " found at position ", POS.

EXIT. [End the search as the item is found].

[End of If structure]

b. Set PTR := NEXTLINK[PTR] [Move to the next node].

c. Set POS := POS + 1 [Increment the position counter].

[END of Loop]

4. [REPORT IF ITEM WAS NOT FOUND]

Write: "Element ", ITEM, " not found in the list." [This line is only reached if the loop finishes].

5. Exit

Doubly Linked List – Length Of Linked List And Time Complexity.

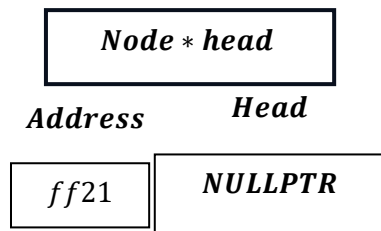
Program :

```
int length()
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return 0;
    }
    int count = 0;
    Node *temp = head;
    while (temp != nullptr)
    {
        count++;
        temp = temp->next;
    }
    return count;
}
...
case 9:
cout << "Length of the list is: " << length() << endl;
break;
```

Program Analysis

A. When LIST is Empty

```
if (head == nullptr)
{
    cout << "List is empty." << endl;
    return 0;
}
```

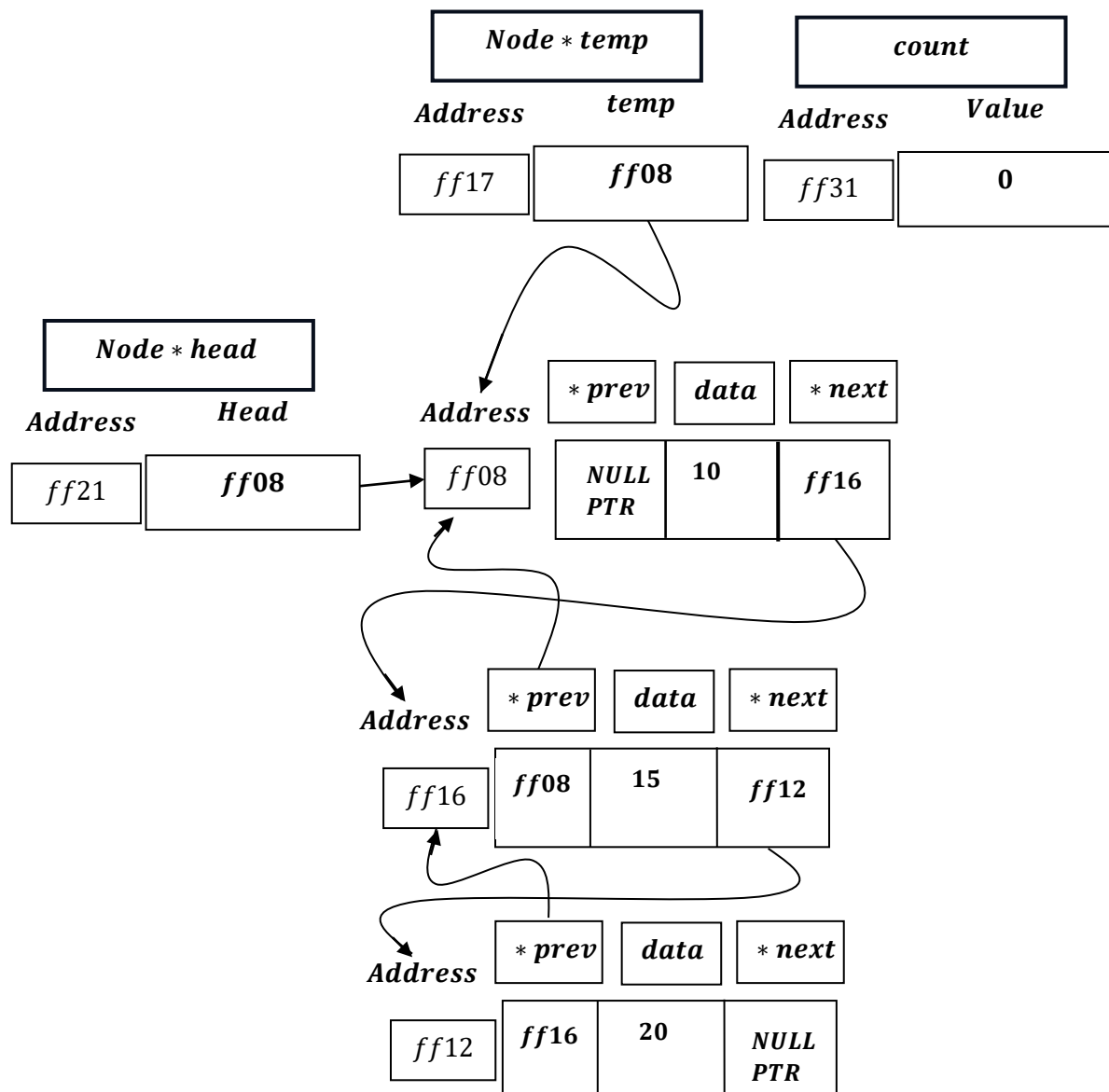


Then, it will print statement : `` List is empty.``

And, return statement will exit from the function.

B. When LIST is not Empty

```
int count = 0;
Node *temp = head;
```



```

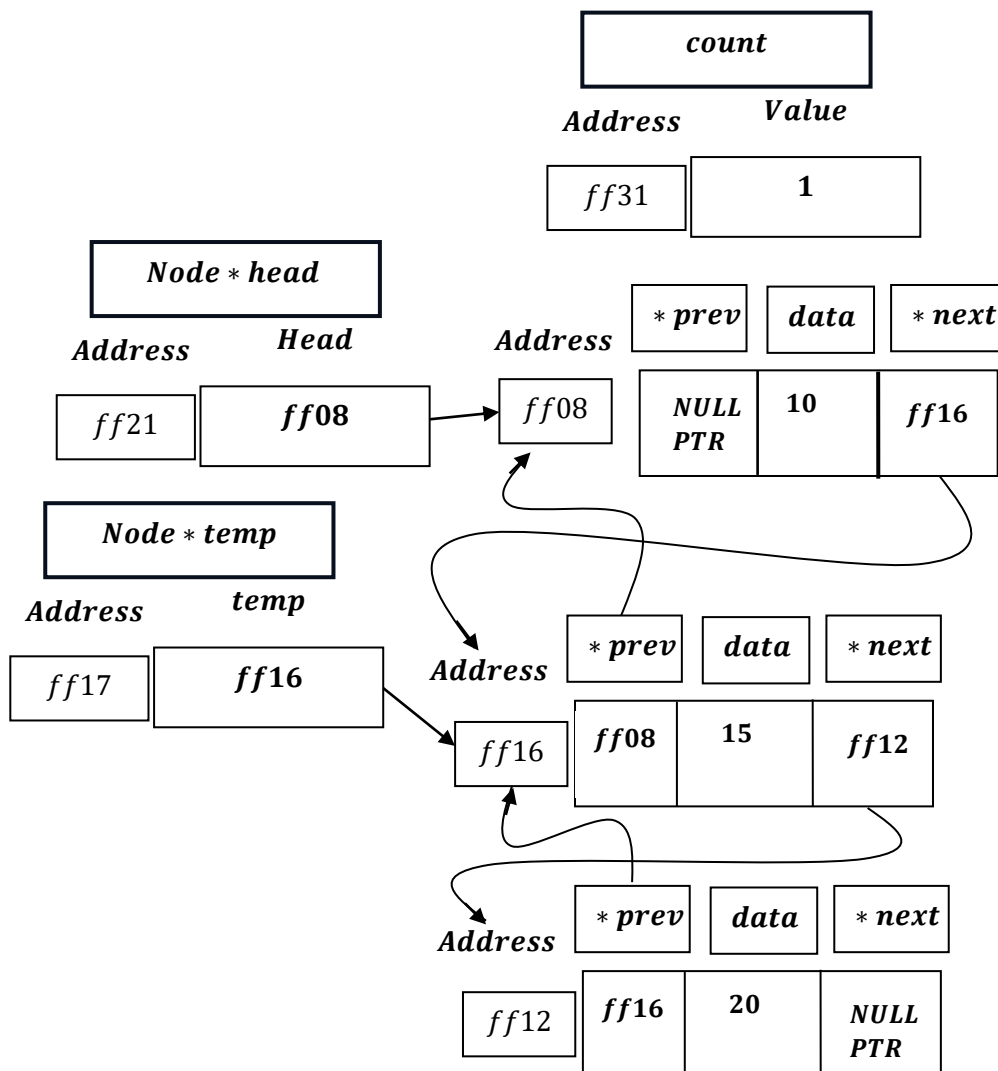
while (temp != nullptr)
{
    count++;
    temp = temp->next;
}

```

Temp $\neq$ *NULLPTR* is true , hence:

count = *count* + 1 = 0 + 1 = 1;

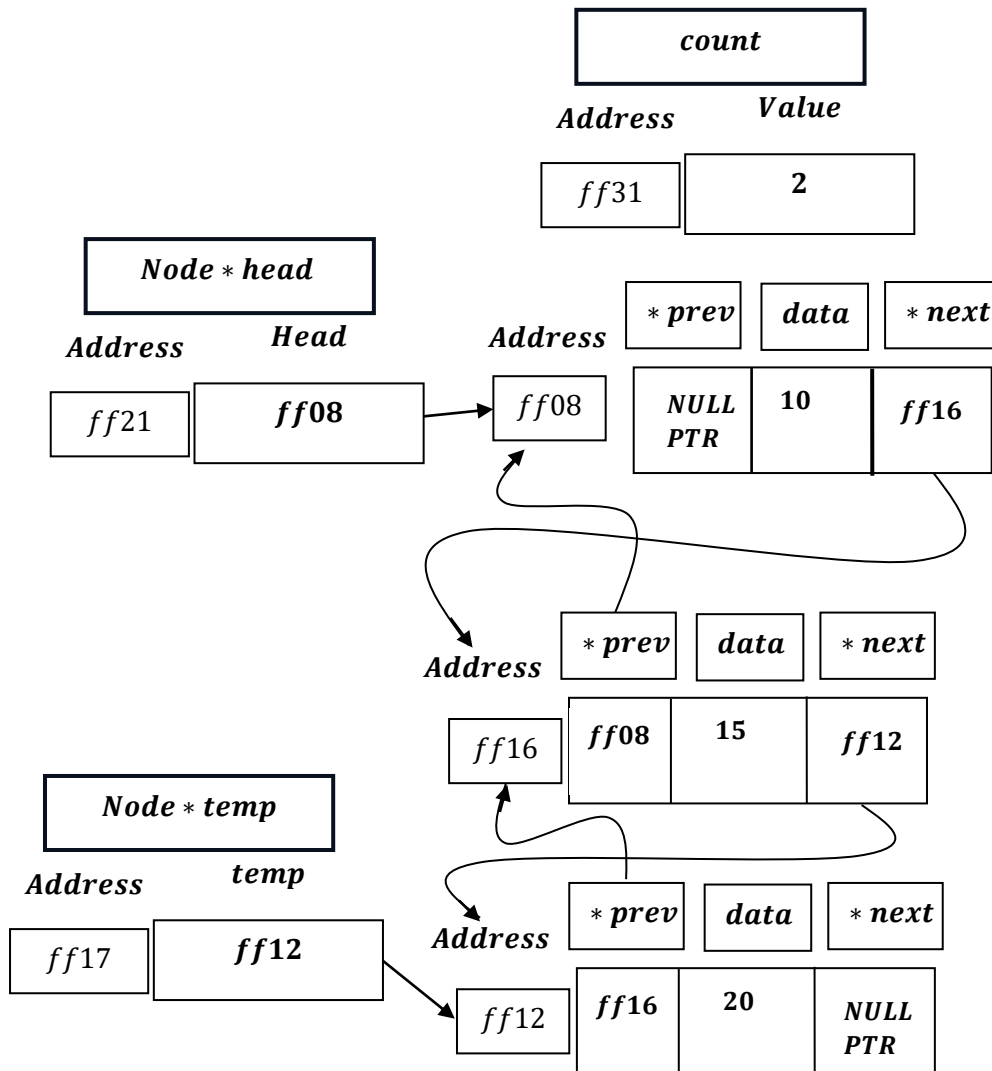
temp = *temp* $\rightarrow$ *next* = *ff16* .



Temp $\neq$ *NULLPTR* is true , hence:

count = *count* + 1 = 1 + 1 = 2;

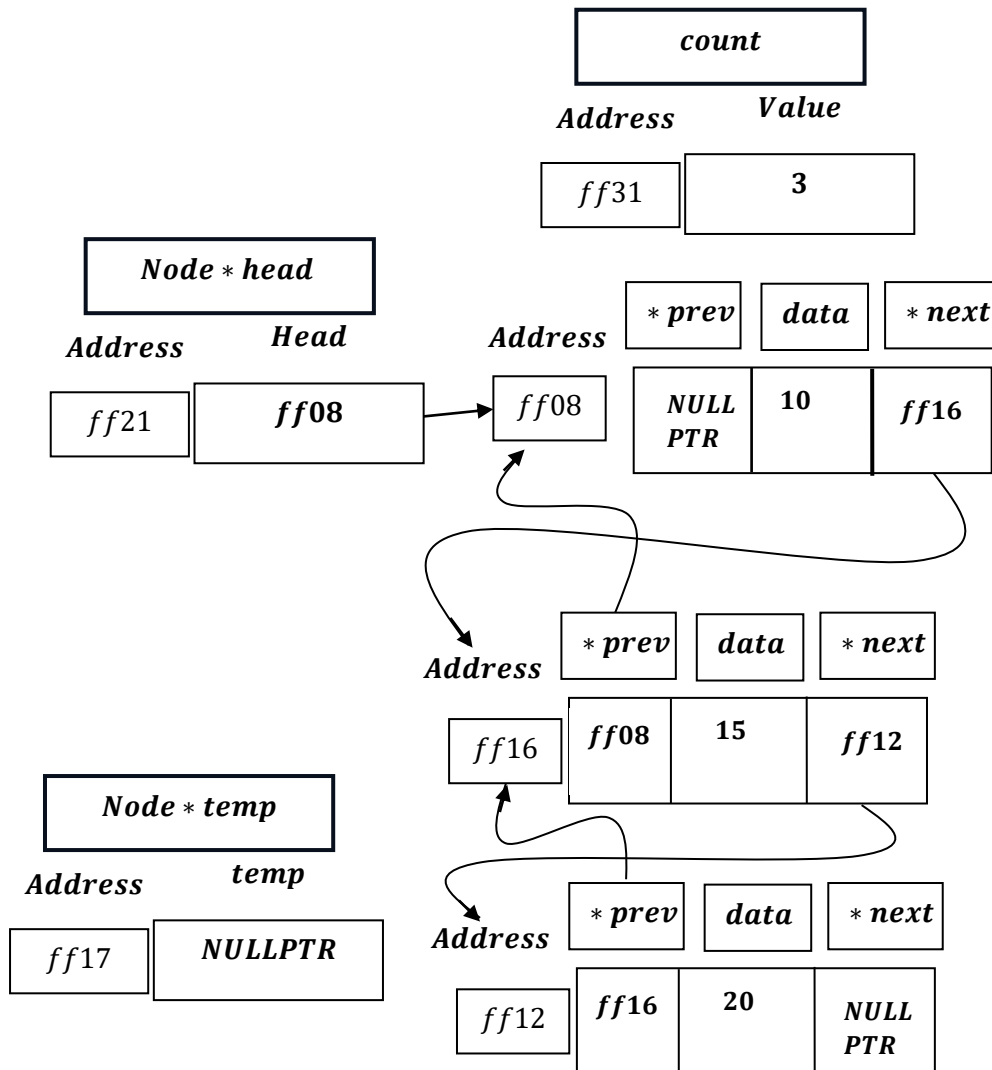
temp = *temp* $\rightarrow$ *next* = *ff12* .



***Temp* $\neq$ *NULLPTR* is true , hence:**

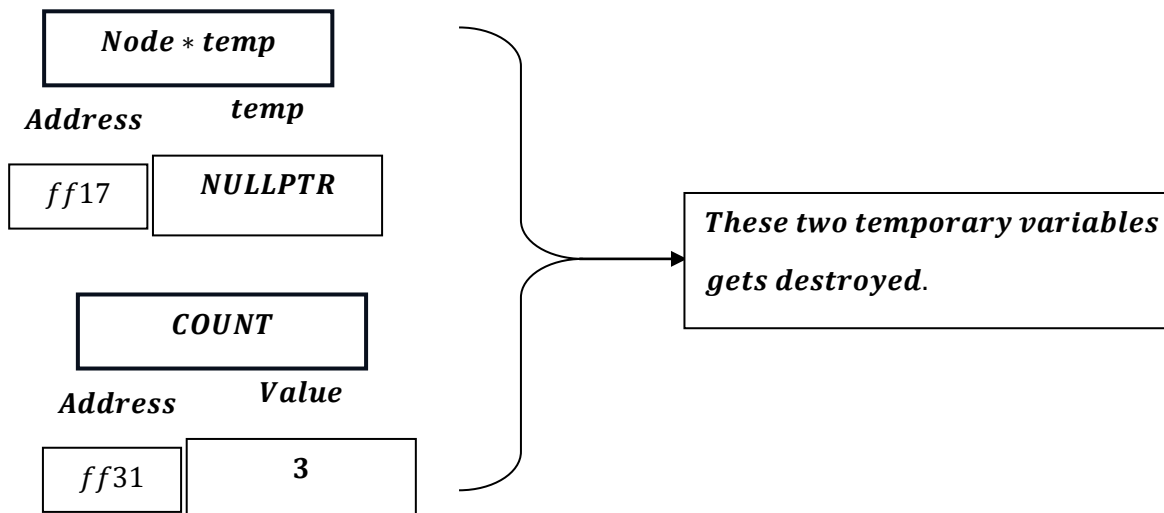
***count* = *count* + 1 = 2 + 1 = 3;**

***temp* = *temp* $\rightarrow$ *next* = *NULLPTR* .**



***Now, TEMP* = *NULLPTR* ,hence while (*TEMP* $\neq$ *NULLPTR*) $\rightarrow$ condition fails .**

As function ends ,Temp pointer along with pos variable gets destroyed:



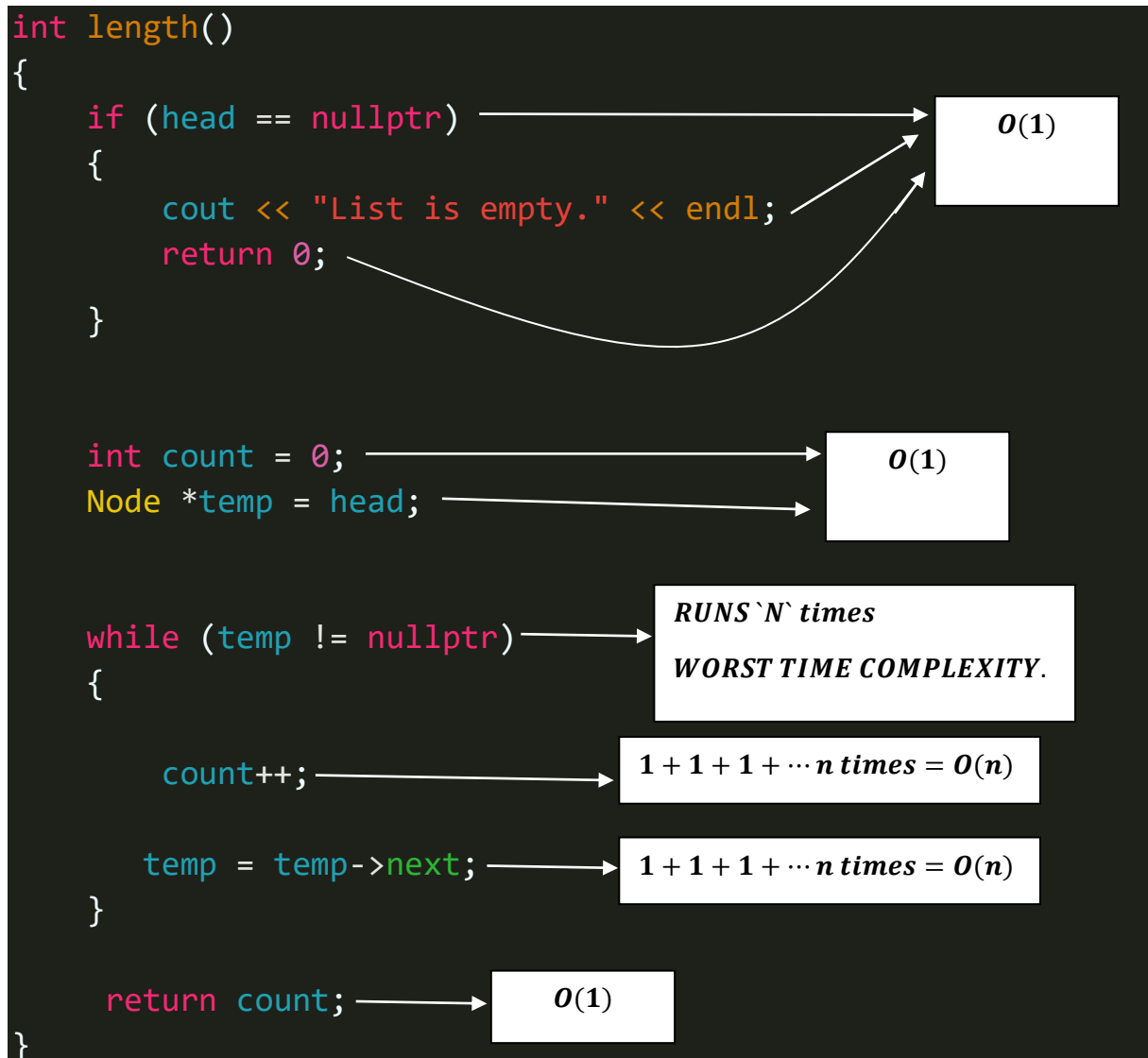
Doubly Linked List – Length Of Linked List [Working Summary]:

This C++ function, `length`, correctly calculates the number of nodes (the length) in a singly linked list.

Working Summary

- Handles Empty List:** It first checks if the list is empty by seeing if the `head` pointer is `nullptr`. If it is, it prints "List is empty." and correctly returns 0 as the length.
 - Initializes for Traversal:** If the list isn't empty, it initializes a counter variable `count` to 0 and creates a temporary pointer `temp` that starts at the head of the list.
 - Iterates and Counts:** The function enters a `while` loop that continues as long as `temp` is not `nullptr` (meaning it hasn't gone past the last node). Inside the loop, it does two things:
 - Increments the `count` by one.
 - Moves the `temp` pointer to the next node in the list (`temp = temp->next`).
 - Returns the Total Count:** Once the loop finishes (after visiting every node), the function returns the final value of `count`, which accurately represents the total number of nodes in the list.
-

Time Complexity Of Length Of Linked List Function Module



Program Analysis

Absolute Best Case:

When List is Empty (head = NULLPTR)

→ If `head = nullptr`, this checking takes constant time. The `cout` and `return` statements take constant time. Hence overall process takes, $O(1)$ time.

Worst Case(When List is not Empty)

→ *int count = 0; this assignment takes constant time i.e. $O(1)$.*

→ *Node * temp = * head; this assignment takes constant time i.e. $O(1)$.*

→ *while(temp != nullptr) checking becomes true , then:*

→ *count++ or count = count + 1, increment of length takes, $1 + 1 + \dots n$ times = n ;*

→ *temp = temp → next; such assignment takes, $1 + 1 + \dots n$ times = n ;*

Therefore ,total loop time complexity: $2n$ or $O(n)$.

→ *return count; returns `count = n` , which takes constant time i.e. $O(1)$.*

Hence total time complexity : $O(1) + O(1) + O(n) + O(1) = O(n)$.

Therefore takes $O(n)$ time .

Average Case

We can say , if List is Empty , we get our best case scenario, and when list is not empty , we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine list get traversed from beginning to end , hence we can say average case is our worst case here i.e. always the entire list is traversed i.e. $O(n) = \Theta(n)$.

This means the execution time is solely dependent on the length of the list.

If the list is empty, the execution time is constant (best case).

If the list is not empty, the execution time is proportional to the length of the list (worst case).

*There's no element of randomness or varying input data to consider.
Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here.
Hence , lengthOfList function doesn't have a true average case.*

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$$

<i>Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Length Of List</i>	<i>1. Best Case</i>	$\Omega(1)$
	<i>2. Worst Case</i>	$O(n)$
	<i>3. Average Case (= Worst Case)</i>	$\Theta(n)$

Doubly Linked List – Length Of Linked List –Space Complexity and Algorithm.

Program :

```
int length()
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return 0;
    }
    int count = 0;
    Node *temp = head;
    while (temp != nullptr)
    {
        count++;
        temp = temp->next;
    }
    return count;
}
...
case 9:
cout << "Length of the list is: " << length() << endl;
break;
```

Space Complexity

Auxiliary Space

Auxiliary space is the temporary or extra memory used by a function, not including the input data.

In the length function, the new variables created are:

int count: A single integer variable to store the count. An integer occupies a constant amount of space.

Node *temp: A single temporary pointer to traverse the list. A pointer also occupies a constant amount of space.

Since the amount of extra memory is fixed and does not grow with the list's size, the auxiliary space complexity is $O(1)$.

Auxiliary Space: $O(1)$

Input Space

Input space is the memory used to store the function's inputs. We can analyze this from two perspectives.

1. Parameter – only Definition

This view considers only the space taken by the function's direct parameters. The function signature is `int length()`.

The function takes no parameters.

Under this definition, the input space is constant.

Input Space (Parameter-only): $O(1)$

2. Full – Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list.

If the linked list contains n nodes, the total memory it occupies is proportional to n .

Under this definition, the input space is linear with respect to the size of the list.

Input Space (Full-Data): $O(n)$

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$Space = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, the "space complexity" of a function refers to its auxiliary space. Therefore, the most common and expected answer for this function is $O(1)$. The while loop that traverses the list affects the function's time complexity ($O(n)$), but not its space complexity.

Algorithm: Find Length of List

DLENGTH(START)

1. [CHECK IF LIST IS EMPTY]

If START = NULL, then:

Write: "List is empty."

Return 0 and EXIT.

[End of If structure.]

2. [INITIALIZE COUNTER AND POINTER]

a. Set COUNT := 0 [Initialize a counter to zero].

b. Set PTR := START [PTR is a pointer used to traverse the list].

3. **[TRAVERSE THE LIST AND INCREMENT COUNTER]**

Repeat while $PTR \neq NULL$:

a. Set $COUNT := COUNT + 1$ [Increment the counter for each node].

b. Set $PTR := NEXTLINK[PTR]$ [Move to the next node].

[END of Loop]

4. **[RETURN FINAL COUNT]**

Return $COUNT$.

5. **Exit**

Doubly Linked List – Traverse In Order And Time Complexity

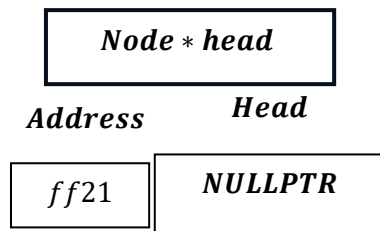
Program :

```
void traverseInOrder()
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    Node *temp = head;
    cout << "List (Forward): NULL <- ";
    while (temp != nullptr)
    {
        cout << temp->data << " <-> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}
... .
case 10:
    traverseInOrder();
    break;
```

Program Analysis

A. When LIST is Empty

```
if (head == nullptr)
{
    cout << "List is empty." << endl;
    return ;
}
```

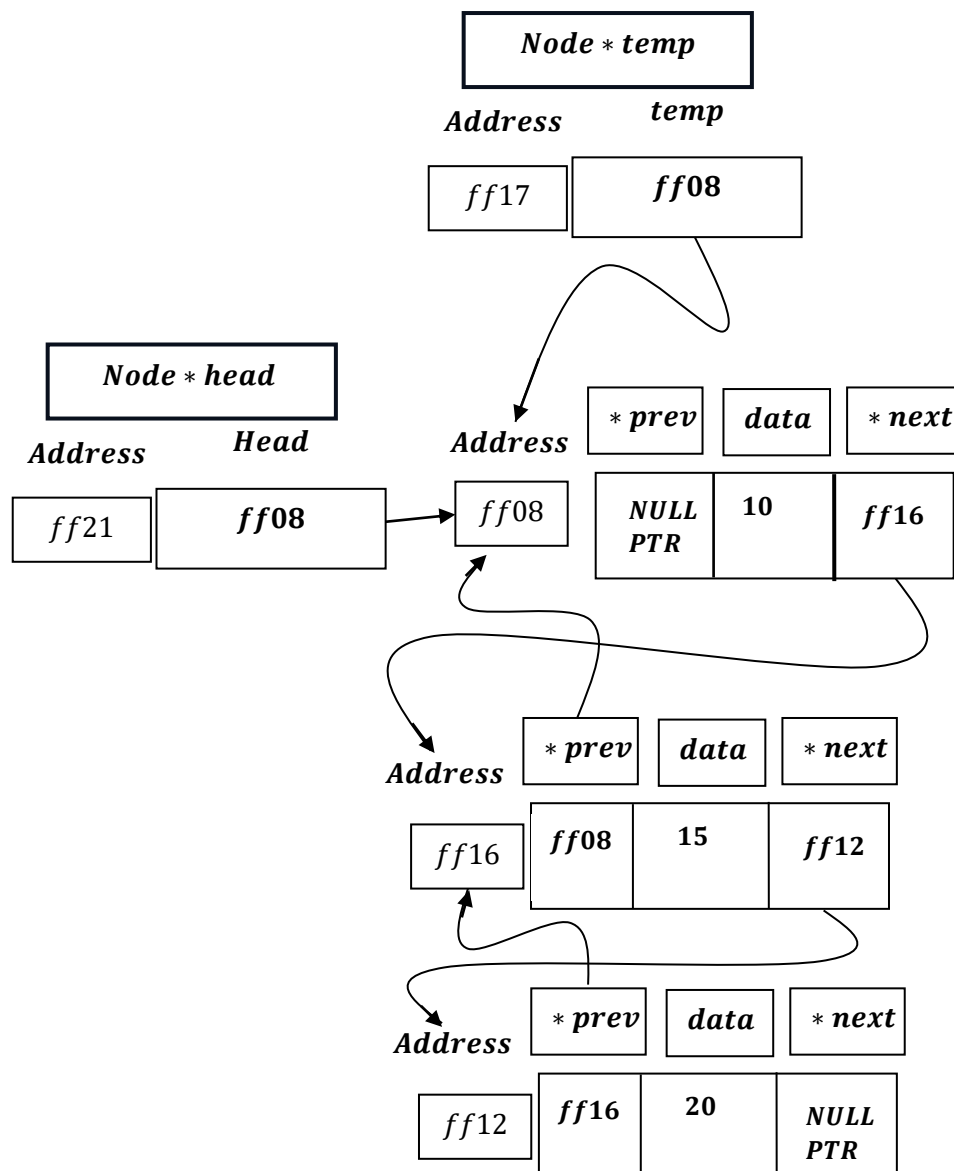


Then, it will print statement : `` List is empty.``

And, return statement will exit from the function.

B. When LIST is not Empty

Node *temp = head;




```

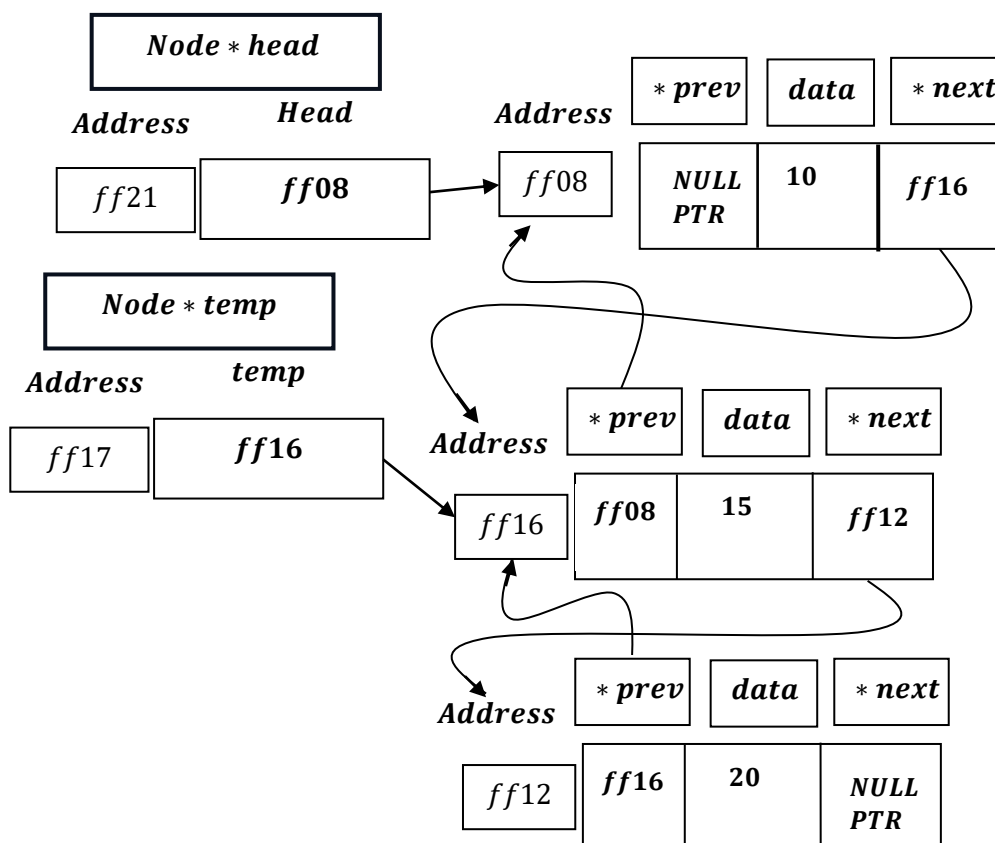
cout << "List (Forward): NULL <- ";
while (temp != nullptr)
{
    cout << temp->data << " <-> ";
    temp = temp->next;
}
cout << "NULL" << endl;
}

```

Temp ≠ *NULLPTR* is true , hence:

cout << *temp* → *data* << <->; ⇒ Output: *NULL* < -10 <->

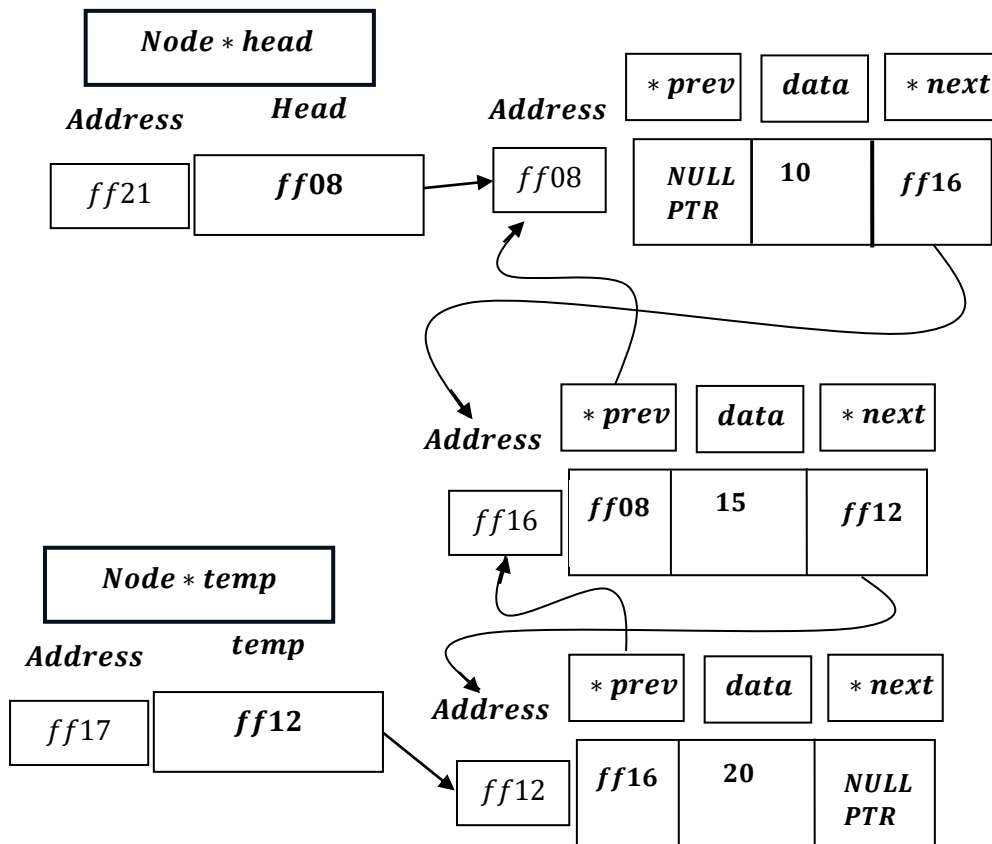
temp = *temp* → *next* = *ff16* .



Temp ≠ *NULLPTR* is true , hence:

cout << *temp* → *data* << <->; ⇒ Output: *NULL* < -10 <-> 15 <->

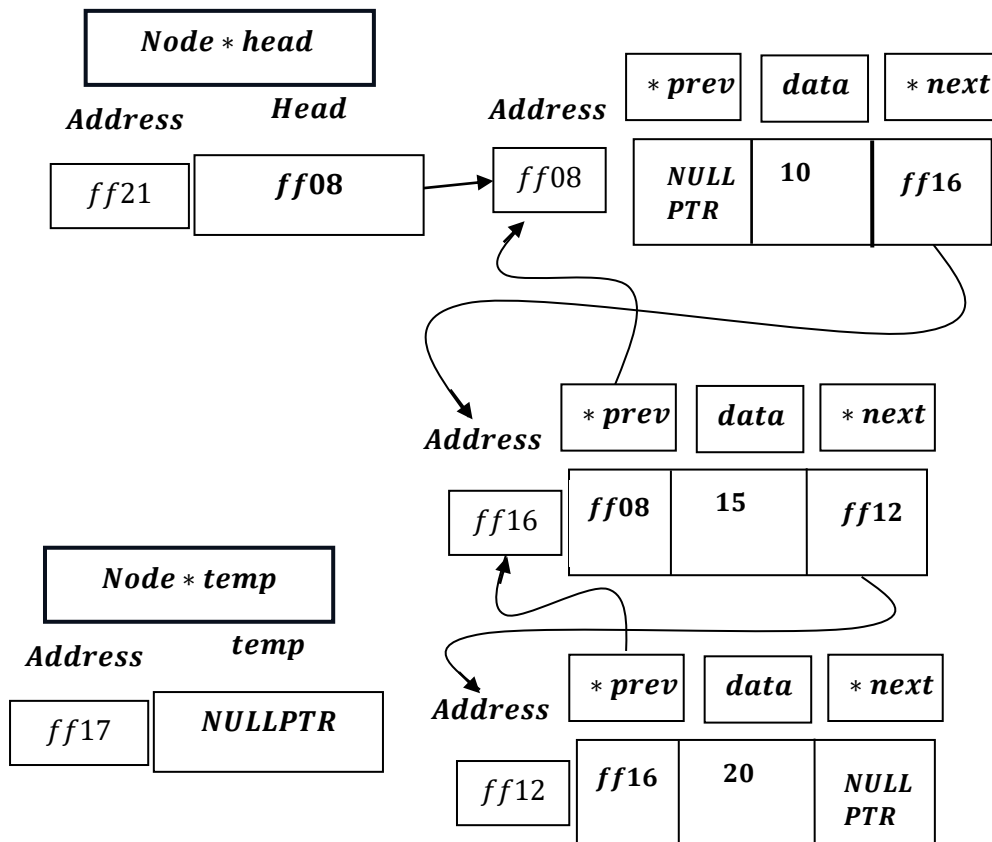
temp = *temp* → *next* = *ff12* .



***Temp* ≠ *NULLPTR* is true , hence:**

***cout* << *temp* → *data* << <-> ; ⇒ Output: *NULL* < -10 <-> 15 <-> 20 <->**

***temp* = *temp* → *next* = *NULLPTR* .**

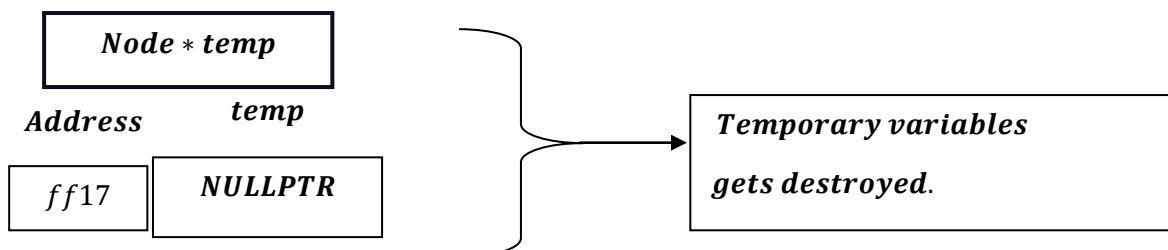


Now, $TEMP = NULLPTR$, hence while ($TEMP \neq NULLPTR$) $\rightarrow$ condition fails .

```
cout << "NULL" << endl;
```

$cout << "NULL" << <->; \Rightarrow$ Output: $NULL <-> 10 <-> 15 <-> 20 <-> NULL$

As function ends ,Temp pointer gets destroyed:



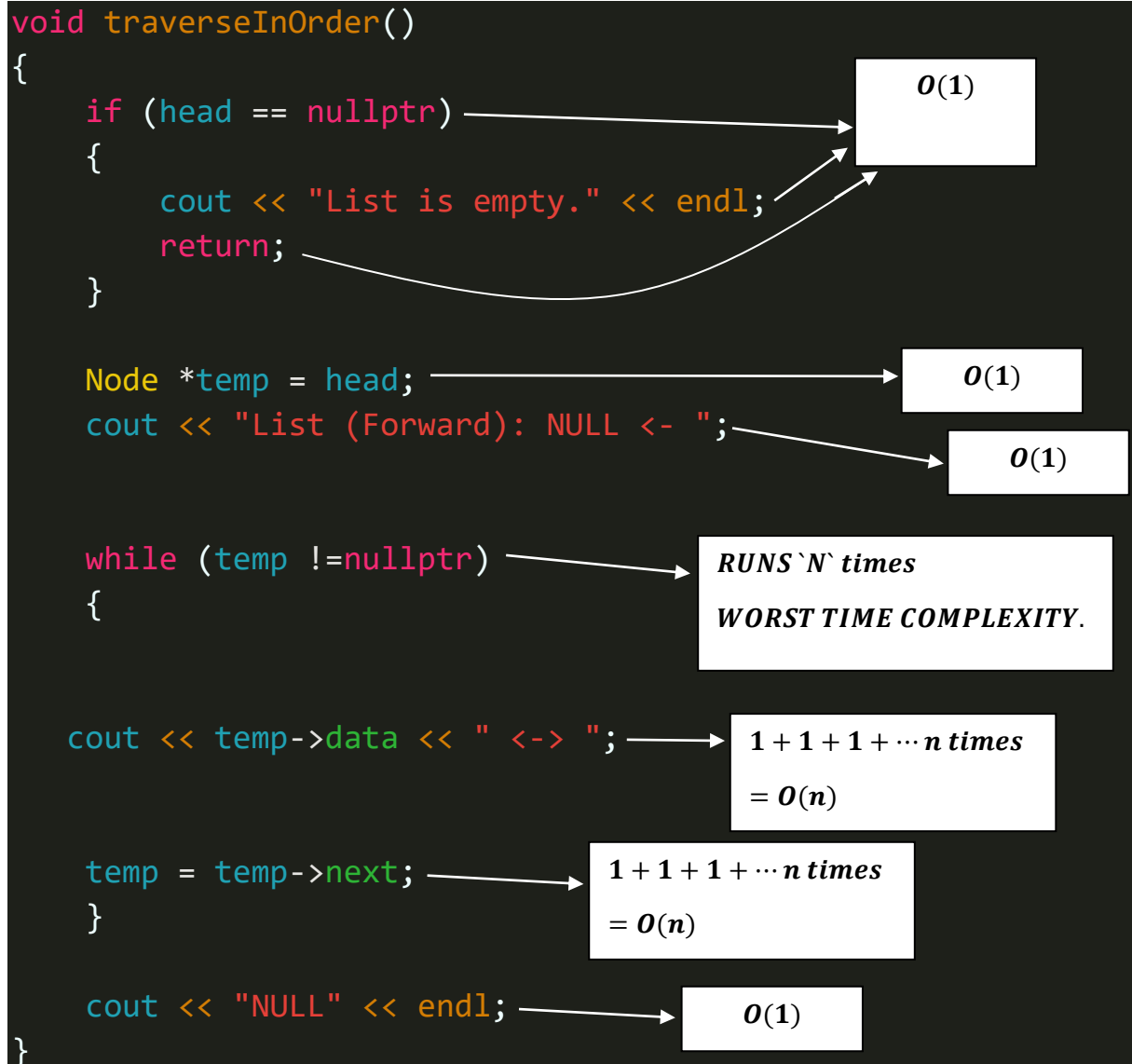
Doubly Linked List – Traverse In Order [Working Summary]:

This C++ function, `traverseInOrder`, is designed to print all the elements of a linked list from beginning to end.

Working Summary

1. **Handles Empty List:** It first checks if the list is empty by verifying if the `head` pointer is `nullptr`. If so, it prints "List is empty." and exits.
 2. **Starts Traversal:** If the list is not empty, it creates a temporary pointer `temp` initialized to the `head` of the list. It then prints a header string `List (Forward): NULL <-` to indicate the start of the list.
 3. **Iterates and Prints:** The function enters a `while` loop that continues as long as `temp` has not reached the end of the list (`temp != nullptr`). In each iteration:
 - It prints the `data` of the current node.
 - It prints `<->` to visually represent the links between nodes.
 - It moves the `temp` pointer to the next node (`temp = temp->next`).
 4. **Finishes Output:** Once the loop completes, it prints `NULL` to signify the end of the list, completing the visual representation. For a list containing 10, 20, and 30, the output would look like this:
 5. `List (Forward): NULL <- 10 <-> 20 <-> 30 <-> NULL`
-

Time Complexity Of Traverse In Order Function Module



Program Analysis

Absolute Best Case:

When List is Empty (head = NULLPTR)

→ If head = nullptr, this checking takes constant time. The cout and return statements takes constant time. Hence overall process takes, $O(1)$ time.

Worst Case(When List is not Empty)

→ Node * temp = * head; this assignment takes constant time i.e. $O(1)$.

→ cout << "List (Forward) : NULL < - "; takes constant time to execute i.e. $O(1)$.

→ while(temp != nullptr) checking becomes true, then:

→ cout << temp → data << " < -> ";, this statement gets executed: $1 + 1 + \dots n$ times = n ;

→ temp = temp → next; such assignment takes, $1 + 1 + \dots n$ times = n ;

Therefore, total loop time complexity: $2n$ or $O(n)$.

→ cout << "NULL " takes constant time to execute i.e. $O(1)$.

Hence total time complexity : $O(1) + O(1) + O(n) + O(1) = O(n)$.

Therefore takes $O(n)$ time.

Average Case

We can say , if List is Empty , we get our best case scenario, and when list is not empty , we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine list get traversed from beginning to end , hence we can say average case is our worst case here i. e. always the entire list is traversed i. e. $O(n) = \Theta(n)$.

This means the execution time is solely dependent on the length of the list.

If the list is empty, the execution time is constant (best case).

If the list is not empty, the execution time is proportional to the length of the list (worst case).

There's no element of randomness or varying input data to consider.

Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here.

Hence , Traverse In Order function doesn't have a true average case.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$$

<i>Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Traverse In Order</i>	1. Best Case	$\Omega(1)$
	2. Worst Case	$O(n)$
	3. Average Case (= Worst Case)	$\Theta(n)$

Doubly Linked List – Traverse In Order

–Space Complexity and Algorithm

Program :

```
void traverseInOrder()
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    Node *temp = head;
    cout << "List (Forward): NULL <- ";
    while (temp != nullptr)
    {
        cout << temp->data << " <-> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}
... .
case 10:
    traverseInOrder();
    break;
```

Space Complexity

Auxiliary Space

Auxiliary space is the extra or temporary memory used by a function, not including the input data itself.

For the traverseInOrder function, the only new variable created is:

Node *temp: A single temporary pointer is used to iterate through the list. The memory required for a pointer is constant.

The function does not allocate any new nodes or data structures. Since the amount of extra memory is fixed, the auxiliary space complexity is $O(1)$.

Auxiliary Space: $O(1)$

Input Space

Input space is the memory used to store the function's inputs. This can be viewed in two ways.

1. Parameter – only Definition

This perspective considers only the space taken by the function's direct parameters. The function signature is `void traverseInOrder()`.

The function takes no parameters.

Under this definition, the input space is constant.

Input Space (Parameter-only): $O(1)$

2. Full – Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list.

If the linked list contains n nodes, the total memory it occupies is proportional to n .

Under this definition, the input space is linear with respect to the size of the list.

Input Space (Full-Data): $O(n)$

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, the "space complexity" of a function refers to its auxiliary space. Therefore, the most common and expected answer for this function is $O(1)$. The while loop that travels the list affects the function's time complexity ($O(n)$), but not its space complexity.

Algorithm: Traverse and Display List (Forward)

DTRAVERSE(START)

1. ***[CHECK IF LIST IS EMPTY]***

If START = NULL, then:

Write: "List is empty."

EXIT.

[End of If structure.]

2. ***[INITIALIZE POINTER AND PRINT HEADER]***

a. Set PTR := START [PTR is a pointer used to traverse the list].

b. Write: "List (Forward): NULL < – "

3. ***[TRAVERSE THE LIST AND PRINT EACH ELEMENT]***

Repeat while PTR ≠ NULL:

a. Write: INFO[PTR], " < –> " [Print the data of the current node].

b. Set PTR := NEXTLINK[PTR] [Move to the next node].

[END of Loop]

4. ***[PRINT THE LIST TERMINATOR]***

Write: "NULL"

5. ***Exit***

Doubly Linked List – Traverse In Reverse And Time Complexity

Program :

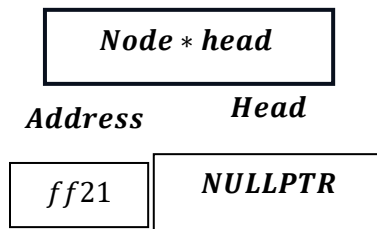
```
void traverseInReverse()
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    Node *temp = head;
    while (temp->next != nullptr)
    { // Go to the last node
        temp = temp->next;
    }

    cout << "List (Reverse): NULL <- ";
    while (temp != nullptr)
    {
        cout << temp->data << " <-> ";
        temp = temp->prev;
    }
    cout << "NULL" << endl;
}
...
case 11:
    traverseInReverse();
    break;
```

Program Analysis

A. When LIST is Empty

```
if (head == nullptr)
{
    cout << "List is empty." << endl;
    return ;
}
```

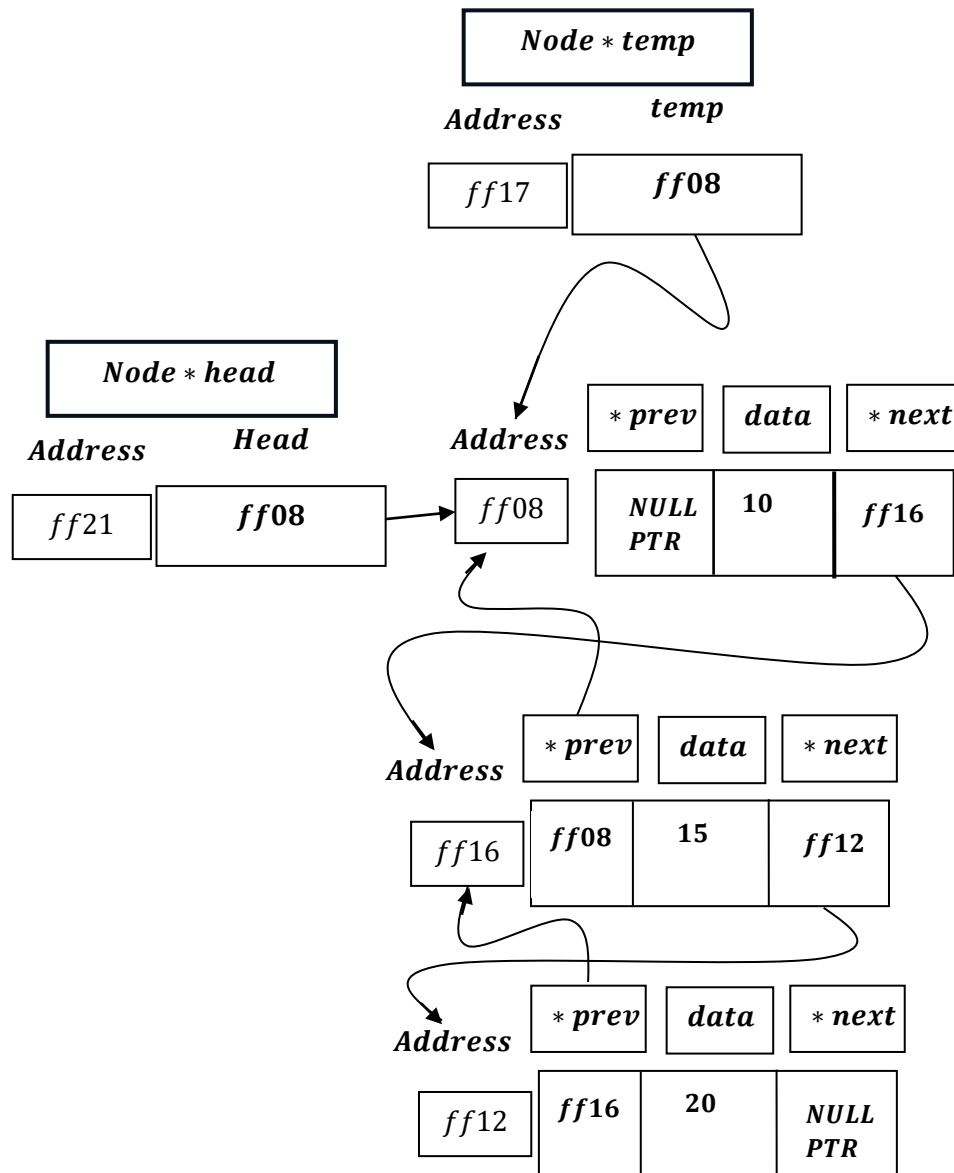


Then, it will print statement : `` List is empty.``

And, return statement will exit from the function.

B. When LIST is not Empty

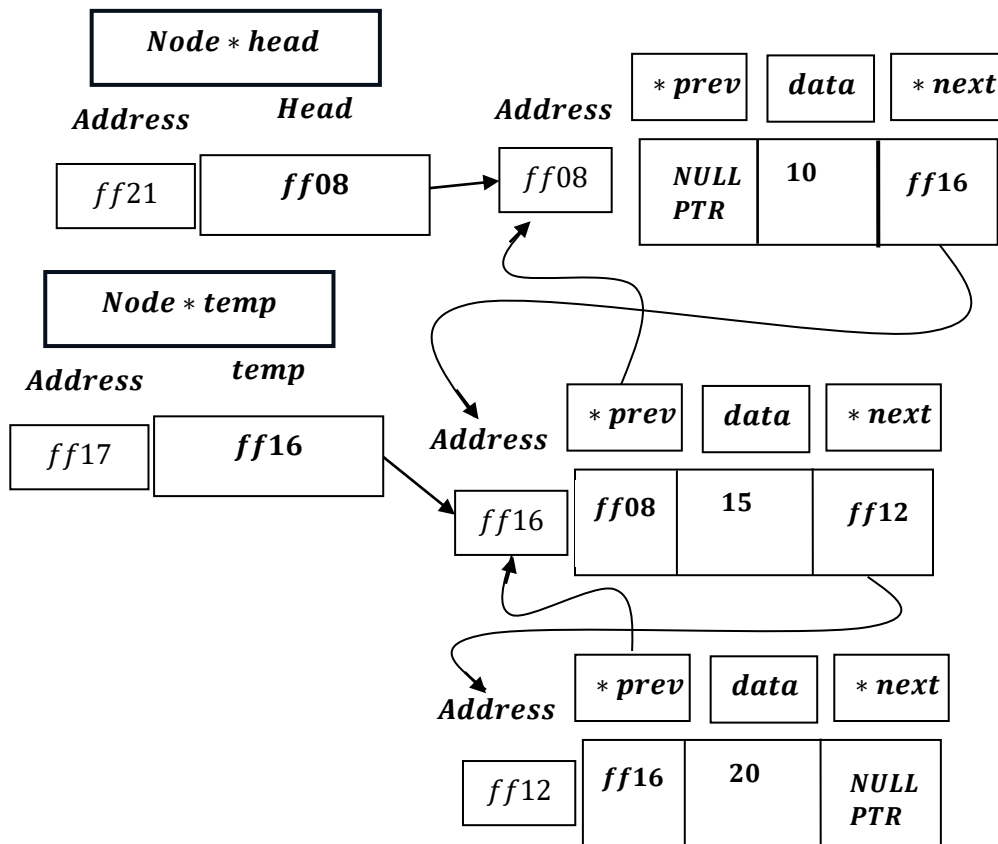
```
Node *temp = head;
```



```
while (temp->next != nullptr)
{ // Go to the last node
    temp = temp->next;
}
```

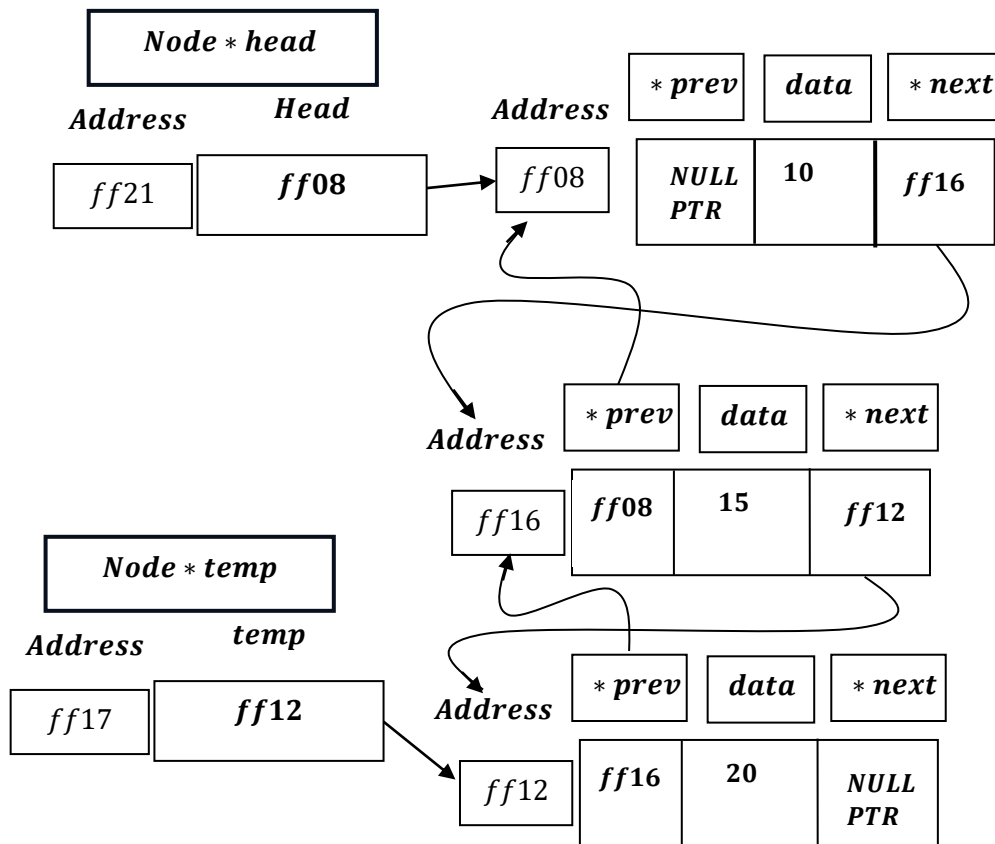
Temp $\neq$ *NULLPTR* is true , hence:

temp = *temp* $\rightarrow$ *next* = *ff16* .



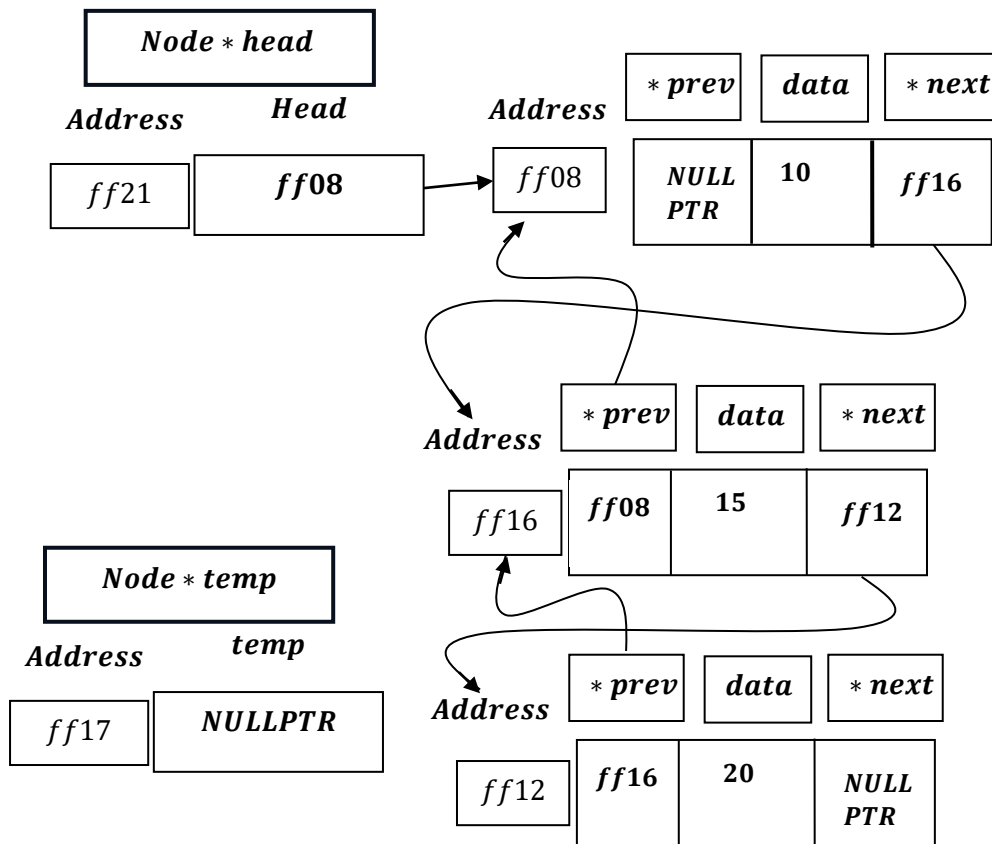
Temp $\neq$ *NULLPTR* is true , hence:

temp = *temp* $\rightarrow$ *next* = *ff12* .



Temp $\neq$ NULLPTR is true , hence:

temp = temp $\rightarrow$ next = NULLPTR .



Now, TEMP = NULLPTR , hence while (TEMP ≠ NULLPTR) → condition fails .

```

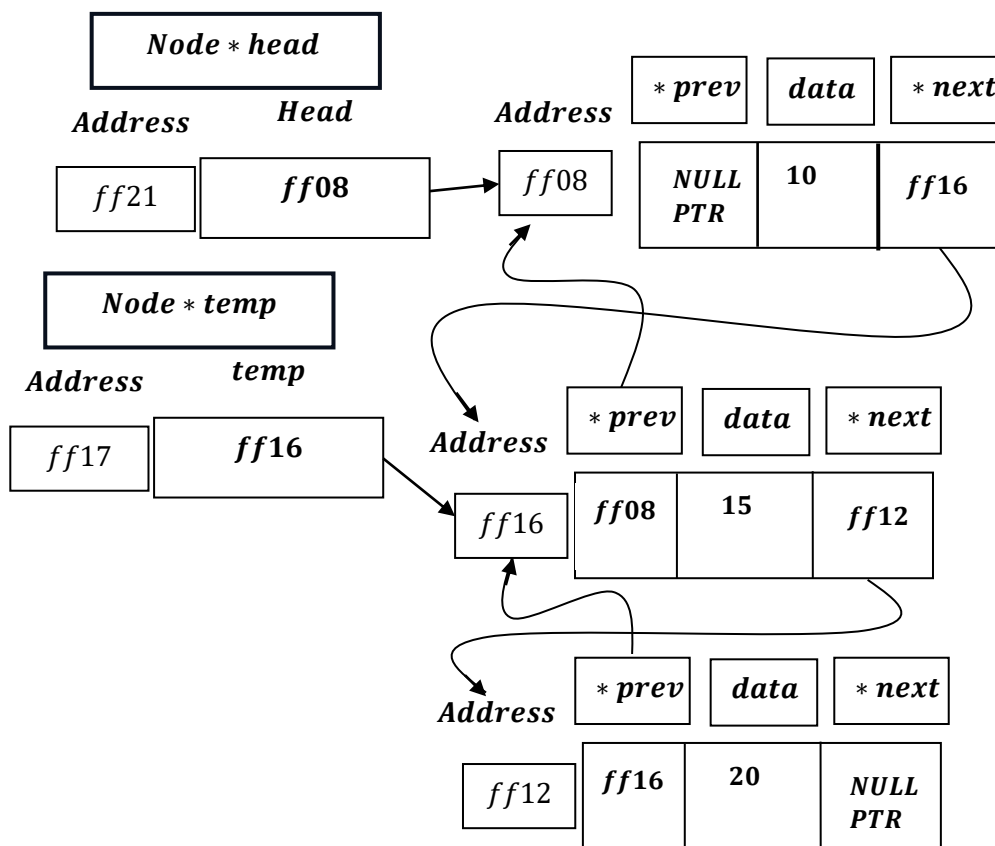
cout << "List (Reverse): NULL <- ";
while (temp != nullptr)
{
    cout << temp->data << " <-> ";
    temp = temp->prev;
}
cout << "NULL" << endl;

```

Temp ≠ NULLPTR is true , hence:

cout << *temp* → *data* << <->; ⇒ Output: NULL < -20 < ->

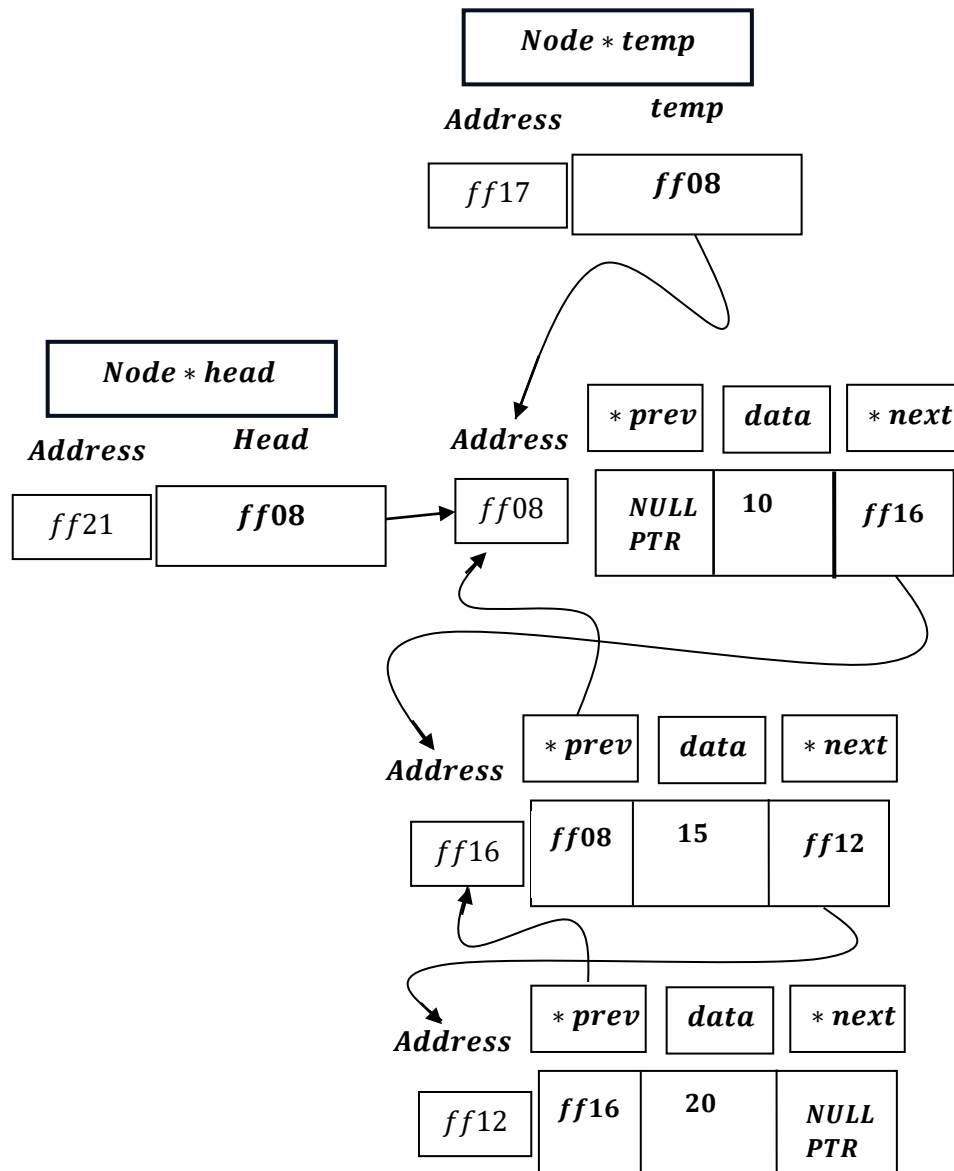
temp = *temp* → *prev* = *ff16* .



Temp ≠ NULLPTR is true , hence:

cout << *temp* → *data* << <->; ⇒ Output: NULL < -20 < -> 15 < ->

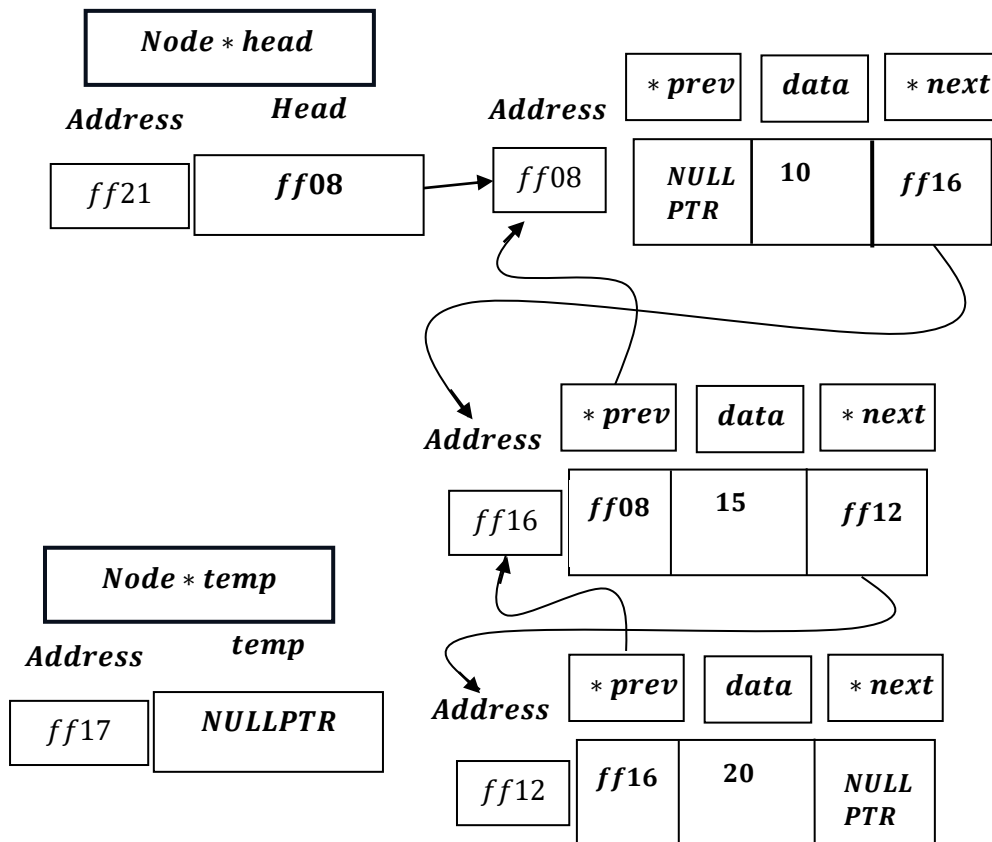
temp = *temp* → *prev* = *ff08* .



***Temp* $\neq$ *NULLPTR* is true , hence:**

***cout* $\ll$ *temp* $\rightarrow$ *data* $\ll$ `<->`; $\Rightarrow$ *Output:* `NULL` `<` `-20` `<->` `15` `<->` `10` `<->`**

***temp* = *temp* $\rightarrow$ *prev* = *NULLPTR* .**

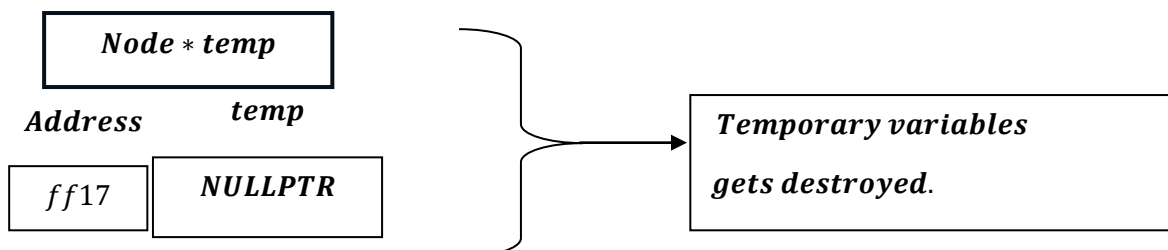


Now, $TEMP = NULLPTR$, hence while ($TEMP \neq NULLPTR$) $\rightarrow$ condition fails .

```
cout << "NULL" << endl;
```

$cout << "NULL" << <->; \Rightarrow$ Output: $NULL <-> -20 <-> 15 <-> 10 <-> NULL$

As function ends ,Temp pointer gets destroyed:



Doubly Linked List – Traverse In Reverse [Working Summary]:

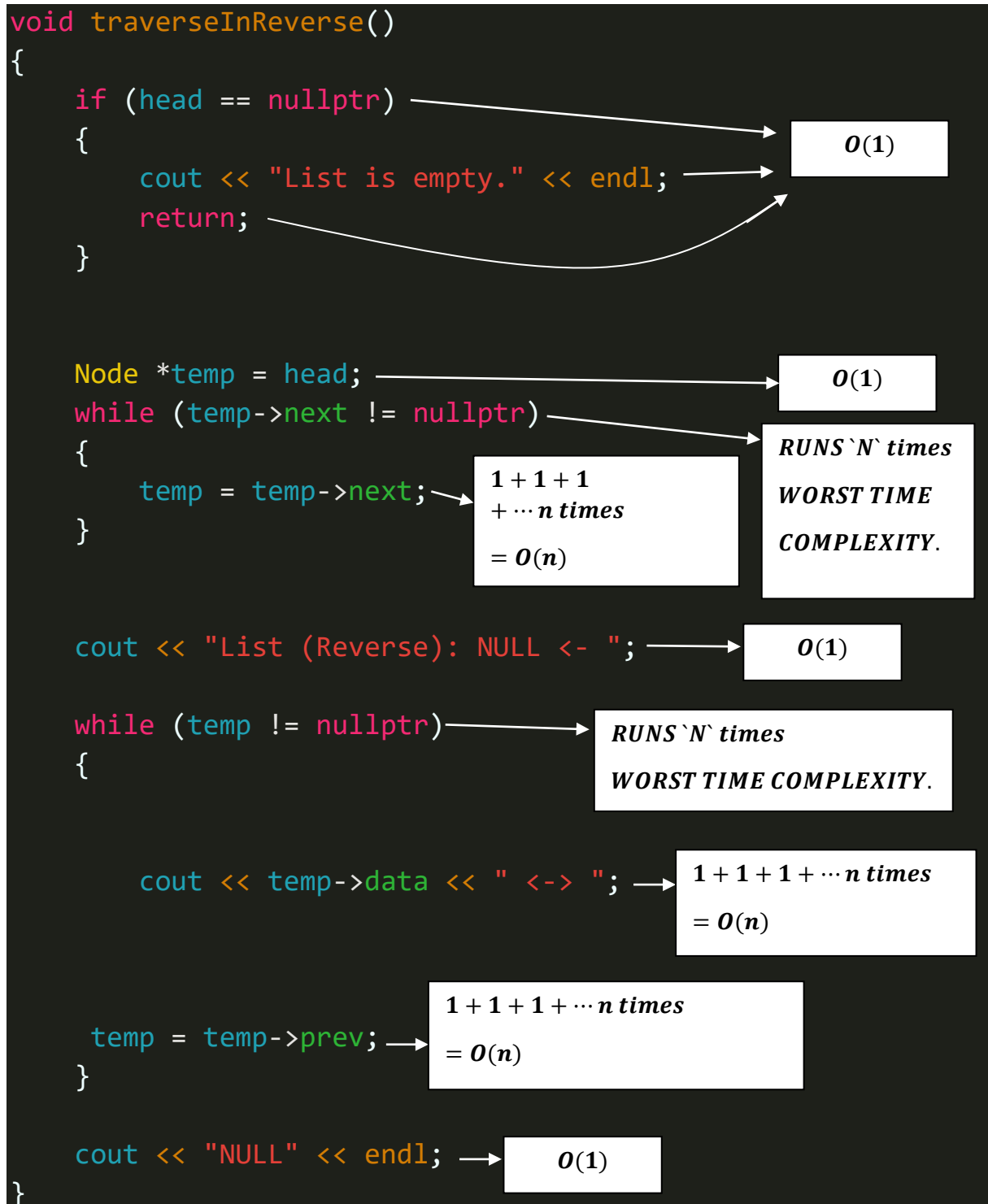
This C++ function, `traverseInReverse`, prints all the elements of a **doubly linked list** in reverse order, from tail to head.

Working Summary

1. **Handles Empty List:** The function first checks if the list is empty (`head == nullptr`). If it is, it prints an "List is empty." message and stops.
2. **Finds the Last Node:** It initializes a temporary pointer `temp` at the `head`. A `while` loop then traverses the list until it reaches the very last node (where `temp->next` is `nullptr`).
3. **Initiates Reverse Print:** After finding the tail, it prints a header `List (Reverse):` `NULL <-` to start the output.
4. **Traverses Backward and Prints:** The function enters a second `while` loop that continues as long as `temp` is not `nullptr`. In each iteration:
 - o It prints the `data` of the current node.
 - o It moves backward to the **previous** node using the `prev` pointer (`temp = temp->prev`). This is the key step that makes reverse traversal possible and indicates the structure is a doubly linked list.
5. **Finishes Output:** Once the loop finishes (after reaching the node before the original `head`), it prints `NULL` to signify the end of the reverse-printed list. For a list with 10, 20, and 30, the output would be:

```
List (Reverse): NULL <- 30 <-> 20 <-> 10 <-> NULL
```

Time Complexity Of Traverse In Reverse Function Module



Program Analysis

Absolute Best Case:

When List is Empty (head = NULLPTR)

→ If head = nullptr, this checking takes constant time. The cout and return statements takes constant time. Hence overall process takes, $O(1)$ time.

Worst Case(When List is not Empty)

→ Node * temp = * head; this assignment takes constant time i. e. $O(1)$.

→ while(temp != nullptr) checking becomes true, then:

→ temp = temp → next; such assignment takes, $1 + 1 + \dots n$ times = n ;

Therefore, total loop time complexity: $O(n)$.

→ cout << "List (Reverse) : NULL < - "; takes constant time to execute i. e. $O(1)$.

→ while(temp != nullptr) checking becomes true, then:

→ cout << temp → data << " < -> ";, this statement gets executed: $1 + 1 + \dots n$ times = n ;

→ temp = temp → prev; such assignment takes, $1 + 1 + \dots n$ times = n ;

Therefore, total loop time complexity: $2n$ or $O(n)$.

→ cout << "NULL " takes constant time to execute i. e. $O(1)$.

Hence total time complexity : $O(1) + O(n) + O(1) + O(n) + O(1) = O(n)$.

Therefore takes $O(n)$ time .

Average Case

We can say , if List is Empty , we get our best case scenario, and when list is not empty , we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine list get traversed from beginning to end , hence we can say average case is our worst case here i. e. always the entire list is traversed i. e. $O(n) = \Theta(n)$.

This means the execution time is solely dependent on the length of the list.

If the list is empty, the execution time is constant (best case).

If the list is not empty, the execution time is proportional to the length of the list (worst case).

There's no element of randomness or varying input data to consider.

Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here.

Hence , Traverse In Reverse function doesn't have a true average case.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$$

<i>Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Traverse In Reverse</i>	1. Best Case	$\Omega(1)$
	2. Worst Case	$O(n)$
	3. Average Case (= Worst Case)	$\Theta(n)$

Doubly Linked List – Traverse In Reverse And Space Complexity

Program :

```
void traverseInReverse()
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    Node *temp = head;
    while (temp->next != nullptr)
    { // Go to the last node
        temp = temp->next;
    }

    cout << "List (Reverse): NULL <- ";
    while (temp != nullptr)
    {
        cout << temp->data << " <-> ";
        temp = temp->prev;
    }
    cout << "NULL" << endl;
}
...
case 11:
    traverseInReverse();
    break;
```

Space Complexity

Auxiliary Space

Auxiliary space is the extra or temporary memory used by a function, excluding the space taken by the inputs themselves.

In the traverseInReverse function, the only new variable created is:

*Node *temp: A single temporary pointer is used to find the end of the list and then traverse backward. The memory required for a single pointer is constant.*

The function does not allocate any new nodes or data structures. Since the amount of extra memory used is fixed, the auxiliary space complexity is $O(1)$.

Auxiliary Space: $O(1)$

Input Space

Input space is the memory used to store the function's inputs. This can be analyzed from two perspectives.

1. Parameter – only Definition

This view considers only the space taken by the function's direct parameters. The function signature is void traverseInReverse().

The function takes no parameters.

Under this definition, the input space is constant.

Input Space (Parameter-only): $O(1)$

2. Full – Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list.

If the linked list contains n nodes, the total memory it occupies is proportional to n .

Under this definition, the input space is linear with respect to the size of the list.

Input Space (Full-Data): $O(n)$

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, the "space complexity" of a function refers to its auxiliary space. Therefore, the most common and expected answer for this function is $O(1)$. The two while loops that travel the list affect the function's time complexity ($O(n)$), but not its space complexity.

Algorithm: Traverse and Display List (Reverse)

This algorithm traverses a doubly linked list from end to beginning and prints the data of each node. The list starts at the node pointed to by START.

DREVERSETRAVERSE(START)

1. ***[CHECK IF LIST IS EMPTY]***

If START = NULL, then:

Write: "List is empty."

EXIT.

[End of If structure.]

2. ***[TRAVERSE TO THE LAST NODE OF THE LIST]***

a. Set PTR := START [PTR is a pointer used to traverse the list].

b. Repeat while NEXTLINK[PTR] ≠ NULL:

Set PTR := NEXTLINK[PTR] [Move forward to the next node].

[END of Loop]

3. ***[PRINT HEADER AND BEGIN REVERSE TRAVERSAL]***

a. Write: "List (Reverse): NULL < – "

b. Repeat while PTR ≠ NULL:

i. Write: INFO[PTR], " < -> " [Print the data of the current node].

ii. Set PTR := PREVLINK[PTR] [Move backward to the previous node].

[END of Loop]

4. ***[PRINT THE LIST TERMINATOR]***

Write: "NULL"

5. ***Exit***
-

Doubly Linked List – Destroy List And Time Complexity

Program :

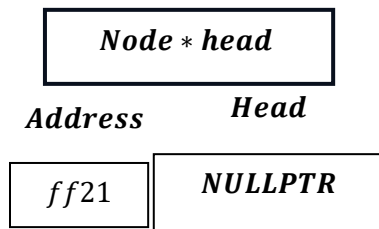
```
void destroyList()
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    Node *current = head;
    Node *nextNode;
    while (current != nullptr)
    {
        nextNode = current->next;
        free(current);
        current = nextNode;
    }
    head = nullptr;
    cout << "List destroyed." << endl;
}

...
case 12:
    destroyList();
    break;
```

Program Analysis

A. When LIST is Empty

```
if (head == nullptr)
{
    cout << "List is empty." << endl;
    return ;
}
```

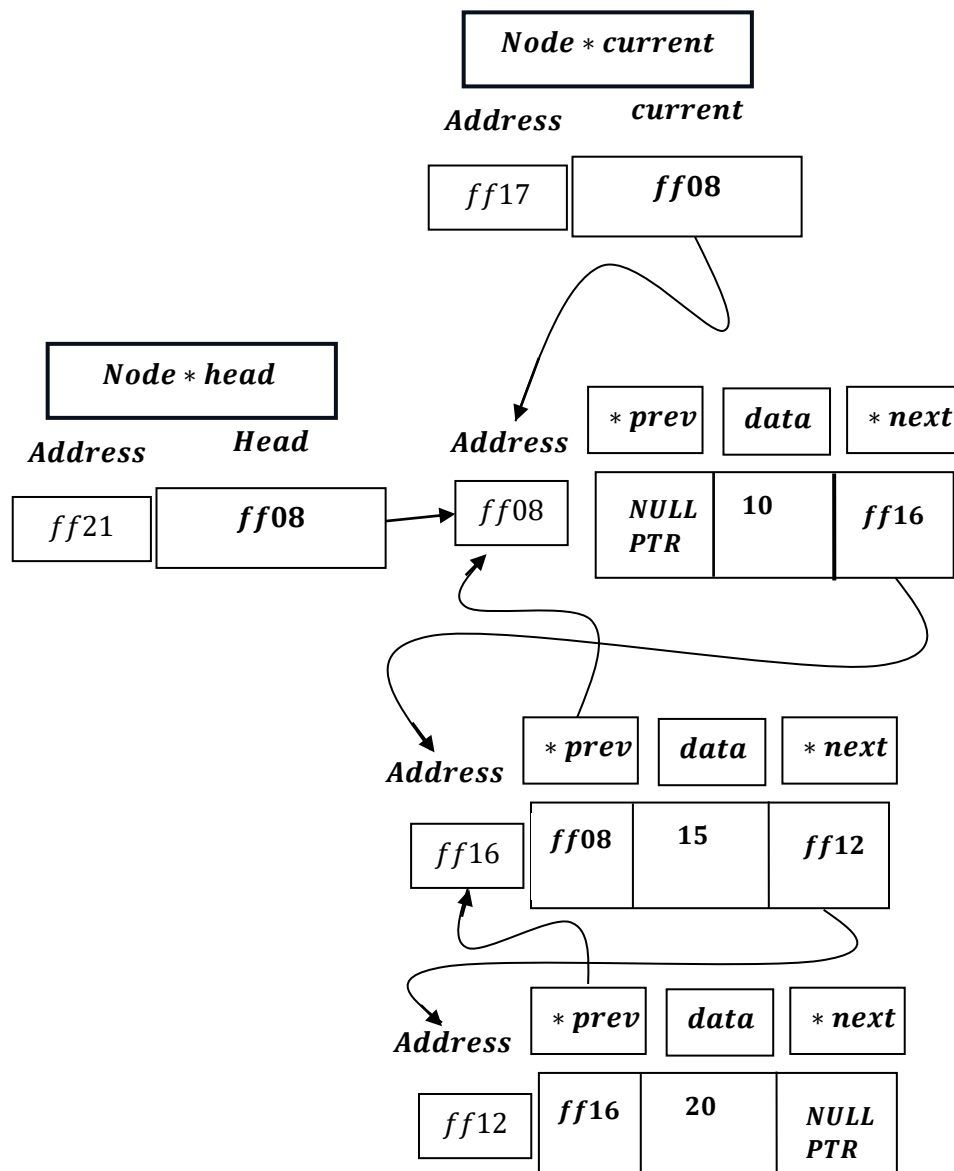


Then, it will print statement : `` List is empty.``

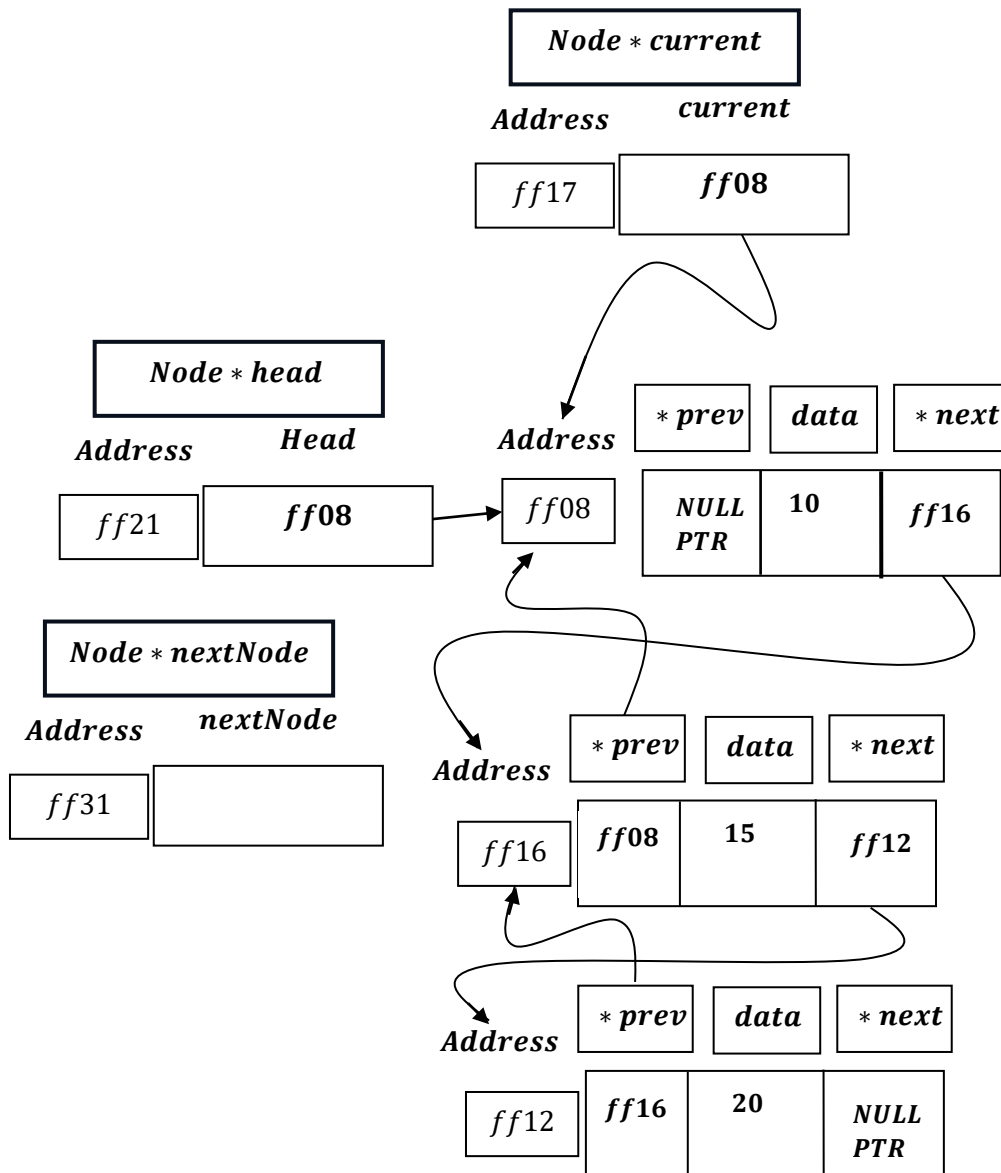
And, return statement will exit from the function.

B. When LIST is not Empty

```
Node *current = head;
```

```
Node *nextNode;
```



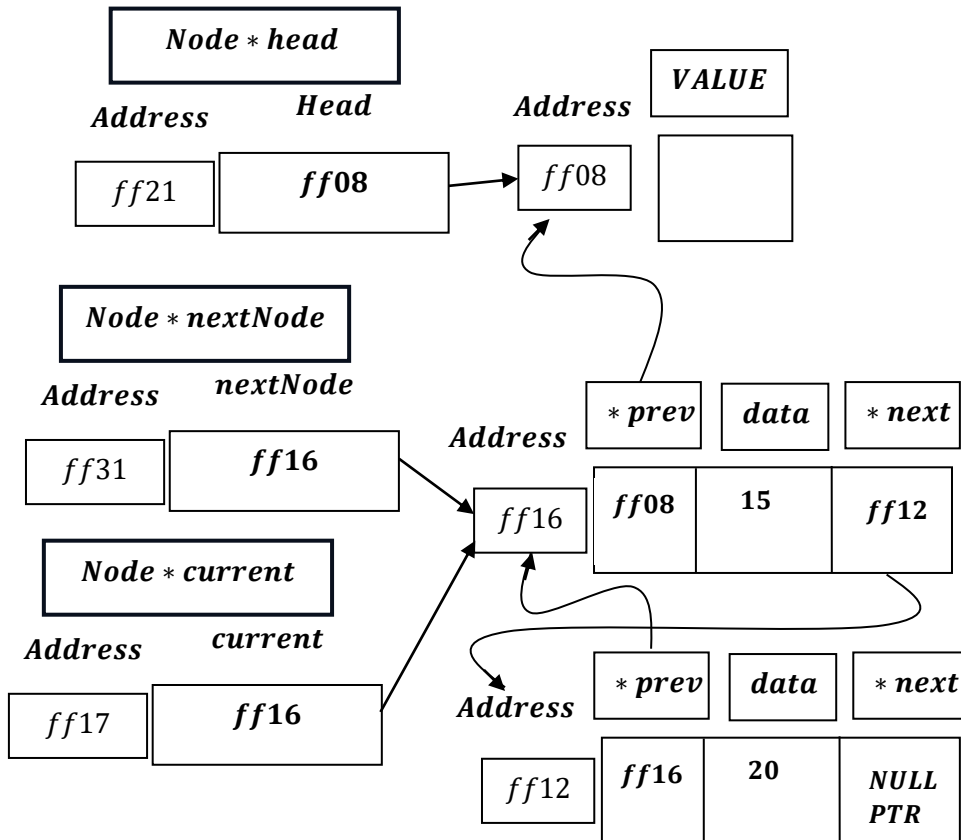
```
while (current != nullptr)
{
    nextNode = current->next;
    free(current);
    current = nextNode;
}
```

***current* $\neq$ NULLPTR is true ,hence:**

***nextNode* = *current* $\rightarrow$ *next* = *ff16*.**

free(current);

***current* = *nextNode* = *ff16*.**

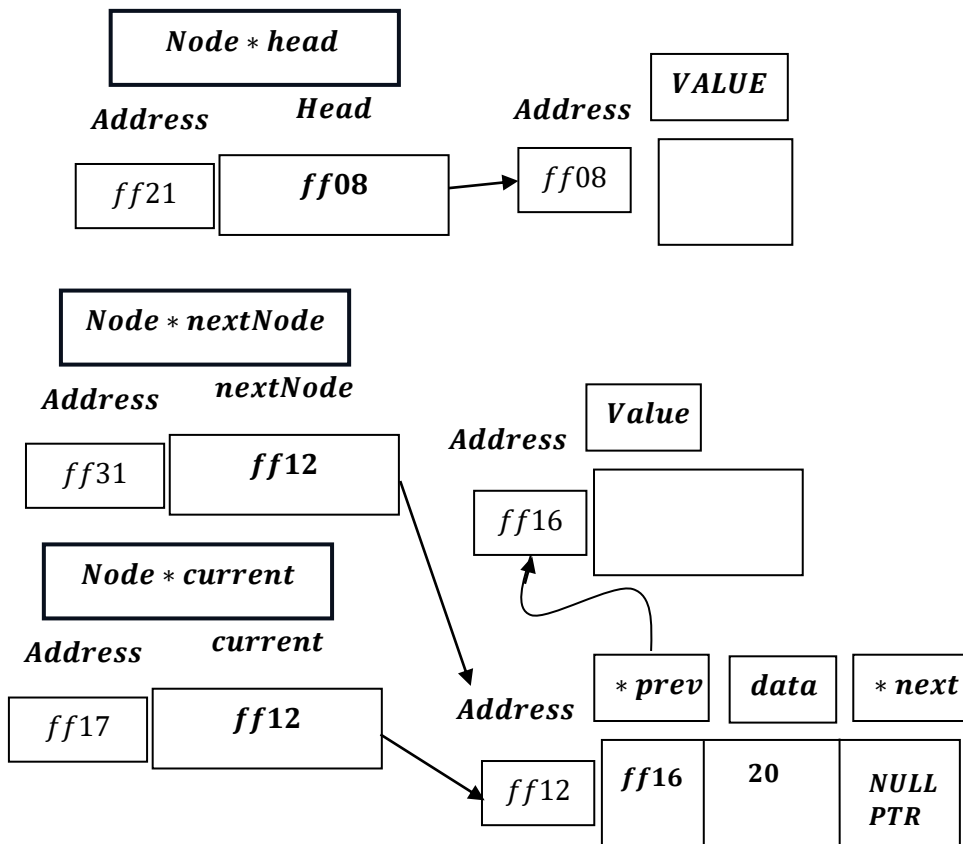


***current* $\neq$ NULLPTR is true ,hence:**

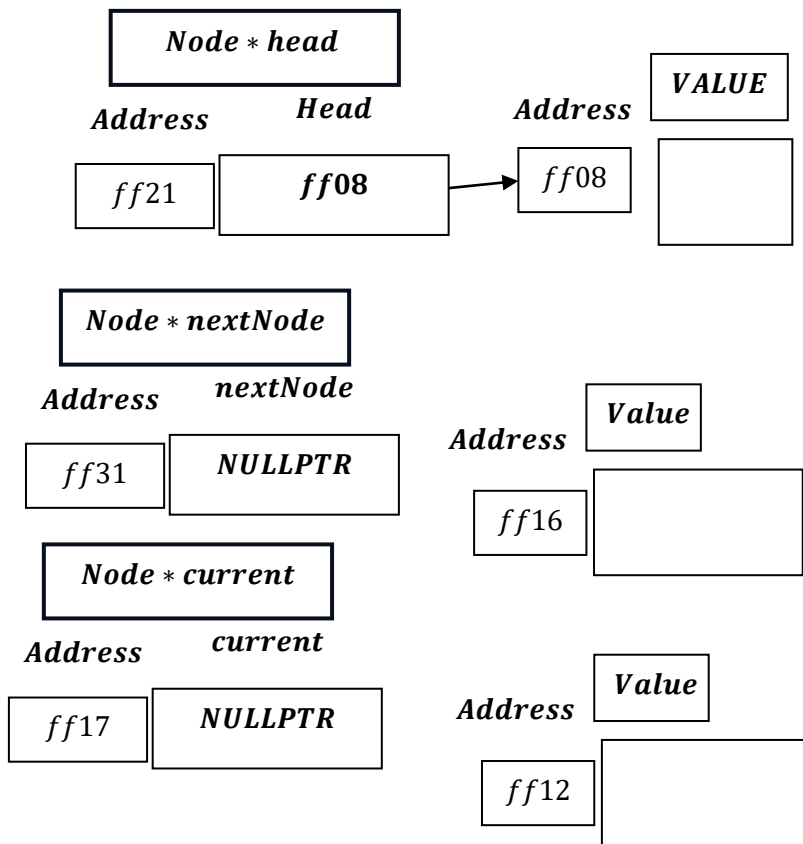
***nextNode* = *current* $\rightarrow$ *next* = *ff12*.**

free(current);

***current* = *nextNode* = *ff12*.**

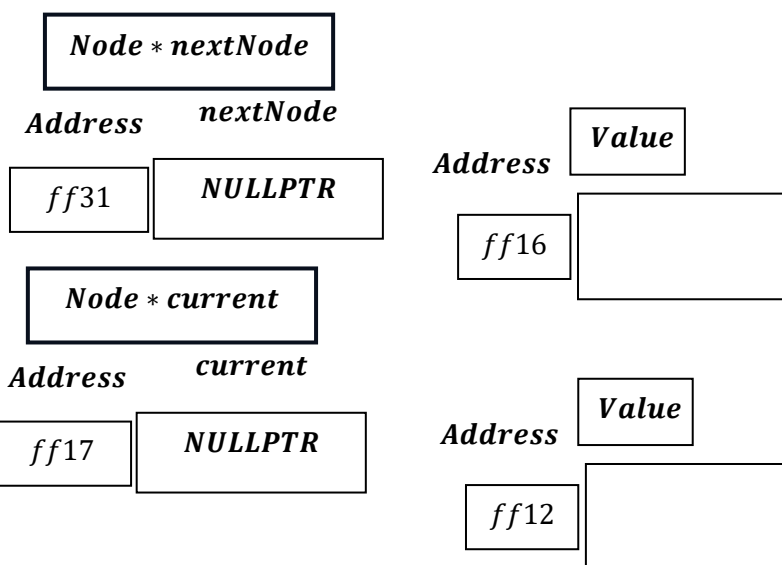
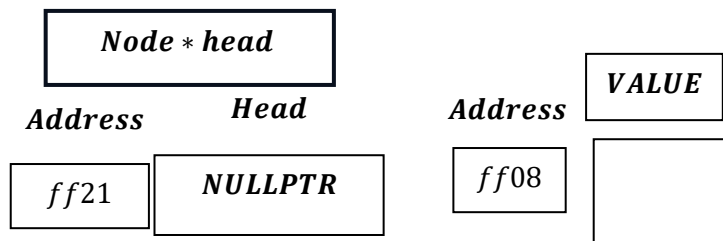


current $\neq$ *NULLPTR* is true , hence:
nextNode = *current* $\rightarrow$ *next* = *NULLPTR*.
free(current);
current = *nextNode* = *NULLPTR*.

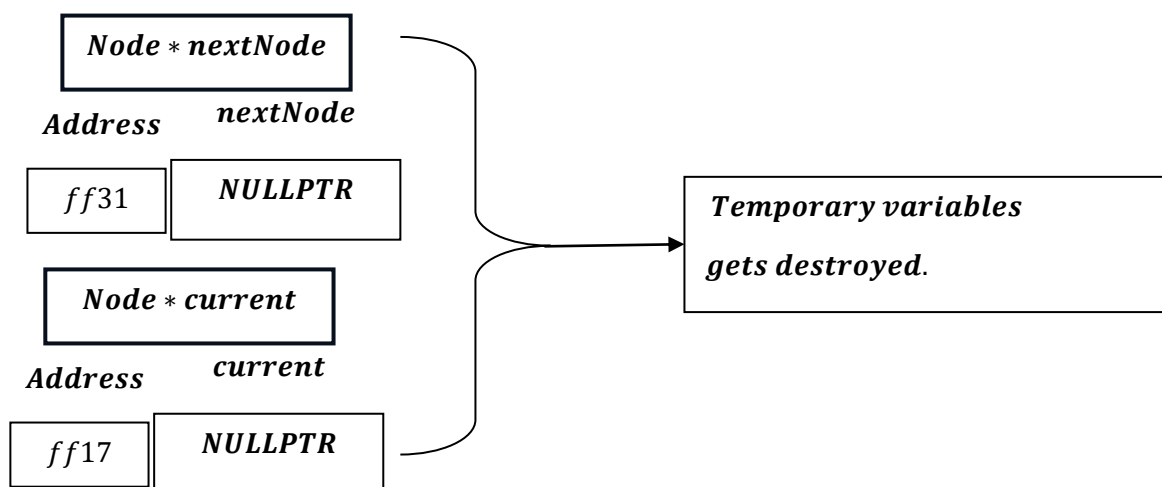


Now, $CURRENT = NULLPTR$, hence while ($TEMP \neq NULLPTR$) $\rightarrow$ condition fails .

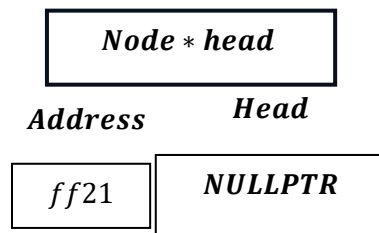
```
head = nullptr;
```



As function ends ,nextNode and current pointer, gets destroyed:



And we are left with:



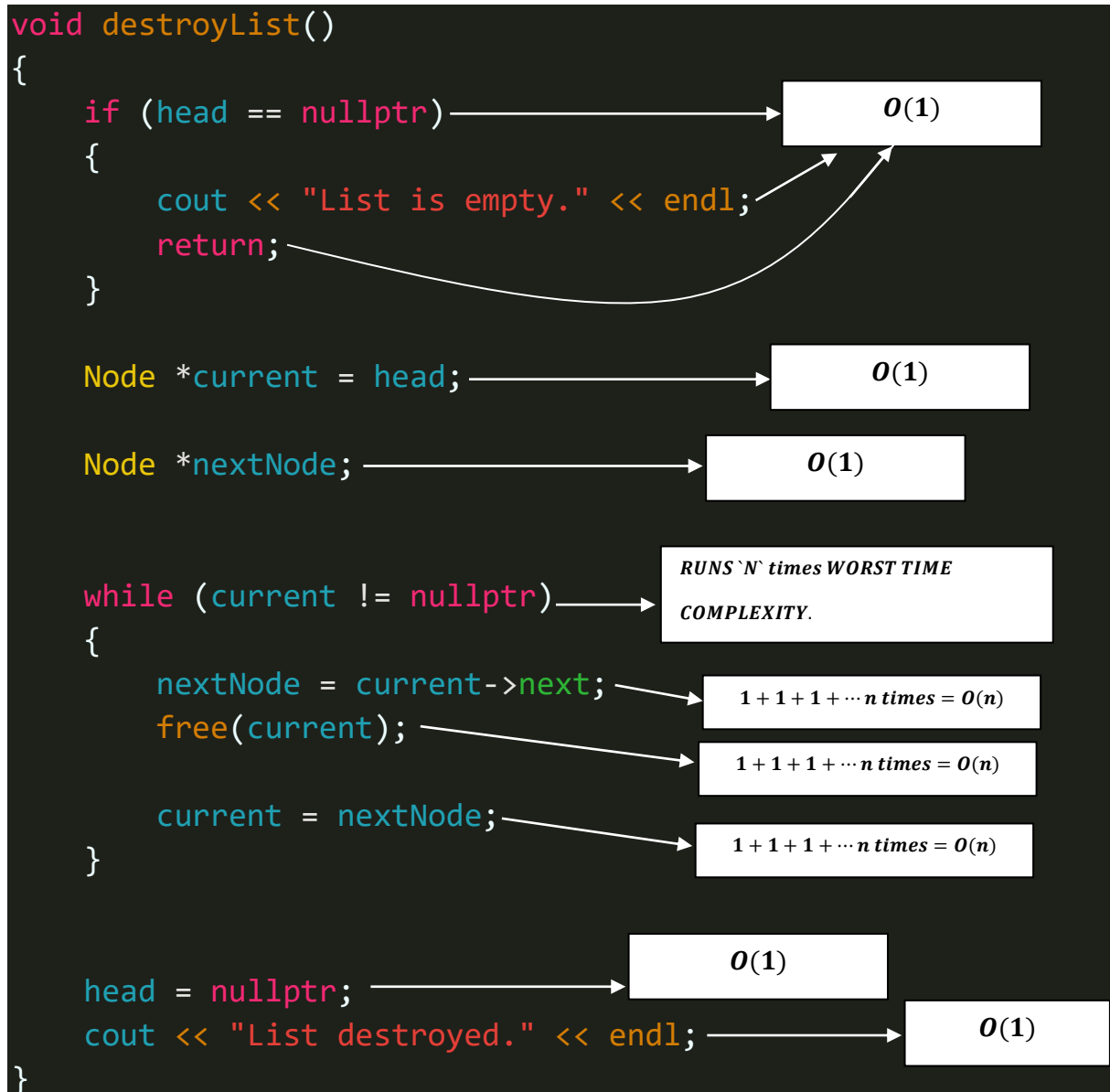
Doubly Linked List – Destroy List [Working Summary]:

This C++ function, `destroyList`, is designed to safely deallocate all the memory used by a linked list, effectively deleting it.

Working Summary

1. **Handles Empty List:** It first checks if the list is already empty (`head == nullptr`). If so, it prints a message and exits, as there's nothing to destroy.
 2. **Initializes Pointers:** If the list exists, it creates two pointers:
 - `current`: Starts at the `head` of the list.
 - `nextNode`: Is used to temporarily store the link to the next node.
 3. **Iterates and Deletes:** The function enters a `while` loop that continues as long as `current` is not `nullptr`. In each iteration:
 - It **saves the location** of the next node (`nextNode = current->next`) before deleting the current one. This is crucial to avoid losing the rest of the list.
 - It **frees the memory** occupied by the `current` node using `free(current)`.
 - It moves forward by setting `current` to the saved `nextNode`.
 4. **Resets Head Pointer:** After the loop has deleted every node, it sets the `head` pointer to `nullptr`. This is an important final step to ensure the list is officially marked as empty and prevents any dangling pointer issues.
 5. **Confirms Destruction:** Finally, it prints "List destroyed." to confirm the operation was completed.
-

Time Complexity Of Destroy List Function Module



Program Analysis

Absolute Best Case:

When List is Empty (head = NULLPTR)

→ If `head = nullptr`, this checking takes constant time. The `cout` and `return` statements take constant time. Hence overall process takes, $O(1)$ time.

Worst Case(When List is not Empty)

→ ***Node * current = * head; this assignment takes constant time i.e. $O(1)$.***

→ ***Declaration of Node * nextNode ; takes constant time i.e. $O(1)$.***

→ ***while(current != nullptr) checking becomes true , then:***

→ ***nextNode = current → next; such assignment takes, $1 + 1 + \dots n$ times = n ;***

→ ***free(current) ; freeing memory pointed by current pointer;***

$1 + 1 + \dots n$ times = n ;

→ ***current = nextNode; such assignment takes, $1 + 1 + \dots n$ times = n ;***

Therefore ,total loop time complexity: $n + n + n = 3n = 3 \times O(n) = O(n)$.

→ ***head = nullptr; this assignment takes constant time i.e. $O(1)$.***

→ ***cout << "List destroyed."; takes constant time to execute i.e. $O(1)$.***

Hence total time complexity : $O(1) + O(1) + O(n) + O(1) + O(1) = O(n)$.

Therefore takes $O(n)$ time .

Average Case

We can say , if List is Empty , we get our best case scenario, and when list is not empty , we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine list get traversed from beginning to end , hence we can say average case is our worst case here i.e. always the entire list is traversed i.e. $O(n) = \Theta(n)$.

This means the execution time is solely dependent on the length of the list. If the list is empty, the execution time is constant (best case). If the list is not empty, the execution time is proportional to the length of the list (worst case).

There's no element of randomness or varying input data to consider. Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here. Hence , Destroy List function doesn't have a true average case.

Note ,by relation: $\Omega \leq \Theta \leq O$, we get:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$$

<i>Doubly Linked List</i>	<i>Cases</i>	<i>Time Complexity</i>
<i>Destroy List</i>	1. Best Case	$\Omega(1)$
	2. Worst Case	$O(n)$
	3. Average Case (= Worst Case)	$\Theta(n)$

Doubly Linked List – Destroy List
– Space Complexity and Algorithm

Program :

```
void destroyList()
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    Node *current = head;
    Node *nextNode;
    while (current != nullptr)
    {
        nextNode = current->next;
        free(current);
        current = nextNode;
    }
    head = nullptr;
    cout << "List destroyed." << endl;
}
...
case 12:
    destroyList();
    break;
```

Space Complexity

Auxiliary Space

Auxiliary space is the extra or temporary memory used by a function, not including the input data itself.

In the `destroyList` function, the new variables created are:

`Node *current`: A pointer to keep track of the node currently being processed. The memory for a pointer is constant.

`Node *nextNode`: A pointer to store the address of the next node before the current one is deleted. This also requires a constant amount of space.

The function's main purpose is to free memory, not allocate it. Since the amount of extra memory used is fixed (just two pointers), the auxiliary space complexity is $O(1)$.

Auxiliary Space: $O(1)$

Input Space

Input space is the memory used to store the function's inputs. This can be analyzed from two perspectives.

1. Parameter-only Definition

This view considers only the space taken by the function's direct parameters. The function signature is `void destroyList()`.

The function takes no parameters.

Under this definition, the input space is constant.

Input Space (Parameter-only): $O(1)$

2. Full-Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list.

If the linked list contains n nodes, the total memory it occupies is proportional to n .

Under this definition, the input space is linear with respect to the size of the list.

Input Space (Full-Data): $O(n)$

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, the "space complexity" of a function refers to its auxiliary space. Therefore, the most common and expected answer for this function is $O(1)$. The while loop that iterates through the list affects the function's time complexity ($O(n)$), but not its space complexity.

Algorithm: Destroy List

DDESTROY(*START*)

1. [CHECK IF LIST IS EMPTY]

 If *START* = *NULL*, then:

 Write: "List is empty."

 EXIT.

 [End of If structure.]

2. [INITIALIZE POINTERS]

 a. Set *PTR* := *START* [*PTR* is a pointer to the current node being processed].

 b. Declare *NEXT_NODE* [A pointer to hold the next node's address].

3. [TRAVERSE THE LIST AND FREE EACH NODE]

 Repeat while *PTR* ≠ *NULL*:

 a. Set *NEXT_NODE* := *NEXTLINK*[*PTR*] (*Save the address of the next node before deleting the current one*).

 b. free(*PTR*) [*Deallocate the memory of the current node*].

 c. Set *PTR* := *NEXT_NODE* [*Move the pointer to the next node*].

 [END of Loop]

4. [RESET THE START POINTER]

 Set *START* := *NULL* [*Mark the list as empty*].

5. [Output Statement That List Destroyed]

 Write: "List Destroyed."

6. Exit

15. Doubly Linked List – MEMORY REFERENCING AND FREEING MEMORY

```
#include <cstdlib>
#include <iostream>

using namespace std;

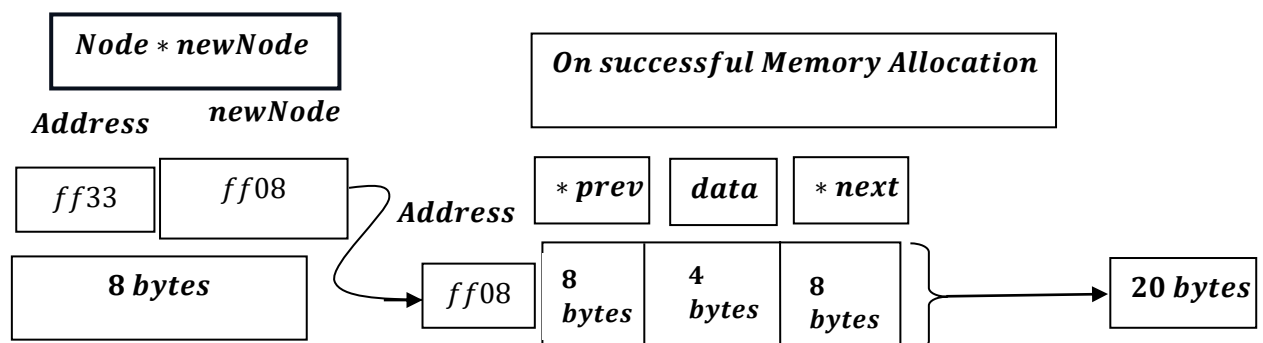
typedef struct DLL
{
    int data;
    DLL *next;
    DLL *prev;
}Node;
```

What the above code depicts: –

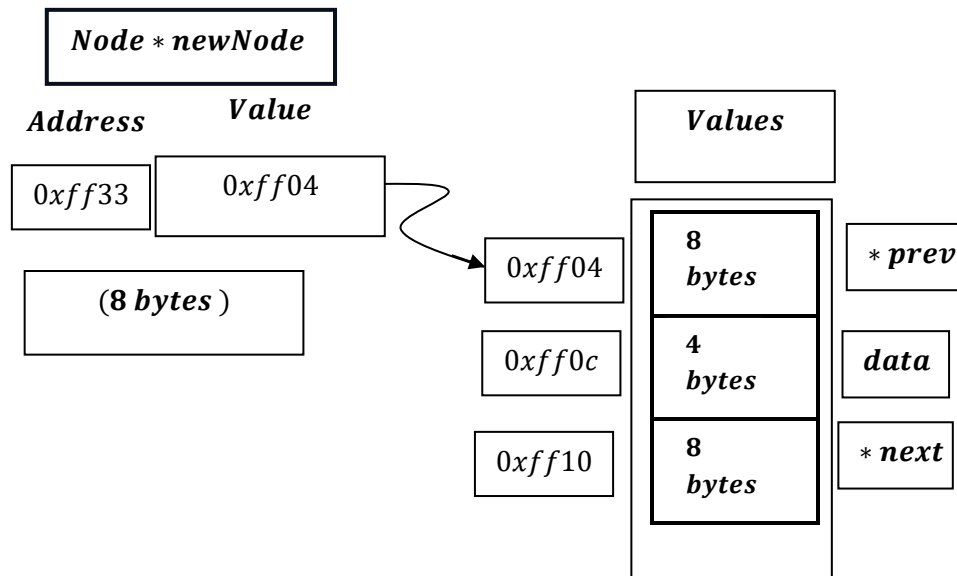
- struct whose name is DLL is renamed with Node through `typedef` keyword.
- for struct Node, there will be a memory allocation for integer variable `data`.
- for struct Node, there will be a memory allocation for pointer next.
- for struct Node, there will be a memory allocation for pointer prev.

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

We know[Based on 64 bit system]:



But memory allocation doesn't take like this, actually, this is for understanding, and for explanation, memory allocation a little bit more complex:



At now we see newNode pointer , only pointing 0xff04 , but it should update Data part and Next pointer part, how that happens?

→ The compiler knows that the data member is at the very beginning of the struct. Its offset from the start is 0 bytes.

such as: newNode → prev = nullptr.

$$\rightarrow \text{baseaddress} + \text{offset} = 0xff04 + 0 = 0xff04.$$

The Compiler's "Secret" Calculation

When the compiler sees newNode->prev = nullptr;, it thinks:

- "Get the base memory address stored in the pointer newNode." (e.g., `0xff04`)
- "Look up the definition of the Node struct. The prev member is at the beginning, so its offset is 0."

- "Calculate the final address: base address + offset => 0xff04 + 0 => 0xff04."
- "Write the value `nullptr` or NULL basically (whose output is 0)[pointer points to nothing] to the memory location starting at 0xff04."

Next is Data, an integer variable :

This comes after the prev pointer (8 bytes). Its offset is 8 bytes.

say, newNode → data = 10;

→ baseaddress + offset = 0xff04 + 8 bytes = 0xff0c [12 → c in hex].

→ And then write head pointer's value to address 0xff0c.

When it sees newNode->data = integer_value;, it thinks:

1. "Get the base memory address stored in the pointer newNode." (e.g., 0xff04)
2. "Look up the definition of the Node struct. The next member comes after prev pointer, which is 8 bytes long. So, the offset for next is 8."
3. "Calculate the final address: base address + offset => 0xff04 + 8 => 0xff0c."
4. "Write the integer_value to the memory location starting at 0xff0c."

But this doesn't mean, value of newNode pointer gets changed and it starts to point to address of data variable, it continues to point to prev pointer.

Next is next, a pointer variable :

This comes after prev (8 bytes) and data (4 bytes). Its offset is 12 bytes (8 + 4).

say, newNode → next = nullptr;

→ baseaddress + offset = 0xff04 + 12 bytes = 0xff10 $\left[\begin{array}{l} 04 + 12 = 16 \\ \text{And integer 16} \approx \text{hex 10} \end{array} \right]$.

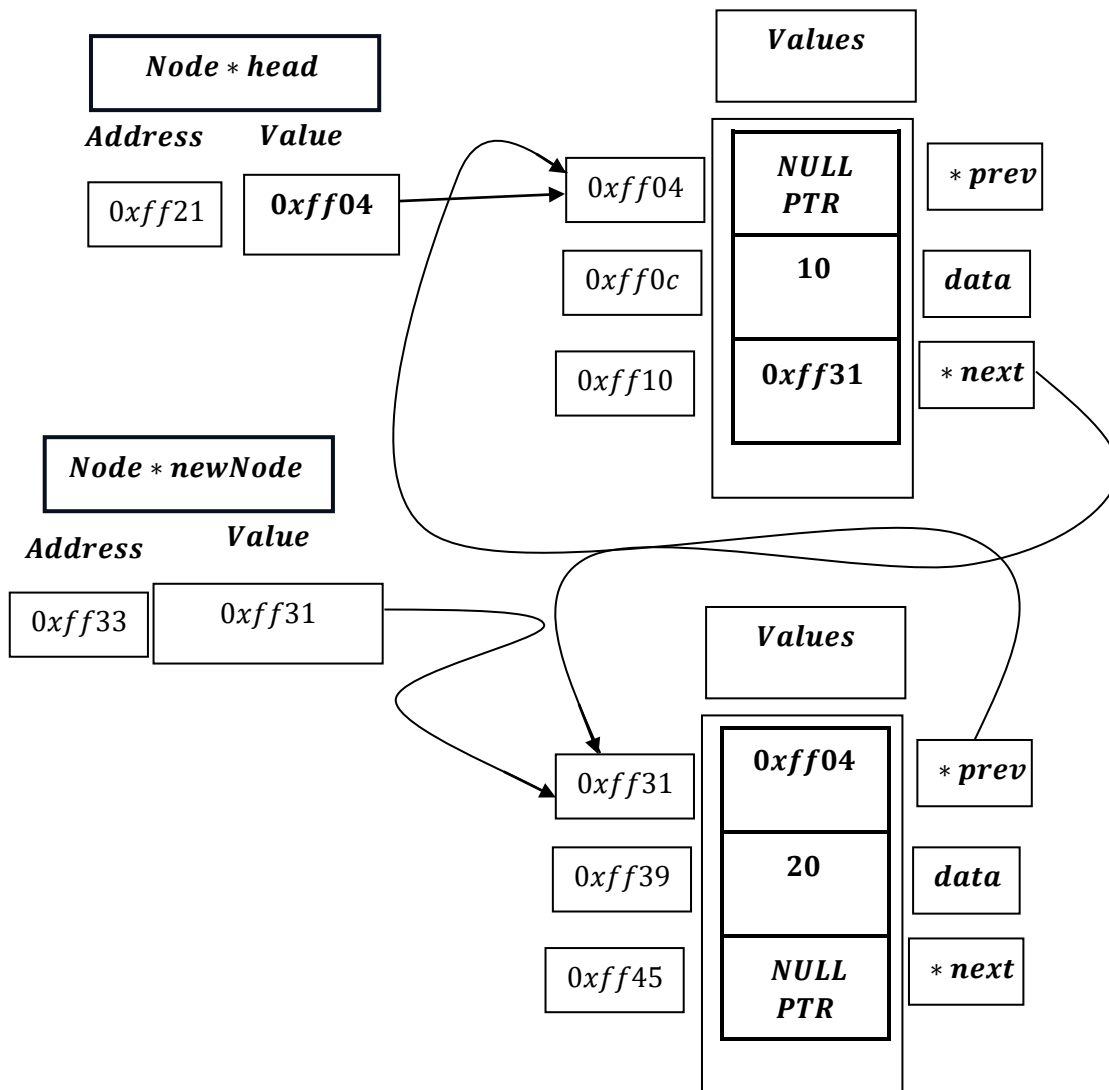
→ And then write head pointer's value to address 0xff10.

When it sees `newNode->next = nullptr;`, it thinks:

1. "Get the base memory address stored in the pointer `newNode`." (e.g., 0xff04)
2. "Look up the definition of the `Node` struct. The next member comes after prev pointer and integer data, which is (8+4) bytes long. So, the offset for next is (8+4)=12."
3. "Calculate the final address: base address + offset => 0xff04 + 12 => 0xff10."
4. "Write the value `nullptr` or `NULL` basically (whose output is 0)[pointer points to nothing] to the memory location starting at 0xff10."

But this doesn't mean, value of `newNode` pointer gets changed and it starts to point to address of next pointer variable, it continues to point to prev pointer.

Now, accessing from prev pointer from `newNode` pointer:



newNode → prev → prev; Now there is two path:

→ newNode → prev: 0xff31 + 0 = 0xff04 and new → prev → prev: (0xff04) → prev: 0xff04 + 0 = 0xff04.

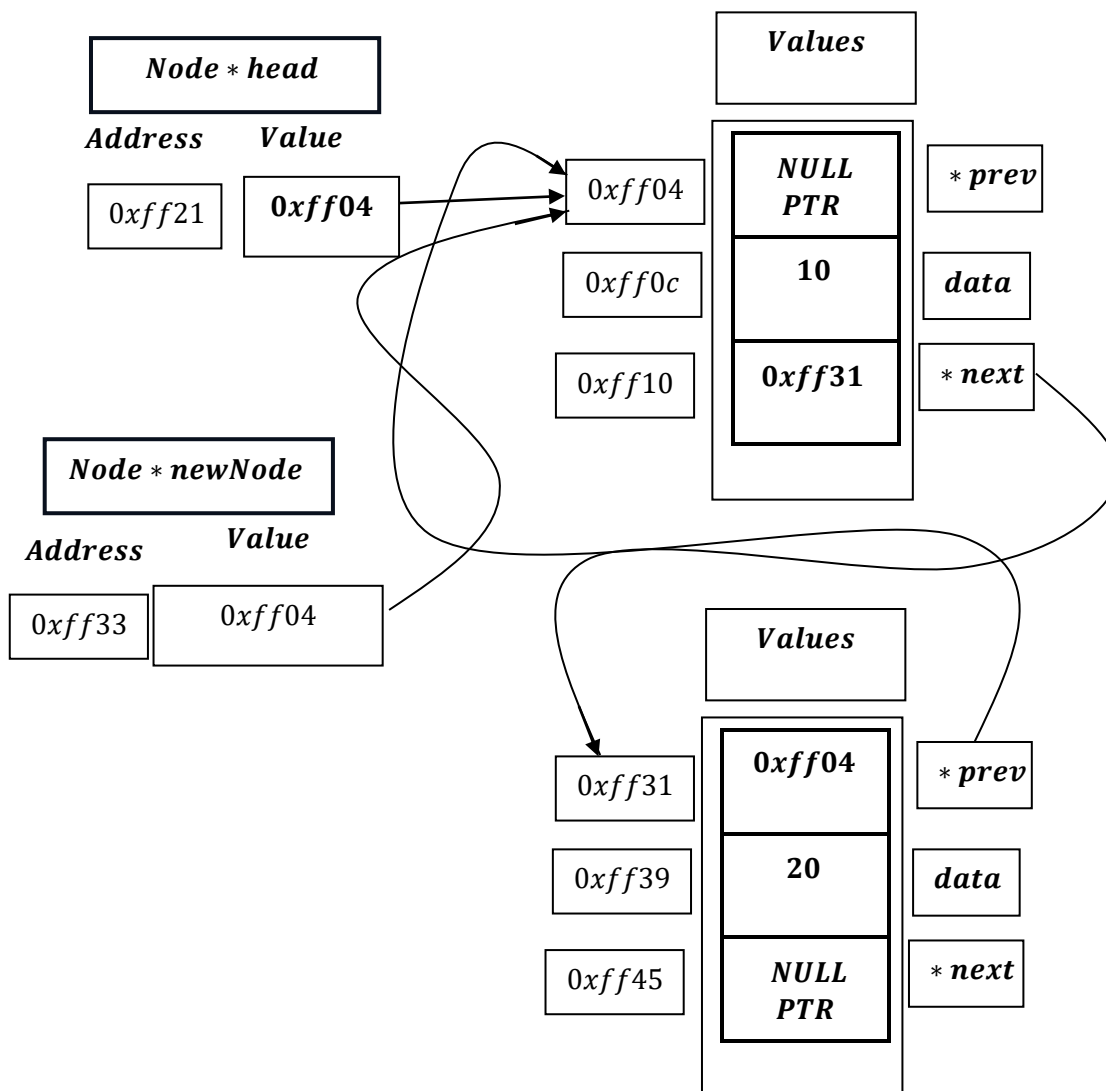
newNode → prev → data; Now there is two path:

→ newNode → prev: 0xff31 + 0 = 0xff04 and new → prev → data: (0xff04) → data: 0xff04 + 8 = 0xff0c.

newNode → prev → next; Now there is two path:

→ newNode → prev: 0xff31 + 0 = 0xff04 and new → prev → next: (0xff04) → next: 0xff04 + 12 = 0xff10.

Again,



newNode → next → prev; Now there is two path:

→ newNode → next: $0xff04 + 12 = 0xff31$ and new → next → prev: $(0xff31) \rightarrow prev: 0xff31 + 0 = 0xff31$.

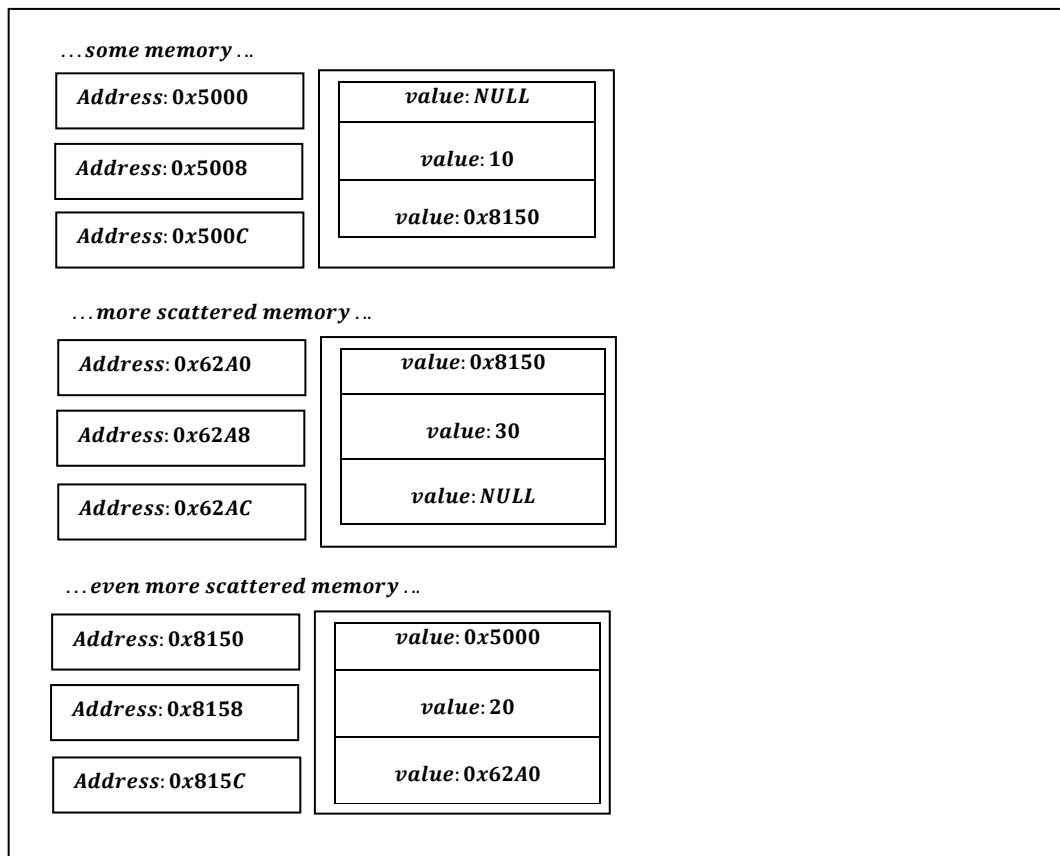
newNode → next → data; Now there is two path:

→ newNode → next: $0xff04 + 12 = 0xff31$ and new → next → data: $(0xff31) \rightarrow data: 0xff31 + 8 = 0xff39$.

newNode → next → next; Now there is two path:

→ newNode → next: $0xff04 + 12 = 0xff31$ and new → next → next: $(0xff31) \rightarrow next: 0xff31 + 12 = 0xff45$.

How does Heap memory looks like : –



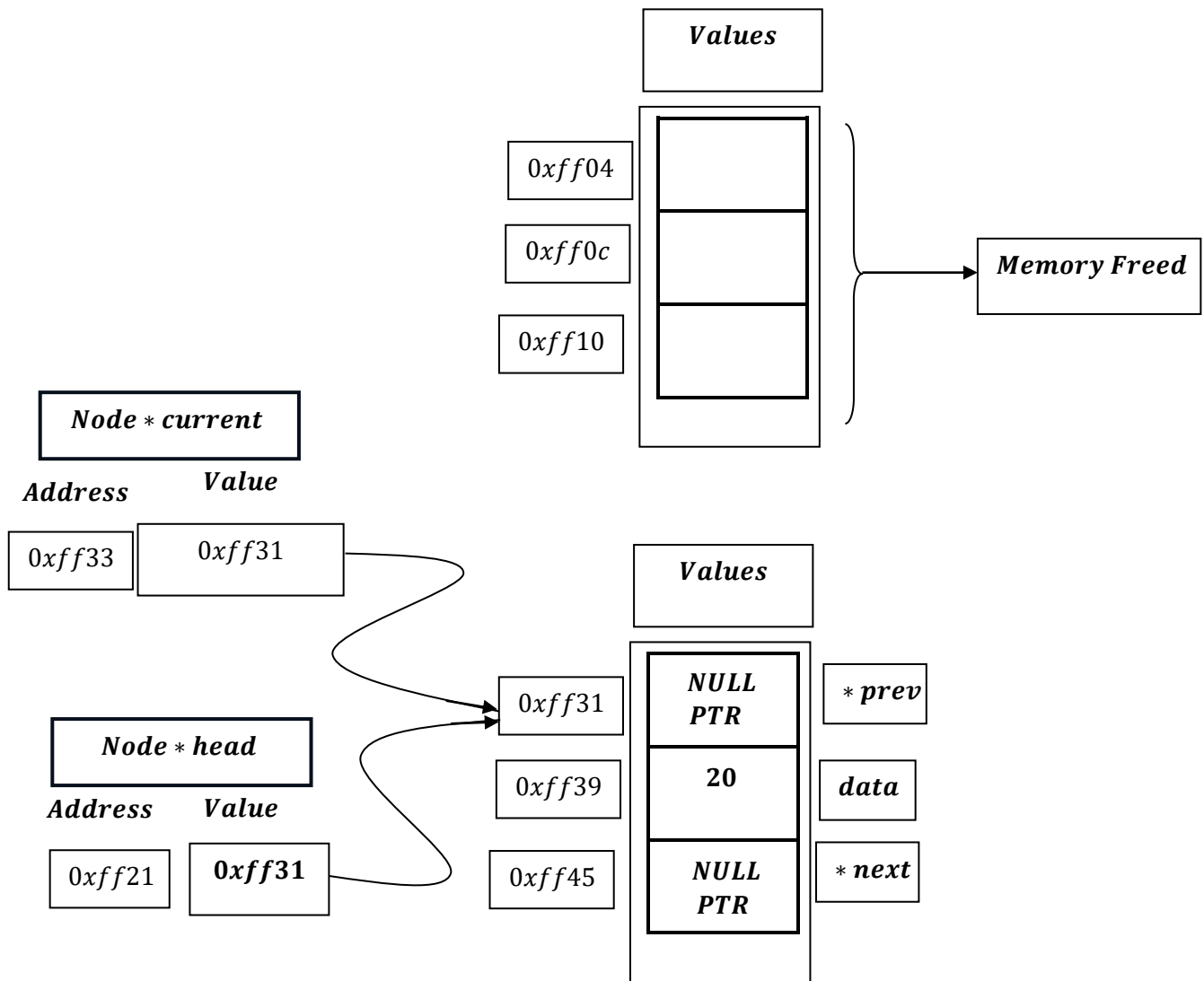
Heap Memory showing how Linked List is formed.

Freeing up the Memory

```
free(*current);
```

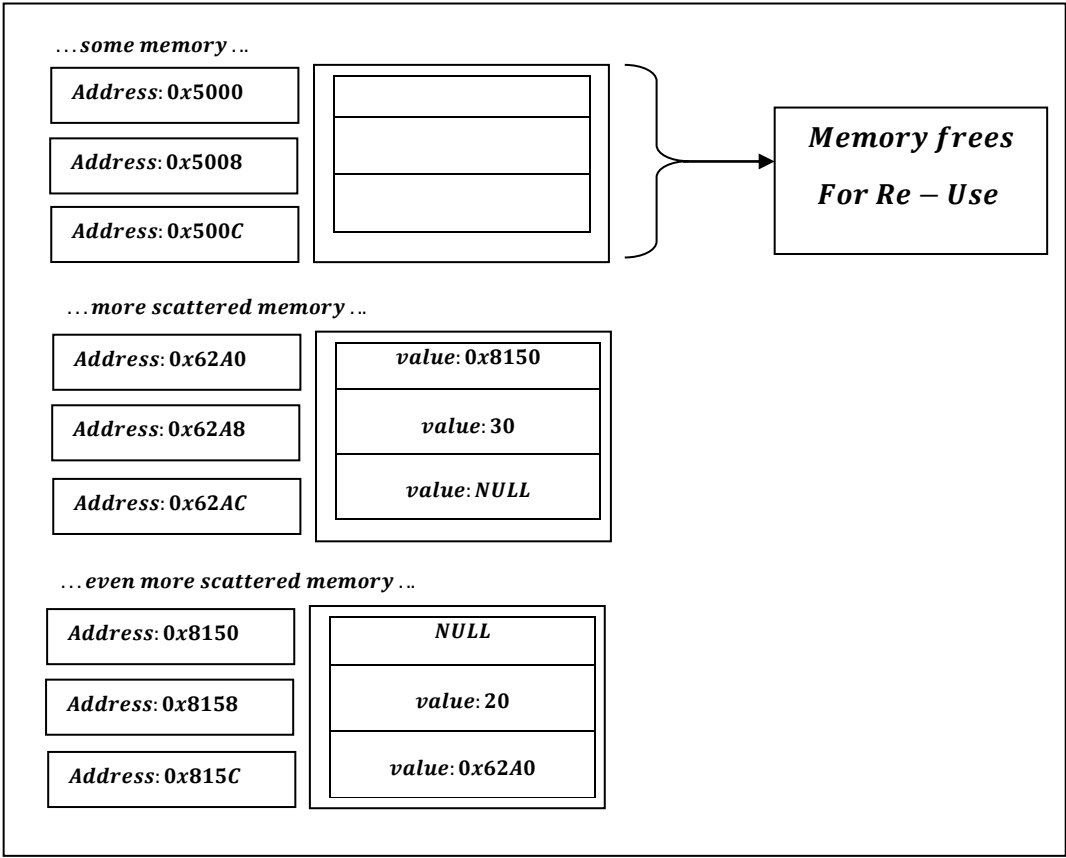
The free(current) command frees the entire memory block allocated for that node, which includes the space for both prev pointer, integer data value and the next pointer.

free () does not know or care that the memory block is being used for a struct with different members. It only knows the starting address (current) and the size of the block that was originally allocated by malloc. It returns that entire chunk of memory to the operating system's heap for reuse.



A simple demonstration of deletion from first node.

How heap memory looks like?



*Doubly Linked with Class –
Destroy List[Tweak Only]*

At case 12: when we call →

```
case 12:
    list.destroyList();
    break;
```

Then :

```
private:
    // Helper flag to avoid double messages when destructor
    is called
    bool isDestroying = false;

    // This method is called by the destructor
    void prepareForDestruction()
    {
        isDestroying = true;
    }
};
```

→ isDestroying remain false .

```
void destroyList()
{
    if (head == nullptr )
    {
        // Only print a message if it's a user action.
        if (!isDestroying)
        {
            // isDestroying flag prevents double "List
            is empty" message from destructor
            cout << "List is already empty." << endl;
        }
    }
}
```

```

        return;
    }
.....
    if (!isDestroying){
        cout << "List destroyed." << endl;
    }
}

```

Here , !isDestroying means isDestroying = False ∴

it will first check head = NULL , if the condition is true , then:

! isDestroying [isDestroying = false] is True, then:

print message: "List is already empty";

Retrun → Exit from the function.

If List is not empty, then , List will get emptied and head will get assigned to

NULL and as ! isDestroying [isDestroying = false] is True, then:

print message: "List destroyed";

→ This is Manual Destruction of List.

Automatic Destruction

But when the Program exits , the driver main function finishes , the list object goes out of scope.

```

DoublyLinkedList list; -> Goes out of scope as Program
exits.

```

C++ automatically calls its destructor (~DoublyLinkedList()) to prevent memory leaks. This is a background clean up operation and Printing "List destroyed." at this point will be redundant and confusing for the user hence,

```
~DoublyLinkedList()
{
    prepareForDestruction();
    destroyList();
}
```

Now, prepareForDestruction() ; gets executed.

```
private:
    // Helper flag to avoid double messages when destructor
    is called
    bool isDestroying = false;

    // This method is called by the destructor
    void prepareForDestruction()
    {
        isDestroying = true;
    }
};
```

isDestroying will be set to true ,hence,

```
void destroyList()
{
    if (head == nullptr )
    {
        // Only print a message if it's a user action.
        if (!isDestroying)
        {
```

```

        // isDestroying flag prevents double "List
is empty" message from destructor
        cout << "List is already empty." << endl;
    }
    return;
}
.....
    if (!isDestroying){
        cout << "List destroyed." << endl;
    }
}

```

it will first check head = NULL ,if the condition is true ,then:

*!isDestroying [isDestroying = false]is False, the inner
if statement doesn't get executed.*

Retrun → Exit from the function.

if List is not Empty or Head ≠ NULL then:

```

void destroyList()
{
    if (head == nullptr ){....}

    Node *current = head;
    Node *nextNode;
    while (current != nullptr)
    {
        nextNode = current->next;
        delete current;
        current = nextNode;
    }
    head = nullptr;
    if (!isDestroying){....}
}

```

Line of Codes Gets Executed:

```
Node *current = head;
    Node *nextNode;
    while (current != nullptr)
    {
        nextNode = current->next;
        delete current;
        current = nextNode;
    }
    head = nullptr;
```

And then again:

```
    if (!isDestroying){
        cout << "List destroyed." << endl;
    }
```

if(! isDestroying){...}is aborted as :

! isDestroying [isDestroying = false]is False.

i. e. silently destroying the List without printing any message to user.

Doubly Linked List – Memory Allocation

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

→ The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.

→ The size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

→ The size of the Node structure itself is determined by the sum of the sizes of its members:

→ data(an integer) is typically 4 bytes.

→ next(a pointer to another Node) is typically 8 bytes.

→ prev(a pointer to previous Node) is typically 8 bytes.

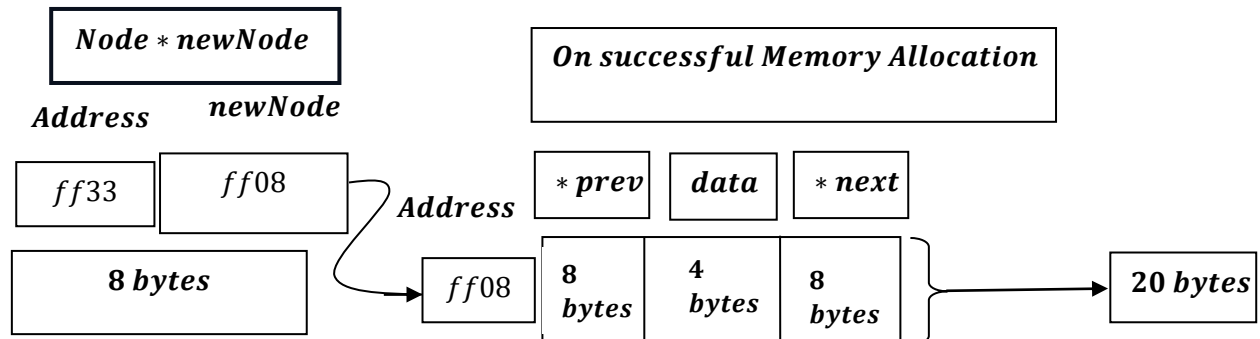
→ Therefore ,the size of the Node structure is typically 20 bytes(4 bytes for data + 8 bytes for next + 8 bytes for prev) .

```
Node *newNode = (Node *)malloc(sizeof(Node));
```

What does it mean?

→ newNode will try to allocate memory for data i.e. 4 bytes ,next pointer variable i.e. typically 8 bytes, and prev pointer variable that is typically 8 bytes i.e. total 20 bytes, as discussed earlier , the size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.

Say,



The above is Theoretical , But if we do a reality check:

Theoretical vs. Real – World Allocation

In a real programming environment (like the C ++ environments),

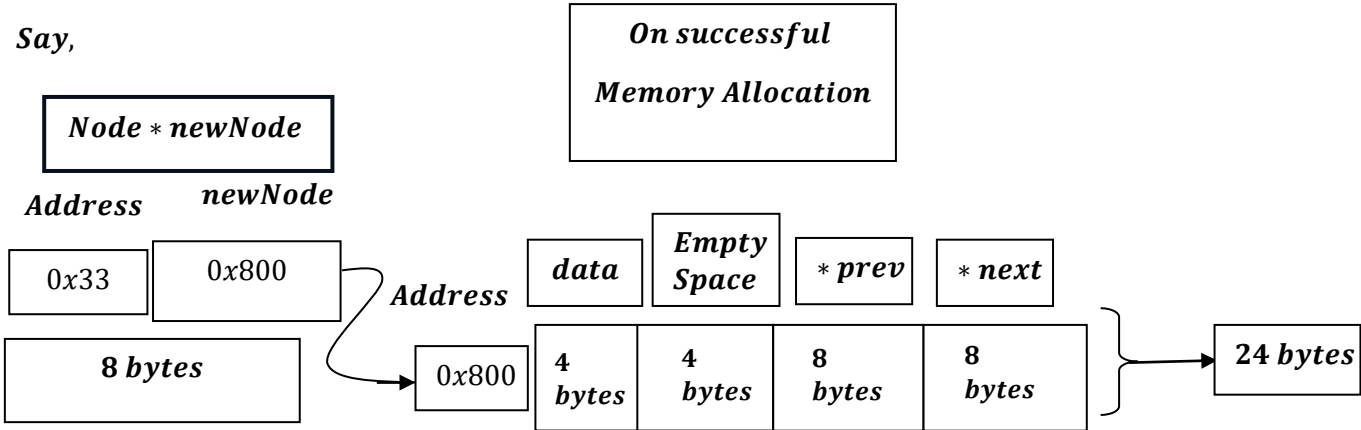
*the allocation might not be exactly **20 bytes** because of how CPUs access memory:*

- **Memory Alignment:** Most 64 – bit compilers align data to **8 – byte boundaries** to improve performance.
- **Padding:** To align the 8 – byte `next` pointer, the compiler will often add **4 bytes of "padding"** after the 4 – byte `data` integer.

Additionally, since there is another 8 – byte pointer (`prev`), the structure must maintain proper alignment for both pointers.

- **Actual Size:** Consequently, `sizeof(Node)` would likely return **24 bytes** on your system, rather than the theoretical **20 bytes**.

<i>Offset</i>	<i>Size</i>	<i>Content</i>	<i>Description</i>
0	4 bytes	<i>data</i>	<i>Integer value</i>
4	4 bytes	<i>padding</i>	<i>Empty space (alignment)</i>
8	8 bytes	<i>next</i>	<i>Address of next node</i>
16	8 bytes	<i>prev</i>	<i>Address of previous node</i>



In a 64 – bit environment, the compiler aligns memory to the size of the largest primitive member in a structure – in this case, the 8 – byte next pointer.

Because the data integer only occupies 4 bytes, the compiler inserts empty space (padding) to ensure the pointer starts at an address that is a multiple of 8.

Such as:

$$0x500 = 5 \times 16^2 + 0 \times 16^1 + 0 \times 16^0 = 5 \times 256 = 1280$$

$$1280 \div 8 = 160 \text{ (no remainder)}$$

$$0x300 = 3 \times 16^2 + 0 \times 16^1 + 0 \times 16^0 = 3 \times 256 = 768$$

$$768 \div 8 = 96 \text{ (no remainder)}$$
